

CURSO COMPLETO

PYTHON

IMPRESSONADOR

PARTE 4 • AVANÇADO

SUMÁRIO

MÓDULO 31 RPA COM PYTHON – AUTOMAÇÕES DE PROCESSOS

2 4 6	Introdução	6
2 4 7	Automatizando Processos e Sistemas	7
2 4 8	Automação de ERPs (Totvs, SAP e outros) com Python	27

MÓDULO 32 REGEX – EXPRESSÕES REGULARES

2 4 9	Introdução a Regex	43
2 5 0	Caracteres Especiais do Regex	47
2 5 1	O bom uso de Regex – Encontrando Padrões	50
2 5 2	Como analisar a resposta que o regex vai encontrar	53
2 5 3	Outros exemplos para treinar	56
2 5 4	Sequência de Caracteres	58
2 5 5	Diferentes formas de resposta no Regex	63
2 5 6	Exercício Desafiador – Extrairlink de um texto	68
2 5 7	Outros operadores de Regex	73



SUMÁRIO

MÓDULO 33 INTEGRAÇÃO PYTHON COM APIS E JSON

258	Python e Integrações de API	78
259	API de Cotação de Moeda	82
260	Mais exercícios de API com Cotação de Moeda	84
261	API com Login – Envio de SMS com Python e Twilio	88
262	Rodar códigos com o computador desligado, agendar diariamente, semanalmente, mensalmente	92
263	API de CEP e Busca de Endereços	98
264	API de Preço de Ações e Informações Econômicas	102
265	API do IBGE – Query Builder	108
266	API do Banco Central – APIs com Paginação	111
267	API do Google Sheets – Integrar Python com Google Sheets	114
268	Trade de Criptomoedas com Python – API da Binance	122
269	Firebase e Python – Introdução a REST API do Firebase	127
270	RapidAPI – Como usar outras APIs de 1 só lugar	134

MÓDULO 34 INTEGRAÇÃO PYTHON PARA FINANÇAS

271	Python para Finanças	138
272	Retorno IBOV e Média Móvel	141
273	Pegando Cotações Carteira com Python	143
274	Tratando Problemas de Importação para o Dataframe e Normalização....	148
275	Criando Valor da Carteira e Puxando Ibovespa	151
276	Comparando a Carteira com Ibovespa	153

MÓDULO 35 AMBIENTES VIRTUAIS

277	O que é um Ambiente Virtual?	157
278	Navegando no Prompt de Comando	158
279	Criando, Ativando e Removendo Ambientes Virtuais	161
280	Observações Importantes de Ambientes Virtuais	164
281	Ambientes Virtuais no PyCharm	165
282	Ambientes Virtuais no VSCode	168



SUMÁRIO

MÓDULO 36

INTEGRAÇÃO DE PYTHON COM POWER BI

283	Como funciona a Integração de Python com o Power BI	172
284	Configurações Importantes – Python e Power BI	173
285	Usando o Python para gerar Base de Dados no Power BI	177
286	Usando o Python para Editar Tabelas do Power BI	179
287	Usando Python para criar Gráficos no Power BI	183
288	Gráficos Seaborn no Power BI.....	189

MÓDULO 37

TRANSFORMANDO PYTHON EM EXE

289	Instruções Importantes	194
290	Python para exe com Arquivos Simples	195
291	Diminuindo Tamanho de Arquivos Executáveis	199
292	Adaptando o Código de Arquivos Complexos	200
293	Python para exe com Arquivos Complexos	204
294	Executando Códigos em Python Automaticamente (Diariamente, Semanalmente, Mensalmente)	209





PYTHON

Módulo 31

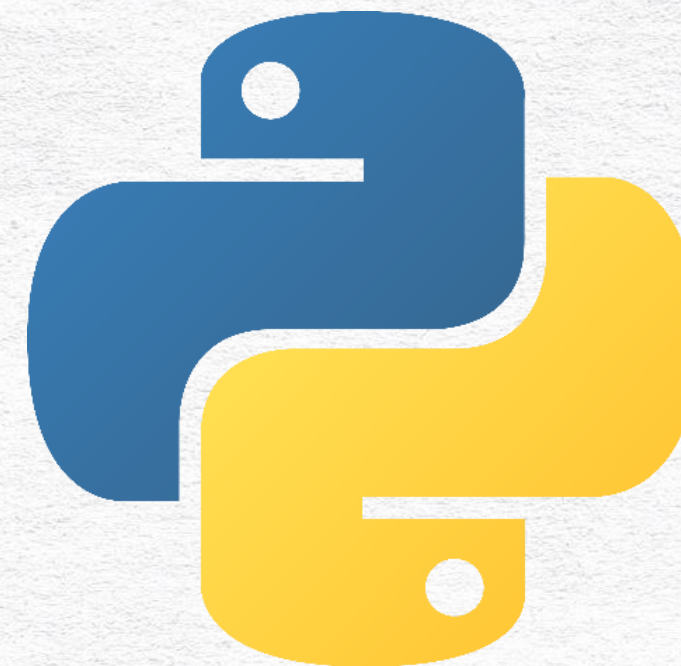
RPA com Python – Automações de Processos



Neste módulo vamos aprender a criar "**robôs de automação**" para os nossos códigos usando o PyAutoGUI.

É importante ressaltar que, para aproveitar e entender ao máximo este projeto, você já tenha uma base sólida de Python. Essa base consiste em:

- Todos os nossos módulos até “**Functions usadas como parâmetro e Expressões Lambda**”, que são a “base geral do Python”.



Se você ainda não possui esse conhecimento prévio, é possível que fique perdido em certas etapas e possa ter dificuldades em compreender algumas partes desse módulo.

No entanto, caso possua os pré-requisitos citados, pode começar esse módulo sem receio, que tudo vai estar explicado bem passo a passo.

O que é RPA?

RPA, ou **Automação de Processos Robóticos** (Robotic Process Automation), refere-se à automação de tarefas repetitivas e manuais em sistemas de software por meio de robôs ou bots.

Através do **RPA** você pode automatizar tarefas e execuções no seu computador de forma rápida e eficiente. Esses recursos são extremamente úteis para lidar com tarefas repetitivas e demoradas.

Por que o RPA é diferente do Selenium/Web-Scraping?

Tudo o que você consegue fazer com o teclado e o mouse do seu computador é possível de ser automatizado por meio do **RPA**. Enquanto com o **Selenium** conseguimos automatizar apenas processos que ocorrem na internet.

O **ponto negativo do RPA** é que, durante a execução do programa, você não pode usar o computador simultaneamente, pois ele estará controlando seu mouse e teclado nesse momento.

Outro ponto de **atenção** com o RPA é que ele não é **100% aplicável de um computador para outro**. Ao criar sua automação em uma máquina e exportá-la para outra, é necessário fazer algumas adaptações para que o programa continue funcionando corretamente.



Biblioteca PyAutoGUI

A biblioteca **PyAutoGUI** é uma das principais bibliotecas de RPA em Python. Você pode acessar a **documentação completa** dela [aqui](#) e acessar os principais comandos a partir do **Quickstart** dela disponível [aqui](#).

Desafio Prático de RPA

Para que possamos visualizar as aplicações do **PyAutoGUI** em Python na criação de uma **RPA**, separei um desafio prático em que construiremos uma automação para extrair informações de um sistema e, a partir disso, enviar um relatório por e-mail.

Passo 1 – Entrar no Sistema

Para simular nosso sistema, utilizaremos o **serviço de e-mail do Gmail**, mas essa automação pode ser adaptada para **qualquer sistema** que você tiver de trabalhar e automatizar.

Primeiramente, é necessário instalar a biblioteca **PyAutoGUI**.

No terminal, você pode utilizar o seguinte comando:

```
pip install pyautogui
```

Ou, se estiver usando o Jupyter Notebook utilize:

```
!pip install pyautogui
```

```
!pip install pyautogui
```

Os principais comando que utilizaremos do **PyAutoGUI** serão:

- **pyautogui.write()**: Escreve/digita algo.
- **pyautogui.click()**: Clica em um elemento na tela.
- **pyautogui.locateOnScreen()**: Identifica uma imagem na tela.
- **pyautogui.hotkey()**: Usa atalhos do teclado (combinação de teclas).
- **pyautogui.press()**: Aperta um botão do teclado.

Visto isso, vamos importar a biblioteca do PyAutoGUI para o nosso código.

```
import pyautogui
```

Agora podemos partir para a automação do nosso primeiro passo. Precisaremos abrir o **sistema** (nesse exemplo o navegador **Google Chrome**) e em seguida acessar o **Gmail**.

Sempre que realizarmos uma automação com o **PyAutoGUI**, é importante pensar em como faríamos isso manualmente primeiro.

Por exemplo, neste caso, poderíamos apertar a tecla **Windows**, em seguida, digitar **chrome** e pressionar **Enter**.

Vamos automatizar esse processo agora. Utilizaremos o **PyAutoGUI** para simular as ações de abrir o navegador.

```
import pyautogui

# apertar o windows do teclado
pyautogui.press('win')
# digitar chrome
pyautogui.write('chrome')
# apertar enter
pyautogui.press('enter')
```

Ao executar esse código, você verá que o **PyAutoGUI** realiza as ações rapidamente, sem tempo de espera entre um comando e outro.

Embora isso possa não ser um problema em alguns casos, é recomendável definir um intervalo entre as ações. Para isso, podemos utilizar o atributo **.PAUSE**.

```
import pyautogui

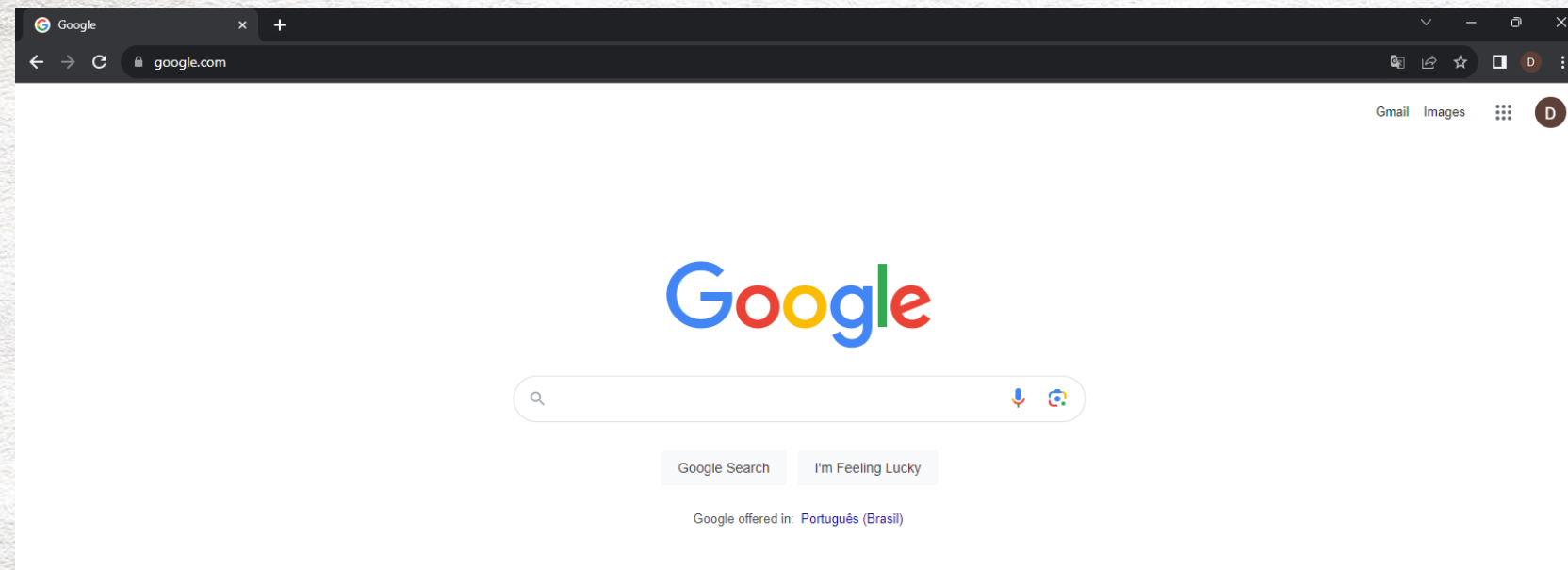
pyautogui.PAUSE = 1
# apertar o windows do teclado
pyautogui.press('win')
# digitar chrome
pyautogui.write("chrome")
# apertar enter
pyautogui.press('enter')
```

Com esse código estamos definindo uma pausa de 1 segundo entre as ações do **PyAutoGUI**. Isso significa que ele executará um comando, aguardará **1 segundo**, e executará o próximo. Se for necessário acelerar a execução, você pode ajustar o valor para **0.5** segundos, conforme a sua necessidade.

Módulo 31 – RPA com Python – Automações de Processos – Automatizando Processos e Sistemas

10

Ao executar esse código, o Chrome será aberto rapidamente em seu computador.

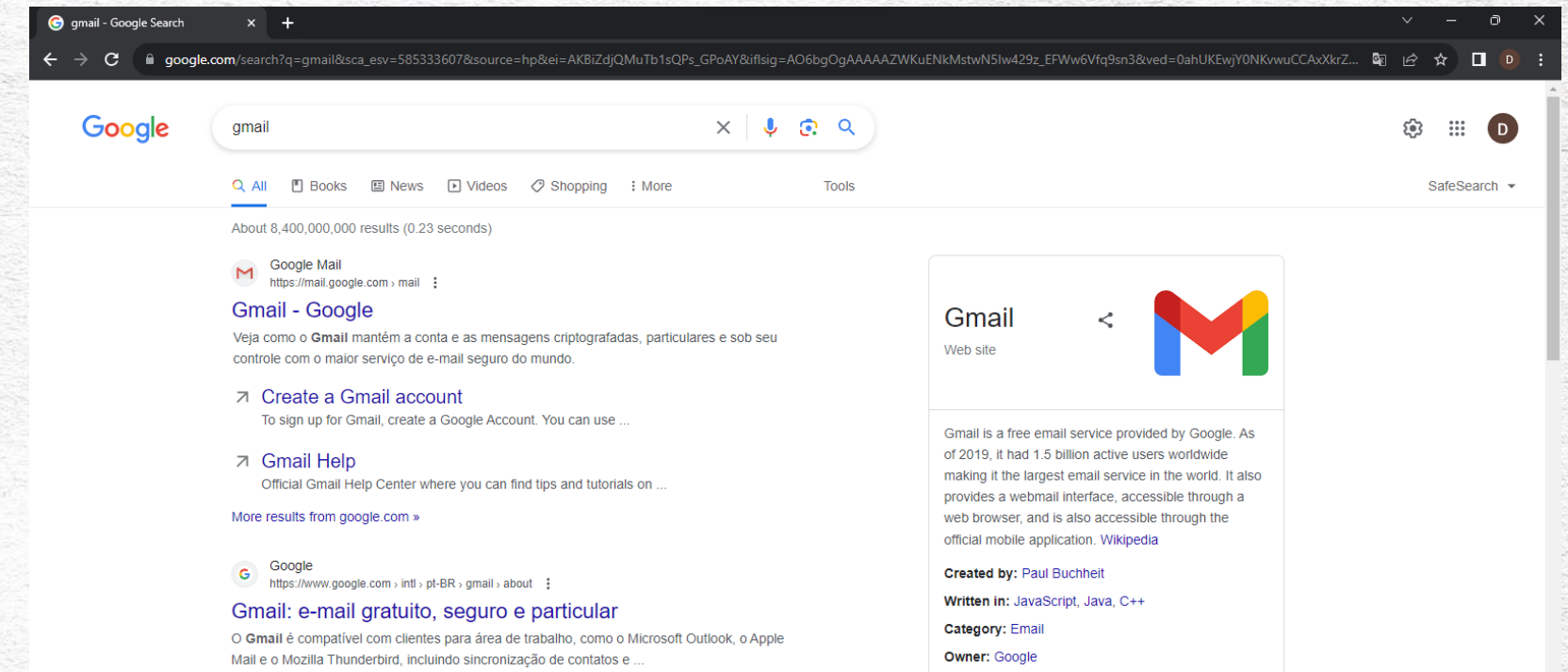


Agora, para acessar o **Gmail**, se fizéssemos isso manualmente, teríamos de digitar **gmail** na barra do navegador e apertar **Enter**. Vamos traduzir essas ações para o **PyAutoGUI**.

```
import pyautogui

pyautogui.PAUSE = 1
# apertar o windows do teclado
pyautogui.press('win')
# digitar chrome
pyautogui.write("chrome")
# apertar enter
pyautogui.press('enter')

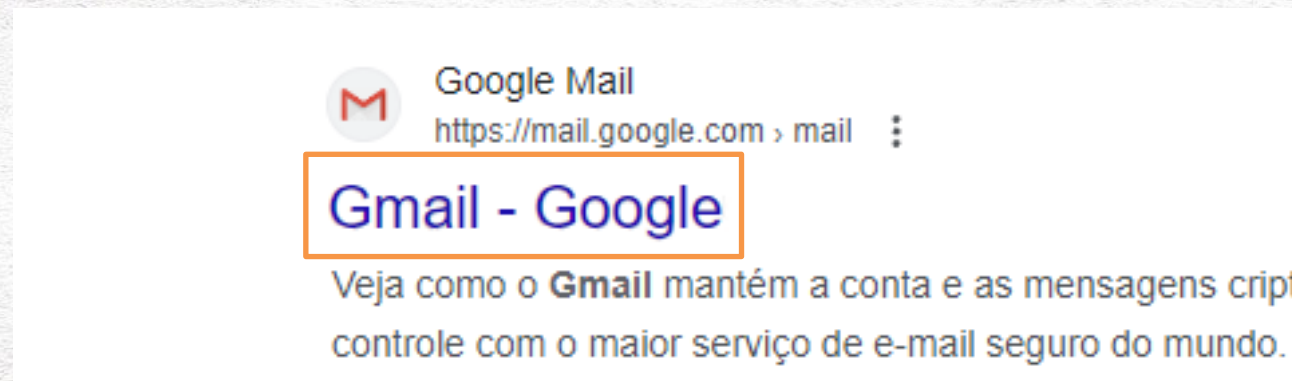
# entrar no Gmail
pyautogui.write('gmail')
pyautogui.press('enter')
```



Uma alternativa mais direta seria digitar o **endereço do Gmail diretamente**. No entanto, para proporcionar uma compreensão mais completa da automação, optaremos por um procedimento diferente, permitindo que você aprenda a automatizar diferentes comandos.

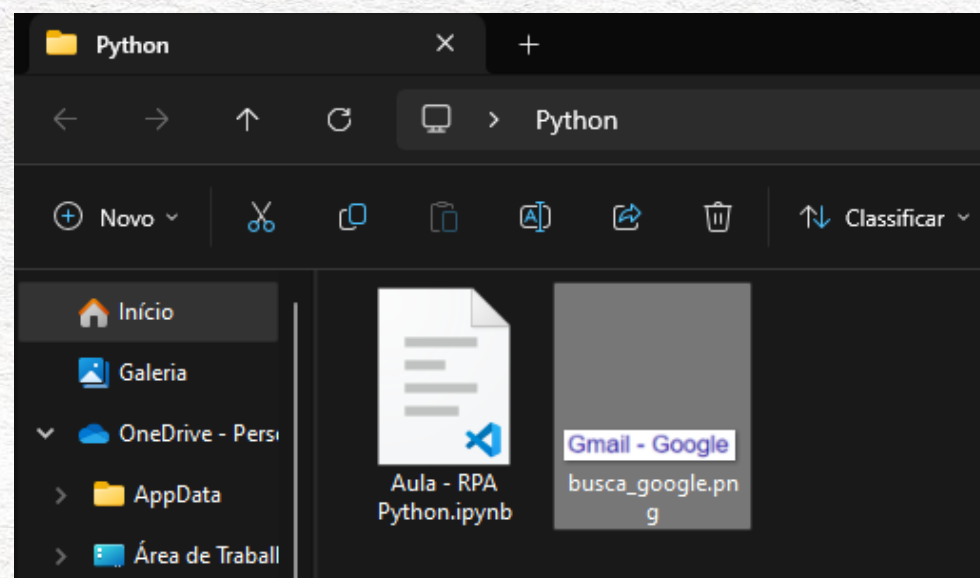


Então, a partir da tela anterior, o próximo passo será o seguinte: Tirar um **print screen** da sua tela selecionando apenas a região onde está escrito **Gmail - Google**.



Tire o print apenas da parte destacada.

Após criar a imagem, tirando o print da região destacada, salve-a na mesma pasta onde está o código da sua automação. Certifique-se de que ela seja salva como um arquivo PNG.



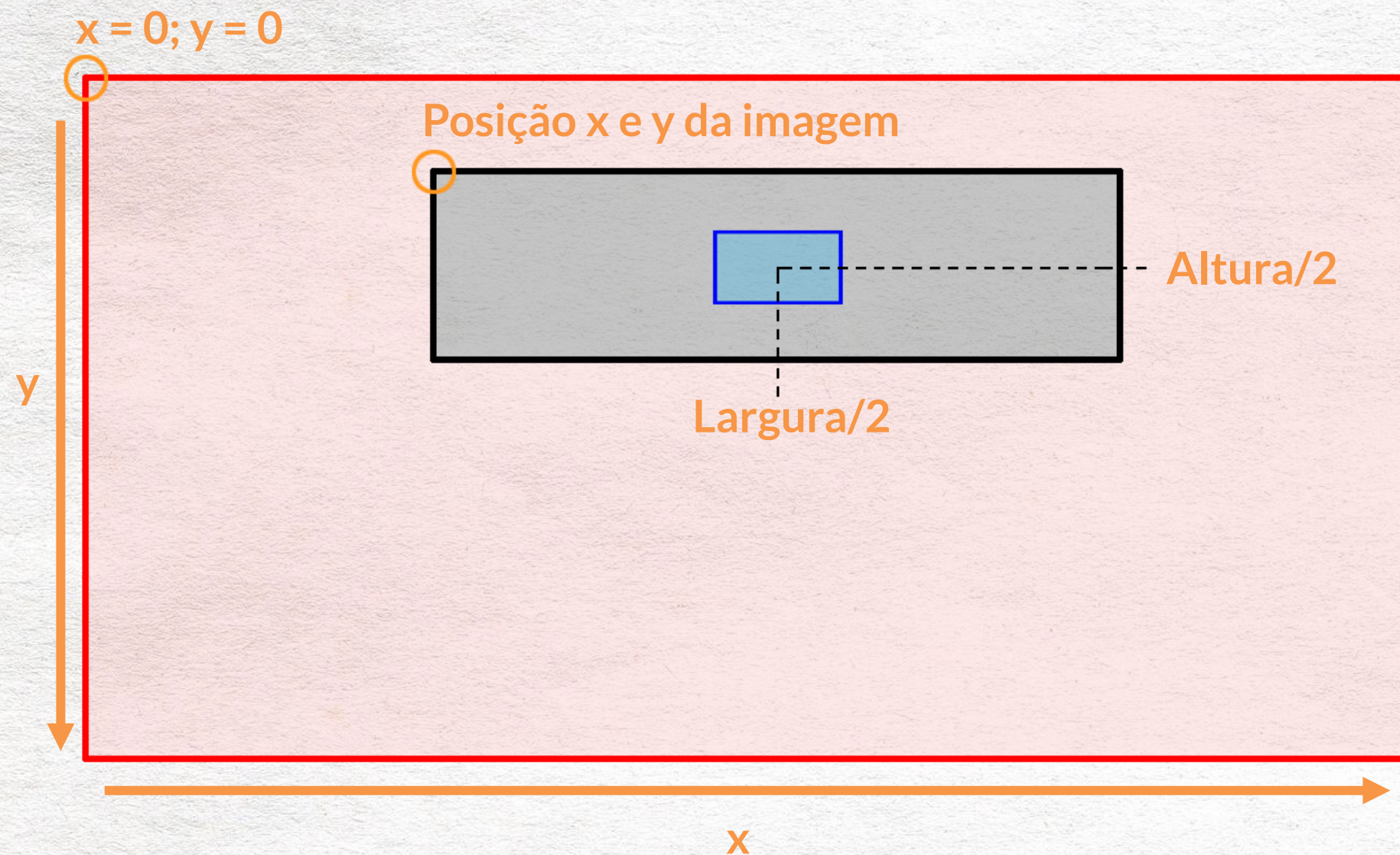
Com a **imagem salva na mesma pasta do seu código**, vamos usá-la para localizar as coordenadas na página do Google e clicar no meio dessa imagem. Isso nos permitirá acessar a página do Gmail.

Quando o **PyAutoGUI** reconhece e localiza uma imagem, ele retorna **4 informações**: posição x, posição y, largura e altura.

O **PyAutoGUI** considera o canto superior esquerdo da sua tela como sendo **x = 0** e **y = 0**.

O valor de **x** aumenta da esquerda para a direita e o valor de **y** aumenta de cima para baixo. As informações fornecidas pelo PyAutoGUI referem-se ao **canto superior esquerdo da imagem**.

Logo, para clicarmos no centro correspondente à nossa imagem, precisamos pegar a **posição x da imagem + largura/2**. E a **posição y + altura/2**.



Nessa imagem, considere o retângulo vermelho como a tela do seu computador, o retângulo preto como a área da nossa imagem e a marcação em azul como o centro da imagem (a região que queremos clicar).

Dessa forma, podemos utilizar a função `locateOnScreen()` do `PyAutoGUI`, passando o nome da nossa captura de tela como argumento.

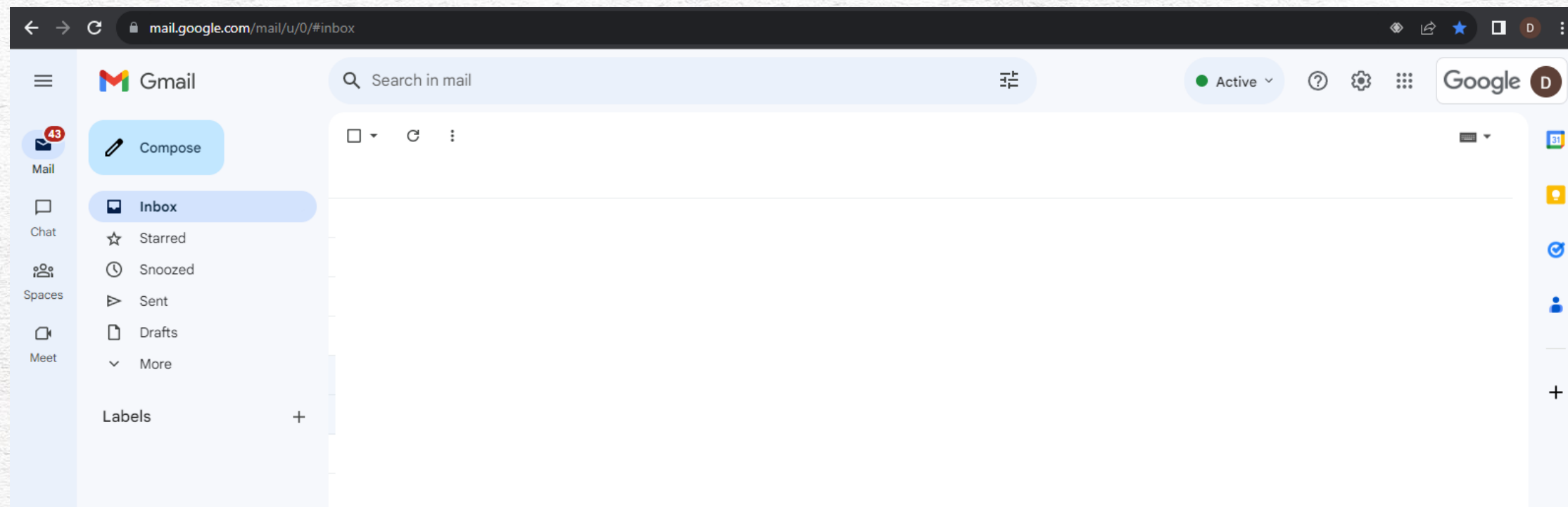
Essa função retorna uma **tupla** contendo 4 valores: **posição x**, **posição y**, **largura** e **altura**. Assim, podemos desempacotar essa tupla em 4 variáveis nessa ordem.

Para clicar no meio da imagem vamos utilizar a função `click()`, passando as coordenadas x e y . Como queremos clicar no centro da imagem, aplicaremos a lógica que vimos anteriormente:

$x + largura/2$ e $y + altura/2$

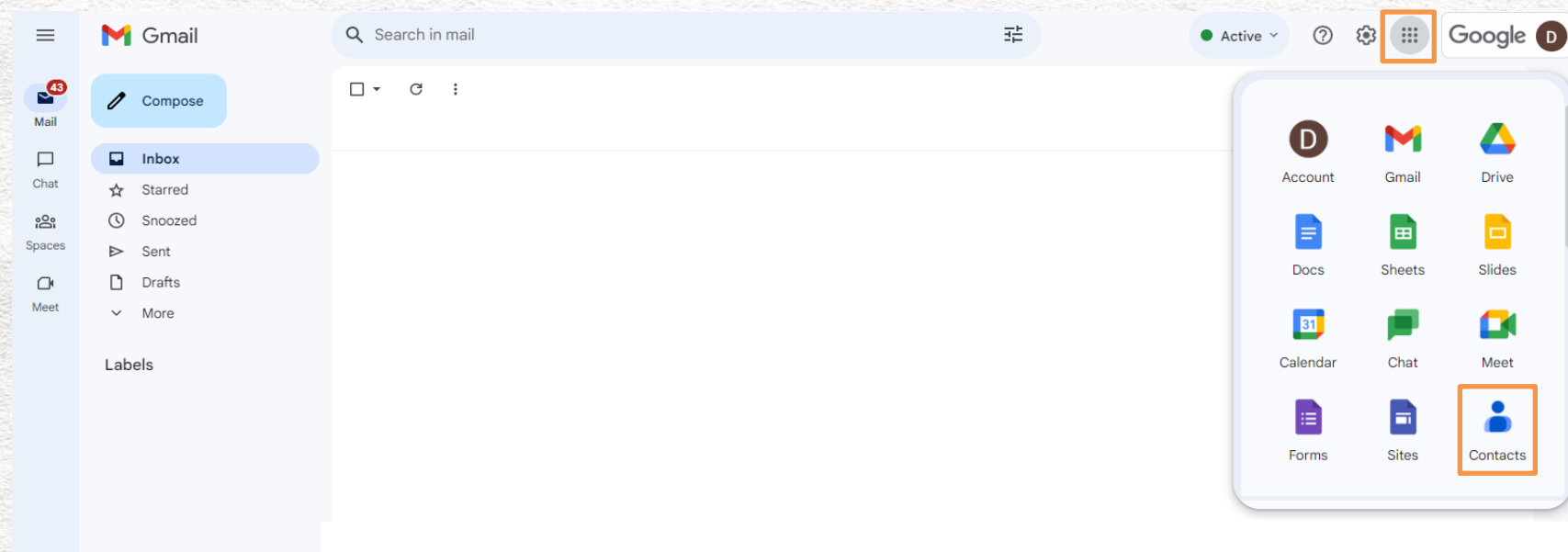
```
# Localizar a imagem -> vai te dar 4 informações: posição x, posição y, largura e altura
x, y, largura, altura = pyautogui.locateOnScreen('busca_google.png')
# clicar no meio da imagem
pyautogui.click(x + largura/2, y + altura/2)
```

Executando todo o nosso código até este ponto, o **PyAutoGUI** abrirá o Chrome, realizará uma pesquisa pelo Gmail, clicará no resultado da pesquisa associado ao Gmail e, assim, acessará a página correspondente.



Passo 2 – Entrar em uma Aba Específica do Sistema – Aba Contatos

Após acessar o nosso sistema, nesse caso, o Gmail, precisamos clicar no menu do **Google Apps** e acessar a aba **Contatos**.



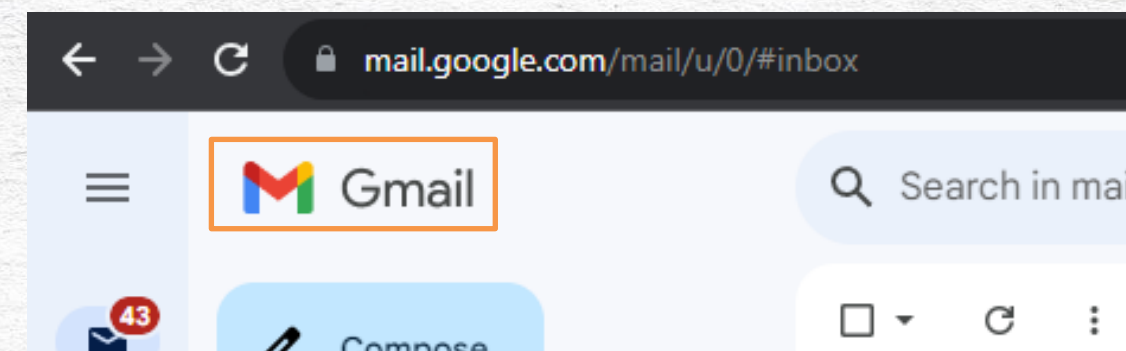
No entanto, a tela do Gmail leva um tempo para ser **totalmente carregada**. Esse intervalo pode variar de computador para computador e de acordo com a velocidade da conexão com a internet.

Portanto, vamos empregar a biblioteca **PyAutoGUI** em conjunto com a biblioteca **Time** para aguardar até que a tela do Gmail seja carregada antes de prosseguirmos com os próximos comandos.

Vamos importar a biblioteca **Time** para o nosso código.

```
import pyautogui
import time
```

Tirar um **print screen** apenas da **logo do Gmail** com o nome **Gmail** ao lado.



Essa será a imagem que utilizaremos dentro da função **locateOnScreen**. Isso ocorre porque, ao localizar a imagem na tela, essa função retorna as **coordenadas**, além da **altura** e **largura**. Entretanto, quando não encontra a imagem, ela retorna **None**.

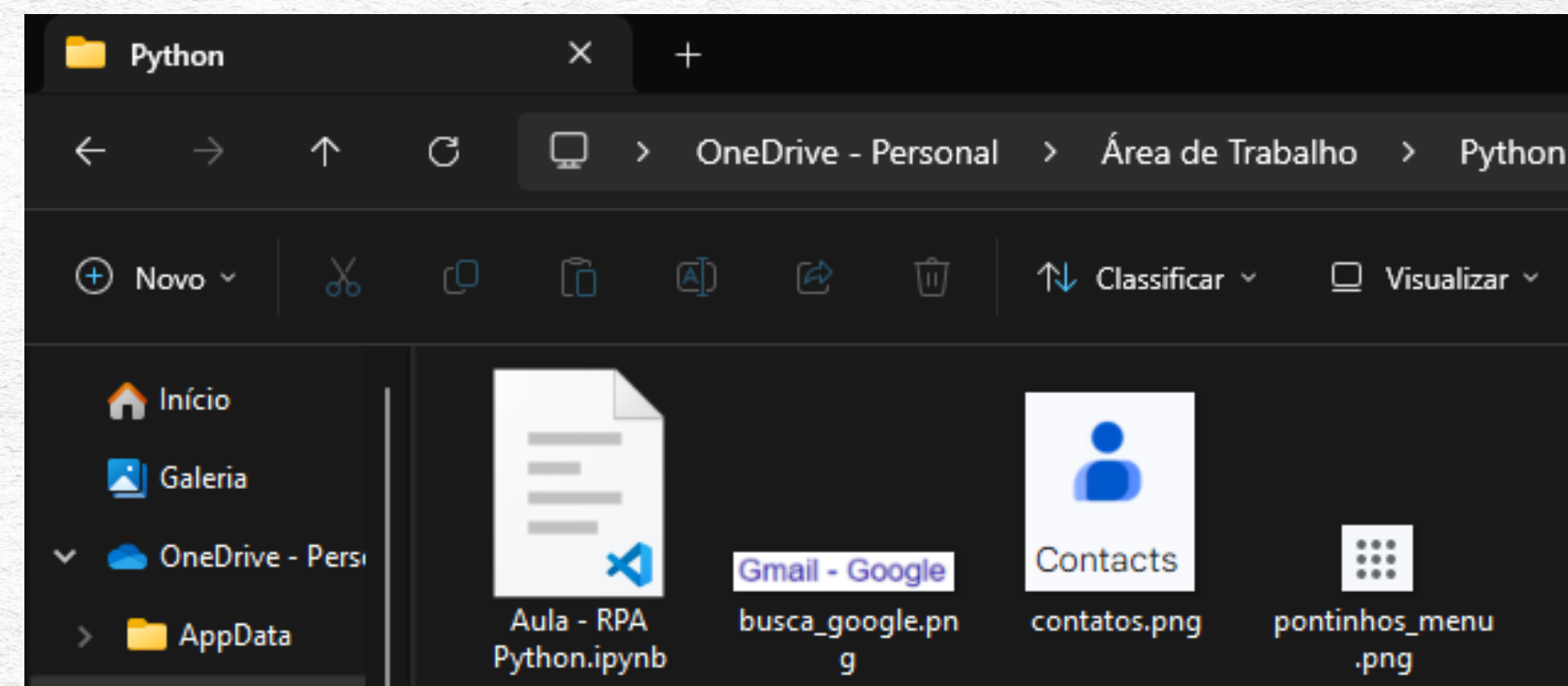
Dessa forma, podemos empregar um loop **while** juntamente com a função **locateOnScreen** para esperar **1 segundo** até que a imagem do **logo do Gmail** seja encontrada.

```
#esperar o gmail
while not pyautogui.locateOnScreen('logo_gmail.png'):
    time.sleep(1)
```

Feito isso, garantimos que o nosso código só continuará a ser executado após a localização da logo do Gmail na tela, indicando que a **tela está carregada**.

Agora, podemos avançar para a próxima etapa, que consiste em encontrar o **menu do Google Apps** (os pontinhos no canto da tela) e clicar em **Contatos**.

Podemos realizar essa tarefa utilizando a mesma lógica que acabamos de discutir, utilizando as funções **locateOnScreen** e **click**. Vamos **capturar a imagem do menu** por meio de um **print screen**, em seguida, faremos o mesmo para o **ícone de contatos**, salvando ambas as imagens na pasta do código.

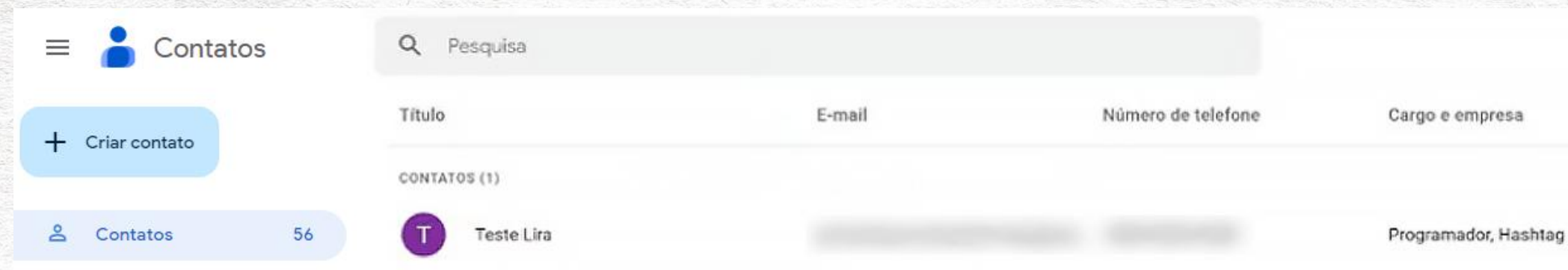


Quanto mais **precisa e delimitada** for a sua captura de tela, melhor será para o PyAutoGUI **localizar** a correspondência da imagem.

Após essa etapa, podemos empregar a função `locateOnScreen` e `click` para localizar e clicar no menu, bem como na opção desejada.

```
# entrar em contatos
x, y, largura, altura = pyautogui.locateOnScreen('pontinhos_menu.png')
pyautogui.click(x + largura/2, y + altura/2)

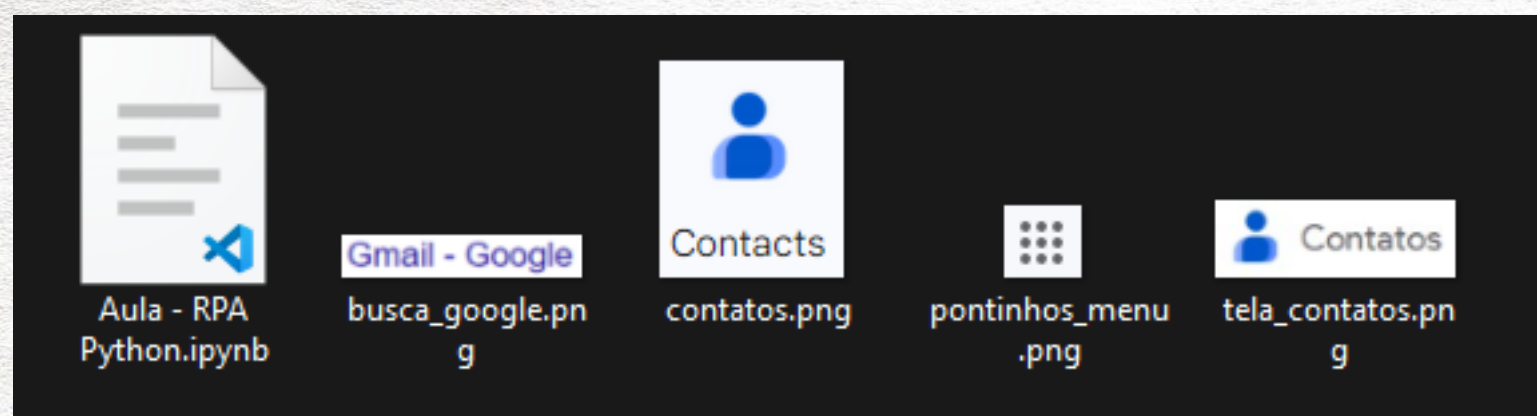
time.sleep(1)
x, y, largura, altura = pyautogui.locateOnScreen('contatos.png')
pyautogui.click(x + largura/2, y + altura/2)
```



Passo 3 – Exportar o Relatório – Exportar Contatos

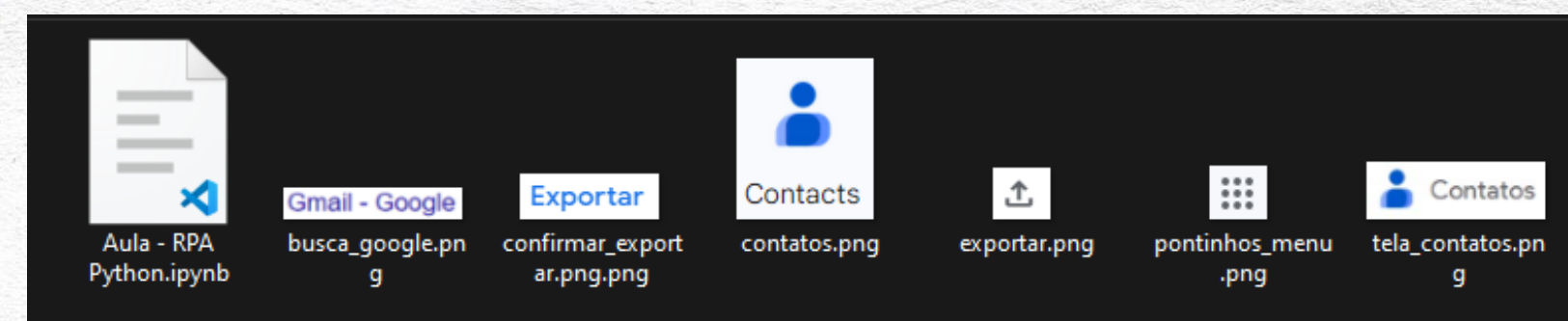
Agora que já acessamos os contatos, podemos exportar a nossa base de dados.

Para isso, é necessário aguardar primeiro que a tela de contatos seja carregada. Utilizaremos a mesma lógica mencionada anteriormente. Na tela de contatos, faremos uma captura de imagem do **ícone e do nome Contatos** para utilizar dentro do loop **while** e aguardar a tela ser carregada.



```
#esperar o contatos
while not pyautogui.locateOnScreen('tela_contatos.png'):
    time.sleep(1)
```

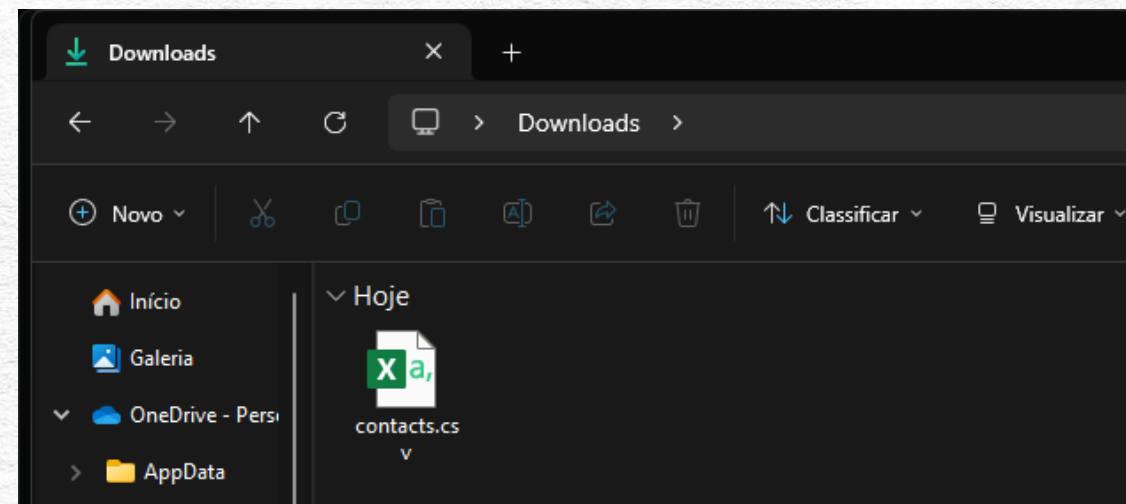
Em seguida, faremos um **print screen** do **ícone de exportar** e do **botão de exportar** que é aberto na sequência, confirmando assim a exportação dos contatos



E aplicaremos novamente os comandos para que o **PyAutoGUI** localize a correspondência dessas imagens na tela e clique sobre elas.

```
# exportar os contatos
x, y, largura, altura = pyautogui.locateOnScreen('exportar.png')
pyautogui.click(x + largura/2, y + altura/2)
x, y, largura, altura = pyautogui.locateOnScreen('confirmar_exportar.png')
pyautogui.click(x + largura/2, y + altura/2)
```

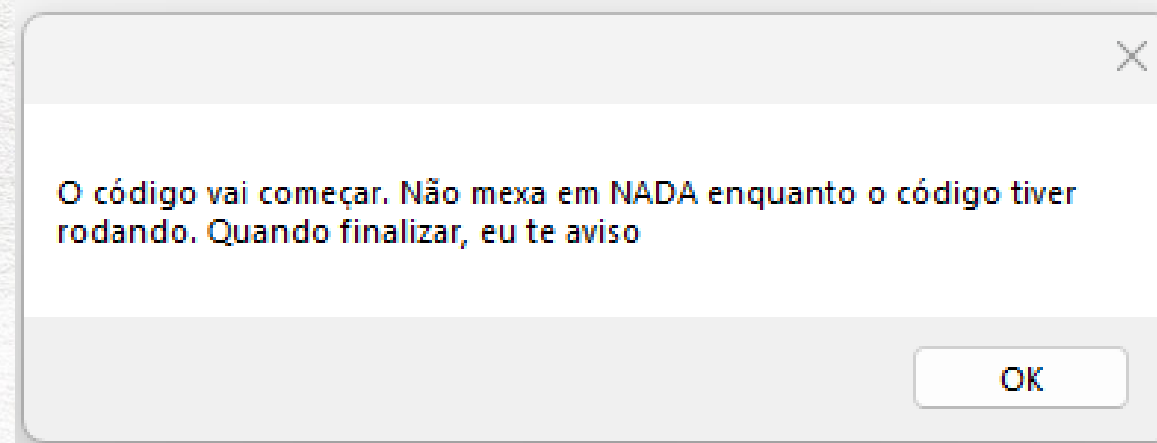
Ao executarmos nossa automação até o momento, ela abrirá o Chrome, buscará por Gmail, acessará o Gmail, abrirá a guia de Contatos e exportará os dados dos contatos para um arquivo CSV.



Dado que durante a execução do PyAutoGUI **não podemos realizar interações com o mouse e o teclado**, é possível definir um **alerta** no início da automação para informar ao usuário que ele não poderá interagir com o computador durante esse processo.

```
pyautogui.alert('O código vai começar. Não mexa em NADA enquanto o código tiver rodando. Quando finalizar, eu te aviso')
```

Assim, no início do código, será exibido um alerta ao usuário:



E a automação só terá início quando o usuário clicar em **OK**.

Passo 4 – Ler o Relatório, Tratar e Pegar as Informações

Agora que conseguimos extrair as informações do nosso sistema e temos a nossa base de dados, podemos começar a trabalhar com ela para enviar o e-mail posteriormente.

Primeiro, precisamos ler o arquivo CSV que foi salvo em nosso computador. Para isso, carregaremos esses dados para um DataFrame (df) utilizando a função read_csv da biblioteca Pandas.

```
import pandas as pd

df = pd.read_csv(r'C://Users/joaop/Downloads/contacts.csv')
```

Observe que, antes do caminho onde está salvo o arquivo, adicionamos a letra r para indicar ao interpretador de Python que essa string deve ser interpretada como uma string bruta (raw string). Dessa forma, os caracteres de escape (como , \n ou \t) não serão interpretados. Isso garantirá que o Python consiga localizar corretamente o local do seu arquivo.

Podemos visualizar essa base de dados através da função display().

```
import pandas as pd

df = pd.read_csv(r'C://Users/joaop/Downloads/contacts.csv')
display(df)
```

	Name	Given Name	Additional Name	Family Name	Yomi Name	Given Name Yomi	Additional Name Yomi	Family Name Yomi	Name Prefix	Name Suffix	...	Phone 1 - Type	Phone 1 - Value	Organization 1 - Type	Organization 1 - Name	Organization 1 - Yomi Name	Organi...
0	Teste Lira	Teste	NaN	Lira	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN		NaN	Hashtag	NaN	Progr...

Ao examinar essa base de dados, notamos a presença de algumas colunas vazias. Portanto, podemos removê-las do nosso DataFrame utilizando a função dropna, passando como parâmetro o axis=1.

A função dropna do Pandas é utilizada para remover valores nulos do DataFrame. O parâmetro axis pode receber os valores 0 e 1, sendo 0 as linhas vazias e 1 as colunas vazias.

Após a exclusão dessas linhas, nossa tabela ficará assim:

```
import pandas as pd

df = pd.read_csv(r'C://Users/joaop/Downloads/contacts.csv')
df = df.dropna(axis=1)
display(df)
```

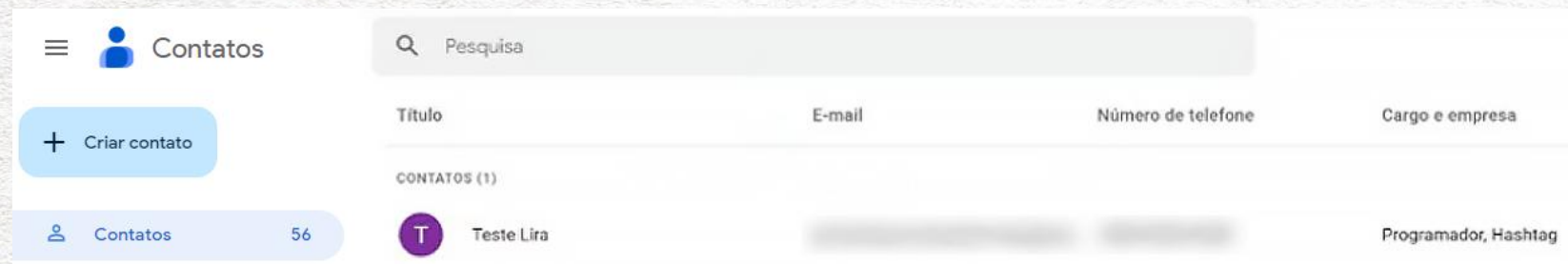
	Name	Given Name	Family Name	Group Membership	E-mail 1 - Type	E-mail 1 - Value	Phone 1 - Value	Organization 1 - Name	Organization 1 - Title
0	Teste Lira	Teste	Lira	* myContacts	* Home			Hashtag	Programador

Passo 5 – Preencher/Atualizar Informações do Sistema – Criar e Enviar o E-mail

Com nossa base de dados devidamente tratada, podemos avançar para a etapa final deste desafio, que é **criar** e **enviar** o **e-mail** para os contatos registrados na nossa tabela.

Até o momento, ao executar nossa automação, ela abre o Chrome, busca o Gmail, acessa o Gmail, abre a guia de Contatos e exporta os dados dos contatos em um arquivo CSV.

Em outras palavras, a última tela da nossa automação é a aba de Contatos.



No entanto, em nosso navegador, a página da caixa de entrada do Gmail continua aberta. Então, o próximo passo será **alterar** a **guia do navegador**, **criar** e **enviar** o e-mail.

Para retornar à guia do Gmail a partir da aba de **Contatos**, podemos pressionar as **teclas de atalho Ctrl + Page Up**. Para utilizar essa combinação de teclas com o PyAutoGUI, usaremos a função **hotkey**.

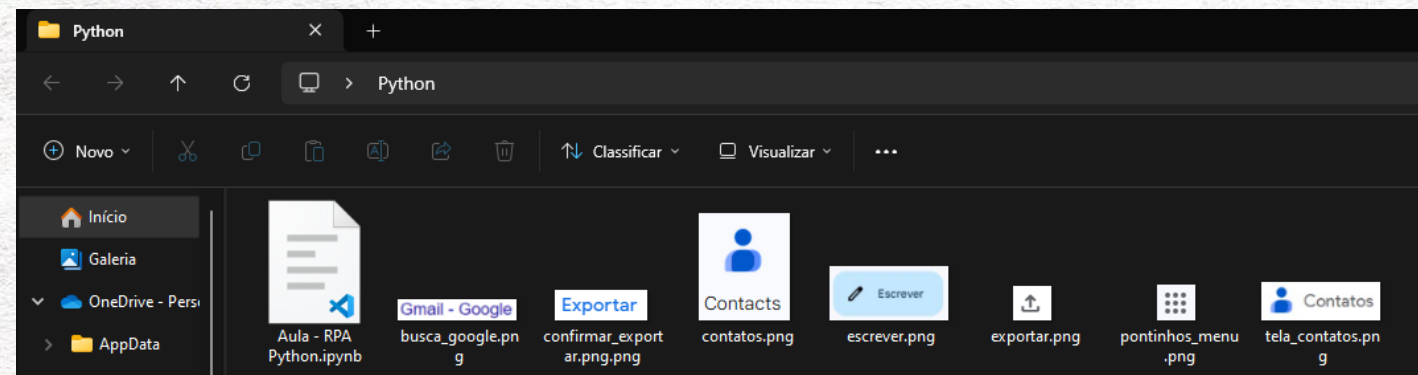
```
import pandas as pd

df = pd.read_csv(r'C://Users/joaop/Downloads/contacts.csv')
df = df.dropna(axis=1)
display(df)

pyautogui.hotkey('ctrl', 'pgup')
```

Após esse comando, nossa automação estará de volta à página do Gmail, e poderemos redigir nosso e-mail. Para isso, precisamos clicar em **Escrever**.

Como já discutimos anteriormente, para clicar em alguma região da tela, primeiro precisamos capturar um **print** dessa região e salvar na pasta junto com o nosso código.



Com a imagem salva, podemos agora pensar na lógica do envio de e-mails. Nesse exemplo, estarei enviando apenas um único e-mail, pois exportei apenas um contato do meu Gmail.

No entanto, em um exemplo prático, lidando com uma **base de dados real**, teríamos vários contatos com e-mails diferentes. Para enviar um e-mail para cada um deles, precisamos trabalhar com uma estrutura de **loop for**.

Faremos um **loop for**, percorrendo cada e-mail presente na coluna **E-mail 1 - Value**, e para cada um iremos **clicar no botão de escrever, escrever o e-mail e enviar**.

O primeiro passo será clicar no ícone de escrever, e já vimos como fazer isso:

```
import pandas as pd

df = pd.read_csv(r'C://Users/joaop/Downloads/contacts.csv')
df = df.dropna(axis=1)
display(df)

pyautogui.hotkey('ctrl', 'pgup')

for email in df['E-mail 1 - Value']:
    #clicar no botão escrever
    time.sleep(1)
    x, y, largura, altura = pyautogui.locateOnScreen('escrever.png')
    pyautogui.click(x + largura/2, y + altura/2)
    time.sleep(1)
```

Na janela que será aberta, precisamos inserir o **destinatário** do e-mail, o **assunto** e o **corpo** do e-mail.

Quando a janela for aberta, o **campo do destinatário** já estará selecionado, então só precisamos **inserir o e-mail** e pressionar **Enter**.

```
for email in df['E-mail 1 - Value']:
    #clicar no botão escrever
    time.sleep(1)
    x, y, largura, altura = pyautogui.locateOnScreen('escrever.png')
    pyautogui.click(x + largura/2, y + altura/2)
    time.sleep(1)
    # escrever o email
    pyautogui.write(email)
    # enter
    pyautogui.press('enter')
```

Como estamos **iterando** sobre cada **e-mail** da **coluna de e-mails** do nosso **DataFrame**, passamos a função **write** tendo como argumento a variável **email**.

Em seguida, precisamos pressionar **Tab** para trocar do campo de destinatário para o **campo de assunto** e **inserir o assunto** desejado.

```
for email in df['E-mail 1 - Value']:
    #clicar no botão escrever
    time.sleep(1)
    x, y, largura, altura = pyautogui.locateOnScreen('escrever.png')
    pyautogui.click(x + largura/2, y + altura/2)
    time.sleep(1)
    # escrever o email
    pyautogui.write(email)
    # enter
    pyautogui.press('enter')
    #tab para o assunto do email
    pyautogui.press('tab')
    pyautogui.write('Lira Caloteiro')
```

Feito isso, agora podemos escrever o **corpo do e-mail**. Para navegar até o **corpo do e-mail**, vamos pressionar **Tab** novamente.

No entanto, para escrever o **corpo do nosso e-mail**, não vamos utilizar diretamente a função **write** do **PyAutoGUI**, mas sim a biblioteca **Pyperclip**. Faremos isso porque o **PyAutoGUI** tem como padrão o **teclado americano**, então **caracteres especiais** como acento agudo ou til não são escritos corretamente pela automação.

A biblioteca **Pyperclip** nos permitirá **copiar o texto** que estará contido na variável texto e, assim, com o **PyAutoGUI**, podemos **colar esse texto** no campo do corpo do e-mail utilizando a função **hotkey**.

Portanto, precisamos importar a biblioteca **Pyperclip** para o nosso código e adicionar essa parte da automação dentro do **loop for**.

```
import pandas as pd
import pyperclip

df = pd.read_csv(r'C://Users/joaop/Downloads/contacts.csv')
df = df.dropna(axis=1)
display(df)

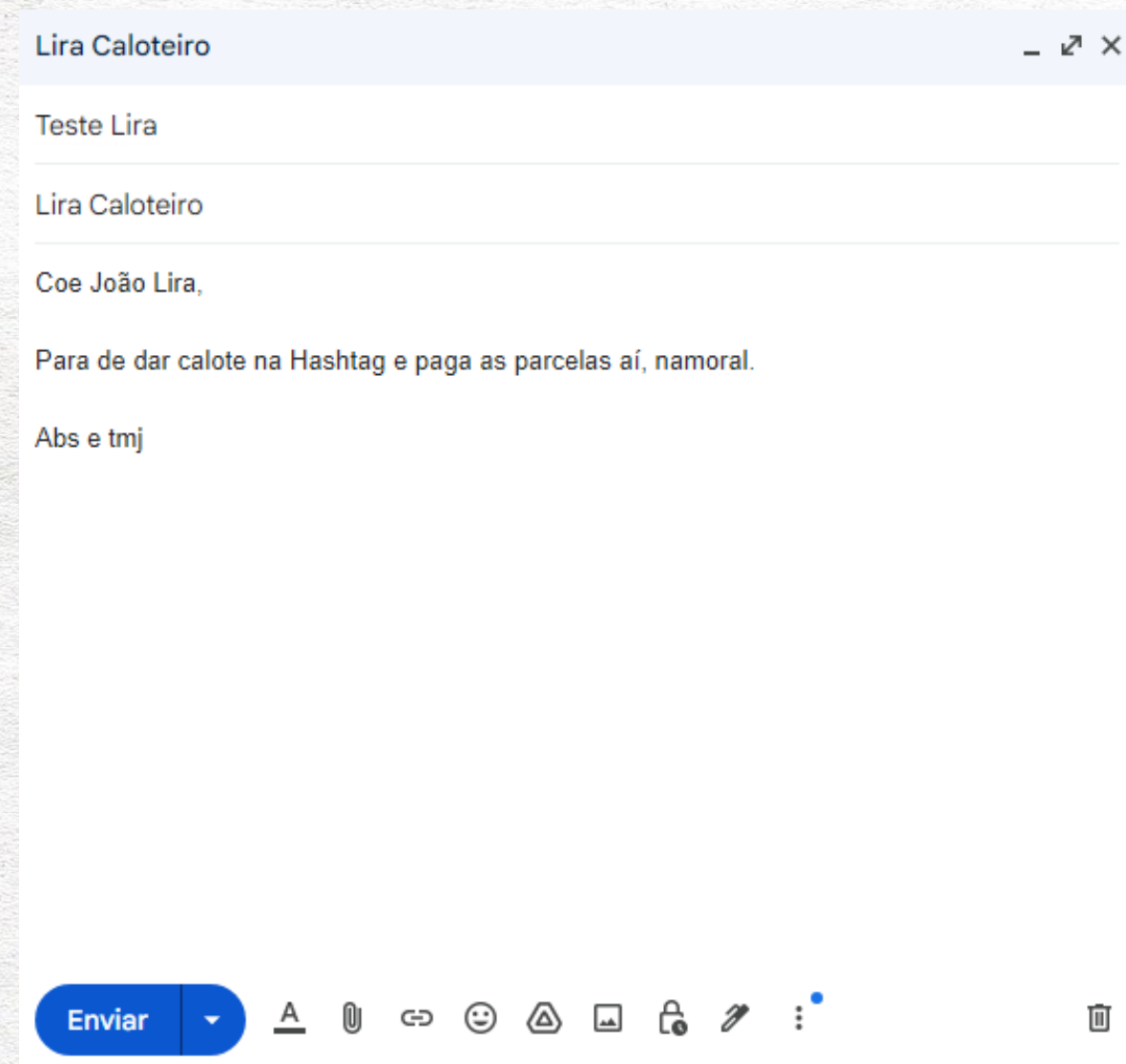
pyautogui.hotkey('ctrl', 'pgup')

for email in df['E-mail 1 - Value']:
    #clicar no botão escrever
    time.sleep(1)
    x, y, largura, altura = pyautogui.locateOnScreen('escrever.png')
    pyautogui.click(x + largura/2, y + altura/2)
    time.sleep(1)
    # escrever o email
    pyautogui.write(email)
    # enter
    pyautogui.press('enter')
    #tab para o assunto do email
    pyautogui.press('tab')
    pyautogui.write('Lira Caloteiro')
    #tab para o corpo do email
    pyautogui.press('tab')
    texto = """
    Coe João Lira,

    Para de dar calote na Hashtag e paga as parcelas aí, namoral.

    Abs e tmj"""
    pyperclip.copy(texto)
    pyautogui.hotkey('ctrl', 'v')
```

Com isso, a janela para enviar o e-mail ficará preenchida da seguinte forma:



Para enviar, podemos utilizar novamente a função **hotkey** do **PyAutoGUI**, passando a combinação das teclas **Ctrl + Enter**.

Com isso, a janela para enviar o e-mail ficará preenchida da seguinte forma:

```
import pandas as pd
import pyperclip

time.sleep(2)
df = pd.read_csv(r'C://Users/joaop/Downloads/contacts.csv')
df = df.dropna(axis=1)
display(df)

pyautogui.hotkey('ctrl', 'pgup')

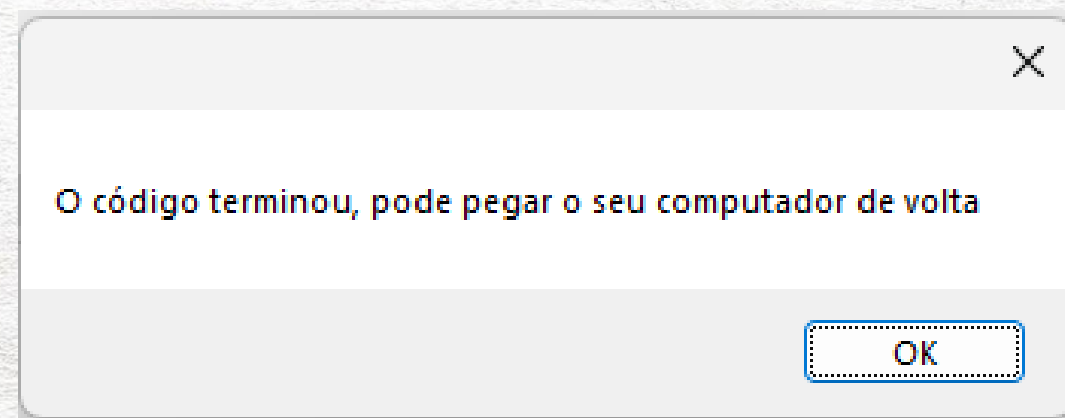
for email in df['E-mail 1 - Value']:
    #clicar no botão escrever
    time.sleep(1)
    x, y, largura, altura = pyautogui.locateOnScreen('escrever.png')
    pyautogui.click(x + largura/2, y + altura/2)
    time.sleep(1)
    # escrever o email
    pyautogui.write(email)
    # enter
    pyautogui.press('enter')
    #tab para o assunto do email
    pyautogui.press('tab')
    pyautogui.write('Lira Caloteiro')
    #tab para o corpo do email
    pyautogui.press('tab')
    texto = """
    Coe João Lira,

    Para de dar calote na Hashtag e paga as parcelas aí, namoral.

    Abs e tmj"""
    pyperclip.copy(texto)
    pyautogui.hotkey('ctrl', 'v')
    pyautogui.hotkey('ctrl', 'enter')
```

E para sinalizar que o código foi concluído, podemos adicionar um novo alerta do PyAutoGUI.

```
pyautogui.alert('O código terminou, pode pegar o seu computador de volta')
```



Com isso, finalizamos o nosso desafio prático para construir uma automação que extrai informações de um sistema e, a partir disso, envia um relatório por e-mail.

Um ponto importante a destacar é que, como esta é uma automação que pode variar dependendo da **conexão com a internet** e da **velocidade do computador**, pode ser necessário adicionar a função **time.sleep()** entre algumas etapas, como **antes da leitura do arquivo CSV**, para garantir que houve tempo para o **download ser concluído**.

```
import pandas as pd
import pyperclip

time.sleep(2)
df = pd.read_csv(r'C://Users/joaop/Downloads/contacts.csv')
df = df.dropna(axis=1)
display(df)
```

Também é possível adicionar mais algum loop **while** em conjunto com a função **locateOnScreen** para esperar **1 segundo** até que alguma página termine de carregar, como a página inicial do Google.

```
#esperar carregar o google
while not pyautogui.locateOnScreen('busca_google.png'):
    time.sleep(1)
```

Esse foi um projeto completo de **RPA**, ou **Automação de Processos Robóticos** (Robotic Process Automation), que permite automatizar tarefas repetitivas e manuais em sistemas de software por meio de robôs.

Para finalizar, existe um comando interessante na biblioteca **PyAutoGUI** que pode ser muito útil na construção de **RPA**, que é como descobrir a posição em que o cursor do seu mouse está.

Para obter a posição do seu mouse, você pode utilizar a função **position** do **PyAutoGUI**.

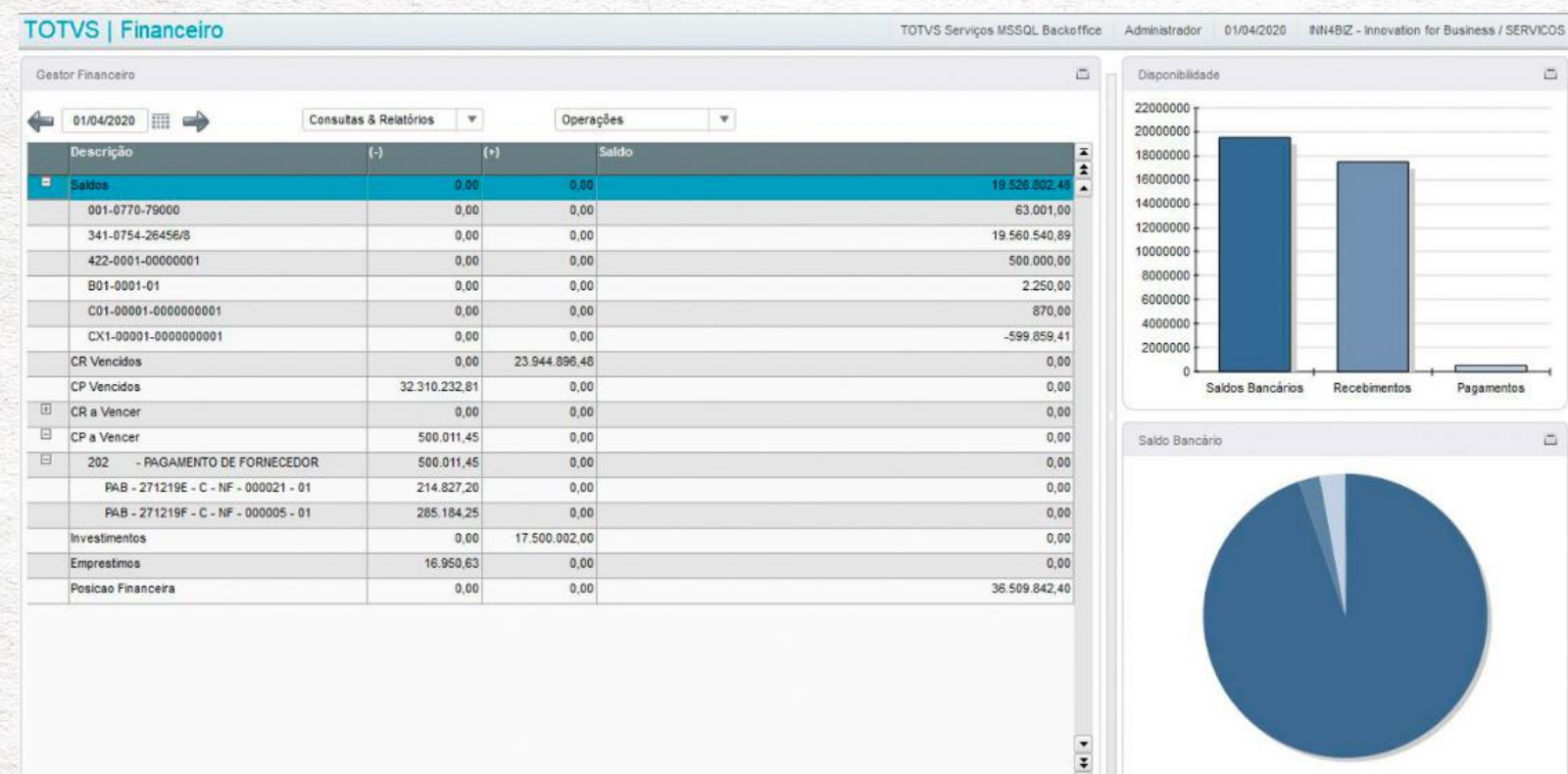
```
time.sleep(3)
print(pyautogui.position())

Point(x=1342, y=266)
```

É importante adicionar um **time.sleep()** antes de executar o **pyautogui.position()** para dar tempo de posicionar o seu mouse antes de o **PyAutoGUI** **capturar** a posição dele.

Veremos agora como aplicar automações de processos em sistemas **ERPs**, como o **Totvs**, **SAP**, entre outros, utilizando o **Python** e a biblioteca **PyAutoGUI**. Você poderá aplicar os conhecimentos desse módulo independentemente do sistema **ERP** que estiver usando.

ERP (Enterprise Resource Planning - Planejamento dos Recursos Empresariais) é um software de gestão empresarial que integra e gerencia os processos de coleta, armazenamento, gerenciamento e interpretação dos dados daquela empresa. Eles abrangem áreas como finanças, contabilidade, recursos humanos, produção, compras, vendas, entre outras.



Com um **ERP**, você pode registrar as notas fiscais da empresa, inserir as vendas, cadastrar produtos, registrar clientes e assim por diante.

Essencialmente, existem duas formas de integrarmos o **Python** com sistemas **ERP**:

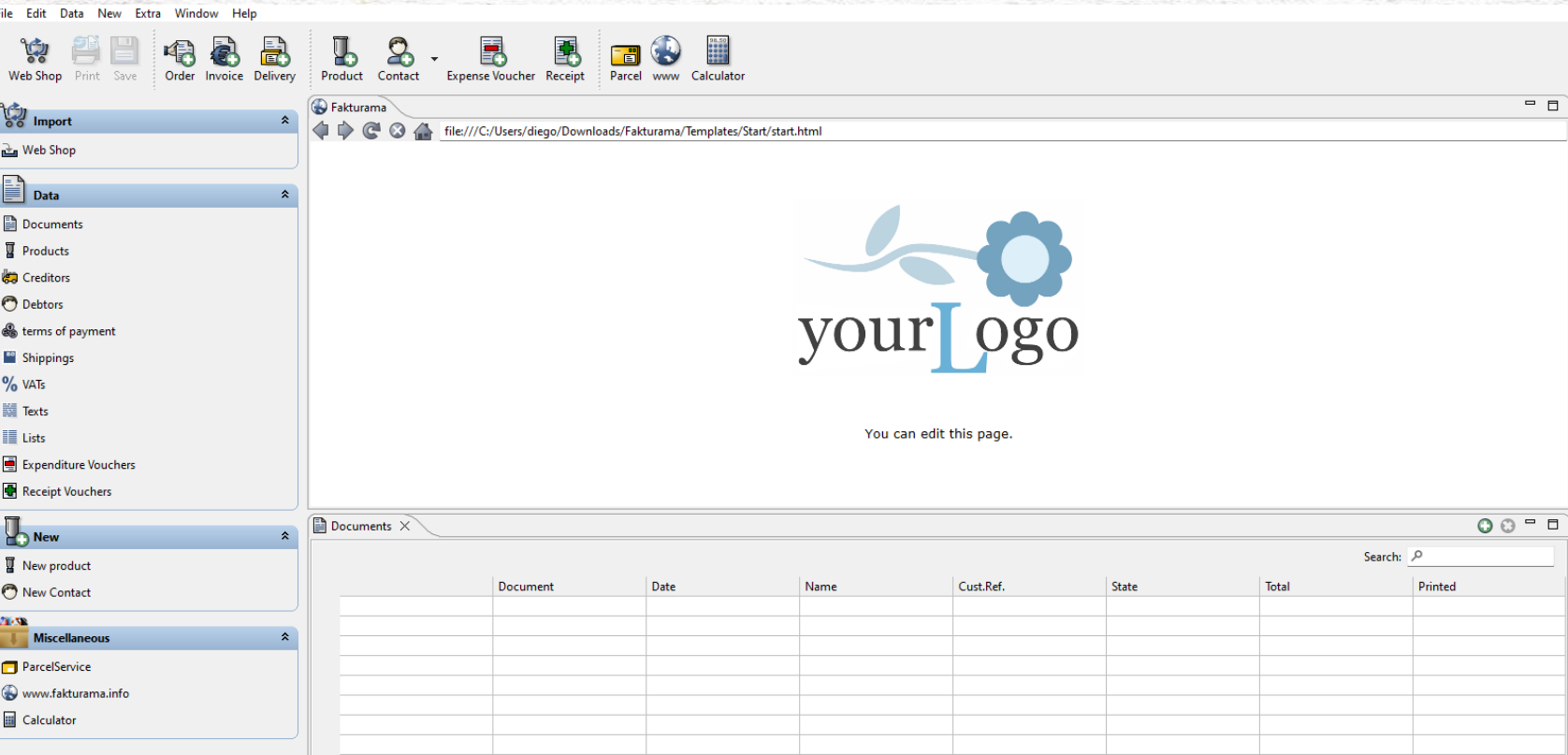
A primeira forma seria realizar uma integração de **Python com SQL** para ter acesso ao banco de dados, podendo consultá-lo e editá-lo.

No entanto, esse método dificilmente estará disponível para você se não for o administrador do banco de dados da empresa.

Por isso, iremos ver como fazer a segunda forma, que será uma **automação via desktop**. Simulando o uso do seu computador para automatizar o processo. As duas ferramentas principais para isso são o **Selenium** e o **PyAutoGUI**.

Nesta aula, utilizaremos o **PyAutoGUI**, pois a maioria dos **ERPs** são softwares executados dentro do seu computador e não na internet através do navegador. Por isso, nesse caso, não poderemos utilizar o Selenium.

Para exemplificar e trabalhar ao longo desse módulo, vamos utilizar o sistema do **Fakturama**, que é um **software ERP gratuito**. Então, você pode fazer o download e instalação dele para acompanhar cada passo.



Antes de partirmos para o desenvolvimento da automação, o primeiro passo será pensar e escrever a **lógica** do que iremos desenvolver em Português, o **passo a passo**.

Nossa automação precisará **abrir o ERP (Fakturama); clicar no menu New; Depois em New Product; Preencher todos os campos de cadastro; Selecionar a imagem; Clicar em salvar.**

Esse é o processo que iremos automatizar. No material disponível na [aula](#), você encontrará tanto o gabarito desse processo quanto as imagens e a base de dados que iremos utilizar para essa automação.

Para realizar essa automação, vamos precisar, primeiramente, instalar 3 bibliotecas: PyAutoGUI, Pillow e OpenCV-Python. No terminal, você pode utilizar o seguinte comando:

```
pip install pyautogui pillow opencv-python
```

Ou, se estiver usando o Jupyter Notebook, utilize:

```
!pip install pyautogui pillow opencv-python
```

Feito isso, vamos começar o passo a passo para nossa automação.



Abrir o ERP (Fakturama):

Vamos importar a biblioteca do **PyAutoGUI** e a biblioteca **subprocess** (essa já vem instalada por padrão).

O primeiro comando que vamos adicionar ao nosso código será o **FAILSAFE**.

```
1 import pyautogui
2 import subprocess
3
4 pyautogui.FAILSAFE = True
5
```

Esse comando garante que, se em algum momento nossa automação der um **problema** no meio do caminho ou precisarmos **pausá-la**, basta arrastar o cursor do mouse para alguma das **extremidades da tela**. Isso irá interromper os processos sendo executados.

Em seguida, vamos abrir o **Fakturama** através da biblioteca **subprocess**.

```
1 import pyautogui
2 import subprocess
3
4 pyautogui.FAILSAFE = True
5
6 # Abrir o ERP (Fakturama)
7 subprocess.Popen([r"C:\Program Files\Fakturama2\Fakturama.exe"])
```

Poderíamos abrir o **Fakturama** utilizando o **PyAutoGUI** e **automatizando** o processo de abrir o menu no computador, buscar pelo Fakturama e clicar para executá-lo.

No entanto, com a biblioteca **subprocess**, ao passarmos o caminho completo do programa no computador, a automação fica mais robusta, segura e menos suscetível a erros.

Lembre-se de trocar o **caminho do programa** pelo **caminho do programa no seu computador**.

Agora, precisamos definir uma **pausa** para aguardar o **Fakturama** terminar de abrir no computador, antes de continuarmos com a automação.

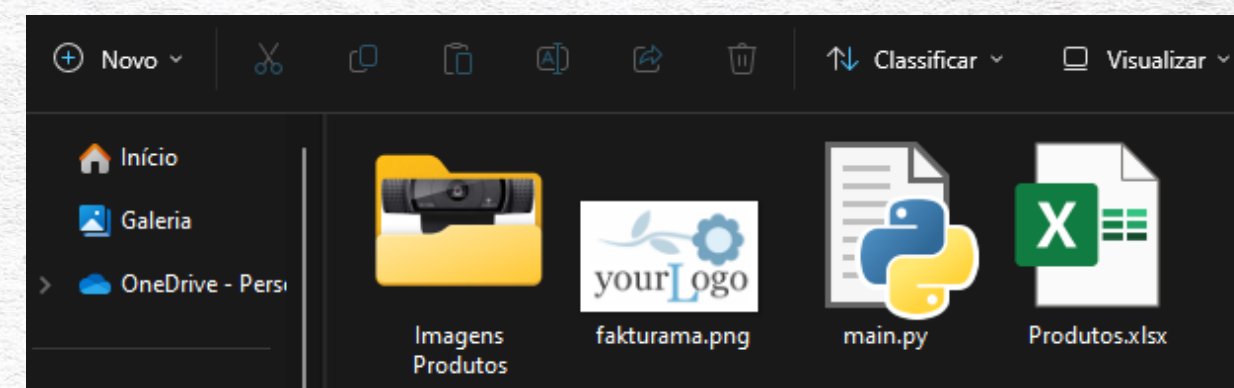
Para isso, vamos usar a função **locateOnScreen** do **PyAutoGUI** que vimos na aula anterior, com a biblioteca **Time**.

```
1 import pyautogui
2 import subprocess
3 import time
```

Primeiro, tiraremos um **print screen** de uma parte da tela do **Fakturama** aberto, algo que identifique que o programa já carregou. É importante escolher uma **imagem única** que só apareça quando o programa já estiver **totalmente carregado**.



Salvaremos essa imagem na **mesma pasta do nosso código**.



E vamos passar o nome dessa captura de tela para a função **locateOnScreen** com os parâmetros **grayscale=True** e **confidence=0.9**.

```
pyautogui.locateOnScreen("fakturama.png", grayscale=True, confidence=0.9)
```

Esses parâmetros vão auxiliar a função a encontrar a **referência mais próxima da imagem** que passamos, com **90% de semelhança (confidence=0.9)**.

Isso é importante para garantir que mesmo que haja alguma pequena diferença entre o seu print e a imagem na tela de fato, a função consiga localizar a imagem.

Essa função retorna uma **tupla** contendo 4 valores: **posição x**, **posição y**, **largura** e **altura da imagem**.

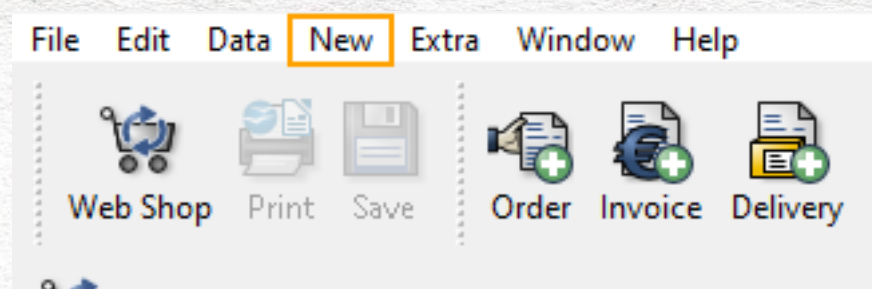
Logo, para aguardarmos o Fakturama carregar, podemos utilizar essa função dentro de um **loop while**.

```
1 import pyautogui
2 import subprocess
3 import time
4
5 pyautogui.FAILSAFE = True
6
7 # Abrir o ERP (Fakturama)
8 subprocess.Popen(["C:\Program Files\Fakturama2\Fakturama.exe"])
9
10 while not pyautogui.locateOnScreen("fakturama.png", grayscale=True, confidence=0.9): # reconhecimento de imagem
11     time.sleep(1)
```

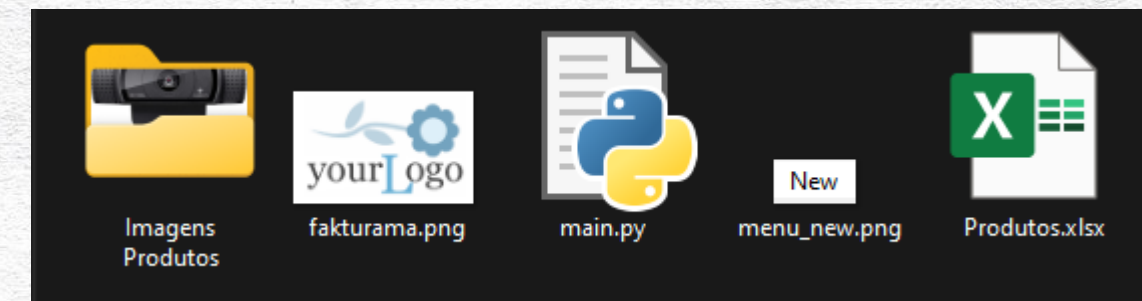
Enquanto a função não retornar nada, ou seja, enquanto o resultado de **locateOnScreen("fakturama.png")** for **None**, o loop continuará rodando e aguardando um segundo (**time.sleep(1)**).

Clicar no menu New:

Com o Fakturama aberto, o próximo passo é esperar o botão **New** estar visível e clicar nele.



Vamos seguir o mesmo procedimento, tirar um **print** do botão e usar a função **locateOnScreen** para esperar o botão estar visível.



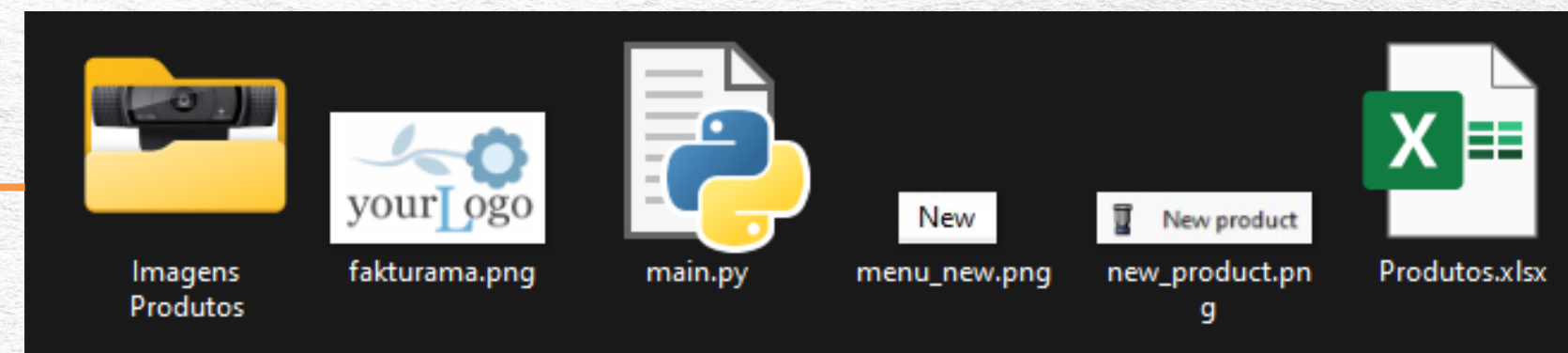
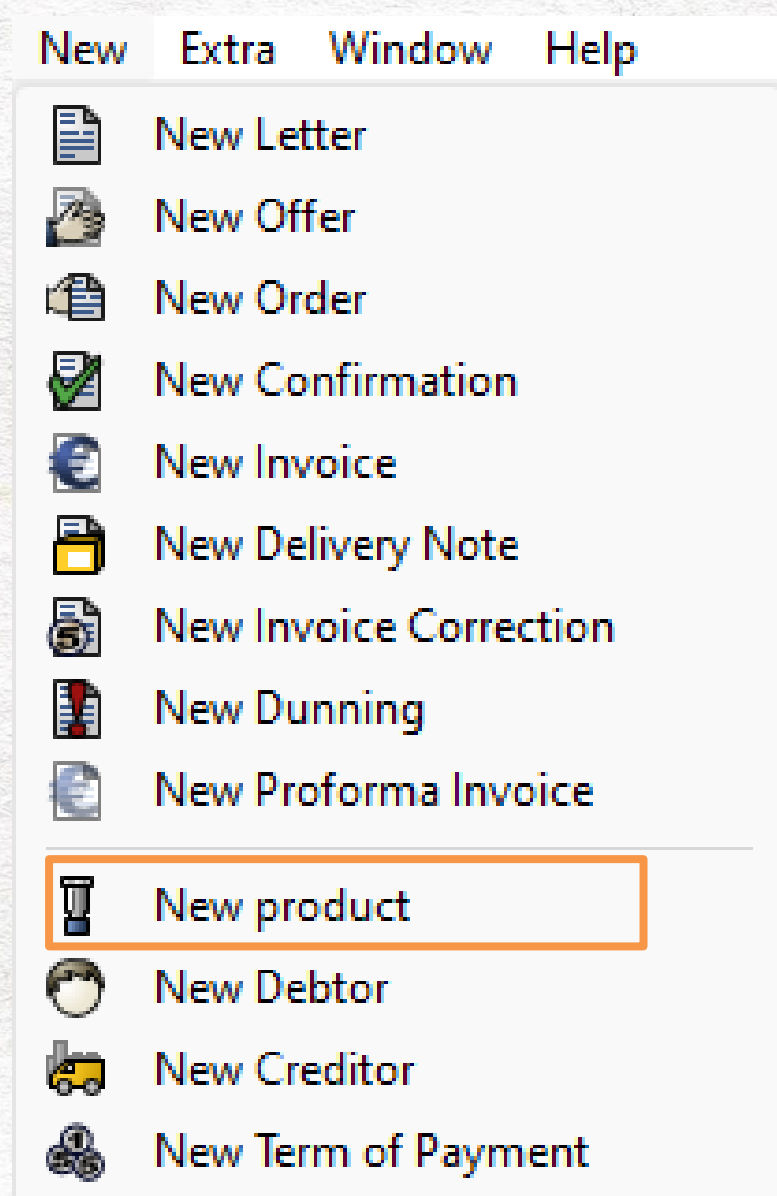
Só que agora, além de **aguardar o botão ficar visível**, iremos **clicar no meio dele**. Para isso, vamos armazenar o resultado da função **locateOnScreen** na variável **encontrou**, que vai conter a tupla com as **coordenadas** e a **altura** e **largura** da imagem encontrada na tela.

Vamos passar essa variável para a função **center** para determinar o centro da correspondência dela na tela e utilizar a função **click** do **PyAutoGUI** para clicar sobre ela.

```
# Clicar no menu New
while not pyautogui.locateOnScreen("menu_new.png", grayscale=True, confidence=0.9):
    time.sleep(1)
encontrou = pyautogui.locateOnScreen("menu_new.png", grayscale=True, confidence=0.9)
pyautogui.click(pyautogui.center(encontrou))
```

Clicar em New Product:

Agora que abrimos o menu **New**, vamos clicar na opção **New Product**, realizando o mesmo processo. Tirar um **print do botão** e utilizar a função **locateOnScreen** com a função **click**.



```
# Clicar em New Product
while not pyautogui.locateOnScreen("new_product.png", grayscale=True, confidence=0.9):
    time.sleep(1)
encontrou = pyautogui.locateOnScreen("new_product.png", grayscale=True, confidence=0.9)
pyautogui.click(pyautogui.center(encontrou))
```

Feito isso, carregaremos a tela de registro de produto do **Fakturama**.

Criando a Função para Localizar Imagem:

Repare que estamos **repetindo** várias vezes esse processo de localizar a correspondência de uma imagem na tela e esperar ela carregar. Isso significa que podemos **otimizar** esse processo, transformando esse código em uma **função** ao invés de ficar repetindo-o várias e várias vezes.

```
6
7 def encontrar_imagem(imagem):
8     while not pyautogui.locateOnScreen(imagem, grayscale=True, confidence=0.9):
9         time.sleep(1)
10    encontrou = pyautogui.locateOnScreen(imagem, grayscale=True, confidence=0.9)
11    return encontrou
12
13 # Abrir o ERP (Fakturama)
14 subprocess.Popen([r"C:\Program Files\Fakturama2\Fakturama.exe"])
15
16 encontrou = encontrar_imagem("fakturama.png")
17
18 # Clicar no menu New
19 encontrou = encontrar_imagem("menu_new.png")
20 pyautogui.click(pyautogui.center(encontrou))
21
22 # Clicar em New Product
23 encontrou = encontrar_imagem("new_product.png")
24 pyautogui.click(pyautogui.center(encontrou))
```

Preenchendo os campos:

Nossa automação até o momento está clicando em **New Product (Novo Produto)** e abrindo a janela para preencher as informações dele.

Vamos **automatizar** esse processo de preenchimento agora. Começando pelo **Item Number**, vamos tirar um **print** dessa parte da imagem, pegando o nome **Item Number** e o **começo da caixa de texto** onde iremos inserir as informações. Isso porque não vamos querer clicar em cima de Item Number, mas sim na **caixa de texto**.

Item Number

Vamos salvar essa imagem como **item_number.png** na pasta do nosso código junto com as outras imagens.

Como queremos clicar na caixa de texto que está à direita da imagem e precisamos fazer isso algumas vezes nesse processo de cadastro de informações, vamos criar a **função direita** para localizar essa região para o clique.

Para pegarmos a posição mais à direita e no meio, precisamos pegar a **posição x da imagem mais a largura toda dela e a posição y + metade da altura**. Como vimos, a função `locateOnScreen` retorna uma **tupla** contendo 4 valores: **posição x, posição y, largura e altura da imagem**.

Logo, para passarmos nossa lógica para dentro da fórmula, podemos utilizar o **índice** para acessar esses valores:

posição x: `posicoes_imagem[0]`

posição y: `posicoes_imagem[1]`

largura: `posicoes_imagem[2]`

altura: `posicoes_imagem[3]`

```
def direita(posicoes_imagem):  
    return posicoes_imagem[0] + posicoes_imagem[2], posicoes_imagem[1] + posicoes_imagem[3]/2
```

Com a função criada, podemos aplicá-la ao campo **item number**.

Com isso, já estamos selecionando a caixa de texto onde iremos digitar as informações. Para escrever de forma automática com o PyAutoGUI, utilizaremos a função **write()**.

```
#Preenchendo os campos  
encontrou = encontrar_imagem("item_number.png")  
pyautogui.click(direita(encontrou))  
pyautogui.write("2")
```

Teremos de repetir esse processo para todos os outros campos. Podemos alterar cada campo apertando a tecla **Tab** utilizando o PyAutoGUI.

`pyautogui.press("tab")`

Ou realizando o processo de **localizar a imagem novamente, selecionar a caixa de texto e digitar a informação desejada**. Iremos utilizar essa abordagem, então para facilitar, tire um **print** de cada parte do campo de texto com o nome do campo, como fizemos para o **Item Number**, e salve na pasta com o seu código.

Feito isso, vamos passar uma informação para cada campo, através da função **write()**, e verificar se, ao executar, todos os campos serão preenchidos.

```
29 #Preenchendo os campos
30 encontrou = encontrar_imagem("item_number.png")
31 pyautogui.click(direita(encontrou))
32 pyautogui.write("2")
33
34 encontrou = encontrar_imagem("name_produto.png")
35 pyautogui.click(direita(encontrou))
36 pyautogui.write("Computador")
37
38 encontrou = encontrar_imagem("category_produto.png")
39 pyautogui.click(direita(encontrou))
40 pyautogui.write("Acessórios")
41
42 encontrou = encontrar_imagem("GTIN_produto.png")
43 pyautogui.click(direita(encontrou))
44 pyautogui.write("123456789")
45
46 encontrou = encontrar_imagem("supplier_produto.png")
47 pyautogui.click(direita(encontrou))
48 pyautogui.write("127")
49
```

```
49
50 encontrou = encontrar_imagem("description_produto.png")
51 pyautogui.click(direita(encontrou))
52 pyautogui.write("Computador novo do Lira")
53
54 encontrou = encontrar_imagem("preco_produto.png")
55 pyautogui.click(direita(encontrou))
56 pyautogui.write("100,00")
57
58 encontrou = encontrar_imagem("custo_produto.png")
59 pyautogui.click(direita(encontrou))
60 pyautogui.write("50,00")
61
62 encontrou = encontrar_imagem("stock_produto.png")
63 pyautogui.click(direita(encontrou))
64 pyautogui.write("5,00")
```

Fakturama *New product X

Description

Item Number 2

Name Computador

Category Acessórios

GTIN 123456789

supplier code 127

Description Computador novo do Lira

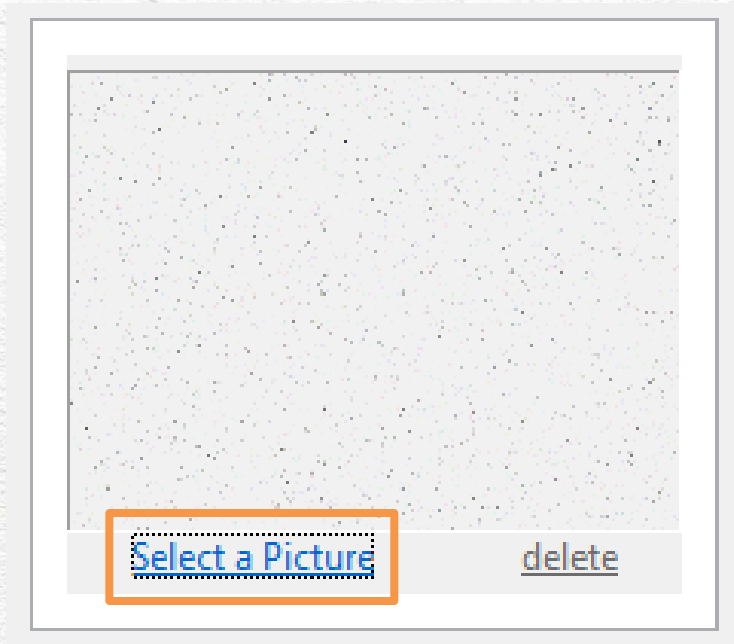
Price (gross) R\$ 100,00

cost price (net) R\$ 50,00 allowance

VAT Free of Tax

Stock 5,00

Agora, vamos carregar a imagem do nosso produto. Para isso, precisamos utilizar a função **localizar_imagem** para encontrar a região da tela que precisamos clicar.



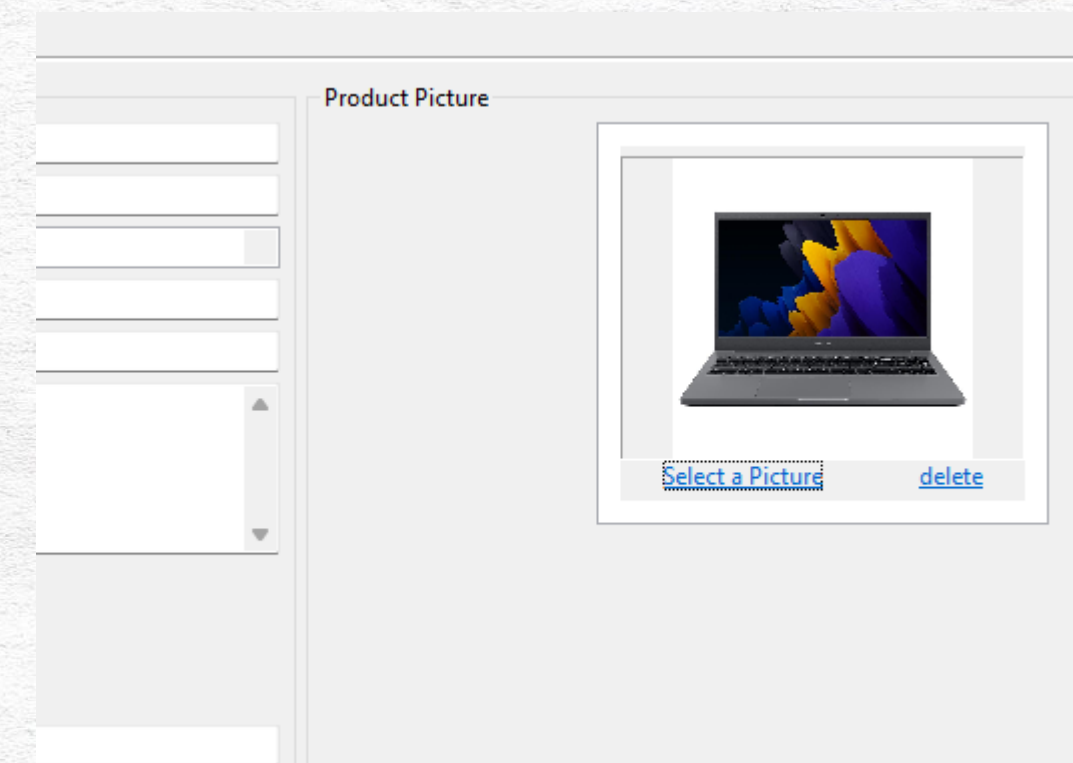
Após fazermos o clique em **Select a Picture**, vamos aguardar até que o campo **Nome**, para inserirmos o **caminho completo da imagem**, esteja carregado. E da mesma forma como estávamos localizando e aguardando até agora, também utilizaremos a função **localizar_imagem**.



Quando esse campo estiver disponível para ser preenchido, vamos utilizar a função **write** para passar o caminho completo até a imagem que queremos carregar e **pressionar Enter**.

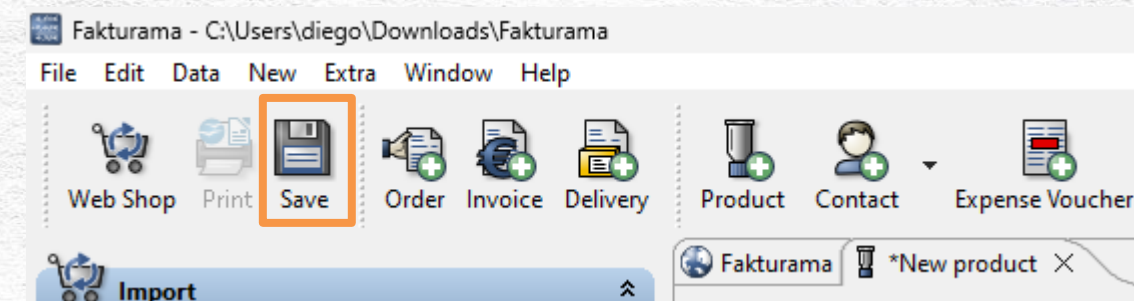
```
# selecionar a imagem
encontrou = encontrar_imagem("select_picture.png")
pyautogui.click(pyautogui.center(encontrou))

encontrou = encontrar_imagem("nome_arquivo.png")
escrever_texto(r"C:\Users\joaol\Downloads\notebook.jpg") #aqui você vai colocar o caminho no seu computador
pyautogui.press("enter")
```



Com isso, só faltará salvar esse produto. Faremos isso com o mesmo procedimento que utilizamos até agora. Tiraremos um print do botão **Save**, vamos salvar a imagem na pasta com o código e utilizar as funções **localizar_imagem** para encontrar a correspondência na tela, e a **click()** para clicar no centro dele.

```
74 # clicar em salvar
75 encontrou = encontrar_imagem("botaosave.png")
76 pyautogui.click(pyautogui.center(encontrou))
77
```



Executando essa automação toda, teremos o nosso produto cadastrado com sucesso.

A screenshot of the Fakturama application showing the product registration form and the resulting product list. The form is titled 'Computador' and contains fields for Item Number (2), Name (Computador), Category (Acessórios), GTIN (123456789), supplier code (127), and Description (Computador novo do Lira). The Price (gross) is set to R\$ 100,00, and the cost price (net) is R\$ 50,00. The VAT is set to Free of Tax. The Product Picture field shows a laptop image. Below the form, the 'Products' tab is active, displaying a table with the following data:

Item No.	Name	Description	Stock	Price	VAT
2	Computador	Computador novo do Lira	5,0	R\$ 100,00	Tax-free (0,0%)

Fazendo Esse Processo Para Vários Produtos:

Terminado esse processo, podemos clicar com o botão direito do mouse sobre o nome do produto e deletá-lo do banco de dados, pois agora vamos fazer isso para diversos produtos.

Item No.	Name	Description	Stock	Price	VAT
2	Comput		5,0	R\$ 100,00	Tax-free (0,0%)
		+ New product			
		✖ delete Ctrl+D			

Para fazermos esse processo para todos os produtos da nossa tabela, vamos precisar utilizar o **Pandas**. Então, instalaremos as bibliotecas **Pandas**, **Numpy** e **OpenPyXL**.

No **terminal**, você pode utilizar o seguinte comando:

```
pip install pandas numpy openpyxl
```

Ou, se estiver usando o **Jupyter Notebook**, não será necessário instalar, pois já vem por padrão.

```
PS C:\Users\diego\Downloads\Mentoria Automação de ERP> pip install pandas numpy openpyxl
```

Feita a instalação, importe o Pandas no início do seu código.

```
import pyautogui
import subprocess
import time
import pandas as pd
```

Com o Pandas instalado, podemos carregar a tabela no nosso código.

```
# Fazer Isso Para Vários Produtos:
tabela_produtos = pd.read_excel("Produtos.xlsx")
```

O arquivo com a tabela deve estar na mesma pasta que seu código. É nessa tabela que estão todas as informações dos produtos que iremos cadastrar no **Fakturama**.

	A	B	C	D	E	F	G	H	I	J
1	ID	Nome	Categoria	GTIN	Supplier	Descrição	Imagem	Preço	Custo	Estoque
2	2	Câmera	Acessorios	6620916	127	Câmera 1080p Logitech USB	camera.jpg	500	55,6	800
3	3	Teclado	Acessorios	9882385	350	Teclado Lg Branco Sem Fio	teclado.jpg	300	33,3	300
4	4	Microfone	Acessorios	1202910	332	HyperX SoloCast - Microfone condensador USB	mic.jpg	450	56,3	400
5	5	Notebook	Principal	8124907	127	Notebook Acer Aspire 5 Intel Core i5-10210U, 4GB RAM, SSD 256	notebook.jpg	3600	400	200
6	6	Monitor	Principal	6930427	350	Monitor 27 LG Full hd, LED ips, 75 Hz, 5ms, FreeSync, hdmi, vga,	monitor.jpg	1200	240	100
7	7	Hub USB	Acessorios	9561480	332	Dock Hub Usb Tipo C Para Usb 3.0 Com Hdmi 4k	hubusb.jpg	100	11,1	600
8	8	Adaptador de Rede	Acessorios	8735119	127	Adaptador de rede USB RJ45 fêmea Wi272 Multilaser CX 1 UM	adaptadorderede.jpg	50	5,6	900
9	9	Ring Light	Acessorios	5936723	350	Kit Iluminador Led Ring Light Tripé Suporte Celular Estética	ringlight.jpg	99	12,4	500
10	10	Celular	Principal	4848751	332	Iphone 6 Plus, 128GB, Cinza Espacial	iphone.jpg	3000	600	100

Agora só precisamos percorrer cada linha da nossa tabela utilizando um **loop for** e extrair de cada coluna a informação que iremos precisar para preencher os campos do Fakturama.

Para localizarmos essas informações, podemos utilizar a função **loc** do **Pandas**.

```
# Fazer Isso Para Vários Produtos:

tabela_produtos = pd.read_excel("Produtos.xlsx")
for linha in tabela_produtos.index:
    nome = tabela_produtos.loc[linha, "Nome"]
    id = tabela_produtos.loc[linha, "ID"]
    categoria = tabela_produtos.loc[linha, "Categoria"]
    gtin = tabela_produtos.loc[linha, "GTIN"]
    supplier = tabela_produtos.loc[linha, "Supplier"]
    descricao = tabela_produtos.loc[linha, "Descrição"]
    imagem = tabela_produtos.loc[linha, "Imagem"]
    preco = tabela_produtos.loc[linha, "Preço"]
    custo = tabela_produtos.loc[linha, "Custo"]
    estoque = tabela_produtos.loc[linha, "Estoque"]
```

Agora só precisamos **identificar** todos os processos de preencher as informações dos produtos para dentro do **for** e substituir onde estávamos passando as informações manualmente, pelas variáveis que acabamos de criar e salvar.

No entanto, o **PyAutoGUI** tem como padrão o teclado americano, então caracteres especiais como acento agudo ou til não são escritos corretamente pela automação. Para contornarmos essa questão, vamos importar o **pyperclip** para o nosso código e criar uma função de escrita para ele, que iremos utilizar no lugar da função **write** do **PyAutoGUI**.

```
import pyautogui
import subprocess
import time
import pandas as pd
import pyperclip
```

```
def escrever_texto(texto):
    pyperclip.copy(texto)
    pyautogui.hotkey("ctrl", "v")
```

Essa função basicamente vai pegar as informações das nossas variáveis com a função **copy** e colar com um comando **ctrl + v** utilizando o **pyautogui**.

Repare também que, quando passávamos os valores individualmente, estávamos passando todos eles como **string**, entre aspas. Por isso, é importante agora utilizar a função **str** para converter a informação que está vindo da tabela para um texto.

Além disso, nos valores referentes a preço, vamos precisar formatar esses valores, porque precisamos passá-los para dentro do **Fakturama** com duas casas decimais. No entanto, ao utilizar a formatação para número **float**, o Python automaticamente utilizará **ponto** ao invés de **vírgula**. Então, além de formatar para duas casas decimais, vamos utilizar a função **replace** para trocar o ponto por vírgula.

```
28 # Fazer Isso Para Vários Produtos:
29
30 tabela_produtos = pd.read_excel("Produtos.xlsx")
31 for linha in tabela_produtos.index:
32     nome = tabela_produtos.loc[linha, "Nome"]
33     id = tabela_produtos.loc[linha, "ID"]
34     categoria = tabela_produtos.loc[linha, "Categoria"]
35     gtin = tabela_produtos.loc[linha, "GTIN"]
36     supplier = tabela_produtos.loc[linha, "Supplier"]
37     descricao = tabela_produtos.loc[linha, "Descrição"]
38     imagem = tabela_produtos.loc[linha, "Imagem"]
39     preco = tabela_produtos.loc[linha, "Preço"]
40     custo = tabela_produtos.loc[linha, "Custo"]
41     estoque = tabela_produtos.loc[linha, "Estoque"]
42
43     # Clicar no menu New
44     encontrou = encontrar_imagem("menu_new.png")
45     pyautogui.click(pyautogui.center(encontrou))
46
47     # Clicar em New Product
48     encontrou = encontrar_imagem("new_product.png")
49     pyautogui.click(pyautogui.center(encontrou))
50
51     # preencher todos os campos
52     encontrou = encontrar_imagem("item_number.png")
53     pyautogui.click(direita(encontrou))
54     escrever_texto(str(id))
55
56     encontrou = encontrar_imagem("name_produto.png")
57     pyautogui.click(direita(encontrou))
58     escrever_texto(str(nome))
59
```

```
60 encontrou = encontrar_imagem("category_produto.png")
61 pyautogui.click(direita(encontrou))
62 escrever_texto(str(categoria))
63
64 encontrou = encontrar_imagem("GTIN_produto.png")
65 pyautogui.click(direita(encontrou))
66 escrever_texto(str(gtin))
67
68 encontrou = encontrar_imagem("supplier_produto.png")
69 pyautogui.click(direita(encontrou))
70 escrever_texto(str(supplier))
71
72 encontrou = encontrar_imagem("description_produto.png")
73 pyautogui.click(direita(encontrou))
74 escrever_texto(str(descricao))
75
76 encontrou = encontrar_imagem("preco_produto.png")
77 pyautogui.click(direita(encontrou))
78 preco_texto = f"{preco:.2f}".replace(".", ",")
79 escrever_texto(str(preco_texto))
80
81 encontrou = encontrar_imagem("custo_produto.png")
82 pyautogui.click(direita(encontrou))
83 custo_texto = f"{custo:.2f}".replace(".", ",")
84 escrever_texto(str(custo_texto))
85
86 encontrou = encontrar_imagem("stock_produto.png")
87 pyautogui.click(direita(encontrou))
88 estoque_texto = f"{estoque:.2f}".replace(".", ",")
89 escrever_texto(str(estoque_texto))
90
91 # selecionar a imagem
92 encontrou = encontrar_imagem("select_picture.png")
93 pyautogui.click(pyautogui.center(encontrou))
94
95 encontrou = encontrar_imagem("nome_arquivo.png")
96 escrever_texto(rf"C:\Users\joaol\Downloads\{str(imagem)}")
97 pyautogui.press("enter")
98
99 # clicar em salvar
100 encontrou = encontrar_imagem("botaosave.png")
101 pyautogui.click(pyautogui.center(encontrou))
102
```

Com isso, nossa automação estará completa e registrando todos os produtos dentro da nossa ERP.

DocumentsCreditorsDebtorsProductsShippings

allAcessoriosPrincipal

Item No.	Name	Description	Stock	Price	VAT
1	Mouse	Mouse Logitech BluetooH	5,0	R\$ 100,00	Tax-free (0,0%)
2	Câmera	Câmera 1080p Logitech USB	800,0	R\$ 500,00	Tax-free (0,0%)
3	Teclado	Teclado Lg Branco Sem Fio	300,0	R\$ 300,00	Tax-free (0,0%)
4	Microfone	HyperX SoloCast -	400,0	R\$ 450,00	Tax-free (0,0%)
5	Notebook	Notebook Acer Aspire 5 Intel ...	200,0	R\$ 3.600,00	Tax-free (0,0%)
6	Monitor	Monitor 27 LG Full hd, LED ips...	100,0	R\$ 1.200,00	Tax-free (0,0%)
7	Hub USB	Dock Hub Usb Tipo C Para Us...	600,0	R\$ 100,00	Tax-free (0,0%)
8	Adaptador de Rede	Adaptador de rede USB RJ45 ...	900,0	R\$ 50,00	Tax-free (0,0%)
9	Ring Light	Kit Iluminador Led Ring Light ...	500,0	R\$ 99,00	Tax-free (0,0%)
10	Celular	Iphone 6 Plus, 128GB, Cinza E...	100,0	R\$ 3.000,00	Tax-free (0,0%)



PYTHON

Módulo 32

REGEX

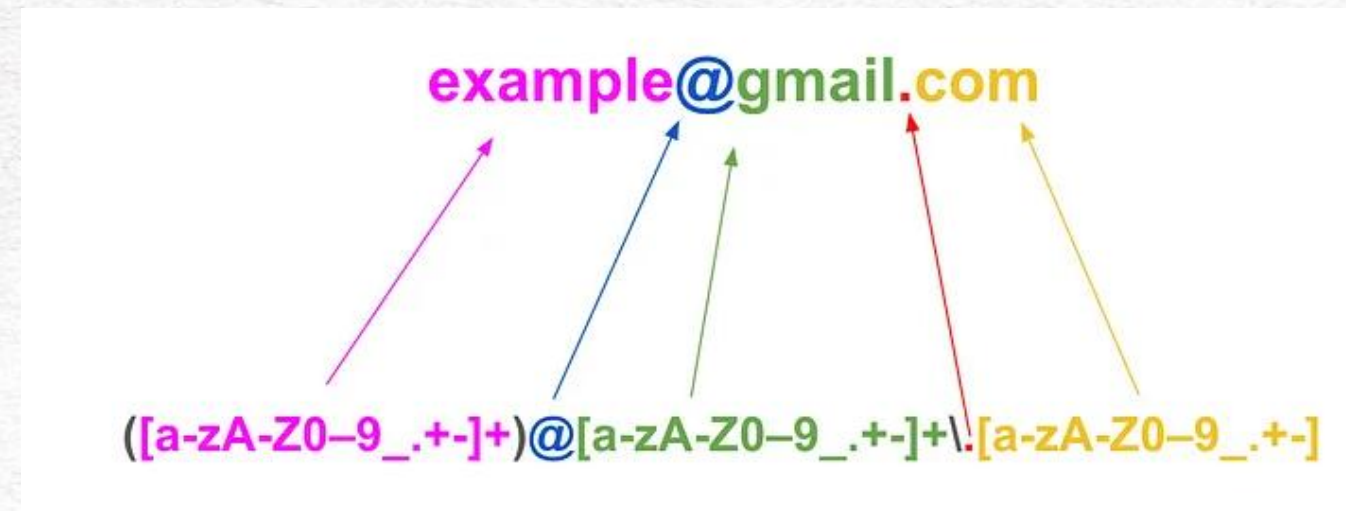
EXPRESSÕES REGULARES



O que é Regex?

Regex é a abreviação de expressão regular (**regular expression**). Basicamente, é uma sequência de caracteres que define um padrão de buscas em texto.

O objetivo principal em usar regex é conseguir escrever padrões de texto, e encontrar esses padrões dentro um texto qualquer.



É importante saber que o regex não funciona por si só. Ele é uma ferramenta que nos ajuda em outras aplicações, como a extração de informações de uma coluna de texto em análise de dados ou a obtenção de uma informação específica de um texto advindo de web-scraping.

Nesse módulo, iremos utilizar alguns textos que estão disponíveis para download abaixo da primeira aula.

Faça o download e abra o arquivo no Jupyter Notebook. No notebook, além dos textos, você também encontrará uma cartilha resumindo os operadores, caracteres e funções que serão abordados nesse módulo.

```
texto = """
Bom dia,

Seguem os orçamentos solicitados:

Cerveja importada (330 ml) - R$12,30598615178 - bebida
Cerveja nacional (0,5 litros) - R$6,10 - bebida
Garrafa de vinho (750ml) - R$39,90 - bebida
Água (garrafa de 1,5 litros) - R$3,30 - bebida
Alface (1 unidade) - R$3,50 - comida
Cebolas (1kg) - R$5,10 - comida
Batatas (1 kg) - R$5,20 - comida
Tomates (1 kg) - R$7,90 - comida
Laranjas (1 kg) - R$4,70 - comida
Bananas (1kg) - R$5,50 - comida
Maçãs (1 kg) - R$8,30 - comida
Queijo fresco (1 kg) - R$42,90 - comida
Uma dúzia de ovos(12) - R$9,80 - comida
Arroz (1 kg) - R$5,70 - comida
Um quilo de pão (1 kg) - R$7,20 - comida
Leite (1 litro) - R$5,20 - bebida
Azeite (1 unidade) - R$20 - tempero
Pimenta Reino (20g) - R$5 - tempero

Favor informar as quantidades desejadas
para emissão da Nota Fiscal.

Att., """
```

Imagine que recebemos um e-mail de orçamentos com o texto da imagem ao lado.

Perceba que ele tem várias informações, que a princípio não são tão simples de extrair. Mas note que apesar disso cada uma das informações segue algum tipo de padrão.

Por exemplo, o preço tem "R\$" antes, e tem casas decimais separadas por vírgula. Enquanto que a quantidade está entre parênteses.

Esses são os padrões que conseguimos buscar de maneira bem eficiente usando o regex.

Vamos iniciar com um uso mais simples do regex, que você poderia resolver até mesmo usando métodos de string.

Nosso objetivo inicial será contar quantas vezes as palavras **"comida"** e **"bebida"** aparecem no texto.

O **primeiro passo** é **importar a biblioteca do regex**, e isso pode ser feito com o código `import re`. Não é necessário instalar a biblioteca, porque o python, assim como a maioria das linguagens de programação, já vem com essa biblioteca instalada.

```
import re
```

O **segundo passo** é **definir o padrão que queremos encontrar**, e para isso, vamos passar esse padrão de texto dentro da função `re.compile()`. Como no nosso caso o padrão é, por exemplo, exatamente a palavra **"comida"**, vamos simplesmente passar ela dentro da função.

```
padrao = re.compile("comida")
```

E **por último**, vamos **usar uma função que efetivamente procure aquele padrão** e retorne algo. No caso, usaremos a `re.findall()`, passando o padrão procurado, e em seguida o texto. Essa função vai retornar uma lista com todos os textos correspondentes aquele padrão.

```
resultado = re.findall(padrao, texto)
```

E ao imprimir o que ficou dentro de resultado, veremos essa lista.

```
['comida', 'comida', 'comida', 'comida', 'comida', 'comida', 'comida', 'comida', 'comida', 'comida', 'comida']
```

Como nosso objetivo é saber a quantidade de vezes que comida e bebida aparecem no texto do e-mail, podemos simplesmente printar o tamanho da lista e em seguida replicar a solução para a palavra bebida.

```
import re

# quantas comidas
padrao = re.compile("comida")
resultado = re.findall(padrao, texto)
print(len(resultado))

# quantas bebidas
padrao = re.compile("bebida")
resultado = re.findall(padrao, texto)
print(len(resultado))
```

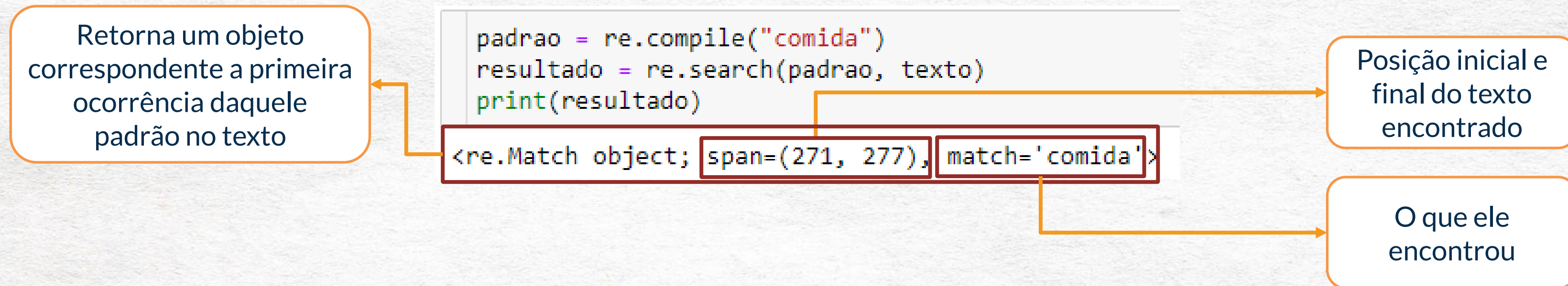
```
11
5
```

Com isso, temos 11 comidas e 5 bebidas listadas no nosso e-mail.

No exemplo anterior, usamos a `re.findall()`. Mas existem outras funções para procurar padrões em texto, e vamos agora saber as diferenças que cada uma delas têm.

`re.search()`

Voltando a busca da comida, vamos agora visualizar o que encontrados como resultado, ao usar o `re.search` para buscar.



E podemos acessar exatamente o texto que foi encontrado usando o `.group(0)`.

```
print(resultado)
print(resultado.group(0))
```

<re.Match object; span=(271, 277), match='comida'>
comida

re.finditer()

O **finditer** vai encontrar, e criar um iterador. Ou seja, vai retornar um objeto que você consegue percorrer com um loop.

```
# quantas comidas
padrao = re.compile("comida")
resultado = re.finditer(padrao, texto)
print(resultado)

<callable_iterator object at 0x0000027E2A98D850>
```

E ao iterar esse objeto, você observará que cada um dos itens será aquele mesmo objeto que o **search** retorna. Então, é possível ver a posição, o texto encontrado, etc.

```
# quantas comidas
padrao = re.compile("comida")
resultado = re.finditer(padrao, texto)
# print(resultado)
for item in resultado:
    print(item)

<re.Match object; span=(271, 277), match='comida'>
<re.Match object; span=(303, 309), match='comida'>
<re.Match object; span=(336, 342), match='comida'>
<re.Match object; span=(369, 375), match='comida'>
<re.Match object; span=(403, 409), match='comida'>
<re.Match object; span=(435, 441), match='comida'>
<re.Match object; span=(466, 472), match='comida'>
<re.Match object; span=(506, 512), match='comida'>
<re.Match object; span=(546, 552), match='comida'>
<re.Match object; span=(577, 583), match='comida'>
<re.Match object; span=(618, 624), match='comida'>
```

No início da cartilha disponibilizada na aula, têm alguns tópicos com operados e caracteres. Eles serão essenciais para criarmos os padrões que serão buscadas. Mas antes de falar sobre eles, precisamos falar de uma observação.

Note que, por exemplo, o caractere "\$" é um operador para buscar elementos no final da string. Ou seja, por padrão ele não é interpretado como uma letra que estamos buscando dentro do texto.

Então, para esses casos, se quisermos realmente o caractere, e não a sua "função", precisamos passar "\" antes desse caractere.

Para exemplificar isso, podemos tentar buscar quantos itens têm no texto do e-mail.

Têm várias formas de se fazer isso, mas uma possível é contar a quantidade de "\$", já que cada linha com produto têm esse caractere. E como queremos buscar realmente o caractere "\$", e não ele como um operador, precisamos passar "\\\$".

Operadores

- [] - conjunto de caracteres;
- \\ - sequência especial de caracteres;
- ^ - buscar elementos no início da string;
- \$ - buscar elementos no final da string;
- * - buscar zero ou mais repetições de uma substring;
- + - uma ou mais aparições de uma substring;
- ? - zero ou uma aparição;
- | - busca um caractere ou outro.
- { } - quantidade específica de caracteres
- [^] - diferente de um caractere especificado logo após o ^
- () - apenas para agrupar regras e definir ordem de aplicação (igual matemática)

▼ Especificando caracteres:

- . - qualquer caractere
- \\d - qualquer dígito
- \\D - não é dígito
- \\w - qualquer alfanumérico
- \\W - não é alfanumérico
- \\s - espaço em branco
- \\S - não é espaço em branco

```
# quantos itens
padrao = re.compile("\\$")
resultado = re.findall(padrao, texto)
print(len(resultado))
```

Agora vamos iniciar com exemplos mais desafiadores, em que vamos criar padrões mais complexos de busca.

Nosso desafio inicial será **extrair todos os números** do texto do e-mail.

Para resolver esse desafio, voltaremos aos caracteres e operadores apresentados no início da nossa cartilha, a lógica será de:

- Com os **caracteres** indicaremos o que queremos que **apareça naquele padrão**.
- Com os **operadores** indicaremos **quantas vezes eles vão aparecer ou as regras de aparição**.

Especificando caracteres:

. - qualquer caractere
\d - qualquer dígito
\D - não é dígito
\w - qualquer alfanumérico
\W - não é alfanumérico
\s - espaço em branco
\S - não é espaço em branco

Quais caracteres
devem aparecer

Regras de
aparição

Operadores

[] - conjunto de caracteres;
\ - sequência especial de caracteres;
^ - buscar elementos no início da string;
\$ - buscar elementos no final da string;
* - buscar zero ou mais repetições de uma substring;
+ - uma ou mais aparições de uma substring;
? - zero ou uma aparição;
| - busca um caractere ou outro.
{ } - quantidade específica de caracteres
[^] - diferente de um caractere especificado logo após o ^
() - apenas para agrupar regras e definir ordem de aplicação (igual matemática)

Uma recomendação importante em regex, é usar o **r**, de **raw string**, antes de passar o texto com o padrão. Note que temos vários vezes a **"\"** nos caracteres apresentados acima, e no python muitas vezes esses caracteres unidos a outros podem significar algo ("**\n**" = pular linha, "**\t**" = tab, etc), e como queremos ignorar esse padrões do python, é uma boa pratica usar o **r** antes da string com o padrão.

Voltando agora ao desafio de **extrair todos os números** do texto do e-mail.

Consultado a cartilha, temos que o `"\d"` busca todos os dígitos, então vamos testa-lo.

Ex: Extrair números

```
padrao = re.compile(r"\d")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['3', '3', '0', '1', '2', '3', '0', '5', '9', '8', '6', '1', '5', '1', '7', '8', '0', '5', '6', '1', '0', '7', '5', '0', '3',  
'9', '9', '0', '1', '5', '3', '3', '0', '1', '3', '5', '0', '1', '5', '1', '0', '1', '5', '2', '0', '1', '7', '9', '0', '1',  
'4', '7', '0', '1', '5', '5', '0', '1', '8', '3', '0', '1', '4', '2', '9', '0', '1', '2', '9', '8', '0', '1', '5', '7', '0',  
'1', '7', '2', '0', '1', '5', '2', '0', '1', '2', '0', '2', '0', '5']
```

Note que como não passamos algo que indique quantas vezes deve aparecer o número, ele vai retornar todas as vezes que ele encontrou um dígito. Consultando novamente a cartilha, temos que o `+`, representa **“uma ou mais aparições de uma substring”**, então vamos dar uma olhada no resultado ao passar `"\d+"`.

```
padrao = re.compile(r"\d+")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['330', '12', '30598615178', '0', '5', '6', '10', '750', '39', '90', '1', '5', '3', '30', '1', '3', '50', '1', '5', '10', '1',  
'5', '20', '1', '7', '90', '1', '4', '70', '1', '5', '50', '1', '8', '30', '1', '42', '90', '12', '9', '80', '1', '5', '70',  
'1', '7', '20', '1', '5', '20', '1', '20', '20', '5']
```

Nosso resultado já começa a tomar forma, mas agora temos que lidar com a vírgula nos números.

Note que alguns números do texto tem a vírgula, enquanto outros não. Então se passarmos, por exemplo, “\d+,\d+” só encontraremos os números que tem decimais.

```
Cerveja importada (330 ml) - R$12,30598615178 - bebida
Cerveja nacional (0,5 litros) - R$6,10 - bebida
Garrafa de vinho (750ml) - R$39,90 - bebida
Água (garrafa de 1,5 litros) - R$3,30 - bebida
```

Para esse caso, temos o operador “?” que representa “zero ou uma aparição”, ou seja, pode ter ou não o que vier antes dele. Com isso, temos o padrão “\d+” para encontrar dígitos, “?” para representar que podemos ou não ter uma vírgula, e em seguida novamente o padrão “+\d” para representar temos mais dígitos depois.

```
padrao = re.compile(r"\d+,?\d+")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['330', '12,30598615178', '0,5', '6,10', '750', '39,90', '1,5', '3,30', '3,50', '5,10', '5,20', '7,90', '4,70', '5,50', '8,30', '42,90', '12', '9,80', '5,70', '7,20', '5,20', '20', '20']
```

Note que ainda está **faltando alguns números aparecerem na nossa lista**, por exemplo, o 1. Por enquanto, vamos deixar dessa forma, e seguir para um exemplo em outro texto que pode nos ajudar a entender melhor a lógica do nosso problema.

```
Cebolas (1kg) - R$5,10 - comida
Batatas (1 kg) - R$5,20 - comida
Tomates (1 kg) - R$7,90 - comida
```

Nesse exemplo, usaremos o texto "Nome: Lira Idade: 29 Salario: 1500". Os operadores usados serão sempre consultado a cartilha. Para iniciar, crie um nova célula, e monte a seguinte estrutura:

```
novo_texto = "Nome: Lira Idade: 29 Salario: 1500"  
padrao = re.compile(r"")  
resultado = re.findall(padrao, novo_texto)  
print(resultado)
```

Extraindo sequência de números:

- "\d+"

```
novo_texto = "Nome: Lira Idade: 29 Salario: 1500"  
padrao = re.compile(r"\d+")  
resultado = re.findall(padrao, novo_texto)  
print(resultado)
```

```
['29', '1500']
```

Extraindo sequência de qualquer caracteres alfanuméricos:

- "\w+"

```
novo_texto = "Nome: Lira Idade: 29 Salario: 1500"  
padrao = re.compile(r"\w+")  
resultado = re.findall(padrao, novo_texto)  
print(resultado)
```

```
['Nome', 'Lira', 'Idade', '29', 'Salario', '1500']
```

Extraindo sequência de textos que não tenham dígitos:

- "\D+"

```
novo_texto = "Nome: Lira Idade: 29 Salario: 1500"  
padrao = re.compile(r"\D+")  
resultado = re.findall(padrao, novo_texto)  
print(resultado)
```

```
['Nome: Lira Idade: ', ' Salario: ']
```

Extraindo qualquer sequência de caracteres:

- ".+"

```
novo_texto = "Nome: Lira Idade: 29 Salario: 1500"  
padrao = re.compile(r".+")  
resultado = re.findall(padrao, novo_texto)  
print(resultado)
```

```
['Nome: Lira Idade: 29 Salario: 1500']
```

Partindo para padrões mais elaborados, como faríamos para extrair apenas os rótulos nome, idade e salário? Nesse caso poderíamos passar “\w+.” para representar qualquer sequência de alfanuméricos que finalize com “.”.

```
novو_texto = "Nome: Lira Idade: 29 Salario: 1500"
padrao = re.compile(r"\w+:")
resultado = re.findall(padrao, novo_texto)
print(resultado)

['Nome:', 'Idade:', 'Salario:']
```

Note que a resposta também tem o “.” no final. Se quisermos, por exemplo, a resposta sem esse caractere no final, podemos usar o parênteses. Que tem basicamente duas funções:

- Se você colocar algo dentro do parênteses e em seguida passar um operador, vai ser entendido que tudo que está dentro do parênteses deve seguir a regra do operador.
- Ou no nosso caso, em que não há um operador depois do parênteses, ele vai continuar seguindo o padrão normalmente, mas a função **findall** só vai retornar aquilo que está dentro do parênteses.

```
novو_texto = "Nome: Lira Idade: 29 Salario: 1500"
padrao = re.compile(r"(\w+):")
resultado = re.findall(padrao, novo_texto)
print(resultado)

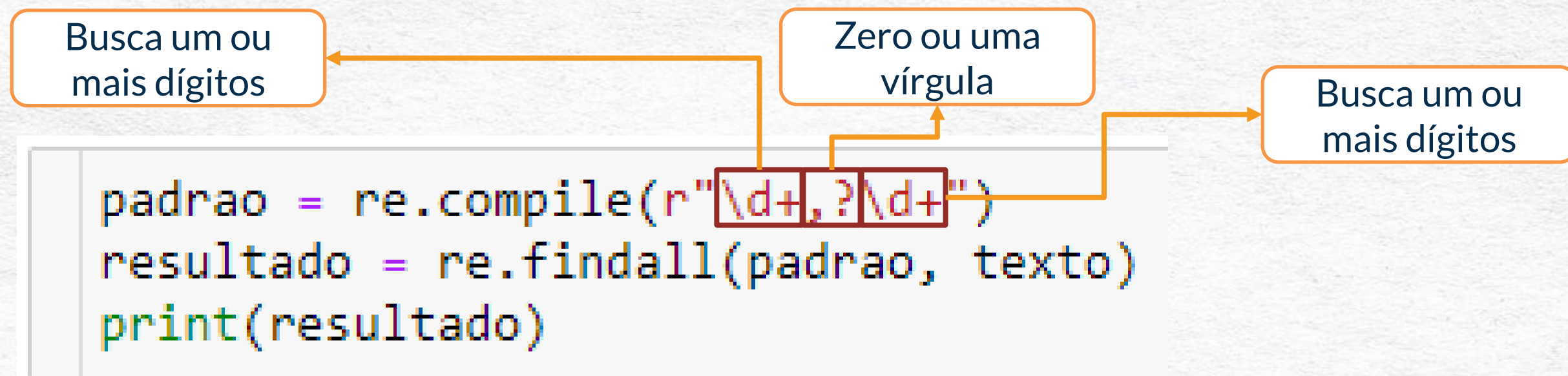
['Nome', 'Idade', 'Salario']
```

Observação: Não se preocupe se não tiver ficado tão claro o uso do parênteses, voltaremos a ele nos próximos desafios.

Veremos agora alguns exemplos com as diferenças de uso no “*”, “+” e “?”.

- * - buscar zero ou mais repetições de uma substring;
- + - uma ou mais aparições de uma substring;
- ? - zero ou uma aparição;

Para isso **voltaremos ao texto do e-mail**, no nosso desafio de **buscar todos os números**. E vamos destrinchar o que cada parte está fazendo novamente.



Imagine a situação em que não há vírgula no nosso número. Na prática, teríamos “`\d+\d+`”, ou seja, estamos buscando **um ou mais dígitos, seguido de um ou mais dígitos (no mínimo forma um número com dois dígitos)**. Por isso, precisamos usar o “*” como a regra da quantidade de dígitos depois da vírgula. E só então que vamos buscar um ou mais dígitos, que depois podem ter ou não vírgula, e em seguida podemos ter ou não outros dígitos.

Ex: Extrair números

```
padrao = re.compile(r"\d+,?\d*")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['330', '12,30598615178', '0,5', '6,10', '750', '39,90', '1,5', '3,30', '1', '3,50', '1', '5,10', '1', '5,20', '1', '7,90',  
'1', '4,70', '1', '5,50', '1', '8,30', '1', '42,90', '12', '9,80', '1', '5,70', '1', '7,20', '1', '5,20', '1', '20', '20', '5']
```



Vamos agora treinar extraindo **quantos ml têm o primeiro item do e-mail**. Para esse fim, usaremos a função `re.search`, que só encontra a primeira correspondência ao padrão.

```
Cerveja importada (330 ml) - R$12,30598615178 - bebida
Cerveja nacional (0,5 litros) - R$6,10 - bebida
Garrafa de vinho (750ml) - R$39,90 - bebida
```

Temos algumas formas de realizar esse desafio, e uma delas é procurar algo que comece com “(“, no meio tenha qualquer caractere em qualquer quantidade, e depois acabe com “)”. Antes de partir para o código, lembre-se que, assim como o “\$”, o parênteses é um dos caracteres especiais utilizados em regex, e como queremos procurar realmente pelo caractere, é necessário colocar a barra invertida antes.

Exemplificando a versatilidade que temos ao usar regex, podemos procurar de outra forma, por exemplo, algo que inicie com “(“, depois tenha um ou mais dígitos, depois um espaço, depois um ou mais caracteres alfanuméricos, e no fim, o “)”

Nessa segunda situação, fomos mais específicos no que queríamos dentro dos parêntesis, e isso é algo que em alguns casos pode evitar possíveis correspondências indesejadas

Ex: Extrair quantos ml (do 1º item)

```
padrao = re.compile(r"\(.+\)")
resultado = re.search(padrao, texto)
print(resultado)
```

```
<re.Match object; span=(65, 73), match='(330 ml)'>
```

```
padrao = re.compile(r"\(\d+\s\w+\)")
resultado = re.search(padrao, texto)
print(resultado)
```

```
<re.Match object; span=(65, 73), match='(330 ml)'>
```

Continuaremos agora extraindo o preço do primeiro item do e-mail.

```
Cerveja importada (330 ml) - R$12,30598615178 - bebida
Cerveja nacional (0,5 litros) - R$6,10 - bebida
Garrafa de vinho (750ml) - R$39,90 - bebida
```

Observando o preço, note que ele começa com “R\$”, depois vem **alguns dígitos**, depois a “,” e depois **mais dígitos**.

Ex: Preço do 1º item

```
padrao = re.compile(r"R\$\d+,\d+")
resultado = re.search(padrao, texto)
print(resultado)
print(resultado.group(0))
```

```
<re.Match object; span=(76, 92), match='R$12,30598615178'>
R$12,30598615178
```

Ao compilar e buscar o padrão “R\\$\d+,\d+”, encontramos nossa resposta. Lembrando novamente que, o “\$” é um caractere especial do regex, então é necessário colocar barra invertida antes.

Além disso, como usamos o `re.search`, podemos usar o `.group(0)` para extrair somente o texto que deu match.

Criamos um padrão para encontrar o primeiro preço do e-mail, mas talvez você tenha notado que ele não ficou tão robusto para encontrar todos os preços. Então agora nosso desafio será esse, conseguir extrair **todos os preços do texto do e-mail**.

Vamos um pouco mais por partes nesse desafio. Primeiro, montando nosso código padrão de buscar. Aqui, usaremos o **re.findall** porque queremos encontrar vários valores.

Ex: Todos os preços

```
padrao = re.compile(r"")
resultado = re.findall(padrao, texto)
print(resultado)
```

Agora vamos buscar algo que inicie com “R\$”, lembrando de adicionar a contra barra antes do “\$”.

```
padrao = re.compile(r"R\$")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['R$', 'R$', 'R$', 'R$', 'R$', 'R$', 'R$', 'R$', 'R$', 'R$', 'R$', 'R$', 'R$', 'R$', 'R$', 'R$', 'R$']
```

Continuando, depois do “R\$”, temos um ou mais dígitos, que podem ser representados com “\d+”.

```
padrao = re.compile(r"R\$d+")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['R$12', 'R$6', 'R$39', 'R$3', 'R$3', 'R$5', 'R$5', 'R$7', 'R$4', 'R$5', 'R$8', 'R$42', 'R$9', 'R$5', 'R$7', 'R$5', 'R$20', 'R$5']
```

Logo depois, podemos ter ou não ter vírgula. E para buscar essa situação podemos usar o “?”.

```
padrao = re.compile(r"R\$\d+,?")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['R$12,', 'R$6,', 'R$39,', 'R$3,', 'R$3,', 'R$5,', 'R$5,', 'R$7,', 'R$4,', 'R$5,', 'R$8,', 'R$42,', 'R$9,', 'R$5,', 'R$7,', 'R$5,', 'R$20', 'R$5']
```

E agora vem um desafio um pouco maior em termos de lógica. Lembra da situação anterior em que pegamos todos os números?

Da mesma forma, aqui não podemos usar o “\d+”, porque caso o número não tenha vírgula, só encontraríamos um número com dois ou mais dígitos. E por isso, precisamos usar “\d*”, representando que depois de ter ou não a vírgula, podemos ter qualquer quantidade de dígitos, inclusive nenhum.

```
padrao = re.compile(r"R\$\d+,?\d*")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['R$12,30598615178', 'R$6,10', 'R$39,90', 'R$3,30', 'R$3,50', 'R$5,10', 'R$5,20', 'R$7,90', 'R$4,70', 'R$5,50', 'R$8,30', 'R$42,90', 'R$9,80', 'R$5,70', 'R$7,20', 'R$5,20', 'R$20', 'R$5']
```

Com isso, conseguimos chegar no resultado final.

E agora usaremos esse e outros exemplos como base para aprender alguns outros operadores de regex

Vamos iniciar usando os **colchetes**.

[] - conjunto de caracteres;

Basicamente podemos passar que caracteres específicos queremos que sejam encontrados. Então, se em vez do segundo “\d+”, usarmos “[0-5]”, encontraremos apenas números que finalizem com dígitos de 0 a 5.

```
padrao = re.compile(r"R\$\d+,?[0-5]")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['R$12,3', 'R$6,1', 'R$3,3', 'R$3,5', 'R$5,1', 'R$5,2', 'R$5,5', 'R$8,3', 'R$42', 'R$7,2', 'R$5,2', 'R$20']
```

Da mesma forma, podemos passar letras dentro dos colchetes, observe a imagem abaixo com um dos exemplo que abordamos anteriormente.

```
novo_texto = "Nome: Lira Idade: 29 Salario: 1500"
padrao = re.compile(r"[a-z]+")
resultado = re.findall(padrao, novo_texto)
print(resultado)
```

```
['ome', 'ira', 'dade', 'alario']
```

Note que no exemplo anterior ele pegou corretamente as letras de “a” até “z”, porém não pegou letras maiúsculas (o python é **case sensitive**, ou seja, letras maiúscula e minúsculas são diferentes). Se quisermos incluir todas as letras independente de ser maiúscula ou minúscula, é só passar dentro do colchete as duas formas.

```
novo_texto = "Nome: Lira Idade: 29 Salario: 1500"
padrao = re.compile(r"[a-zA-Z]+")
resultado = re.findall(padrao, novo_texto)
print(resultado)
```

```
['Nome', 'Lira', 'Idade', 'Salario']
```

E podemos, incluir outros caracteres dentro dos colchetes. Como por exemplo, o “:”, de forma que, como estamos passando o “+” depois do conteúdo que está dentro dos colchetes, qualquer caractere daquele conjunto de caracteres pode aparecer uma ou mais vezes.

```
novo_texto = "Nome: Lira Idade: 29 Salario: 1500"
padrao = re.compile(r"[ :a-zA-Z]+")
resultado = re.findall(padrao, novo_texto)
print(resultado)
```

```
['Nome:', 'Lira', 'Idade:', 'Salario:']
```

Vamos falar agora sobre o operador de chaves “{}”.

{} - quantidade específica de caracteres

Até agora usamos bastante o “+”, o “*” e o “?” para informar a quantidade de caracteres. Mas com as chaves também é possível realizar essa tarefa. Isso pode ser feito, basicamente, usando {quantidade mínima, quantidade máxima}.

Lembra do exemplo em que extraímos todos os preços?

Ex: Todos os preços

```
padrao = re.compile(r"R\$\d+,?\d*")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['R$12,30598615178', 'R$6,10', 'R$39,90', 'R$3,30', 'R$3,50', 'R$5,10', 'R$5,20', 'R$7,90', 'R$4,70', 'R$5,50', 'R$8,30', 'R$42,90', 'R$9,80', 'R$5,70', 'R$7,20', 'R$5,20', 'R$20', 'R$5']
```

Note que o primeiro valor, é o “R\$12,30598615178”, um número com várias casas decimais. Se quiséssemos no máximo, por exemplo, duas casas decimais, poderíamos usar as chaves, informando que a quantidade mínima é 0, e a máxima é 2. Resumindo em vez de “*”, usaremos “{0,2}”.

```
padrao = re.compile(r"R\$\d+,?\d{0,2}")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['R$12,30', 'R$6,10', 'R$39,90', 'R$3,30', 'R$3,50', 'R$5,10', 'R$5,20', 'R$7,90', 'R$4,70', 'R$5,50', 'R$8,30', 'R$42,90', 'R$9,80', 'R$5,70', 'R$7,20', 'R$5,20', 'R$20', 'R$5']
```

Vamos agora extrair cada uma das **quantidades dos itens** presentes do texto do e-mail.

```
Cerveja importada (330 ml) - R$12,30598615178 - bebida  
Cerveja nacional (0,5 litros) - R$6,10 - bebida  
Garrafa de vinho (750ml) - R$39,90 - bebida
```

Basicamente, queremos encontrar algo que comece com “(“, no meio tenha qualquer conteúdo em qualquer quantidade, e termine com “)”. Lembrando que, como queremos realmente o caractere parênteses, é necessário usar a barra invertida.

Ex: Quantidades dos itens

```
padrao = re.compile(r"\(.+\)")  
resultado = re.findall(padrao, texto)  
print(resultado)
```

```
['(330 ml)', '(0,5 litros)', '(750ml)', '(garrafa de 1,5 litros)', '(1 unidade)', '(1kg)', '(1 kg)', '(1 kg)', '(1 kg)', '(1k  
g)', '(1 kg)', '(1 kg)', '(12)', '(1 kg)', '(1 kg)', '(1 litro)', '(1 unidade)', '(20g)']
```

Falando em parênteses, lembra que comentamos antes um pouco sobre a sua função como operador em regex?

Vamos aplicar um pouco do uso desse operador.

Para exemplificar esse uso de parênteses, vamos voltar ao exemplo em que coletamos **todos os preços**.

Ex: Todos os preços

```
padrao = re.compile(r"R$\d+,?\d{0,2}")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['R$12,30', 'R$6,10', 'R$39,90', 'R$3,30', 'R$3,50', 'R$5,10', 'R$5,20', 'R$7,90', 'R$4,70', 'R$5,50', 'R$8,30', 'R$42,90', 'R$9,80', 'R$5,70', 'R$7,20', 'R$5,20', 'R$20', 'R$5']
```

Essencialmente, podemos passar só o que queremos que esteja na nossa resposta final dentro do operador de parênteses. Ou seja, o padrão buscado continuará o mesmo, mas a resposta final excluirá o que não tiver dentro do parênteses.

No caso, se quisermos encontrar todos os preços, mas sem incluir o “R\$” na resposta final, basta passar o resto do padrão dentro de parênteses.

No fim, podemos usar algo como "R\$\d+,?\d{0,2})"

```
padrao = re.compile(r"R$(\d+,?\d{0,2})")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['12,30', '6,10', '39,90', '3,30', '3,50', '5,10', '5,20', '7,90', '4,70', '5,50', '8,30', '42,90', '9,80', '5,70', '7,20', '5,20', '20', '5']
```

Voltando agora ao exemplo de encontrar **todas as quantidades**. Você consegue imaginar como podemos fazer para selecionar apenas o conteúdo que está dentro do parênteses?

Ex: Quantidades dos itens

```
padrao = re.compile(r"\(.+\)")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['(330 ml)', '(0,5 litros)', '(750ml)', '(garrafa de 1,5 litros)', '(1 unidade)', '(1kg)', '(1 kg)', '(1 kg)', '(1 kg)', '(1k g)', '(1 kg)', '(1 kg)', '(12)', '(1 kg)', '(1 kg)', '(1 litro)', '(1 unidade)', '(20g)']
```

Podemos passar o “.” dentro de parênteses, lembrando que nesses parênteses não vamos usar a barra invertida, já que queremos realmente o operador de regex.

No fim, o nosso padrão fica “\((.+)\)”.

Ex: Quantidades dos itens

```
padrao = re.compile(r"\((.+)\)")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['330 ml', '0,5 litros', '750ml', 'garrafa de 1,5 litros', '1 unidade', '1kg', '1 kg', '1 kg', '1 kg', '1kg', '1 kg', '1 kg', '12', '1 kg', '1 kg', '1 litro', '1 unidade', '20g']
```

Voltando agora ao exemplo de encontrar **todas as quantidades**. Você consegue imaginar como podemos fazer para selecionar apenas o conteúdo que está dentro do parênteses?

Ex: Quantidades dos itens

```
padrao = re.compile(r"\(.+\)")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['(330 ml)', '(0,5 litros)', '(750ml)', '(garrafa de 1,5 litros)', '(1 unidade)', '(1kg)', '(1 kg)', '(1 kg)', '(1 kg)', '(1k g)', '(1 kg)', '(1 kg)', '(12)', '(1 kg)', '(1 kg)', '(1 litro)', '(1 unidade)', '(20g)']
```

Essencialmente podemos passar o “.” dentro de parênteses, lembrando que nesses parênteses não vamos usar a barra invertida, já que queremos realmente o operador de regex.

No fim, o nosso padrão fica “\((.+)\)”.

Ex: Quantidades dos itens

```
padrao = re.compile(r"\((.+)\)")
resultado = re.findall(padrao, texto)
print(resultado)
```

```
['330 ml', '0,5 litros', '750ml', 'garrafa de 1,5 litros', '1 unidade', '1kg', '1 kg', '1 kg', '1 kg', '1kg', '1 kg', '1 kg', '12', '1 kg', '1 kg', '1 litro', '1 unidade', '20g']
```

Uma observação importante é que esse funcionamento do operador de parênteses só funciona com a função `re.findall`. Veja o exemplo abaixo com o `re.search` e o `re.finditer` na busca de todos os preços.

```
padrao = re.compile(r"R\$(\d+,?\d{0,2})")

resultado = re.findall(padrao, texto)
resultado2 = re.search(padrao, texto)
resultado3 = re.finditer(padrao, texto)

print(resultado) # findall
print(resultado2) # search
for item in resultado3: # finditer
    print(item)
```

`re.findall`

```
['12,30', '6,10', '39,90', '3,30', '3,50', '5,10', '5,20', '7,90', '4,70', '5,50', '8,30', '42,90', '9,80', '5,70', '7,20', '5,20', '20', '5']
```

`re.search`

```
<re.Match object; span=(76, 83), match='R$12,30'>
<re.Match object; span=(76, 83), match='R$12,30'>
<re.Match object; span=(134, 140), match='R$6,10'>
<re.Match object; span=(177, 184), match='R$39,90'>
<re.Match object; span=(225, 231), match='R$3,30'>
<re.Match object; span=(262, 268), match='R$3,50'>
```

`re.finditer`

Note que os resultados do `re.search` e `re.finditer` continuam incluindo o “R\$”.

Nos próximos exemplos veremos uma outra funcionalidade do operador de parênteses, que aí sim, afeta qualquer uma das funções citadas acima.

Utilizaremos agora o texto com os links disponível no final da nossa cartilha.

Basicamente, nosso desafio será extrair todos os links desse texto e usaremos as funções **re.findall** e **re.finditer** para exemplificar as diferenças no seu resultado.

Ex: Links

```
texto2 = """
Olá

Segue o relatório de SEO para os seguintes sites:
https://portalhashtag.com
http://hashtagtreinamentos.com
https://www.wikipedia.org/
www.bcb.gov.br/

Qualquer dúvida estamos à disposição"""

padrao = re.compile(r"")
resultado1 = re.findall(padrao, texto2)
resultado2 = re.finditer(padrao, texto2)

print(resultado1)
for item in resultado2:
    print(item)
```

Veremos que temos várias formas de realizar esse desafio, principalmente sendo mais ou menos restritos no que um link contém.

Inicialmente podemos encontrar usando um padrão menos restritivo e ignorando o links que iniciam com “www”.

Então basicamente buscar algo que comece com “http”, em seguida vem qualquer texto, e finaliza com uma quebra de linha, que em regex pode ser encontrada utilizando o “\s”.

Ao juntar tudo isso, temos o padrão “http.+s”. `padrao = re.compile(r"http.+s")`

```
['https://portalhashtag.com\n', 'http://hashtagtreinamentos.com\n', 'https://www.wikipedia.org/\n']  
<re.Match object; span=(56, 82), match='https://portalhashtag.com\n'>  
<re.Match object; span=(82, 113), match='http://hashtagtreinamentos.com\n'>  
<re.Match object; span=(113, 140), match='https://www.wikipedia.org/\n'>
```

Nesse caso, o “.” não inclui quebra de linha, então é possível passar apenas o “http.+” e obter basicamente o mesmo resultado.

Agora vamos tentar ser **mais restritivos em que regras queremos usar** para encontrar o link.

Então ele vai poder começar com “http”, “https” ou “www”, depois pode ter vários caracteres, depois tem que ter um “.”, e no fim ele tem que finalizar com: “com”, “com.br”, “org” ou “gov.br”.

Lembrando que aqui são nossas escolhas, porque queremos restringir apenas essa finalizações de link.

Como mencionado, nosso objetivo é pegar todos os links, agora de maneira mais restritiva.

```
https://portalhashtag.com  
http://hashtagtreinamentos.com  
https://www.wikipedia.org/  
www.bcb.gov.br/
```

Então, o link inicia com “**http**”, pode ou não ter “**s**”, e depois tem “**://**”. Só que temos um link que simplesmente inicia com “**www**”, e outros que após o “**://**” tem ou não o “**www**”.

Perceba que fica um pouco mais complexo de passar essas regras, é aí que podemos **usar parênteses para passar um operador que vai agir em todo um conjunto de caracteres**.

Vamos analisar como construir o padrão do início de um link, até antes de começar o domínio do site.

```
padrao = re.compile(r"(https?://)?(www\.)?")
```

Ao aplicar o operador “?” depois do parênteses, basicamente aplicamos aquela regra de aparição em toda aquela sequência que está dentro do parênteses. Ou seja pode ou não ter toda aquela sequência de caracteres, que inclusive já contém outras regras internamente, como no primeiro caso, o “s?”.

Vamos continuar criando o nosso padrão de extração, buscando agora o início do domínio dos links.

```
https://portalhashtag.com  
http://hashtagtreinamentos.com  
https://www.wikipedia.org/  
www.bcb.gov.br/
```

No nosso caso, só apareceram letras, mas como também podem ser usados números, vamos buscar por caracteres alfanuméricos com o “\w+”.

```
padrao = re.compile(r"(https?://)?(www\.)?\w+")
```

E depois temos um ponto. Lembrando que como queremos realmente o caractere “.” e não seu operador, devemos passar a barra invertida.

```
padrao = re.compile(r"(https?://)?(www\.)?\w+\.")
```

Sobre o final do link, percebeu que tem várias escolhas em relação a como o link pode acabar?

Nesse caso podemos usar o operador de OU, que é o “|”. Em que vamos representar que o link pode finalizar com: “com”, “org”, “gov.br” e “com.br”, separando essas possibilidades com o “|”, e colocando toda essa regra dentro de parênteses.

```
padrao = re.compile(r"(https?://)?(www\.)?\w+\.(com\.br|com|org|gov\.br)")
```

Já finalizamos o nosso padrão, mas vamos dar uma olhada nos resultados.

```
[('https://', '', 'com'), ('http://', '', 'com'), ('https://', 'www.', 'org'), ('', 'www.', 'gov.br')]  
<re.Match object; span=(56, 81), match='https://portalhashtag.com'>  
<re.Match object; span=(82, 112), match='http://hashtagtreinamentos.com'>  
<re.Match object; span=(113, 138), match='https://www.wikipedia.org'>  
<re.Match object; span=(140, 154), match='www.bcb.gov.br'>
```

re.findall

re.finditer

Note que a solução com **re.findall** não trouxe corretamente o link, justamente porque os parênteses interferem no que ela traz como resultado. Como solução paliativa, podemos usar parênteses em todo o padrão criado, o que faz com que ela consiga também retornar o link inteiro.

```
padrao = re.compile(r"((https?://)?(www\.)?\w+\.(com\.br|com|org|gov\.br))")
```

```
[('https://portalhashtag.com', 'https://', '', 'com'), ('http://hashtagtreinamentos.com', 'http://', '', 'com'), ('https://www.wikipedia.org', 'https://', 'www.', 'org'), ('www.bcb.gov.br', '', 'www.', 'gov.br')]
```

Mas, via de regra esse é um dos casos em que faz simplesmente mais sentido usar apenas o **re.finditer**, que nos trouxe a resposta perfeitamente, e podemos usar, por exemplo, o **.group(0)** para pegar apenas o texto encontrado.

```
padrao = re.compile(r"((https?://)?(www\.)?\w+\.(com\.br|com|org|gov\.br))")
```

```
resultado1 = re.findall(padrao, texto2)
```

```
resultado2 = re.finditer(padrao, texto2)
```

```
print(resultado1)
```

```
for item in resultado2:
```

```
    print(item.group(0))
```

```
[('https://portalhashtag.com', 'https://', '', 'com'), ('http://hashtagtreinamentos.com', 'http://', '', 'com'), ('https://www.wikipedia.org', 'https://', 'www.', 'org'), ('www.bcb.gov.br', '', 'www.', 'gov.br')]
```

```
https://portalhashtag.com  
http://hashtagtreinamentos.com  
https://www.wikipedia.org  
www.bcb.gov.br
```



Agora vamos falar sobre os operadores que faltaram nos exemplos anteriores, que são “^”, “\$” e “[^]”.

Para exemplificar melhor, usaremos novo arquivo, que está disponível para download abaixo da aula.

```
informacoes = [  
    "http://hashtagtreinamentos.com",  
    "https://vimeo.com/",  
    "https://youtube.com/",  
    "Pesquisar no Google: http://google.com",  
    "http://wikipedia.org",  
    "http://google.com",  
    "http://bcb.gov.br/",  
    "Pesquisar no Bing",  
]
```

Iniciando, vamos ver um pouco sobre o “^”, que tem como objetivo “**buscar elementos no início da string**”. E exatamente por isso, não estamos usando os exemplos anteriores, porque apesar de ter textos grandes, eram textos únicos. Já aqui temos uma lista com vários textos, que vai permitir maior versatilidade em visualizar regras de início e fim (para o caso do próximo operador) para cada um dos textos.

Para exemplificar, monte a seguinte estrutura de código.

Vamos:

- Importar a biblioteca regex.
- Definir o padrão.
- E então percorrer a lista de textos, imprimindo o item inteiro e o resultado encontrado.

- ^ – buscar elementos no início da string;

```
import re  
  
padrao = re.compile(r"  
  
for item in informacoes:  
    print('item: ', item)  
  
    resultado = re.findall(padrao, item)  
    print('resultado: ', resultado)
```



Ao utilizar o padrão “**^http**”, veja que todos os textos que comecem com “**http**” vão aparecer no nosso resultado.

```
padrao = re.compile(r"^http")
```



```
item: http://hashtagtreinamentos.com
resultado: ['http']
item: https://vimeo.com/
resultado: ['http']
item: https://youtube.com/
resultado: ['http']
item: Pesquisar no Google: http://google.com
resultado: []
item: http://wikipedia.org
resultado: ['http']
item: http://google.com
resultado: ['http']
item: http://bcb.gov.br/
resultado: ['http']
item: Pesquisar no Bing
resultado: []
```

Um detalhe é que poderíamos passar o “**http**” entre parênteses, de forma que nosso padrão seria “**^(http)**”, ou seja, ele encontrará um texto de inicie efetivamente com “**http**”.

Como passamos “**^http**”, o que ele fez foi buscar algo que inicie com “**h**” e em seguida tenha “**ttp**”, o que na prática dá o mesmo resultado.

Da mesma forma podemos partir para exemplificar o “\$”, que serve para “**buscar elementos no final da string**”. Basicamente podemos copiar o mesmo código que usamos para testar o operador anterior.

- \$ – buscar elementos no final da string;

```
import re

padrao = re.compile(r"")

for item in informacoes:
    print('item: ', item)

    resultado = re.findall(padrao, item)
    print('resultado: ', resultado)
```

Em seguida, podemos procurar, por exemplo, apenas elementos que terminem com “.com”, ou que terminem com “.com” e possam ou não ter uma barra depois.

padrao = re.compile(r"\.com\$")

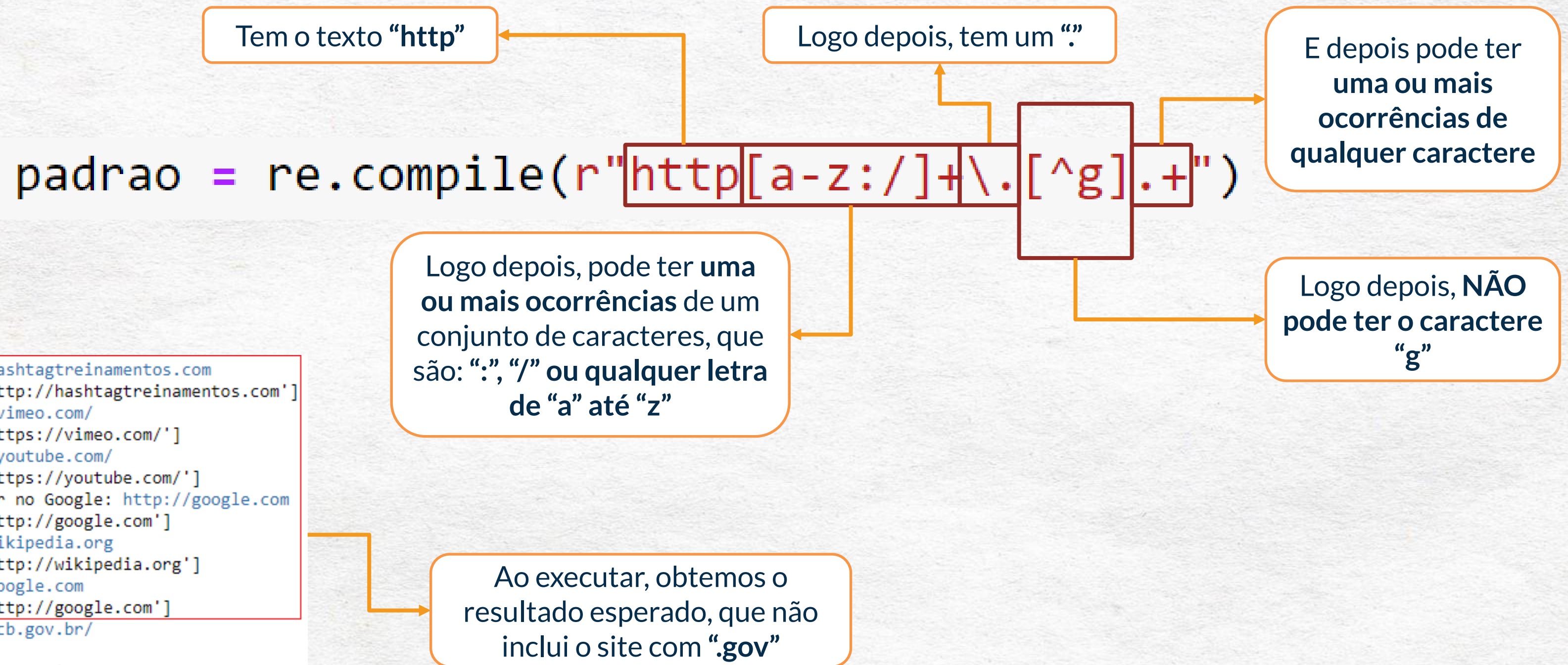
```
item: http://hashtagtreinamentos.com
resultado: ['.com']
item: https://vimeo.com/
resultado: []
item: https://youtube.com/
resultado: []
item: Pesquisar no Google: http://google.com
resultado: ['.com']
item: http://wikipedia.org
resultado: []
item: http://google.com
resultado: ['.com']
item: http://bcb.gov.br/
resultado: []
item: Pesquisar no Bing
resultado: []
```

```
item: http://hashtagtreinamentos.com
resultado: ['.com']
item: https://vimeo.com/
resultado: ['.com/']
item: https://youtube.com/
resultado: ['.com/']
item: Pesquisar no Google: http://google.com
resultado: ['.com']
item: http://wikipedia.org
resultado: []
item: http://google.com
resultado: ['.com']
item: http://bcb.gov.br/
resultado: []
item: Pesquisar no Bing
resultado: []
```

padrao = re.compile(r"\.com/?\$")



Partindo para o último operador, que é “[^]”, e faz a busca por: “diferente de um caractere especificado logo após o ^”. Novamente, podemos copiar o código utilizado nos operadores anteriores. Nosso objetivo com ele será extrair **todos os links**, exceto os que tenham o “gov”. No caso é só passar o que não queremos que seja encontrado após o “^”, tudo dentro de colchetes: “[^NÃO_ENCONTRAR]”.



```
item: http://hashtagtreinamentos.com
resultado: ['http://hashtagtreinamentos.com']
item: https://vimeo.com/
resultado: ['https://vimeo.com/']
item: https://youtube.com/
resultado: ['https://youtube.com/']
item: Pesquisar no Google: http://google.com
resultado: ['http://google.com']
item: http://wikipedia.org
resultado: ['http://wikipedia.org']
item: http://google.com
resultado: ['http://google.com']
item: http://bcb.gov.br/
resultado: []
item: Pesquisar no Bing
resultado: []
```



PYTHON

Módulo 33

INTEGRAÇÃO PYTHON COM APIS E JSON



Módulo 33 – Integração Python com API e JSON – Python e Integrações de API (1/4)

Antes de qualquer coisa... O que é uma API??

API é um conjunto de códigos que uma plataforma, um site, etc fornece para que outros códigos possam usá-los e garantir esse relacionamento de forma segura e precisa.

Nem todas plataformas, oferecem serviço de API, mas existem muitas disponíveis para uso.

Veja o exemplo ao lado. Pegamos aqui a API do Youtube ([link](#)). Aqui, temos códigos já disponibilizados pelo próprio youtube que nos auxiliam a executar esses comandos de forma segura.

Ao invés de desenvolvermos um código novo, simplesmente utilizamos a API.

 YouTube > Data API

Index
Apps Script
Go
Java
JavaScript
.NET
PHP
Python
Python on App Engine
Ruby

Recuperar meus envios (`playlistItems.list`)
Criar uma playlist (`playlists.insert`)
Pesquisar por palavra-chave (`search.list`)
Adicionar inscrição de canal (`subscriptions.insert`)
Envios a retomar (`videos.insert`)

Python

Postar um boletim de canal (`activities.insert`)
Adicionar um vídeo em destaque (`channels.update`)
Recuperar meus envios (`playlistItems.list`)
Criar uma playlist (`playlists.insert`)
Pesquisar por palavra-chave (`search.list`)
Pesquisar por tópico (`search.list`)
Adicionar inscrição de canal (`subscriptions.insert`)
Enviar uma imagem em miniatura do vídeo (`thumbnails.set`)
Enviar um vídeo (`videos.insert`)
Avaliar um vídeo (gostei) (`videos.rate`)
Atualizar um vídeo (`videos.update`)

Módulo 33 – Integração Python com API e JSON – Python e Integrações de API (2/4)

Beleza, entendemos o que é uma API, vamos usar uma na prática.

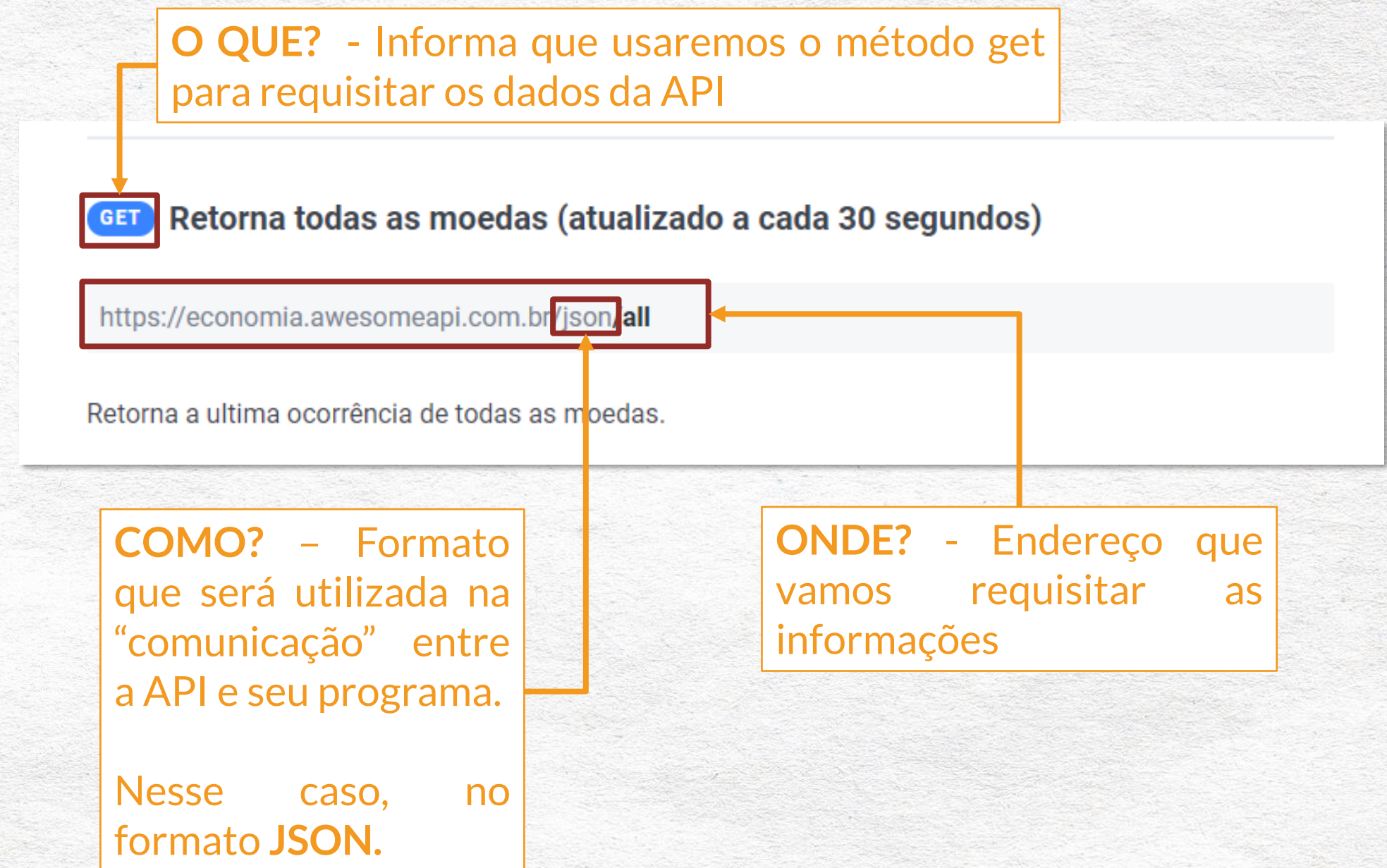
Para esse exemplo, vamos usar uma API gratuita que nos fornecerá a cotação de moedas (dólar, euro, etc).

<https://docs.awesomeapi.com.br/api-de-moedas>

Nesse caso, não temos a necessidade de um login na API, então, podemos ir direto para a documentação da API e entender quais são as etapas a serem seguidas para conseguirmos a informação das cotações.

Conforme apresentado ao lado esses dados serão requisitados através do método GET e serão buscados através do link indicado.

Essa comunicação pode ser feita em diversas “línguas” uma das mais comuns e quando estamos trabalhando com Python é a **JSON**.



Módulo 33 – Integração Python com API e JSON – Python e Integrações de API (3/4)

Vamos para o nosso código! Como já é rotina, vamos começar importando as bibliotecas serão necessárias.

Quando estamos falando de APIs e Formato JSON, vamos utilizar 2 bibliotecas novas:

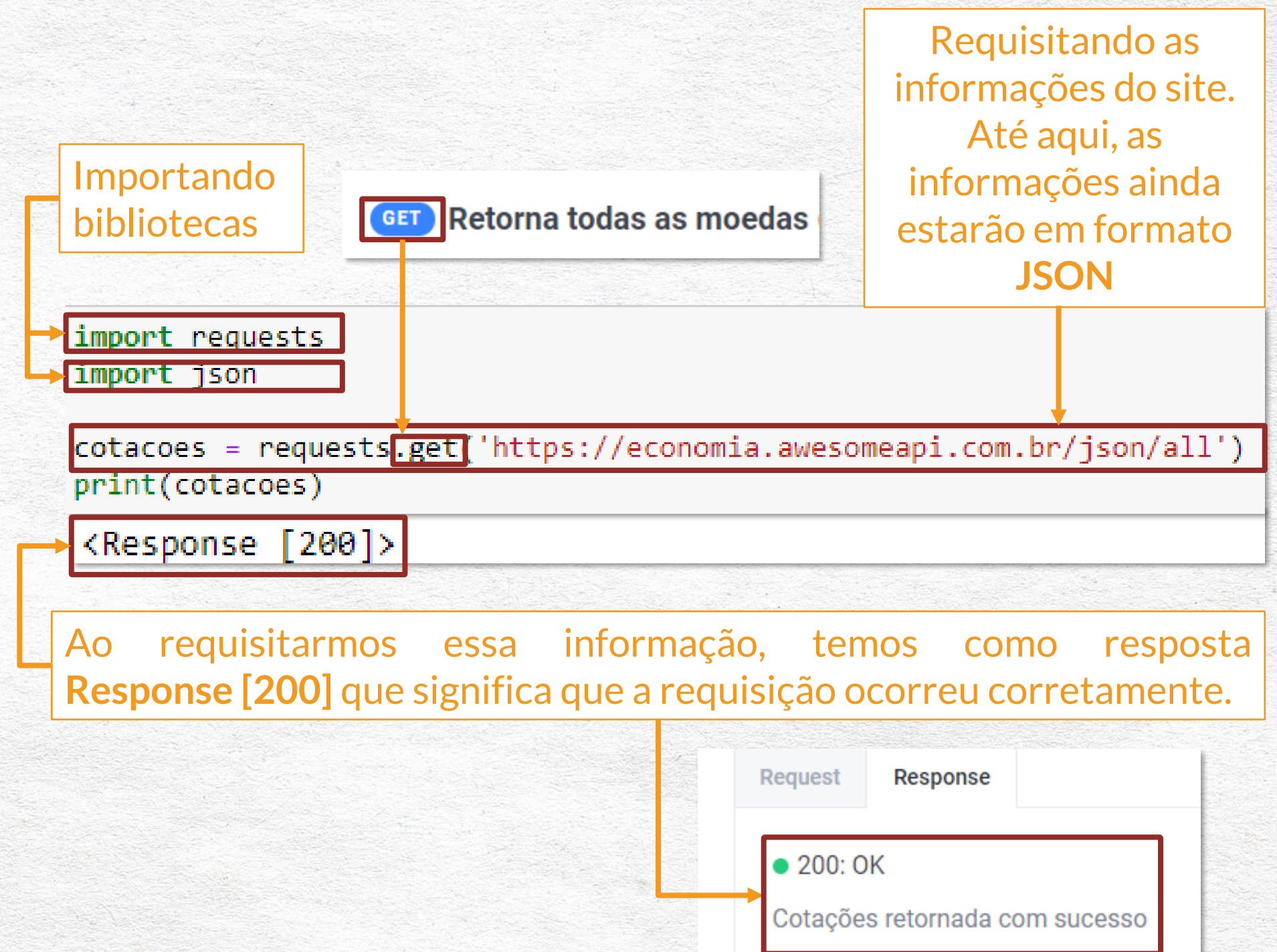
`requests(doc)` `json(doc)`

Primeiro vamos entender a requests. Conforme vimos na documentação da API usaremos o **GET** para requisitar os dados.

No entanto, quando printamos o resultado armazenado na variável `cotacoes` temos:

`<Response [200]>`

Sempre que requisitamos dados temos uma resposta. Essa resposta possui um código 200 indica que a requisição foi bem sucedida.



Sabemos que a requisição está correta, agora precisamos acessar essa informação.

Para isso usaremos o **JSON**. Utilizando essa biblioteca poderemos acessar os dados e convertê-los em um dicionário Python.

Usando a biblioteca **json** conseguimos transformar as informações em um dicionário PYTHON

```
import requests
import json

cotacoes = requests.get('https://economia.awesomeapi.com.br/json/all')
cotacoes_dic = cotacoes.json()
print(cotacoes_dic)
```

```
{'USD': {'code': 'USD', 'codein': 'BRL', 'name': 'Dólar Comercial', 'high': '5.4585', 'low': '5.3662', 'varBid': '-0.0245', 'pctChange': '-0.45', 'bid': '5.4331', 'ask': '5.4343', 'timestamp': '1605540818', 'create_date': '2020-11-16 12:33:39'}, 'USDT': {'code': 'USD', 'codein': 'BRLT', 'name': 'Dólar Turismo', 'high': '5.61', 'low': '5.5', 'varBid': '-0.035', 'pctChange': '-0.62', 'bid': '5.41', 'ask': '5.74', 'timestamp': '1605540240', 'create_date': '2020-11-16 12:24:43'}, 'CAD': {'code': 'CAD', 'codein': 'BRL', 'name': 'Dólar Canadense', 'high': '4.1729', 'low': '4.1014', 'varBid': '0.0008', 'pctChange': '0.02', 'bid': '4.1538', 'ask': '4.1557', 'timestamp': '1605540813', 'create_date': '2020-11-16 12:33:36'}, 'EUR': {'code': 'EUR', 'codein': 'BRL', 'name': 'Euro', 'high': '6.4775', 'low': '6.3551', 'varBid': '-0.0233', 'pctChange': '-0.36', 'bid': '6.4307', 'ask': '6.4342', 'timestamp': '1605540814', 'create_date': '2020-11-16 12:33:37'}, 'GBP': {'code': 'GBP', 'codein': 'BRL', 'name': 'Libra Esterlina', 'high': '7.2275', 'low': '7.0782', 'varBid': '-0.0289', 'pctChange': '-0.4', 'bid': '7.1637', 'ask': '7.169', 'timestamp': '1605540813', 'create_date': '2020-11-16 12:33:36'}, 'ARS': {'code': 'ARS', 'codein': 'BRL', 'name': 'Peso Argentino', 'high': '0.0684', 'low': '0.0673', 'varBid': '-0.0004', 'pctChange': '-0.66', 'bid': '0.0679', 'ask': '0.068', 'timestamp': '1605540818', 'create_date': '2020-11-16 12:33:39'}, 'BTC': {'code': 'BTC', 'codein': 'BRL', 'name': 'Bitcoin', 'high': '89500', 'low': '87251', 'varBid': '2128.6', 'pctChange': '2.45', 'bid': '89210', 'ask': '89406', 'timestamp': '1605540818', 'create_date': '2020-11-16 12:33:39'}}
```

Módulo 33 – Integração Python com API e JSON – API de Cotação de Moeda (1/2)

Como vimos no exemplo anterior, o resultado da nossa requisição é um dicionário.

Se olharmos esse dicionário com um pouco mais detalhe, vamos perceber que cada moeda é uma **chave deste dicionário**.

Se nosso objetivo é conseguir a cotação do dólar (USD), podemos acessar nosso dicionário.

Como não conhecemos essa API, antes de irmos para o código, vamos dar uma olhada em sua documentação para entendermos as nomenclaturas utilizadas pela API.



ATENÇÃO!

Nessa API específica, os valores estarão sempre sendo atualizados, portanto, o valor indicado aqui provavelmente será diferente do valor conseguido por você 😊

Códigos das moedas

- USD (Dólar Americano)
- CAD (Dólar Canadense)
- AUD (Dólar Australiano)
- EUR (Euro)
- GBP (Libra Esterlina)
- ARS (Peso Argentino)

Legendas

key	Label
bid	Compra
ask	Venda
varBid	Variação
pctChange	Porcentagem de Variação
high	Máximo
low	Mínimo

Usando a documentação da API podemos saber o **nome da moeda** e qual a **chave que devemos utilizar para acessar a cotação**

```
print('Dólar: {}'.format(cotacoes_dic['USD']['bid']))
print('Euro: {}'.format(cotacoes_dic['EUR']['bid']))
print('Bitcoin: {}'.format(cotacoes_dic['BTC']['bid']))
```

```
Dólar: 5.4331
Euro: 6.4307
Bitcoin: 89210
```

Aqui temos um dicionário dentro de um dicionário. Para o primeiro, identificamos a **MOEDA** ('BTC') depois, identificamos a **INFORMAÇÃO** daquela moeda que queremos ('bid')



Módulo 33 – Integração Python com API e JSON – API de Cotação de Moeda (2/2)

Vamos dizer que temos interesse em pegar não só a cotação de hoje, mas dos últimos 30 dias?

Como fazer? O primeiro passo, é ler a documentação da API e tentar entender se essa API nos oferece essa opção.

Nesse caso, temos essa opção, o que facilita nosso trabalho. Caso não houvesse, teríamos que programar com os dados disponíveis.

MAS como temos, vamos entender como acessar esses dados no print ao lado.

GET Retorna o fechamento dos últimos dias

`https://economia.awesomeapi.com.br/json/daily/moeda/numero_dias`

`https://economia.awesomeapi.com.br/json/daily/USD-BRL/15`

Request	Response
Path Parameters	
moeda REQUIRED	string Código da moeda Ex: USD-BRL
numero_dias OPTIONAL	integer Números de dias 30/60/90/.../360 dias

Indica que **moeda** deverá ser substituído pelo código da moeda; Indica que **número de dias** será substituído por um número inteiro (integer)

Aqui estamos pegando os dados de conversão ('USD-BRL') dos últimos ('30') dias

For percorre todo o dicionário disponibilizado pela API

```

cotacoes_dolar30d = requests.get('https://economia.awesomeapi.com.br/json/daily/USD-BRL/30')
cotacoes_dolar_dic = cotacoes_dolar30d.json()
lista_cotacoes_dolar = [float(item['bid']) for item in cotacoes_dolar_dic]
print(lista_cotacoes_dolar)

```

List comprehension, inserindo o valor da cotação na lista_cotações_dólar como float, por se tratarem de números

[5.7125, 5.7729, 5.6211, 5.5214, 5.5067, 5.49, 5.5602, 5.5832, 5.6244, 5.6156, 5.5518, 5.5357, 5.672, 5.8025, 5.8755, 5.6908, 5.6693, 5.6193, 5.6761, 5.6418, 5.5976, 5.5308, 5.4409, 5.4659, 5.3823, 5.427, 5.4105, 5.3695, 5.3708, 5.3665]

Módulo 33 – Integração Python com API e JSON – Mais exercícios de API com Cotação de Moeda (1/4)

Nosso objetivo agora será pegar as informações do BitCoin de Jan/20 a Out/20 e plotarmos um gráfico com essas informações.

Como fizemos anteriormente, vamos verificar na documentação da nossa API se é possível requisitar essas informações diretamente.

Como podemos ver ao lado é possível requisitar e é isso que iremos fazer.

Perceba que o link de requisição possui um número maior de parâmetros, mas todos eles estão listados abaixo na própria API

Basta **substituir no link as informações conforme as orientações da API**. E é isso que iremos fazer no próximo slide.

GET Retorna cotações sequenciais de um período específico

`https://economia.awesomeapi.com.br/:moeda:quantidade?start_date=20200301&end_date=20200330`

`https://economia.awesomeapi.com.br/USD-BRL/10?start_date=20200201&end_date=20200229`

Request Response

Path Parameters

:moeda	string	Código da moeda ex.: USD-BRL
REQUIRED		
:quantidade	string	Numero de dados para retornar
OPTIONAL		

Query Parameters

start_date	string	Data de inicio dos resultados no formato YYYYMMDD ex.: 20200201
OPTIONAL		
end_date	string	Data limite dos resultados no formato YYYYMMDD ex.: 20200229
OPTIONAL		

Módulo 33 – Integração Python com API e JSON – Mais exercícios de API com Cotação de Moeda (2/4)

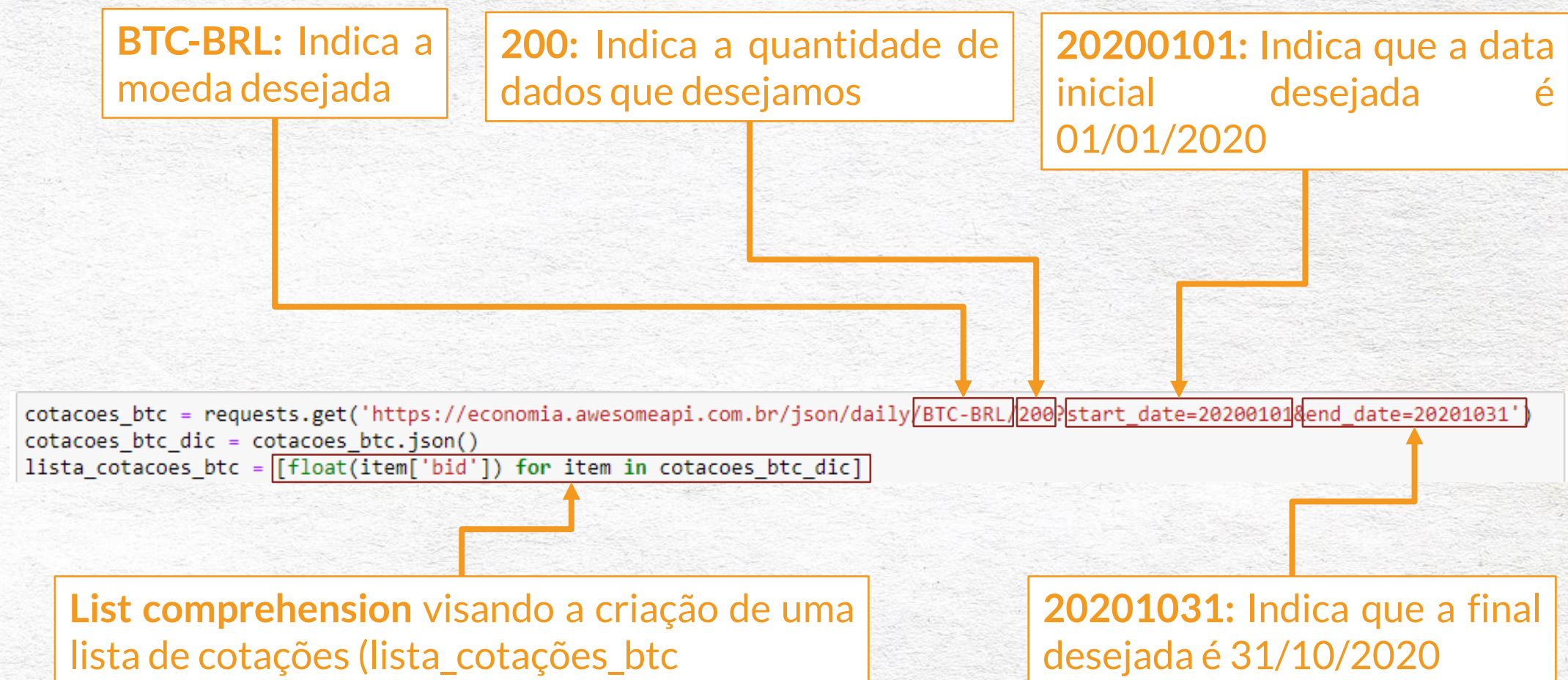
Nosso objetivo agora será pegar as informações do BitCoin de Jan/20 a Out/20 e plotarmos um gráfico com essas informações.

Como fizemos anteriormente, vamos verificar na documentação da nossa API se é possível requisitar essas informações diretamente.

Como podemos ver ao lado é possível requisitar e é isso que iremos fazer.

Perceba que o link de requisição possui um número maior de parâmetros, mas todos eles estão listados abaixo na própria API

Basta substituir no link as informações conforme as orientações da API. E é isso que iremos fazer no próximo slide.



Módulo 33 – Integração Python com API e JSON – Mais exercícios de API com Cotação de Moeda (3/4)

As cotações são extraídas da API da mais atual para a mais antiga. Podemos “corrigir” essa informação usando o método `.reverse()` e ter a informação da mais antiga para a mais nova. Como apresentado abaixo:

```
cotacoes_btc = requests.get('https://economia.awesomeapi.com.br/json/daily/BTC-BRL/200?start_date=20200101&end_date=20201031')
cotacoes_btc_dic = cotacoes_btc.json()
lista_cotacoes_btc = [float(item['bid']) for item in cotacoes_btc_dic]
lista_cotacoes_btc.reverse()
print(lista_cotacoes_btc)
print(len(lista_cotacoes_btc))
```

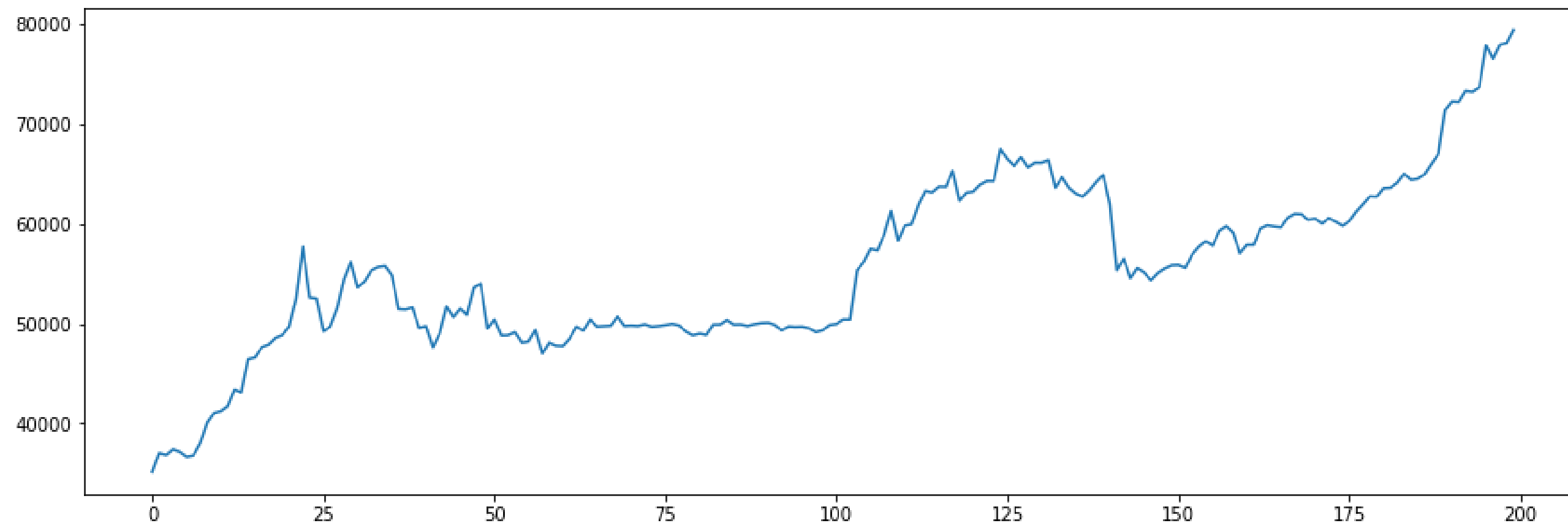
```
[35150.0, 37025.0, 36836.7, 37415.0, 37158.0, 36652.1, 36800.0, 38000.1, 40050.0, 41050.0, 41212.0, 41715.0, 43392.0, 43070.0,
46424.7, 46610.0, 47626.0, 47900.0, 48553.0, 48852.0, 49700.0, 52500.0, 57700.0, 52609.1, 52499.0, 49225.0, 49700.0, 51488.8, 5
4450.0, 56208.0, 53609.0, 54207.4, 55300.0, 55651.1, 55760.8, 54831.8, 51440.0, 51400.0, 51600.0, 49560.0, 49735.0, 47625.1, 49
000.2, 51700.3, 50650.0, 51500.0, 50865.1, 53600.0, 54001.0, 49500.4, 50400.6, 48800.4, 48820.6, 49133.5, 48096.8, 48201.0, 493
50.0, 47000.0, 48100.1, 47788.9, 47750.0, 48501.4, 49679.0, 49290.6, 50405.0, 49700.0, 49750.0, 49782.0, 50710.0, 49759.1, 4980
0.0, 49755.0, 49915.1, 49678.2, 49746.8, 49840.0, 49960.0, 49800.0, 49208.8, 48815.5, 48993.1, 48850.0, 49890.0, 49900.1, 5036
0.1, 49890.0, 49905.9, 49750.0, 49920.0, 50041.3, 50073.4, 49868.0, 49320.0, 49702.1, 49651.8, 49680.0, 49516.5, 49143.0, 4932
1.0, 49850.0, 49951.1, 50400.2, 50402.0, 55300.0, 56161.0, 57490.0, 57330.0, 58950.0, 61298.1, 58296.3, 59800.0, 59951.0, 6190
2.2, 63302.0, 63140.0, 63715.0, 63700.0, 65303.8, 62300.0, 63120.0, 63210.3, 63905.0, 64280.0, 64270.3, 67501.0, 66453.3, 6580
0.0, 66655.0, 65657.4, 66112.0, 66100.0, 66360.9, 63606.0, 64655.2, 63599.9, 63016.2, 62700.0, 63350.9, 64200.0, 64900.0, 6193
9.0, 55336.6, 56500.0, 54555.7, 55550.3, 55102.2, 54348.3, 55059.1, 55511.0, 55821.2, 55856.7, 55566.0, 56908.9, 57700.0, 5825
0.0, 57807.2, 59285.0, 59750.0, 59100.0, 57050.6, 57880.5, 57884.0, 59501.1, 59845.0, 59720.5, 59638.4, 60600.1, 60995.0, 6095
0.1, 60390.7, 60512.5, 60018.1, 60569.2, 60210.3, 59801.0, 60265.0, 61211.5, 61960.0, 62747.2, 62701.0, 63558.3, 63601.7, 6410
0.6, 65000.0, 64400.0, 64500.0, 64950.1, 66000.3, 66952.4, 71401.0, 72251.2, 72202.4, 73300.0, 73200.0, 73685.5, 77888.1, 7655
0.1, 77950.2, 78101.0, 79401.1]
200
```

Módulo 33 – Integração Python com API e JSON – Mais exercícios de API com Cotação de Moeda (4/4)

Em posse dessa lista, podemos utilizá-la na elaboração de um gráfico mostrando a evolução da cotação do bitcoin para o período solicitado.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(15, 5))
plt.plot(lista_cotacoes_btc)
plt.show()
```



Módulo 33 – Integração Python com API e JSON – API com Login – Envio de SMS com Python e Twilio (1/4)

Vamos usar agora uma API que necessita de um cadastro anterior.

O cadastro é bem direto e pode ser feito diretamente pelo [site do Twilio](#)

Feito o cadastro, vamos dar uma olhada na [documentação da API](#) para entendermos como vamos usá-la para enviar SMS via Python

Lendo a API, vemos que 2 informações são necessárias e ainda não as possuímos.

Vamos então descobrir onde identifica-las no próximo slide.

Python Helper Library SMS Test

PYTHON

```
1 from twilio.rest import Client
2
3 # Your Account SID from twilio.com/console
4 account_sid = "AC81ab470d064db528e5dc32901289a685"
5 # Your Auth Token from twilio.com/console
6 auth_token = "your_auth_token"
7
8 client = Client(account_sid, auth_token)
9
10 message = client.messages.create(
11     to="+15558675309",
12     from_="+15017250604",
13     body="Hello from Python!")
14
15 print(message.sid)
```

Importando a biblioteca

Precisamos encontrar o nosso account_sid

Precisamos encontrar o nosso token de autenticação

Módulo 33 – Integração Python com API e JSON – API com Login – Envio de SMS com Python e Twilio (2/4)

Na aba Dashboard do Twilio, podemos encontrar o **account SID** e o **Auth Token**.

Por motivos de segurança censuramos parte do auth number do pobre coitado que está fazendo essa apostila. Sem a censura, já sabemos que muitos SMS serão magicamente enviados 😊.

Em posse dessas informações, vamos para o nosso código desenvolver o código conforme indicado na documentação da API do Twilio.

The screenshot shows the Twilio dashboard titled "My first Twilio account Dashboard". It has two main sections: "ACCOUNT SID" and "AUTH TOKEN".

- ACCOUNT SID:** Displays "AC81ab470d" followed by a redacted area labeled "CENSURADO" and then "89a685". An orange arrow points from a text box to this section.
- AUTH TOKEN:** Displays a "Hide" link, followed by "08065522ec92", a redacted area labeled "CENSURADO", and then "3e". An orange arrow points from a text box to this section.

Annotations on the right side:

- For ACCOUNT SID: "Precisamos encontrar o nosso account_sid"
- For AUTH TOKEN: "Precisamos encontrar o nosso token de autenticação"

```
1 from twilio.rest import Client
2
3 # Your Account SID from twilio.com/console
4 account_sid = "AC81ab470d[CENSURADO]85"
5 # Your Auth Token from twilio.com/console
6 auth_token = "your_auth_token"
7
8 client = Client(account_sid, auth_token)
```

```
from twilio.rest import Client

account_sid = 'AC81ab470d[CENSURADO]89a685'
token = '08065522ec92[CENSURADO]3e'

client = Client(account_sid, token)
```

Módulo 33 – Integração Python com API e JSON – API com Login – Envio de SMS com Python e Twilio (3/4)

Outra informação que precisamos coletar no site do Twilio é o número de telefone que será utilizado para envio dos SMS.

Para isso, basta clicar na opção **Trial number** e **confirmar**. O Twilio lhe dará um número como apresentado ao lado.

```
from twilio.rest import Client

# Your Account SID from twilio.com/console
account_sid = "AC81ab470d064db528e5dc32901289a685"
# Your Auth Token from twilio.com/console
auth_token = "your_auth_token"

client = Client(account_sid, auth_token)

message = client.messages.create(
    to="+15558675309",
    from_="+15017250604",
```

Project Info

TRIAL BALANCE

\$14.50

TRIAL NUMBER

+186452 CENSURADO

Número criado pelo Twilio para envio dos SMSs

```
from twilio.rest import Client

account_sid = 'AC81ab470d CENSURADO a685'
token = '08065522ec92 CENSURADO 3e'

client = Client(account_sid, token)

remetente = '+186452 CENSURADO'
destino = '+552199 CENSURADO'
```



Por fim é só escrevermos nossa mensagem e enviar.

Antes de rodar o código, verifique se a biblioteca do twilio está instalada. Caso não esteja, assim como já fizemos outras vezes, basta usar o :

pip install twilio

```
from twilio.rest import Client

account_sid = 'AC81a[REDACTED]85'
token = '08065[REDACTED]3e'

client = Client(account_sid, token)

remetente = '+186[REDACTED]'
destino = '+552[REDACTED]'

message = client.messages.create(
    to=destino,
    from_=remetente,
    body="Coe, é o Lira aqui!")

print(message.sid)
```

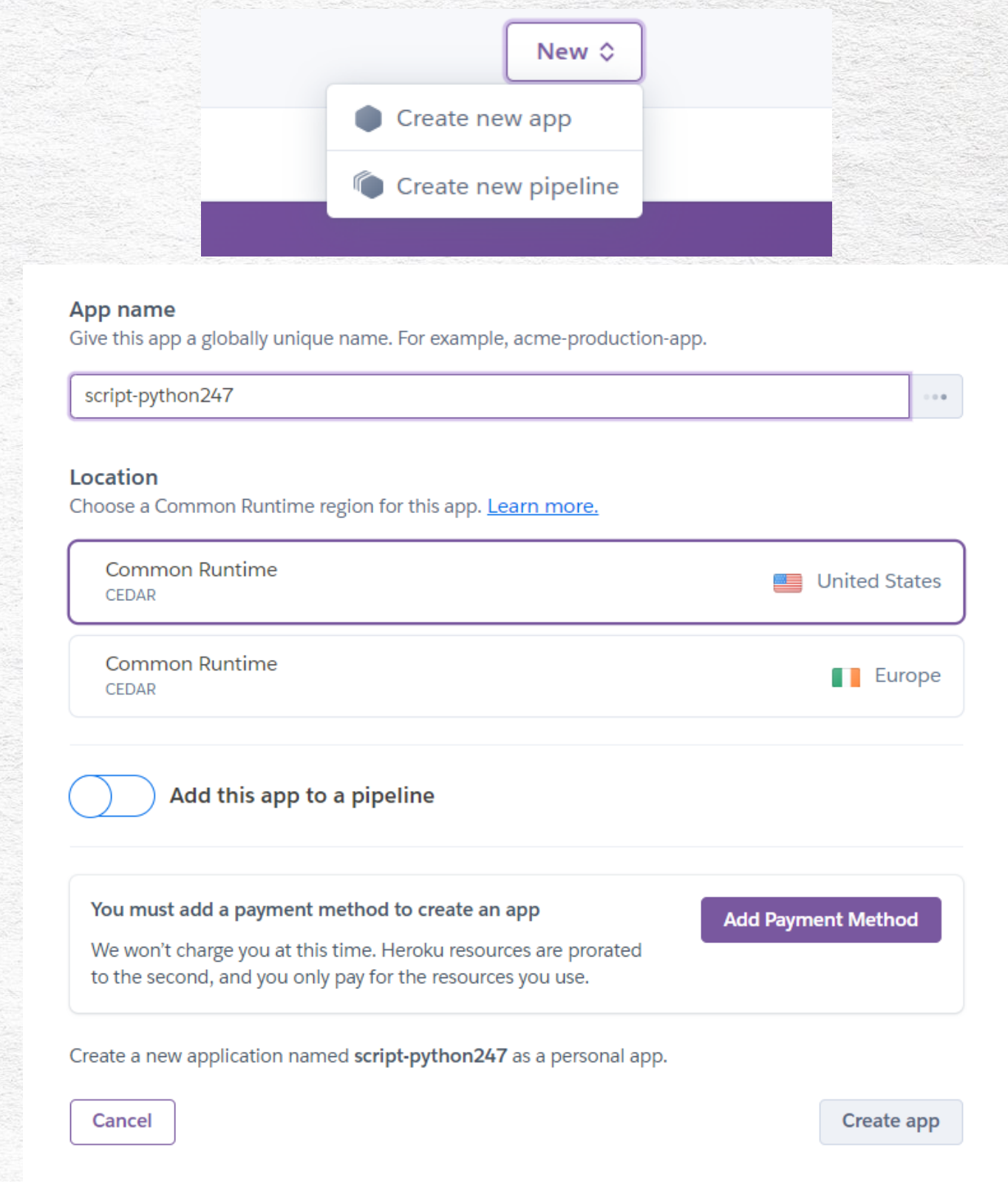
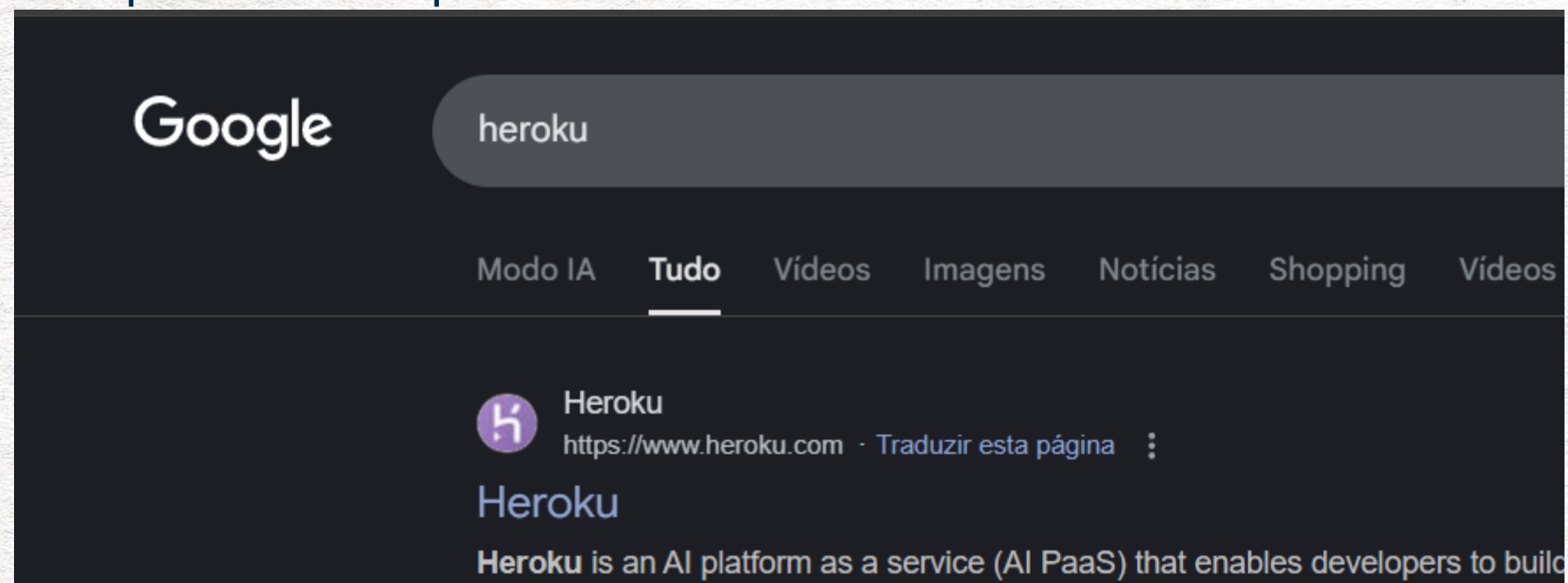


Módulo 33 – Integração Python com APIs e JSON – Rodar códigos com o computador desligado, agendar diariamente, semanalmente, mensalmente (1/6)

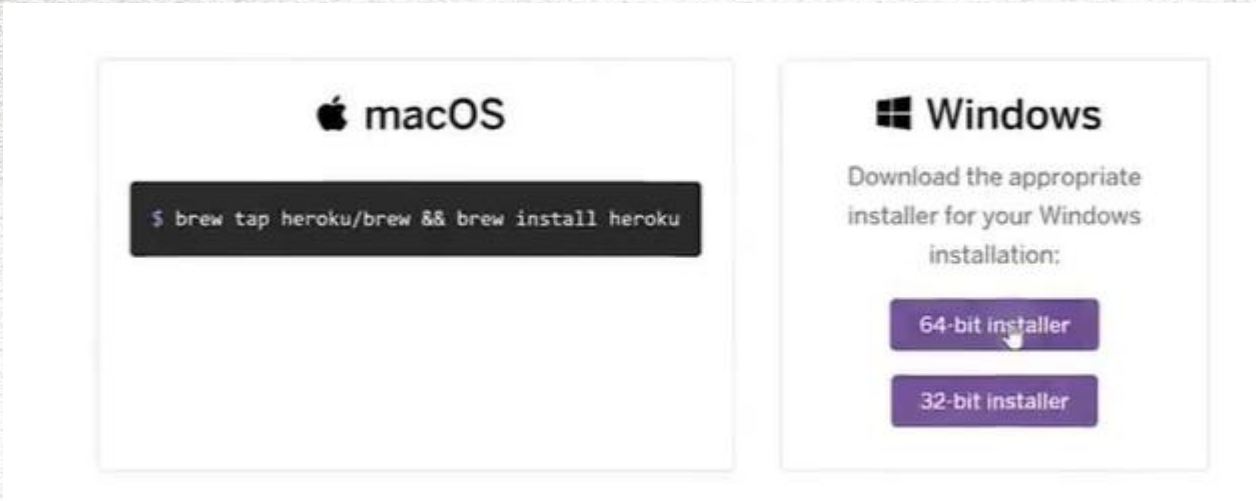
92

Nesta aula, você vai aprender como deixar seus códigos em Python rodando **24 horas por dia**, sete dias por semana, mesmo que o seu computador esteja desligado. Para atingir esse objetivo, você precisará realizar o **deploy** do seu código, ou seja, disponibilizá-lo em um servidor na internet que nunca para de funcionar. Entre as diversas opções de servidores, utilizaremos o **Heroku**, que se destaca por possuir um plano gratuito e ser extremamente simples de configurar.

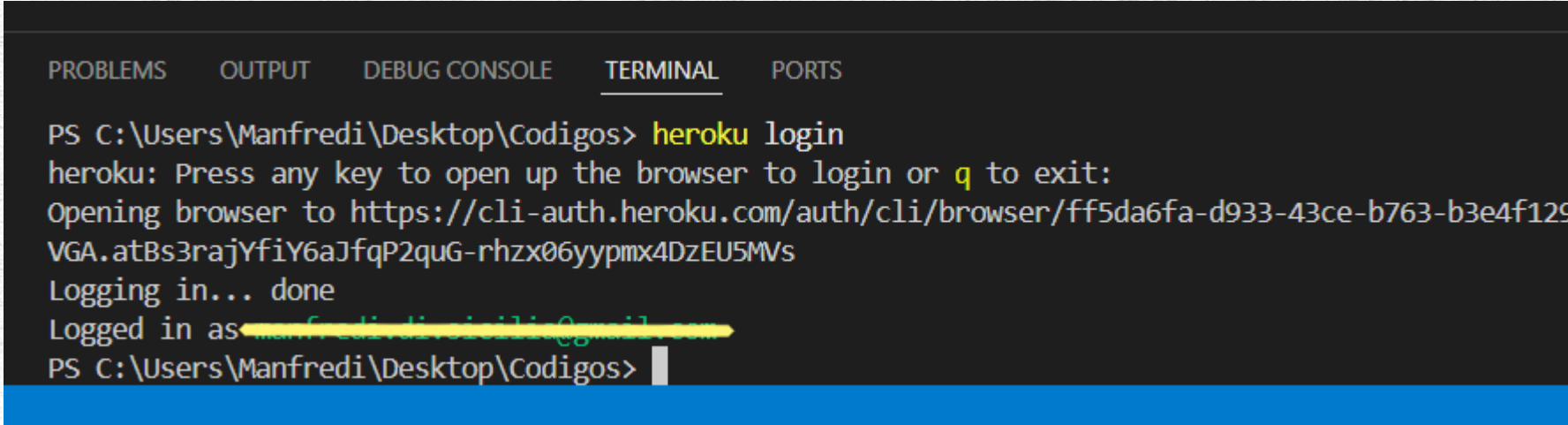
O primeiro passo consiste em criar uma conta gratuita no site do **Heroku** e configurar um novo aplicativo dentro da plataforma. Após essa criação, você deve baixar e instalar o **Heroku CLI**, que é a interface de linha de comando necessária para interagir com o servidor. Além disso, é essencial ter o **Git** instalado em sua máquina, pois ele gerencia o processo de envio das versões do seu código do computador local para o servidor.

A screenshot of the Heroku app creation form. At the top, there's a 'New' button with a dropdown menu showing 'Create new app' and 'Create new pipeline'. The form fields include: 'App name' with the value 'script-python247'; 'Location' with two options: 'Common Runtime CEDAR United States' (selected) and 'Common Runtime CEDAR Europe'; a toggle switch for 'Add this app to a pipeline'; a message 'You must add a payment method to create an app' with an 'Add Payment Method' button; and a final confirmation message 'Create a new application named script-python247 as a personal app.' with 'Cancel' and 'Create app' buttons.

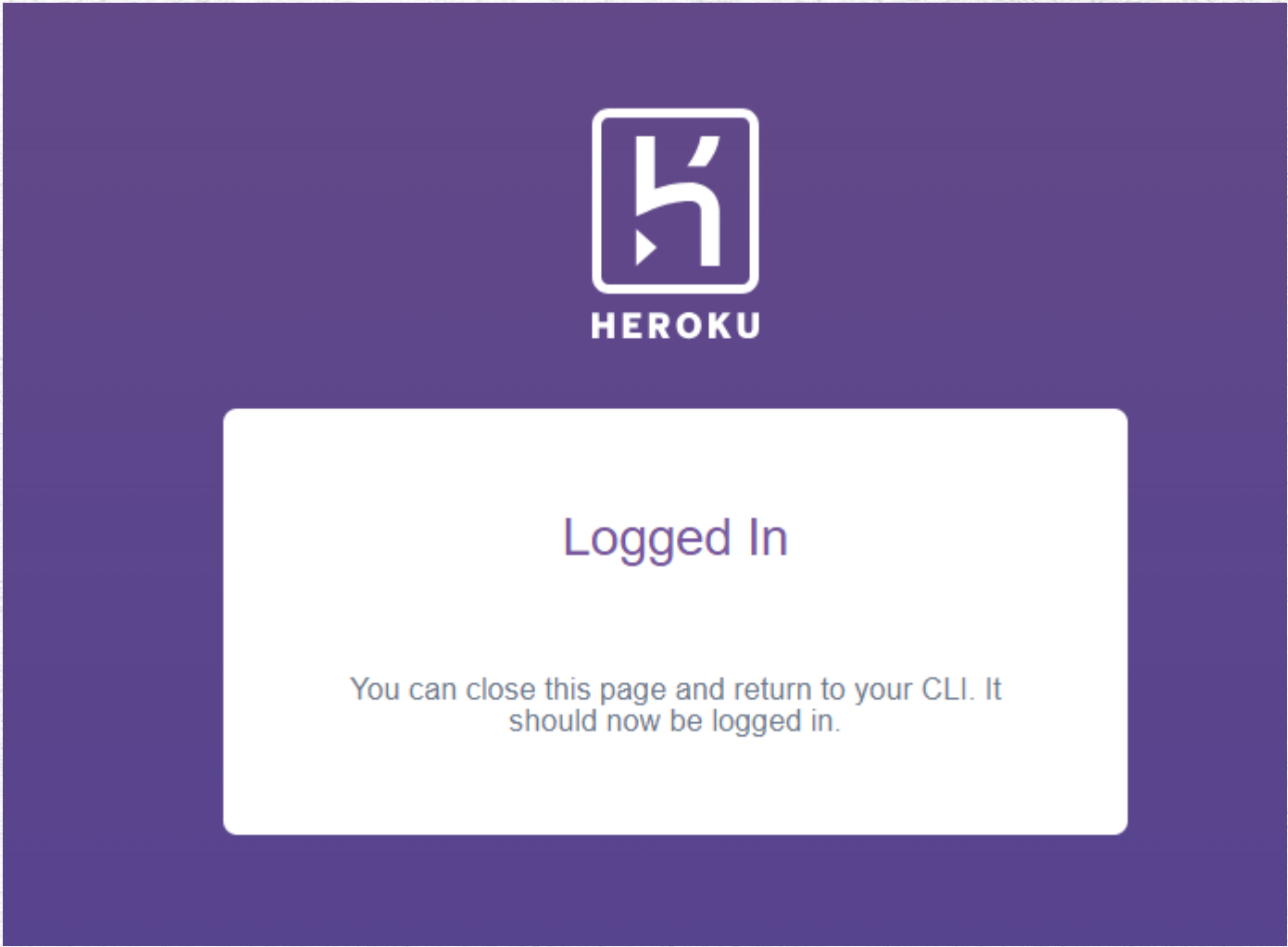
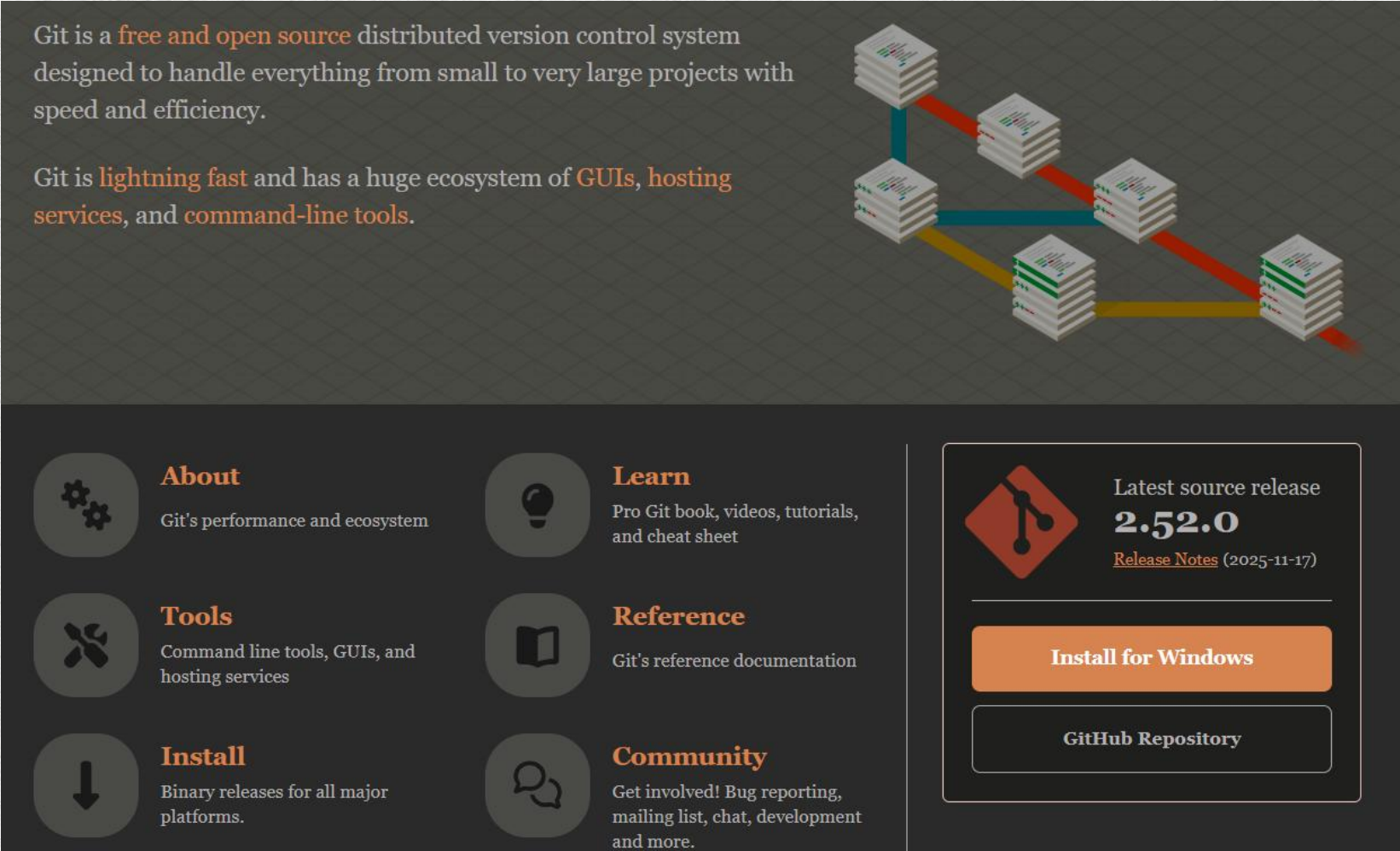
Heroku CLI



Após as intalações volte ao terminal e faça login no heroku.



Git



Após as instalações, você deve preparar o seu projeto criando dois arquivos fundamentais na mesma pasta do seu código Python. O primeiro é o **profile**, um arquivo sem extensão onde você define que o servidor deve executar o comando **worker: python main.py**. O segundo é o **requirements.txt**, que lista todas as bibliotecas necessárias para o seu programa funcionar. Você pode gerar esse arquivo automaticamente no terminal utilizando o comando **pip freeze > requirements.txt**.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Manfredi\Desktop\Codigos>pip freeze > requirements.txt

C:\Users\Manfredi\Desktop\Codigos>
```

Desktop > Codigos				
Sort View ...				
Name	Date modified	Type	Size	
main.py	2026-01-20 17:40	Python Source File	1 KB	
Procfile	2026-01-20 17:40	File	1 KB	
requirements.txt	2026-01-21 19:08	Text Document	2 KB	

Com tudo preparado, você iniciará o processo de envio através dos comandos **git init** e **heroku git:remote -a nome-do-seu-app**. Para finalizar o deploy, você adicionará os arquivos com **git add .**, salvará as alterações com **git commit -am "mensagem"** e enviará tudo com o comando **git push heroku master**. Uma dica importante é sempre trabalhar dentro de um **ambiente virtual** para garantir que apenas as bibliotecas essenciais do projeto sejam enviadas, tornando o processo mais leve e organizado.

```
C:\Users\Manfredi\Desktop\Codigos>git init
Initialized empty Git repository in C:/Users/Manfredi/Desktop/Codigos/.git/

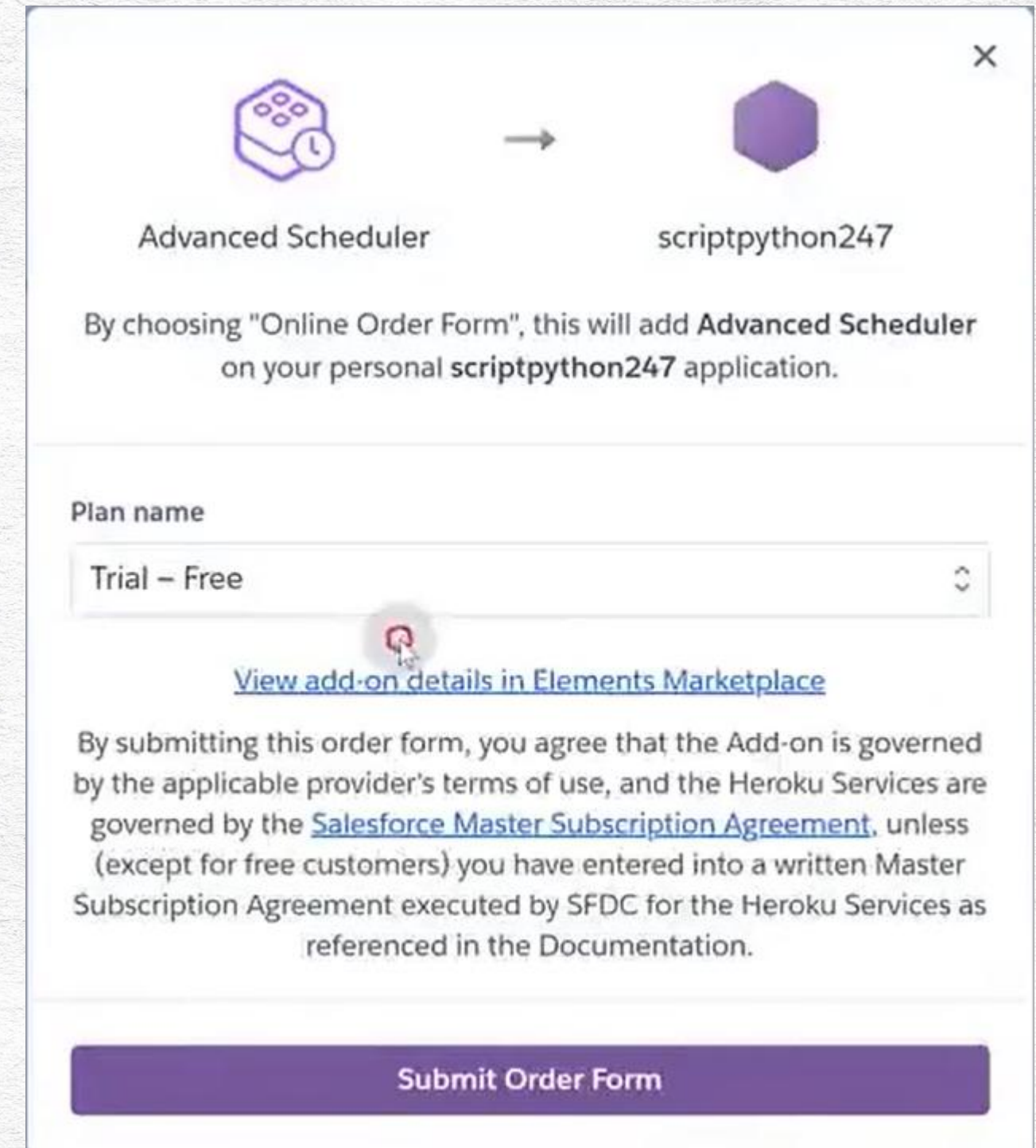
C:\Users\Manfredi\Desktop\Codigos>
```

```
PS G:\Meu Drive\Lira\PC Lira 2021\Projetos PyCharm\ScriptHerokuTeste> heroku git:remote -a scriptpython247
set git remote heroku to https://git.heroku.com/scriptpython247.git
PS G:\Meu Drive\Lira\PC Lira 2021\Projetos PyCharm\ScriptHerokuTeste>
```

```
set git remote heroku to https://git.heroku.com/scriptpython247.git
PS G:\Meu Drive\Lira\PC Lira 2021\Projetos PyCharm\ScriptHerokuTeste> git add .
PS G:\Meu Drive\Lira\PC Lira 2021\Projetos PyCharm\ScriptHerokuTeste> git commit -am "deploy inicial"
[master a9a0a44] deploy inicial
1 file changed, 6 insertions(+)
create mode 100644 .idea/vcs.xml
```

```
create mode 100644 .idea/vcs.xml
PS G:\Meu Drive\Lira\PC Lira 2021\Projetos PyCharm\ScriptHerokuTeste> git push heroku master
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 8 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (16/16), 2.11 KiB | 23.00 KiB/s, done.
Total 16 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Compressing source files... done.
remote: Building source:
```

Após realizar o deploy do seu código no **Heroku**, você precisa configurar como ele será executado de forma automática. Para isso, utilizaremos um adicional gratuito chamado **Advanced Scheduler**, que funciona como um agendador de tarefas dentro do servidor. Ao acessar esse recurso, você criará um **trigger**, definindo o nome da automação e o comando de execução, que no nosso caso é **python main.py**.



The screenshot shows the Heroku interface for adding the 'Advanced Scheduler' add-on to the application 'scriptpython247'. At the top, there's a visual representation of the add-on being added, with an arrow pointing from the 'Advanced Scheduler' icon to the application name. Below this, a message states: 'By choosing "Online Order Form", this will add Advanced Scheduler on your personal scriptpython247 application.' A section for 'Plan name' shows a dropdown menu with 'Trial - Free' selected. A link 'View add-on details in Elements Marketplace' is provided. A paragraph of legal text follows: 'By submitting this order form, you agree that the Add-on is governed by the applicable provider's terms of use, and the Heroku Services are governed by the Salesforce Master Subscription Agreement, unless (except for free customers) you have entered into a written Master Subscription Agreement executed by SFDC for the Heroku Services as referenced in the Documentation.' At the bottom, there is a large purple button labeled 'Submit Order Form'.

Name
Pick a descriptive name for the task.

Automacao Colacoes

Command
Enter the command used to execute the task.

\$ python main.py

ⓘ Do not put sensitive data directly in the command. Use config vars instead.

Timezone
Select the timezone.

America/Sao Paulo

Dentro das configurações do agendador, você deve selecionar o fuso horário correto, como **América/São Paulo**, para garantir que as execuções ocorram no momento desejado. O sistema permite escolher se a tarefa será executada uma única vez ou de forma recorrente, possibilitando intervalos de minutos, horas, dias ou semanas. Você pode, por exemplo, configurar para que o código rode a cada minuto para testar se a automação está funcionando corretamente no servidor.

Select the type of trigger you need.

☒ Recurring

☐ One-off

Schedule
Choose the method to define at what interval the task should be executed.

☒ Schedule helper

☐ Cron Expression

Select a unit of time.

Minute

Select the required interval.

Every 1 minutes

ⓘ Use cron expressions to schedule your tasks with even more flexibility.

Para verificar se o seu script está sendo executado com sucesso, o Heroku disponibiliza a ferramenta **View Logs**, onde você consegue visualizar o histórico de processamento e os resultados de comandos como o **print**. Caso você precise atualizar o seu programa futuramente, basta realizar as alterações no código local e repetir o processo de envio com o **git push heroku master**. Dessa forma, sua automação continuará operando de forma independente, sem depender da energia ou da conexão do seu próprio computador.

Deploy your changes

Make some changes to the code you just cloned and deploy them to Heroku using Git.

```
$ git add .  
$ git commit -am "make it better"  
$ git push heroku master
```

Application Logs

```
2022-06-08T21:14:20.601875+00:00 heroku[worker.1]: State changed from crashed to starting  
2022-06-08T21:14:21.897416+00:00 heroku[worker.1]: Starting process with command `python main.py`  
2022-06-08T21:14:22.562634+00:00 heroku[worker.1]: State changed from starting to up  
2022-06-08T21:14:23.287747+00:00 app[api]: Scaled to worker@0:Free by user pythonimpressionador@gmail.com  
2022-06-08T21:14:23.410060+00:00 heroku[worker.1]: State changed from up to down  
2022-06-08T21:14:23.722564+00:00 app[worker.1]: Cotação Atualizada. 2022-06-08 21:14:23.722481  
2022-06-08T21:14:23.722578+00:00 app[worker.1]: Dólar: R$4.895  
2022-06-08T21:14:23.722579+00:00 app[worker.1]: Euro: R$5.2496  
2022-06-08T21:14:23.722585+00:00 app[worker.1]: BTC: R$148.751  
2022-06-08T21:14:23.894348+00:00 heroku[worker.1]: Process exited with status 0  
2022-06-08T21:19:03.034921+00:00 heroku[advanced-scheduler.8061]: Starting process with command `python main.py`  
2022-06-08T21:19:03.704444+00:00 heroku[advanced-scheduler.8061]: State changed from starting to up  
2022-06-08T21:19:01.812496+00:00 app[api]: Starting process with command `python main.py` by user advanced-scheduler@addons.heroku.com  
2022-06-08T21:19:04.474541+00:00 app[advanced-scheduler.8061]: Cotação Atualizada. 2022-06-08 21:19:04.474472  
2022-06-08T21:19:04.474554+00:00 app[advanced-scheduler.8061]: Dólar: R$4.895  
2022-06-08T21:19:04.474554+00:00 app[advanced-scheduler.8061]: Euro: R$5.2496  
2022-06-08T21:19:04.474559+00:00 app[advanced-scheduler.8061]: BTC: R$148.848  
2022-06-08T21:19:04.598365+00:00 heroku[advanced-scheduler.8061]: Process exited with status 0  
2022-06-08T21:19:04.792884+00:00 heroku[advanced-scheduler.8061]: State changed from up to complete
```



Nesta aula, você aprenderá a utilizar a **API do ViaCEP**, uma ferramenta gratuita e muito prática para consultar informações de endereços no Brasil. Uma API funciona disponibilizando links, chamados de **endpoints**, onde você pode realizar requisições para obter ou enviar dados. No caso do ViaCEP, os links são do tipo **get**, o que significa que eles servem exclusivamente para capturar informações do sistema deles.

Para realizar a comunicação entre o seu código Python e a API, você utilizará a biblioteca **requests**, que pode ser instalada com o comando **pip install requests**. O formato de resposta mais comum e recomendado é o **JSON**, que o Python consegue ler facilmente transformando-o em um dicionário. A estrutura do link de busca segue o padrão **https://viacep.com.br/ws/CEP/json/**, onde você substitui a palavra CEP pelos oito dígitos numéricos que deseja consultar.

```
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\Manfredi\Desktop\Codigos>pip install requests
Collecting requests
  Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\manfredi\appdata\local\programs\python\python39\lib\site-packages (from requests)
Requirement already satisfied: idna<4,>=2.5 in c:\users\manfredi\appdata\local\programs\python\python39\lib\site-packages (from requests)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\manfredi\appdata\local\programs\python\python39\lib\site-packages (from requests)
Requirement already satisfied: certifi<2017.4.17 in c:\users\manfredi\appdata\local\programs\python\python39\lib\site-packages (from requests)
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
Installing collected packages: requests
Successfully installed requests-2.32.5
```

Ao executar o comando **requests.get(link)**, a API devolve um código de status, sendo o **200** o indicativo de que a requisição foi bem-sucedida. Caso o CEP possua menos de oito dígitos ou caracteres inválidos, você receberá um erro **400**, por isso é fundamental tratar os dados antes do envio. Você pode utilizar o método **replace** do Python para remover pontos, traços ou espaços, garantindo que apenas números sejam enviados para a consulta.

```
import requests

cep = "20090002"

link = f'https://viacep.com.br/ws/{cep}/json/'

requisicao = requests.get(link)

print(requisicao)

[2] ✓ 0.6s

... <Response [200]>
```

Uma vez que a resposta é obtida, você utiliza o comando **requisicao.json()** para transformar o conteúdo em um **dicionário Python**. A partir desse dicionário, você consegue extrair campos específicos como a **localidade** para a cidade, o **UF** para o estado e o **logradouro** para a rua.

Essa integração permite que você automatize a busca de endereços de forma extremamente rápida e eficiente dentro de qualquer sistema que esteja construindo.

```
import requests

cep = "20090002"

link = f'https://viacep.com.br/ws/{cep}/json/'

requisicao = requests.get(link)

print(requisicao)

print(requisicao.json())
```

✓ 0.6s

<Response [200]>

{'cep': '20090-002', 'logradouro': 'Avenida Rio Branco', 'complemento': 'de 32 a 62 - lado', 'bairro': 'Centro', 'cidade': 'Rio de Janeiro', 'estado': 'RJ'}

```
import requests

cep = "20090002"

link = f'https://viacep.com.br/ws/{cep}/json/'

requisicao = requests.get(link)

print(requisicao)

dic_requisicao = requisicao.json()

uf = dic_requisicao["uf"]
cidade = dic_requisicao["localidade"]
bairro = dic_requisicao["bairro"]

print(uf, cidade, bairro)
```

[4] ✓ 0.6s

... <Response [200]>

RJ Rio de Janeiro Centro

```
import requests

cep = "20.090-002"

cep = cep.replace("-", "").replace(".", "").replace(" ", "")

if len(cep) == 8:
    link = f'https://viacep.com.br/ws/{cep}/json/'

    requisicao = requests.get(link)

    print(requisicao)

    dic_requisicao = requisicao.json()

    uf = dic_requisicao["uf"]
    cidade = dic_requisicao["localidade"]
    bairro = dic_requisicao["bairro"]

    print(uf, cidade, bairro)

else:
    print("CEP inválido")
```

✓ 0.6s

```
<Response [200]>
RJ Rio de Janeiro Centro
```

Além de consultar um CEP para obter um endereço, a API do ViaCEP também permite fazer o caminho inverso: descobrir o CEP a partir de um nome de rua. Para essa busca, o link de requisição deve conter o estado, a cidade e o nome do logradouro, seguindo o formato <https://viacep.com.br/ws/UF/Cidade/Rua/json/>. É importante notar que tanto a cidade quanto o endereço precisam ter pelo menos três caracteres para que a busca seja processada.

```
uf = "RJ"
cidade = "Rio de Janeiro"
endereco = "Rio Branco"

link = f'https://viacep.com.br/ws/{uf}/{cidade}/{endereco}/json/'

requisicao = requests.get(link)
print(requisicao)

dic_requisicao = requisicao.json()
print(dic_requisicao)
```

[7] ✓ 0.7s

```
... <Response [200]>
[{'cep': '20251-620', 'logradouro': 'Rua Lírio Branco', 'complemento': '', 'unidade':
```

Diferente da consulta individual, essa busca por endereço pode retornar até **50 resultados** diferentes em uma única requisição. Isso acontece porque a API devolve uma **lista de dicionários** contendo todas as correspondências que encontrou na base de dados. Para visualizar esses dados de maneira organizada, você pode importar a biblioteca **pprint** e usar o comando **pprint.pprint(dicionario)**, que formata a exibição no terminal.

```
import pprint

pprint.pprint(dic_requisicao)
```

```
[8] ✓ 0.0s
```

```
[{'bairro': 'Rio Comprido',
  'cep': '20251-620',
  'complemento': '',
  'ddd': '21',
  'estado': 'Rio de Janeiro',
  'gia': '',
  'ibge': '3304557',
  'localidade': 'Rio de Janeiro',
  'logradouro': 'Rua Lírio Branco',
  'regiao': 'Sudeste',
  'siafi': '6001',
  'uf': 'RJ',
  'unidade': ''},
 {'bairro': 'Centro',
  'cep': '20090-001',
  'complemento': 'de 22 a 30 - lado par',
  'ddd': '21',
  'estado': 'Rio de Janeiro',
  'gia': '',
  'ibge': '3304557',
  'localidade': 'Rio de Janeiro',
  'logradouro': 'Avenida Rio Branco',
  'regiao': 'Sudeste',
  'siafi': '6001',
  'uf': 'RJ',
  'unidade': ''}]
```

Uma forma ainda mais profissional de exibir esses múltiplos resultados é utilizando a biblioteca **Pandas** para criar uma tabela. Ao usar o comando **pd.DataFrame(lista_de_dicionarios)**, o Python organiza automaticamente os CEPs, nomes de ruas e bairros em colunas e linhas. Caso esteja utilizando o ambiente **Jupyter**, você pode usar o comando **display(tabela)** para visualizar os dados de forma estilizada e facilitar a análise das informações encontradas.

```
import pandas as pd

tabela = pd.DataFrame(dic_requisicao)
display(tabela)
```

```
[10] ✓ 0.0s
```

	cep	logradouro	complemento	unidade	bair
0	20251-620	Rua Lírio Branco			Rio Comprido
1	20090-001	Avenida Rio Branco	de 22 a 30 - lado par		Centro
2	20040-913	Avenida Rio Branco	91	Edifício São Francisco	Centro
3	20040-001	Avenida Rio Branco	de 102 a 126 - lado par		Centro
4	20040-909	Avenida Rio Branco	138	Edifício Bozano Simonsen	Centro
5	20040-922	Avenida Rio Branco	70	Edifício Banco Real	Centro
6	20090-000	Avenida Rio Branco	até 20 - lado par		Centro
7	20040-907	Avenida Rio Branco	118	Edifício Pedro de Magalhães Correia	Centro
8	20040-920	Avenida Rio Branco	219	Edifício Biblioteca Nacional	Centro
9	20090-907	Avenida Rio Branco	1	Centro Empresarial Internacional Rio	Centro
10	20040-070	Avenida Rio Branco	de 64 a 100 - lado par		Centro
11	20090-904	Avenida Rio Branco	35	Edifício Almare	Centro
12	20040-908	Avenida Rio Branco	103	Edifício Índico	Centro
13	20040-010	Avenida Rio Branco	101	Edifício Século Fronteira	Centro

O acesso a dados financeiros atualizados, como cotações diárias, semanais e mensais de ações, moedas e criptomoedas, é fundamental para análises técnicas e fundamentalistas. Para realizar esse procedimento, você utilizará a **API do Alpha Vantage**, um serviço que disponibiliza diversos **endpoints** para a requisição de informações econômicas. Ao iniciar o trabalho com uma nova **API**, o primeiro passo indispensável é consultar a documentação oficial para compreender os métodos de autenticação e os parâmetros necessários para as consultas.

A comunicação com a **API** ocorre de forma autenticada, funcionando de maneira análoga a um login. Você deve obter uma **chave API** gratuita no site do provedor. Para manter a segurança do seu código e evitar a exposição de sua credencial, recomenda-se salvar essa chave em um arquivo separado, como um **chave.py**, e importá-la no seu script principal utilizando o comando **from chave import chave_api**. Dessa forma, sua chave permanece protegida enquanto você realiza as requisições.

alphavantage.co/support/#api-key

Claim your Free API Key

Claim your free key for the [Alpha Vantage Stock API](#) with lifetime access. We highly recommend that you use a legitimate email address - this is the primary way we will contact you for feature announcements or troubleshooting (e.g., if you lose your API key). By acquiring and using an Alpha Vantage API key, you agree to our [Terms of Service](#) and [Privacy Policy](#).

Which of the following best describes you?

Student

Organization (e.g. company, university, etc.):

Email:

abc@gmail.com

[GET FREE API KEY](#)

A documentação do **Alpha Vantage** divide as funcionalidades em seções como **Stock APIs**, **Forex**, **Criptomoedas** e indicadores econômicos. Se o seu objetivo for extrair cotações semanais, você deve procurar pela função específica de série temporal. É recomendável utilizar a opção **weekly adjusted**, que fornece valores corrigidos por eventos corporativos, como **dividendos** e desdobramentos, garantindo maior precisão em análises de longo prazo. Cada consulta é estruturada através de parâmetros obrigatórios, como **function**, **symbol** e **apikey**.

Para executar as requisições em **Python**, você deve utilizar a biblioteca **requests**. Caso ainda não a possua, realize a instalação via terminal com o comando **pip install requests**. No ambiente **Jupyter**, utilize a exclamação antes do comando, como **!pip install requests**. A requisição propriamente dita é feita através da função **requests.get**, que acessa o link formatado com os parâmetros de consulta e armazena o resultado em uma variável de resposta.

```
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\Manfredi\Desktop\Codigos>pip install requests
```

```
Collecting requests
```

```
Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
```

```
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\manfredi\appdata\local\progr
```

```
Requirement already satisfied: idna<4,>=2.5 in c:\users\manfredi\appdata\local\programs\python\p
```

```
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\manfredi\appdata\local\programs\py
```

```
Requirement already satisfied: certifi>=2017.4.17 in c:\users\manfredi\appdata\local\programs\py
```

```
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
```

```
Installing collected packages: requests
```

```
Successfully installed requests-2.32.5
```

Por padrão, a **API** retorna os dados no formato **JSON**, que o **Python** interpreta como um grande **dicionário** composto por outros dicionários e listas. Para extrair o conteúdo útil da resposta, você deve aplicar o método **r.json** na variável de resposta. A construção da **URL** de consulta pode ser otimizada com o uso de **f-strings**, inserindo variáveis como a **chave API** diretamente no texto do link. É importante observar que o plano gratuito possui limitações, permitindo, por exemplo, até cinco requisições por minuto.

Pegando cotações semanais - JSON

```
from chave import chave_api
import requests

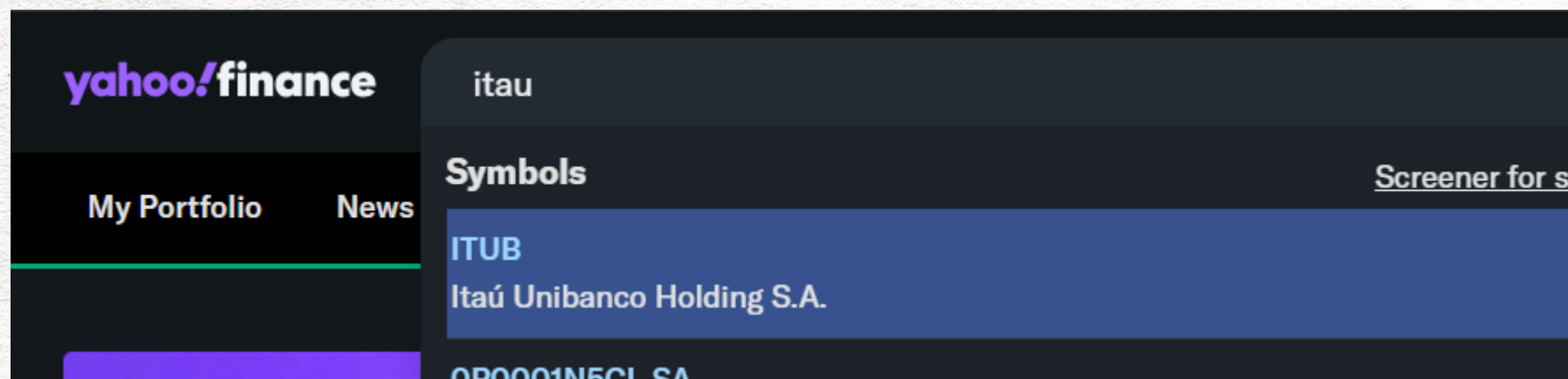
# replace the "demo" apikey below with your own key from https://www.alphavantage.co/support/#api-key
url = f'https://www.alphavantage.co/query?function=TIME_SERIES_WEEKLY_ADJUSTED&symbol=IBM&apikey={chave_api}'
r = requests.get(url)
data = r.json()

print(data)
```

[1] ✓ 1.7s

... {'Meta Data': {'1. Information': 'Weekly Adjusted Prices and Volumes', '2. Symbol': 'IBM', '3. Last Refreshed': '202

Para identificar o código de negociação ou **ticker** de uma empresa, você pode consultar fontes externas como o **Yahoo Finance**. Ações listadas na bolsa brasileira exigem a inclusão do sufixo **.SAO**, como em **ITUB4.SAO**.



```
from chave import chave_api
import requests

# replace the "demo" apikey below with your own key from https://www.alphavantage.co/support/#api-key
url = f'https://www.alphavantage.co/query?function=TIME_SERIES_WEEKLY_ADJUSTED&symbol=ITUB4.SAO&apikey={chave_api}'
r = requests.get(url)
data = r.json()

print(data)
```

[2] ✓ 1.7s

... {'Meta Data': {'1. Information': 'Weekly Adjusted Prices and Volumes', '2. Symbol': 'ITUB4.SAO', '3. Last Refreshed': '202

Além do **JSON**, a **API** permite solicitar dados no formato **CSV** através do parâmetro opcional **datatype=csv**. Para processar um retorno em **CSV** e transformá-lo em uma tabela manipulável, você deve utilizar a biblioteca **pandas**. Como a resposta chega como texto, é necessário utilizar a função **StringIO** da biblioteca **io** para converter esse texto em um objeto que simula um arquivo, permitindo que a função **pd.read_csv** do **pandas** realize a leitura corretamente. A utilização da biblioteca **pandas** em conjunto com o recurso **StringIO** permite que você converta dados textuais recebidos via **API** diretamente em um arquivo virtual para processamento. Uma vez que as informações estejam estruturadas em um **dataframe**, você terá acesso a ferramentas poderosas de análise de dados, podendo calcular médias, valores máximos e outras métricas estatísticas de forma eficiente.

No ambiente **Jupyter**, você pode utilizar a função **display** para visualizar as tabelas de forma estilizada, enquanto em outros editores deve-se optar pelo comando **print** tradicional.

```
import pandas as pd
from io import StringIO

# replace the "demo" apikey below with your own key from https://www.alphavantage.co/support/#api-key
url = f'https://www.alphavantage.co/query?function=TIME_SERIES_WEEKLY_ADJUSTED&symbol=ITUB4.SAO&apikey={chave_api}&datatype=csv'
r = requests.get(url)

tabela = pd.read_csv(StringIO(r.text))
display(tabela)
```

[4] ✓ 8.7s

	timestamp	open	high	low	close	adjusted close	volume	dividend amount
0	2026-01-23	39.7200	43.9100	39.0700	43.5700	43.5700	151356300	0.0000
1	2026-01-16	39.8500	40.2200	39.1700	39.6100	39.6100	88127700	0.0000
2	2026-01-09	39.2400	40.4800	39.0600	39.8500	39.8500	101526000	0.0000
3	2026-01-02	39.6300	39.7700	39.1500	39.1500	39.1500	18846800	0.0000
4	2025-12-30	39.1800	39.6700	38.8600	39.2300	39.2123	24421900	0.0000
...	...	...	...	...	...	...	...	...
1105	2005-02-11	429.9996	437.9984	427.0024	432.9008	1.9699	722173740	0.0000
1106	2005-02-04	384.5026	440.0032	384.5026	437.9984	1.9931	1013533962	0.0024
1107	2005-01-28	387.0003	392.0019	375.4976	379.0011	1.7239	979320690	0.0000
1108	2005-01-21	397.9005	402.9914	386.0970	389.4978	1.7717	960024404	0.0000
1109	2005-01-14	390.0110	398.0099	384.0031	398.0099	1.8104	1067864637	0.0000

1110 rows × 8 columns

Caso sua análise envolva múltiplas ações simultaneamente, o procedimento ideal consiste em criar uma lista com os **tickers** desejados, como **ITUB3.SA** e **ABEV3.SA**. Se você necessitar apenas da cotação mais recente, o **endpoint** adequado é o **Global Quote**, que retorna o preço atual e o volume de negociação em vez do histórico completo. Para automatizar a coleta, utilize uma estrutura de repetição **for acao in acoes**, que percorrerá cada item da sua lista e executará a requisição individualmente.

Para consolidar os resultados de várias empresas em uma única base de dados, você deve inicializar uma tabela vazia utilizando o comando **pd.DataFrame**. Dentro do laço de repetição, após obter a cotação de cada ativo, utilize a função **pd.concat** para concatenar a nova tabela de resultados à tabela principal compilada. Durante esse processo, é fundamental respeitar os limites de requisição do seu plano, implementando pausas se necessário para não exceder o limite de acessos por minuto permitido pela **API**.

```
acoes = ['ITUB4', 'ABEV3', 'BBAS3']

compilada = pd.DataFrame()

for acao in acoes:
    url = f'https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol={acao}.SAO&apikey={chave_api}&datatype=csv'
    r = requests.get(url)
    tabela = pd.read_csv(StringIO(r.text))
    lista_tabelas = [compilada, tabela]
    compilada = pd.concat(lista_tabelas)

display(compilada)
```

[4] ✓ 2.0s

	symbol	open	high	low	price	volume	latestDay	previousClose	change	changePercent	
0	ITUB4.SAO	43.19	43.91	43.04	43.57	41001000.0	2026-01-23	43.08	0.49	1.1374%	NaN
	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	and instantly unlock all premium endpoints"
	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
0	BBAS3.SAO	23.45	24.59	23.43	24.28	70273200.0	2026-01-23	23.45	0.83	3.5394%	NaN

Generate Code Markdown

A **API** também oferece recursos de busca para casos em que você não conhece o código de negociação de um ativo. O **endpoint** de **symbol search** utiliza o parâmetro **keywords** para localizar empresas por nome, retornando uma lista de correspondências em diversas bolsas mundiais, incluindo a bolsa de Londres ou de Frankfurt. O resultado trará códigos, nomes das empresas e uma pontuação de relevância para a sua pesquisa.

Encontrar ativos

```
url = f'https://www.alphavantage.co/query?function=SYMBOL_SEARCH&keywords=amazon&apikey={chave_api}&datatype=csv'
r = requests.get(url)

tabela = pd.read_csv(StringIO(r.text))
display(tabela)
```

✓ 0.7s

	symbol	name	type	region	marketOpen	marketClose	timezone	currency	matchScore
0	AMZN	Amazon.com Inc	Equity	United States	09:30	16:00	UTC-04	USD	0.8000
1	AMZN.TRT	Amazon.com CDR (CAD Hedged)	Equity	Toronto	09:30	16:00	UTC-05	CAD	0.6154
2	AMZ.DEX	Amazon.com Inc	Equity	XETRA	08:00	20:00	UTC+02	EUR	0.6000
3	AMZ.FRK	Amazon.com Inc	Equity	Frankfurt	08:00	20:00	UTC+02	EUR	0.6000
4	AMZO34.SAO	Amazon.com Inc	Equity	Brazil/Sao Paulo	10:00	17:30	UTC-03	BRL	0.6000
5	OR1O.LON	Amazon.com Inc.	Equity	United Kingdom	08:00	16:30	UTC+01	USD	0.5714
6	AMZ1.FRK	AMAZON.COM INC.CDR DL-01	Equity	Frankfurt	08:00	20:00	UTC+02	EUR	0.5000
7	AZFL	Amazonas Florestal Ltd	Equity	United States	09:30	16:00	UTC-04	USD	0.4286

Para dados fundamentalistas, como o histórico de resultados de uma empresa, você utilizará o **endpoint** de **earnings**. Este retorno é disponibilizado exclusivamente em formato **JSON**. Para facilitar a visualização de **dicionários** complexos e aninhados, você pode importar a biblioteca **pprint** e utilizar a função **pprint.pprint**. Se houver uma lista de dicionários dentro do campo **annualEarnings**, por exemplo, você pode convertê-la instantaneamente em uma tabela do **pandas** passando essa chave específica para a função **pd.DataFrame**, o que torna os dados prontos para a etapa de análise técnica ou financeira.

Informações de Resultado

```
url = f'https://www.alphavantage.co/query?function=EARNINGS&symbol=AMZN&apikey={chave_api}'
r = requests.get(url)
data = r.json()

print(data)
```

[6] ✓ 0.8s

```
{'symbol': 'AMZN', 'annualEarnings': [{'fiscalDateEnding': '2025-12-31', 'reportedEPS': '5.22'}, ...]}
```

import pprint

```
pprint.pprint(data)
```

[8] ✓ 0.0s

```
{'annualEarnings': [{'fiscalDateEnding': '2025-12-31', 'reportedEPS': '5.22'},
{'fiscalDateEnding': '2024-12-31', 'reportedEPS': '5.53'},
{'fiscalDateEnding': '2023-12-31', 'reportedEPS': '2.9'},
{'fiscalDateEnding': '2022-12-31', 'reportedEPS': '0.97'},
{'fiscalDateEnding': '2021-12-31', 'reportedEPS': '2.15'},
{'fiscalDateEnding': '2020-12-31', 'reportedEPS': '2.09'},
{'fiscalDateEnding': '2019-12-31', 'reportedEPS': '1.14'},
{'fiscalDateEnding': '2018-12-31', 'reportedEPS': '1'},
{'fiscalDateEnding': '2017-12-31', 'reportedEPS': '0.23'},
{'fiscalDateEnding': '2016-12-31', 'reportedEPS': '0.25'},
{'fiscalDateEnding': '2015-12-31', 'reportedEPS': '0.06'},
{'fiscalDateEnding': '2014-12-31', 'reportedEPS': '-0.03'},
{'fiscalDateEnding': '2013-12-31', 'reportedEPS': '-0.07'},
{'fiscalDateEnding': '2012-12-31', 'reportedEPS': '0.02'},
{'fiscalDateEnding': '2011-12-31', 'reportedEPS': '0.07'},
{'fiscalDateEnding': '2010-12-31', 'reportedEPS': '0.13'},
{'fiscalDateEnding': '2009-12-31', 'reportedEPS': '0.1'},
{'fiscalDateEnding': '2008-12-31', 'reportedEPS': '0.08'},
{'fiscalDateEnding': '2007-12-31', 'reportedEPS': '0.05'},
{'fiscalDateEnding': '2006-12-31', 'reportedEPS': '0.12'},
{'fiscalDateEnding': '2005-12-31', 'reportedEPS': '0.11'},
{'fiscalDateEnding': '2004-12-31', 'reportedEPS': '0.05'},
{'fiscalDateEnding': '2003-12-31', 'reportedEPS': '0.04'}]}
```

Informações de Resultado

```
url = f'https://www.alphavantage.co/query?function=EARNINGS&symbol=AMZN&apikey={chave_api}'
r = requests.get(url)
data = r.json()

print(data)
```

[6] ✓ 0.8s

```
{'symbol': 'AMZN', 'annualEarnings': [{'fiscalDateEnding': '2025-12-31', 'reportedEPS': '5.22'}, ...]}
```

import pprint

```
# pprint.pprint(data)

resultado = pd.DataFrame(data['annualEarnings'])
display(resultado)
```

[7] ✓ 0.0s

	fiscalDateEnding	reportedEPS
0	2025-12-31	5.22
1	2024-12-31	5.53
2	2023-12-31	2.9
3	2022-12-31	0.97
4	2021-12-31	2.15
5	2020-12-31	2.09
6	2019-12-31	1.14
7	2018-12-31	1
8	2017-12-31	0.23
9	2016-12-31	0.25
10	2015-12-31	0.06
11	2014-12-31	-0.03
12	2013-12-31	-0.07

Para integrar a linguagem **Python** com a **API** do **IBGE**, você deve compreender que uma **API** funciona como um sistema online que disponibiliza **links**, tecnicamente chamados de **end points**, para a troca de informações. A **API** do **IBGE** é uma ferramenta pública e gratuita que não exige autenticação, permitindo a extração programática de dados para que você construa códigos capazes de automatizar a coleta de informações atualizadas. Para facilitar o entendimento da estrutura dos dados, você pode utilizar o **query builder**, um editor que funciona como um construtor de **links** de requisição.

No desenvolvimento com **APIs**, existem quatro métodos principais de manipulação: o **get** para buscar informações, o **post** para criar dados, o **put** ou **patch** para edições e o **delete** para exclusões. Como a **API** do **IBGE** disponibiliza dados governamentais protegidos, você utilizará exclusivamente o método **get** para acessar os conteúdos. O **query builder** permite que você selecione variáveis, períodos e localidades, gerando automaticamente a **URL** necessária para a sua requisição.

Para realizar a comunicação técnica, é necessário utilizar a biblioteca **requests** no **Python**. Caso ela não esteja instalada em seu ambiente, você deve executar o comando **pip install requests** no terminal ou, no caso do **Jupyter**, utilizar o prefixo de exclamação antes do comando.

```
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\Manfredi\Desktop\Codigos>pip install requests
```

```
Collecting requests
```

```
Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
```

```
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\manfredi\appdata\local\programs\python\python39\python.exe (3.9.0)
```

```
Requirement already satisfied: idna<4,>=2.5 in c:\users\manfredi\appdata\local\programs\python\python39\python.exe (3.9.0)
```

```
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\manfredi\appdata\local\programs\python\python39\python.exe (3.9.0)
```

```
Requirement already satisfied: certifi>=2017.4.17 in c:\users\manfredi\appdata\local\programs\python\python39\python.exe (3.9.0)
```

```
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
```

```
Installing collected packages: requests
```

```
Successfully installed requests-2.32.5
```

Após a instalação, você importa a biblioteca com o comando **import requests** e utiliza a função **requests.get** passando a **URL** desejada como parâmetro para armazenar a resposta do sistema em uma variável.

A maioria das **APIs** se comunica através do formato **json**, que no **Python** é interpretado como uma estrutura de **dicionário** ou uma **lista** de **dicionários**. Para organizar a visualização desses dados, que podem vir desestruturados, recomenda-se o uso da biblioteca **pprint**, que permite imprimir o conteúdo de forma indentada e legível através da função **pprint.pprint**.

API IBGE

<https://servicodados.ibge.gov.br/api/docs>

```
import requests

link = "https://servicodados.ibge.gov.br/api/v3/agregados/4112/periodos/2006/variaveis/2586?localidades=N1[all]"

requisicao = requests.get(link)
informacoes = requisicao.json()

import pprint

pprint.pprint(informacoes)
```

[2] ✓ 0.5s

```
{'id': '2586',
 'resultados': [{'classificacoes': [{'categoria': {'0': 'Total'},
                                             'id': '637',
                                             'nome': 'Classificação de médio '
                                             'produtor'},
                                     {'categoria': {'116700': 'Total'},
                                             'id': '12650',
                                             'nome': 'Plantel de suínos'}],
 'series': [{'localidade': {'id': '1',
                              'nivel': {'id': 'N1',
                                         'nome': 'Brasil'},
                              'nome': 'Brasil'},
              'serie': {'2006': '1496422'}}]}],
 'unidade': 'Unidades',
 'variavel': 'Número de estabelecimentos agropecuários com suínos'}}
```

A extração das informações específicas é feita navegando pelos níveis da estrutura, utilizando **índices** numéricos para **listas** e **chaves** de texto para os campos do **dicionário**.

API IBGE

<https://servicodados.ibge.gov.br/api/docs>

```
import requests

link = "https://servicodados.ibge.gov.br/api/v3/agregados/4112/periodos/2006/variaveis/2586?localidades=N1[all]"

requisicao = requests.get(link)
informacoes = requisicao.json()

import pprint

pprint.pprint(informacoes[0]['resultados'][0]['series'])
```

[3] ✓ 0.4s

```
{'localidade': {'id': '1',
                 'nivel': {'id': 'N1', 'nome': 'Brasil'},
                 'nome': 'Brasil'},
 'serie': {'2006': '1496422'}}
```

Ao trabalhar com os resultados, você deve observar a hierarquia dos dados, acessando primeiro a **lista** geral e depois os campos internos como **resultados**, **séries** ou **variável**. Essa abordagem permite que você isole os valores desejados e os armazene em variáveis para uso posterior, como o envio de relatórios por e-mail ou a criação de **dashboards**. A documentação da **API** também fornece exemplos de respostas e confirmações, como o código de status **200**, que indica que a requisição foi bem-sucedida.

```
API IBGE

https://servicodados.ibge.gov.br/api/docs

import requests

link = "https://servicodados.ibge.gov.br/api/v3/agregados/7392/periodos/2014/variaveis/10484?localidades=N1[all]"

requisicao = requests.get(link)
informacoes = requisicao.json()

# import pprint

# pprint.pprint(informacoes[0]['resultados'][0]['series'])

item_busca = informacoes[0]['variavel'] # variavel
resultados = informacoes[0]['resultados'][0]['series'] # series

[4] ✓ 0.5s Python

print(item_busca)
print(resultados)

[5] ✓ 0.0s Python

... Número de espécies da fauna e da flora brasileira avaliadas quanto ao risco de extinção
[{'localidade': {'id': '1', 'nivel': {'id': 'N1', 'nome': 'Brasil'}, 'nome': 'Brasil'}, 'serie': {'2014': '16645'}}]
```

A utilização da **API do Banco Central** permite o acesso a uma vasta base de dados públicos, como informações sobre o dinheiro em circulação no país. Para este desenvolvimento, você precisará da biblioteca **requests** para as requisições, além das bibliotecas **pandas**, **numpy** e **openpyxl** caso pretenda realizar análises de dados avançadas e manipulação de tabelas. A instalação da biblioteca de requisições é feita através do comando **pip install requests** via terminal ou prompt de comando.

```
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\Manfredi\Desktop\Codigos>pip install requests
```

```
Collecting requests
```

```
Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
```

```
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\manfredi\appdata\local\progr
```

```
Requirement already satisfied: idna<4,>=2.5 in c:\users\manfredi\appdata\local\programs\python\p
```

```
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\manfredi\appdata\local\programs\py
```

```
Requirement already satisfied: certifi>=2017.4.17 in c:\users\manfredi\appdata\local\programs\py
```

```
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
```

```
Installing collected packages: requests
```

```
Successfully installed requests-2.32.5
```

A **API do Banco Central** disponibiliza diversos **end points** que funcionam como portas de acesso para informações diárias registradas pela instituição. O site oferece um **navegador de dados** e um **ponto de acesso à API** que atua como um **sendbox**, onde você pode testar parâmetros e visualizar a estrutura do **json** de resposta antes de implementar o código definitivo. Assim como outras **APIs** públicas, não há necessidade de autenticação ou uso de **tokens** para o consumo dos dados.

Para capturar as informações, você deve realizar um **requests.get** utilizando o link construído e extrair o conteúdo com o método **.json()**. Como a resposta costuma ser um **dicionário** complexo, a biblioteca **pprint** auxilia na identificação dos parâmetros principais, como o campo **value**, que geralmente contém a **lista de dicionários** padronizados com os dados reais.

API do Banco Central

https://dadosabertos.bcb.gov.br/dataset?res_format=API

```
import requests

link = "https://olinda.bcb.gov.br/olinda/servico/mecir_dinheiro_em_circulacao/versao/v1/odata/informacoes_diarias?$top=10000&$orderby=Data%20desc&$format=json"

requisicao = requests.get(link)
informacoes = requisicao.json()

import pprint

pprint.pprint(informacoes)
```

```
[1] ✓ 5.7s
```

```
{ '@odata.context': 'https://was-p.bcbnet.bcb.gov.br/olinda/servico/mecir_dinheiro_em_circulacao/versao/v1/odata/$metadata#informacoes_diarias',
  'value': [ { 'Data': '2026-01-23',
               'Denominacao': '0.01',
               'Especie': 'Moedas',
               'Quantidade': 3191004790,
               'Valor': 31910047.9 },
            { 'Data': '2026-01-23',
               'Denominacao': '0.05',
               'Especie': 'Moedas',
               'Quantidade': 8346088202,
               'Valor': 417304410.1 },
            { 'Data': '2026-01-23',
               'Denominacao': '0.10',
               'Especie': 'Moedas',
               'Quantidade': 8476528011,
               'Valor': 847652801.1 },
            { 'Data': '2026-01-23',
               'Denominacao': '0.25',
               'Especie': 'Moedas',
               'Quantidade': 3856451190,
               'Valor': 964112797.5 },
            { 'Data': '2026-01-23',
               'Denominacao': '0.50',
               'Especie': 'Moedas',
               'Quantidade': 3856451190,
               'Valor': 964112797.5 } ] }
```

Esses dados padronizados podem ser facilmente convertidos em uma tabela do **Python** utilizando a função **pd.DataFrame** da biblioteca **pandas**. Uma vez que os dados estão no formato de **dataframe**, você pode exibir a tabela com a função **display** ou **print**. Se for necessário formatar visualmente colunas de valores financeiros ou quantidades, você pode aplicar o método **map** com a função **.format**, utilizando códigos como **.2f** para casas decimais ou vírgulas para separadores de milhar. No entanto, é recomendável manter os dados originais em formato numérico caso você pretenda realizar cálculos matemáticos ou análises estatísticas posteriores.

```
import pandas as pd
tabela = pd.DataFrame(informacoes["value"])

tabela["Valor"] = tabela["Valor"].map("R${:,.2f}".format)

display(tabela)
```

[7] ✓ 0.0s

	Data	Quantidade	Valor	Denominacao	Especie
0	2026-01-23	3,191,004,790	R\$31,910,047.90	0.01	Moedas
1	2026-01-23	8,346,088,202	R\$417,304,410.10	0.05	Moedas
2	2026-01-23	8,476,528,011	R\$847,652,801.10	0.10	Moedas
3	2026-01-23	3,856,451,190	R\$964,112,797.50	0.25	Moedas
4	2026-01-23	3,858,524,017	R\$1,929,262,008.50	0.50	Moedas
...	...	...	...	...	...
9995	2023-03-23	123,954,135	R\$24,790,827,000.00	200.00	Cédulas
9996	2023-03-22	3,191,005,234	R\$31,910,052.34	0.01	Moedas
9997	2023-03-22	7,592,145,701	R\$379,607,285.05	0.05	Moedas
9998	2023-03-22	7,812,116,345	R\$781,211,634.50	0.10	Moedas
9999	2023-03-22	3,477,542,290	R\$869,385,572.50	0.25	Moedas

O link de requisição aceita parâmetros de configuração após o símbolo de interrogação, como o **top** para definir a quantidade de registros e o **format** para especificar o tipo de dado. Outro parâmetro essencial é o **order by**, que permite ordenar os resultados por campos específicos, como a data, de forma ascendente ou descendente. Muitas **APIs** possuem o sistema de **paginação**, limitando a quantidade de informações enviadas em uma única requisição, o que exige o uso de parâmetros como o **offset** ou o **skip** para percorrer toda a base de dados.

Quando uma **API** limita a quantidade de dados por requisição, você deve implementar uma lógica de **paginação** para extrair a base completa através de múltiplas chamadas sequenciais. Para isso, você pode utilizar um loop de repetição do tipo **while True**, que manterá o código executando requisições contínuas até que uma condição de parada seja atingida. A regra de parada geralmente é definida ao identificar que uma requisição retornou uma **lista** vazia no campo **value**, indicando que não há mais registros a serem lidos.

Para consolidar os dados de várias requisições em uma única estrutura, você deve criar inicialmente uma tabela vazia com o comando **pd.DataFrame**. A cada iteração do loop, você realiza a nova requisição, transforma os dados em um **dataframe** temporário e utiliza a função **pd.concat** para concatenar a tabela acumulada com os novos dados recebidos. Esse processo garante que, ao final do loop, você possua todas as linhas da **API** reunidas em uma única variável de tabela.

A dinamicidade das requisições é controlada por um parâmetro de salto, muitas vezes chamado de **skip** ou **offset**, que define a partir de qual registro a **API** deve começar a enviar os dados. Você deve criar uma variável de controle, como **pular_indice**, inicializada em zero e incrementada ao final de cada iteração pelo mesmo valor definido no parâmetro **top**. No **Python**, o incremento pode ser simplificado com o operador **+=**, garantindo que cada nova chamada ignore os registros já capturados nas etapas anteriores.

Embora algumas ferramentas permitam puxar grandes volumes de uma só vez, dominar a técnica de **paginação** é fundamental, pois muitas **APIs** profissionais impõem limites rígidos de tráfego por requisição. Essa estrutura de loop com parâmetros autoincrementados e travas de segurança baseadas na resposta do sistema permite que o seu código seja resiliente e capaz de lidar com bases de dados de qualquer escala de forma automática e sequencial.

```
# pegar todas as informacoes com varias requisicoes

tabela_final = pd.DataFrame()
pular_indice = 0

while True:
    link = f'https://olinda.bcb.gov.br/olinda/servico/mecir_dinheiro_em_circulacao/versao/v1/odata/in
    requisicao = requests.get(link)
    informacoes = requisicao.json()
    tabela = pd.DataFrame(informacoes["value"])
    if len(informacoes['value']) < 1:
        break
    tabela_final = pd.concat([tabela_final, tabela])
    pular_indice += 10000

display(tabela_final)
```

[8] ✓ 21.1s

	Data	Quantidade	Valor	Denominacao	Especie
0	2026-01-23	3191004790	3.191005e+07	0.01	Moedas
1	2026-01-23	8346088202	4.173044e+08	0.05	Moedas
2	2026-01-23	8476528011	8.476528e+08	0.10	Moedas
3	2026-01-23	3856451190	9.641128e+08	0.25	Moedas
4	2026-01-23	3858524017	1.929262e+09	0.50	Moedas
...	...	...	...	...	...
9806	1994-10-03	181609358	1.816094e+08	1.00	Moedas
9807	1994-10-03	252922174	1.264611e+09	5.00	Cédulas
9808	1994-10-03	273630983	2.736310e+09	10.00	Cédulas
9809	1994-10-03	28945486	1.447274e+09	50.00	Cédulas
9810	1994-10-03	2841789	2.841789e+08	100.00	Cédulas

99811 rows × 5 columns

A integração entre **Python** e a **API do Google Sheets** permite a leitura, edição e criação de informações em planilhas de forma automatizada. O processo é dividido em uma etapa de configuração de segurança no ambiente do **Google** e o desenvolvimento do código propriamente dito. Para iniciar, é necessário acessar o **Google Developer Console** e criar um novo projeto, que servirá como o aplicativo de conexão entre as ferramentas.

New Project

 You have 12 projects remaining in your quota. Request an increase or delete projects. [Learn more](#)

[Manage Quotas](#)

Project name *

aula-python-goolesheets

?

Project ID: proven-aviary-485319-k7. It cannot be changed later.

Edit

Location *

 No organization

Browse

Parent organization or folder

Create

Cancel

Dentro do console de desenvolvedor, você deve habilitar obrigatoriamente duas bibliotecas de serviço: a **Google Drive API** e a **Google Sheets API**.



Google Drive API

[Google Enterprise API](#)

Create and manage resources in Google Drive.

Enable

Try this API



Google Sheets API

[Google Enterprise API](#)

Read and write Google Sheets data

Enable

Try this API

Após a ativação, o próximo passo é a configuração da tela de permissão **OAuth**, que define como o aplicativo será autenticado. Se você estiver utilizando uma conta comum do **Gmail**, o aplicativo deve ser configurado como **externo**, sendo necessário cadastrar seu próprio e-mail na seção de **usuários de teste** para garantir o acesso durante o desenvolvimento.

Para que o código **Python** consiga se autenticar, você deve criar uma **credencial** do tipo **ID do cliente OAuth 2.0**, selecionando a opção de **aplicativo de área de trabalho**. Ao finalizar a criação, você deve realizar o download do arquivo no formato **json** e renomeá-lo para **credentials.json**, salvando-o na mesma pasta onde o seu código será executado. Este arquivo contém a chave secreta e o identificador necessários para o login inicial.

App information

This shows in the consent screen, and helps end users know who you are and contact you

App name *

integracaogoogle sheets

The name of the app asking for consent

User support email *

marcelo@brasil.com

For users to contact you with questions about their consent. [Learn more](#)

User type

External

Make internal

OAuth user cap

While publishing status is set to "Testing", only test users are able to access the app. Allowed user cap prior to app verification is 100, and is counted over the entire lifetime of the app. [Learn more](#)

1 user (1 test, 0 other) / 100 user cap

Test users

+ Add users

Create OAuth client ID

Go to previous page

A client ID is used to identify a single app to Google's OAuth servers. If your app runs on multiple platforms, each will need its own client ID. See [Setting up OAuth 2.0](#) for more information. [Learn more](#) about OAuth client types.

Application type *

Desktop app

Name *

pc

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

Note: It may take 5 minutes to a few hours for settings to take effect

Create

Cancel

Diferente de **APIs** mais simples que utilizam apenas **URLs**, o **Google** disponibiliza bibliotecas específicas para facilitar o uso de seus serviços. Você deve instalar esses pacotes utilizando o comando **pip install** seguido dos nomes das bibliotecas de cliente e autenticação fornecidos na documentação oficial.

```
“pip install --upgrade google-api-python-client google-auth-httplib2 google-auth-oauthlib”
```

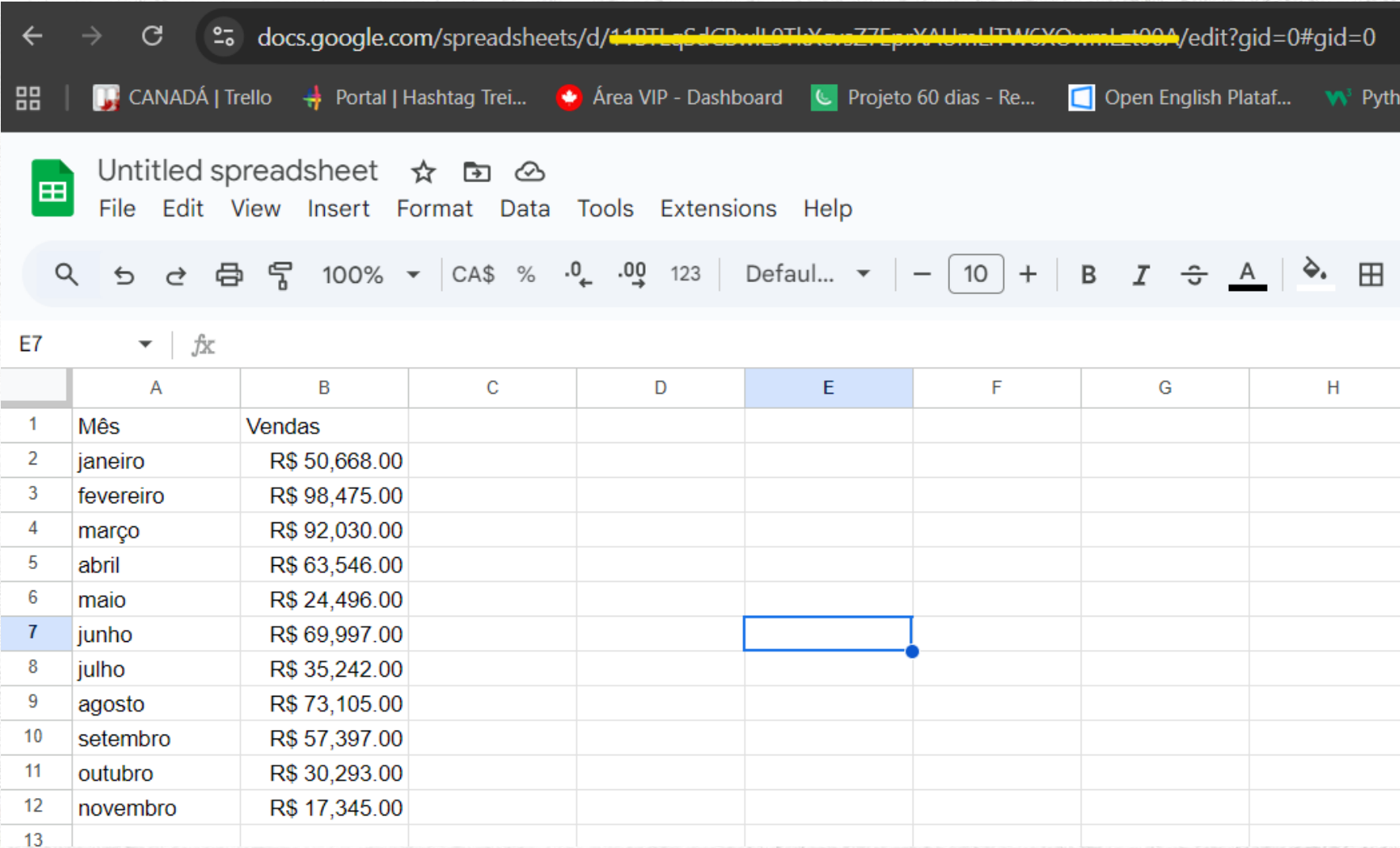
O código base exige a definição de **escopos** de acesso, que determinam se o seu programa poderá apenas ler dados com o parâmetro **read only** ou se terá permissão total para editar e excluir informações.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Manfredi\Desktop\Codigos>pip install --upgrade google-api-python-client google-auth-httplib2 google-auth-oauthlib
Collecting google-api-python-client
  Downloading google_api_python_client-2.188.0-py3-none-any.whl.metadata (7.0 kB)
Collecting google-auth-httplib2
  Downloading google_auth_httplib2-0.3.0-py3-none-any.whl.metadata (3.1 kB)
Collecting google-auth-oauthlib
  Downloading google_auth_oauthlib-1.2.4-py3-none-any.whl.metadata (3.1 kB)
Collecting httplib2<1.0.0,>=0.19.0 (from google-api-python-client)
  Downloading httplib2-0.31.2-py3-none-any.whl.metadata (2.2 kB)
Collecting google-auth!=2.24.0,!>=2.25.0,<3.0.0,>=1.32.0 (from google-api-python-client)
  Downloading google_auth-2.47.0-py3-none-any.whl.metadata (6.4 kB)
Collecting google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0,>=1.31.5 (from google-api-python-client)
  Downloading google_api_core-2.29.0-py3-none-any.whl.metadata (3.3 kB)
Collecting uritemplate<5,>=3.0.1 (from google-api-python-client)
  Downloading uritemplate-4.2.0-py3-none-any.whl.metadata (2.6 kB)
Collecting googleapis-common-protos<2.0.0,>=1.56.2 (from google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0,>=1.31.5)
  Downloading googleapis_common_protos-1.72.0-py3-none-any.whl.metadata (9.4 kB)
Collecting protobuf!=3.20.0,!>=3.20.1,!>=4.21.0,!>=4.21.1,!>=4.21.2,!>=4.21.3,!>=4.21.4,!>=4.21.5,<7.0.0,>=3.19.5 (f
```

Cada planilha do **Google Sheets** possui um identificador exclusivo, o **ID**, que pode ser encontrado na **URL** do navegador entre as barras após o caractere **d**. No código, você deve referenciar esse **ID** e definir o **range** de células desejado, seguindo o padrão do próprio **Excel**, como o nome da aba seguido de exclamação e o intervalo de células. Essa estrutura permite que o **Python** localize exatamente onde deve buscar ou inserir as informações dentro do seu **Google Drive**.



O processo de autenticação na **API do Google Sheets** utiliza um sistema de **tokens** para garantir a segurança e evitar logins repetitivos. Na primeira execução do código, o programa utiliza o arquivo **credentials.json** para abrir uma janela no navegador e solicitar a sua permissão manual. Uma vez autorizada, a **API** gera um arquivo chamado **token.json**, que armazena a credencial de acesso e permite que as execuções futuras ocorram de forma automática, renovando-se silenciosamente conforme necessário.

```
import os.path

from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build
from googleapiclient.errors import HttpError

# If modifying these scopes, delete the file token.json.
SCOPES = ["https://www.googleapis.com/auth/spreadsheets"]

# The ID and range of a sample spreadsheet.
SAMPLE_SPREADSHEET_ID = "118Tlq5dCBw1l9TKXcvsZ7EprXAUmL1TW6X0wmlztB8A"
SAMPLE_RANGE_NAME = "Sheet1!A1:B12"

def main():
    creds = None

    if os.path.exists("token.json"):
        creds = Credentials.from_authorized_user_file("token.json", SCOPES)
    # If there are no (valid) credentials available, let the user log in.
    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
            creds.refresh(Request())
        else:
            flow = InstalledAppFlow.from_client_secrets_file(
                "credentials.json", SCOPES
            )
            creds = flow.run_local_server(port=0)
        # Save the credentials for the next run
        with open("token.json", "w") as token:
            token.write(creds.to_json())

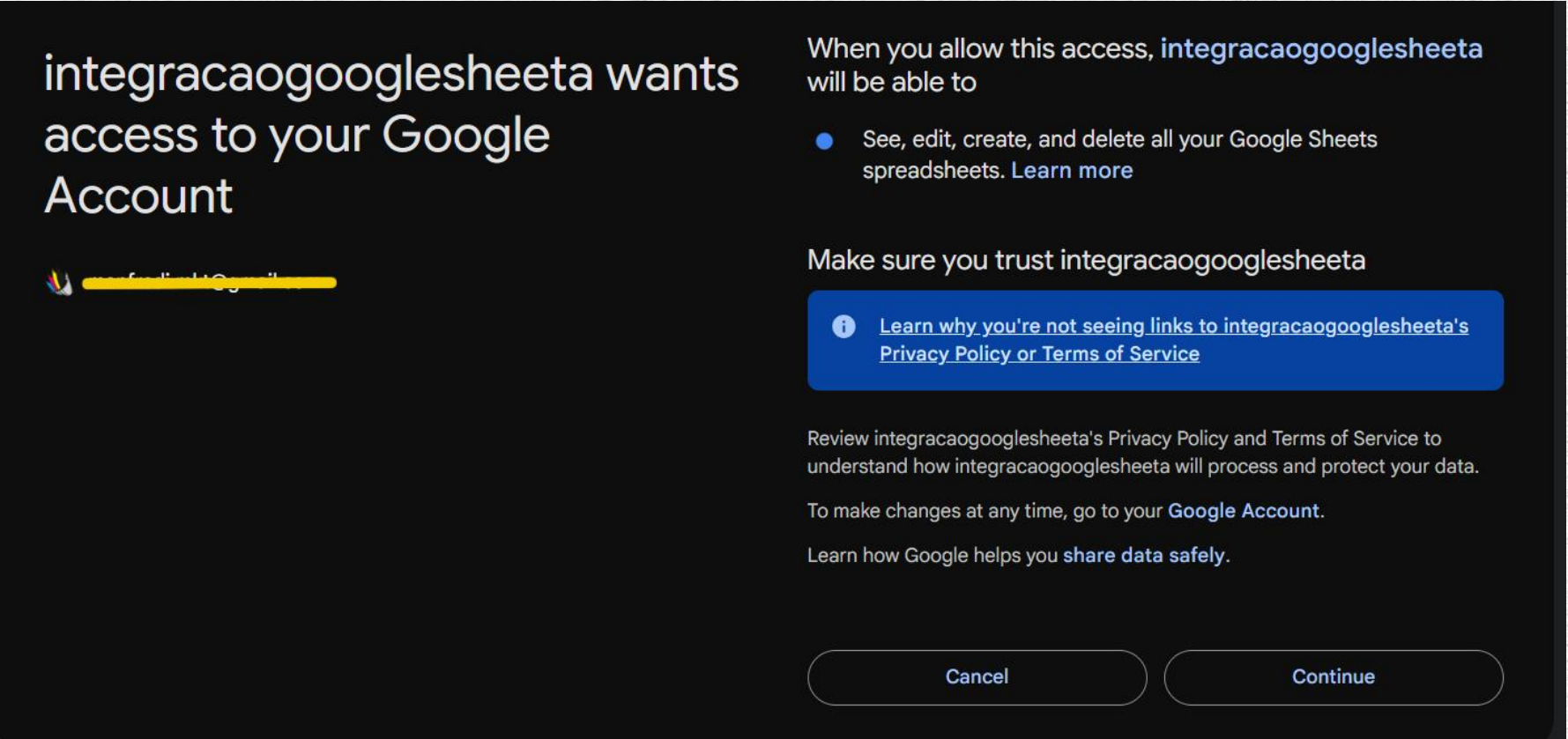
    try:
        service = build("sheets", "v4", credentials=creds)

        # Call the Sheets API
        sheet = service.spreadsheets()
        result = (
            sheet.values()
            .get(spreadsheetId=SAMPLE_SPREADSHEET_ID, range=SAMPLE_RANGE_NAME)
            .execute()
        )
        values = result.get("values", [])

        if not values:
            print("No data found.")
            return

        print("Name, Major:")
        for row in values:
            # Print columns A and E, which correspond to indices 0 and 4.
            print(f"{row[0]}, {row[4]}")
    except HttpError as err:
        print(err)

if __name__ == "__main__":
    main()
```



Para manipular a planilha, o código utiliza a função **build** para criar um serviço de conexão com o **Google Sheets**. A extração de dados é realizada através do método **values().get()**, que exige o **ID** da planilha e o intervalo de células como parâmetros. A resposta dessa operação é um **dicionário** contendo metadados do arquivo e os valores reais, que ficam armazenados em uma **chave** específica chamada **values**.

O processo de autenticação na **API do Google Sheets** utiliza um sistema de **tokens** para garantir a segurança e evitar logins repetitivos. Na primeira execução do código, o programa utiliza o arquivo **credentials.json** para abrir uma janela no navegador e solicitar a sua permissão manual. Uma vez autorizada, a **API** gera um arquivo chamado **token.json**, que armazena a credencial de acesso e permite que as execuções futuras ocorram de forma automática, renovando-se silenciosamente conforme necessário.

```

flow = InstalledAppFlow.from_client_secrets_file(
    'credentials.json', SCOPES)
creds = flow.run_local_server(port=0)
# Save the credentials for the next run
with open('token.json', 'w') as token:
    token.write(creds.to_json())

try:
    service = build('sheets', 'v4', credentials=creds)

    # Call the Sheets API
    sheet = service.spreadsheets()
    result = sheet.values().get(spreadsheetId=SAMPLE_SPREADSHEET_ID,
                                range=SAMPLE_RANGE_NAME).execute()

    print(result)

except HttpError as err:
    print(err)

if __name__ == '__main__':
    main()

```

[3] ✓ 0.9s

... {'range': 'Sheet1!A1:B12', 'majorDimension': 'ROWS', 'values': [['Mês', 'Vendas'], ['janeiro', 'R\$ 50,668.00'], ['fevereiro', 'R\$ 98,475.00'], ['março', 'R\$ 92,030.00']]}

Os dados retornados pelo **Google Sheets** são estruturados no formato de uma **lista de listas**, onde cada item da **lista** principal representa uma linha da planilha. Essa organização permite que você utilize loops do tipo **for** para percorrer a tabela linha por linha, acessando cada coluna através de seus **índices** numéricos, começando pelo zero. É importante notar que células vazias ao final das linhas podem não ser incluídas na lista, o que requer atenção no tratamento dos dados no **Python**.

```

# Save the credentials for the next run
with open('token.json', 'w') as token:
    token.write(creds.to_json())

try:
    service = build('sheets', 'v4', credentials=creds)

    # Call the Sheets API
    sheet = service.spreadsheets()
    result = sheet.values().get(spreadsheetId=SAMPLE_SPREADSHEET_ID,
                                range=SAMPLE_RANGE_NAME).execute()

    valores = result['values']
    print(valores)

except HttpError as err:
    print(err)

if __name__ == '__main__':
    main()

```

[5] ✓ 0.8s

... [['Mês', 'Vendas'], ['janeiro', 'R\$ 50,668.00'], ['fevereiro', 'R\$ 98,475.00'], ['março', 'R\$ 92,030.00']]

Para realizar alterações na planilha, você deve substituir o método de busca pelo método `values().update()`. Esta função exige o parâmetro adicional `valueInputOption`, que aceita as opções **RAW**, para inserir o texto exatamente como enviado, ou **USER_ENTERED**, que simula a digitação de um usuário humano, permitindo que o **Google** reconheça fórmulas, datas e formatos numéricos automaticamente. Os dados a serem enviados devem seguir rigorosamente o padrão de **lista de listas**, mesmo que você deseje alterar apenas uma única célula.

```
try:
    service = build('sheets', 'v4', credentials=creds)

    # Ler informações do Google Sheets
    sheet = service.spreadsheets()
    result = sheet.values().get(spreadsheetId=SAMPLE_SPREADSHEET_ID,
                               range=SAMPLE_RANGE_NAME).execute()

    valores = result['values']
    print(valores)

    # adicionar/editar uma informação
    sheet = service.spreadsheets()
    valores_adicionar = [
        ['dezembro', 127315],
        ['janeiro', 100000]
    ]
    result = sheet.values().update(spreadsheetId=SAMPLE_SPREADSHEET_ID,
                                   range="A13", valueInputOption="USER_ENTERED",
                                   body={"values": valores_adicionar }).execute()

    # for linha in valores:

except HttpError as err:
    print(err)

if __name__ == '__main__':
    main()
```

Python

242.00', ['agosto', 'R\$ 73,105.00'], ['setembro', 'R\$ 57,397.00'], ['outubro', 'R\$ 30,293.00'], ['novembro', 'R\$ 17,345.00'], ['dezembro', 'R\$ 127,315.00'], ['janeiro', 'R\$ 100,000.00']

E7	A	B	C
1	Mês	Vendas	
2	janeiro	R\$ 50,668.00	
3	fevereiro	R\$ 98,475.00	
4	março	R\$ 92,030.00	
5	abril	R\$ 63,546.00	
6	maio	R\$ 24,496.00	
7	junho	R\$ 69,997.00	
8	julho	R\$ 35,242.00	
9	agosto	R\$ 73,105.00	
10	setembro	R\$ 57,397.00	
11	outubro	R\$ 30,293.00	
12	novembro	R\$ 17,345.00	
13	dezembro	R\$ 127,315.00	
14	janeiro	R\$ 100,000.00	

Para adicionar novas informações em uma planilha, você deve definir uma variável contendo uma **lista de listas**, onde cada sublista interna representa uma nova linha a ser escrita. Ao executar o comando **update**, a **API** preenche as células a partir do endereço inicial informado no parâmetro de intervalo, reconhecendo automaticamente a disposição das colunas. Caso a planilha esteja configurada em português, o sistema do **Google** pode converter textos formatados com símbolos de moeda em valores numéricos automaticamente, desde que você utilize a opção **USER_ENTERED**.

A criação de novas colunas baseadas em cálculos, como o cálculo de impostos sobre vendas, exige que você percorra a **lista de listas** original obtida via **get**. Durante a iteração, você pode utilizar a função **enumerate** para obter o **índice** de cada linha, permitindo ignorar o cabeçalho através de uma condição **if**. Como os dados capturados da planilha chegam ao **Python** no formato de **string**, é necessário realizar um tratamento de texto antes dos cálculos, removendo símbolos de moeda, espaços e ajustando separadores de milhar e decimais antes de converter o valor para **float**.

Após processar os cálculos, você deve armazenar os novos resultados em uma nova **lista de listas** utilizando o método **append**. Para criar uma coluna vertical, cada resultado deve ser inserido como um único item dentro de uma sublista. Ao final do processamento, a atualização da planilha é feita em uma única chamada de **update**, passando a nova **lista** para o parâmetro **body** e definindo a célula inicial da nova coluna, como a célula **C1**.

```
if not creds or not creds.valid:
    if creds and creds.expired and creds.refresh_token:
        creds.refresh(Request())
    else:
        flow = InstalledAppFlow.from_client_secrets_file(
            'credentials.json', SCOPES)
        creds = flow.run_local_server(port=0)
    # Save the credentials for the next run
    with open('token.json', 'w') as token:
        token.write(creds.to_json())

try:
    service = build('sheets', 'v4', credentials=creds)

    # Ler informações do Google Sheets
    sheet = service.spreadsheets()
    result = sheet.values().get(spreadsheetId=SAMPLE_SPREADSHEET_ID,
                                range=SAMPLE_RANGE_NAME).execute()

    valores = result['values']
    print(valores)

    valores_adicionar = [
        ["Imposto"]
    ]

    for i, linha in enumerate(valores):
        if i > 0:
            vendas = linha[1]
            vendas = vendas.replace("R$ ", "").replace(".", "")
            vendas = float(vendas.replace(",", "."))
            imposto = vendas * 0.1
            imposto = f'R$ {imposto}'.replace(".", ",")
            valores_adicionar.append([imposto])

    result = sheet.values().update(spreadsheetId=SAMPLE_SPREADSHEET_ID,
                                    range="C1", valueInputOption="USER_ENTERED",
                                    body={'values': valores_adicionar}).execute()
```

Caso ocorram erros de conversão, como falhas ao transformar texto em **float**, verifique se a formatação das células na planilha está padronizada, pois o **Python** lerá exatamente o que estiver visível, incluindo espaços ou símbolos diferentes. Se desejar que os resultados calculados pelo **Python** apareçam com formatação específica na planilha, você pode realizar um **replace** para trocar o ponto decimal por vírgula antes do envio, adequando o dado à língua configurada no seu **Google Drive**.

F12	▼	fx			
	A	B	C	D	E
1	Mês	Vendas	Imposto		
2	janeiro	R\$ 50.668,00	R\$ 5.066,80		
3	fevereiro	R\$ 98.475,00	R\$ 9.847,50		
4	março	R\$ 92.030,00	R\$ 9.203,00		
5	abril	R\$ 63.546,00	R\$ 6.354,60		
6	maio	R\$ 24.496,00	R\$ 2.449,60		
7	junho	R\$ 69.997,00	R\$ 6.999,70		
8	julho	R\$ 35.242,00	R\$ 3.524,20		
9	agosto	R\$ 73.105,00	R\$ 7.310,50		
10	setembro	R\$ 57.397,00	R\$ 5.739,70		
11	outubro	R\$ 30.293,00	R\$ 3.029,30		
12	novembro	R\$ 17.345,00	R\$ 1.734,50		
13	dezembro	R\$ 127.315,00	R\$ 12.731,50		
14	janeiro	R\$ 100.000,00	R\$ 10.000,00		
15					
16					

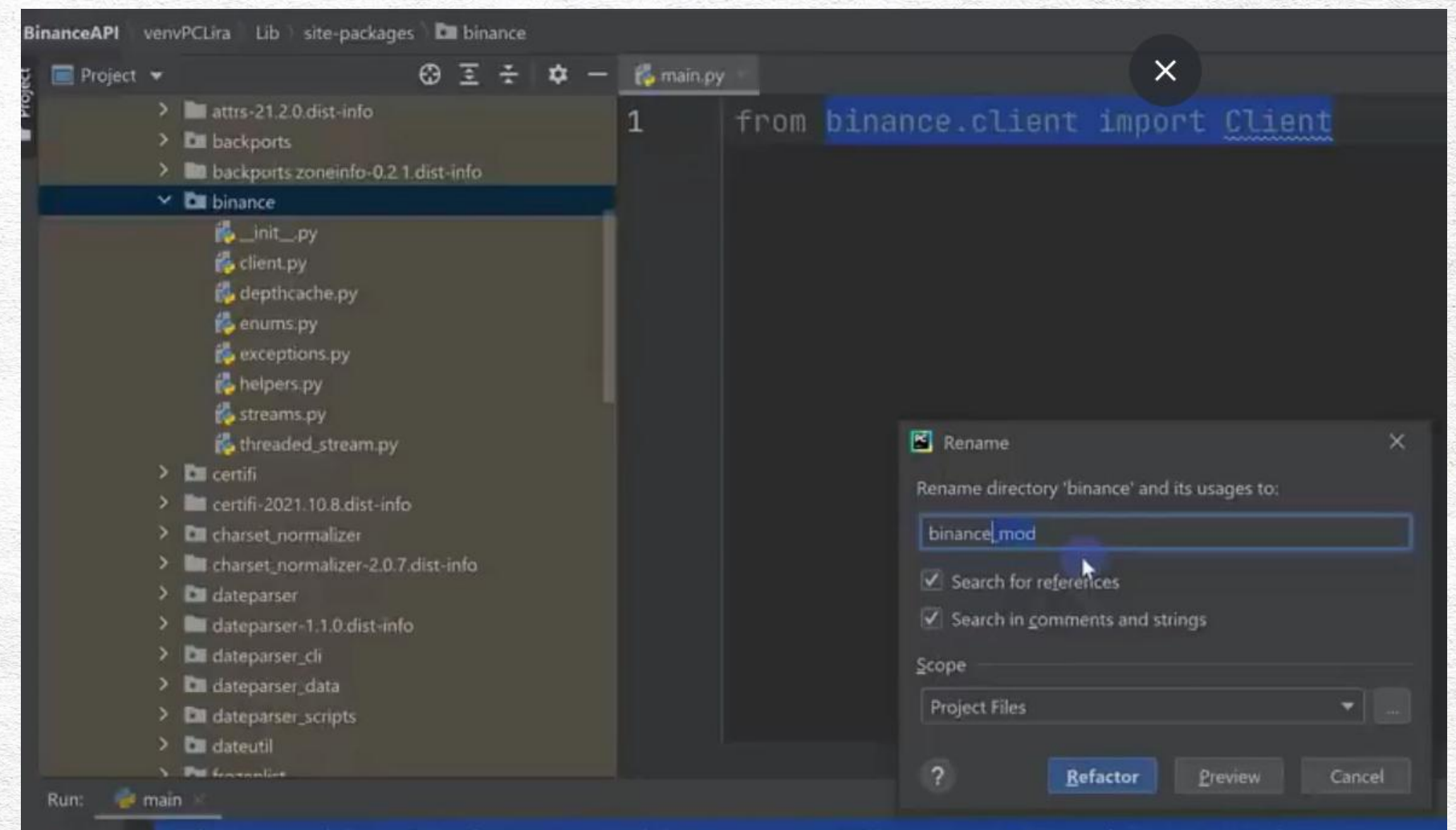
A realização de operações de compra e venda de criptoativos de forma automatizada requer a integração do **Python** com uma plataforma de negociação, sendo a **Binance** uma das opções mais completas devido à robustez de sua **API**. O primeiro passo técnico é a instalação da biblioteca específica através do comando **pip install python-binance**. É fundamental não nomear o seu arquivo ou pasta de projeto como **Binance**, pois isso gera conflitos de importação, impedindo que o **Python** identifique corretamente os módulos da biblioteca instalada.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Manfredi\Desktop\Codigos> pip install python-binance
Collecting python-binance
  Downloading python_binance-1.0.34-py2.py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: requests in c:\users\manfredi\appdata\local\programs\python\python314\lib\site-packages (from python-binance)
Requirement already satisfied: six in c:\users\manfredi\appdata\roaming\python\python314\site-packages (from python-binance)
Collecting dateparser (from python-binance)
  Downloading dateparser-1.2.2-py3-none-any.whl.metadata (29 kB)
Collecting aiohttp (from python-binance)
  Downloading aiohttp-3.13.3-cp314-cp314-win_amd64.whl.metadata (8.4 kB)
Collecting websockets (from python-binance)
  Downloading websockets-16.0-cp314-cp314-win_amd64.whl.metadata (7.0 kB)
Collecting pycryptodome (from python-binance)
  Downloading pycryptodome-3.23.0-cp37-abi3-win_amd64.whl.metadata (3.5 kB)
Collecting aiohappyeyeballs>=2.5.0 (from aiohttp->python-binance)
  Downloading aiohappyeyeballs-2.6.1-py3-none-any.whl.metadata (5.9 kB)
Collecting aiosignal>=1.4.0 (from aiohttp->python-binance)
  Downloading aiosignal-1.4.0-py3-none-any.whl.metadata (3.7 kB)
Collecting attrs>=17.3.0 (from aiohttp->python-binance)
  Downloading attrs-25.4.0-py3-none-any.whl.metadata (10 kB)
Collecting frozenlist>=1.1.1 (from aiohttp->python-binance)
  Downloading frozenlist-1.8.0-cp314-cp314-win_amd64.whl.metadata (21 kB)
Collecting multidict<7.0,>=4.5 (from aiohttp->python-binance)
  Downloading multidict-6.7.0-cp314-cp314-win_amd64.whl.metadata (5.5 kB)
Collecting propcache>=0.2.0 (from aiohttp->python-binance)
  Downloading propcache-0.2.0-cp314-cp314-win_amd64.whl.metadata (5.5 kB)
```

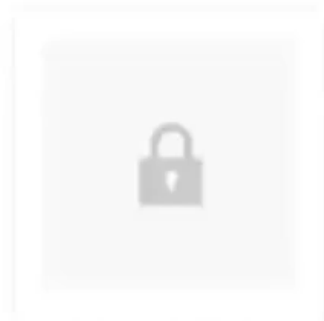
Caso a biblioteca apresente erros de reconhecimento após a instalação, pode ser necessário renomear a pasta do módulo dentro do diretório **site-packages** para algo como **binance_mod** e atualizar as referências nos arquivos internos **init** e **client**. Após garantir a instalação, você deve importar a classe **Client** do módulo **binance.client** para iniciar a comunicação com a plataforma.



A conexão segura com a sua conta exige dois identificadores gerados no painel de gerenciamento da **Binance**: a **API Key** e o **API Secret**. Ao criar essas chaves, você deve habilitar manualmente a permissão de **spot and margin trading** para que o seu código tenha autorização para enviar ordens de compra e venda, além de apenas ler dados. Por segurança, recomenda-se armazenar essas chaves em um arquivo separado, como **secrets.py**, importando-as para o código principal para evitar a exposição acidental de suas senhas.

Lista API

testePython



API Key

UPOz1NvXljG

Copy

Secret Key

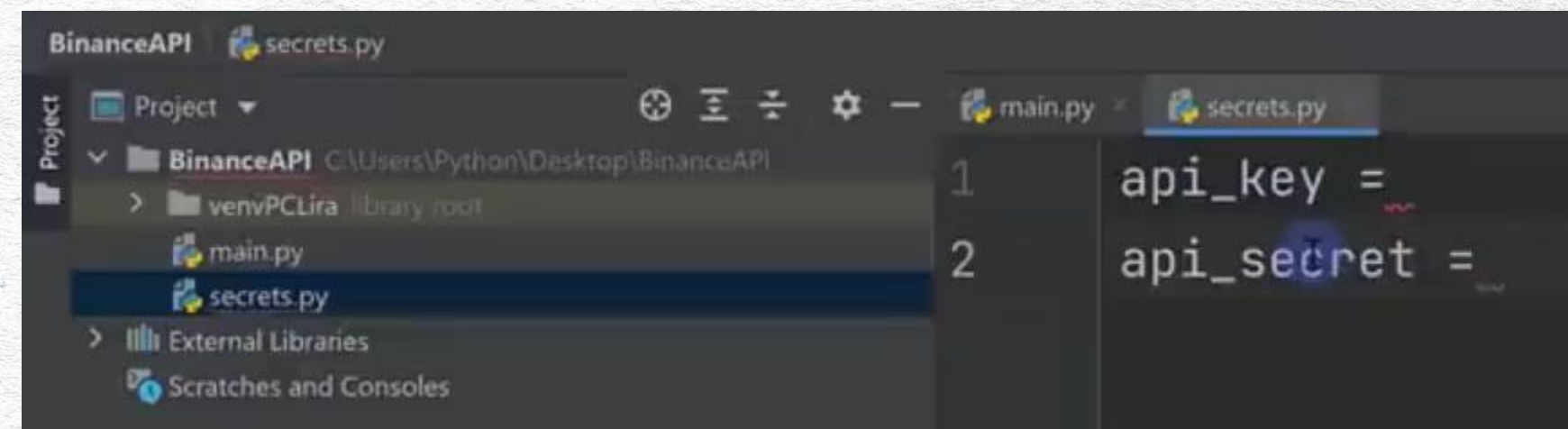
Restrições API:

☒ Habilitar Leitura

☒ Enable Spot & Margin Trading

☐ Habilitar Empréstimo, Reembolso e Transferência de Margem

☐ Permitir Transferência Universal



Para verificar se a conexão foi estabelecida com sucesso, você pode utilizar a função **get_account_status**, que deve retornar o status **normal** para a sua conta. O método **get_account** é utilizado para extrair informações gerais, retornando um **dicionário** que inclui permissões de saque, taxas e o campo **balances**, onde residem os saldos de cada criptoativo. Como a **lista** de ativos é extensa e contém muitos valores zerados, você pode filtrar os resultados utilizando um loop e convertendo o valor do campo **free** para **float** para identificar apenas as moedas em que você possui saldo real.

```

4 client = Client(api_key, api_secret)
5
6 # pegar informações da nossa conta
7 info = client.get_account()
8 for item in info:
9     print(item)

```

```

C:\Users\Python\Desktop\BinanceAPI\venvPCLira\Scripts\python.exe C:/Users/Python/Desktop/Binance
makerCommission
takerCommission
buyerCommission
sellerCommission
canTrade
canWithdraw
canDeposit
updateTime
accountType
balances
permissions
Process finished with exit code 0

```

```

4 client = Client(api_key, api_secret)
5
6 # pegar informações da nossa conta
7 info = client.get_account()
8 for item in info:
9     print(item)
10
11 # ver os saldos dos ativos que temos na conta
12 lista_ativos = info["balances"]
13 # print(lista_ativos)
14 # ativos que temos algum saldo
15 for ativo in lista_ativos:
16     if float(ativo["free"]) > 0:
17         print(ativo)

```

```

Run: main
balances
permissions
{'asset': 'BTC', 'free': '0.00000225', 'locked': '0.00000000'}
{'asset': 'BNB', 'free': '0.02694976', 'locked': '0.00000000'}
{'asset': 'XEM', 'free': '0.25200000', 'locked': '0.00000000'}
{'asset': 'BRL', 'free': '62.77078694', 'locked': '0.00000000'}
{'asset': 'XYM', 'free': '8739.25200000', 'locked': '0.00000000'}

```

A criação de negociações na **Binance** é feita através do envio de ordens ao mercado, utilizando métodos específicos da biblioteca para pedidos de compra ou venda. Antes de iniciar as operações, é necessário importar as constantes do módulo **binance.enums**, que definem parâmetros como o tipo de ordem e o lado da operação. Na plataforma, as negociações ocorrem em pares de moedas, como **BNB BRL**, onde a primeira moeda é o ativo negociado e a segunda é a moeda utilizada para o pagamento ou recebimento.

Você deve consultar no sistema da corretora quais pares estão disponíveis para garantir que o símbolo informado no código seja válido. No caso de uma ordem de compra, utiliza-se o parâmetro **side** configurado como **SIDE_BUY**, indicando que você usará seus reais para adquirir o ativo. Para as operações, o tipo de ordem mais comum para iniciantes é a **ORDER_TYPE_MARKET**, que executa a transação imediatamente ao preço atual de mercado, sem a necessidade de definir um valor máximo ou mínimo de preço.

```

13 # print(lista_ativos)
14 # ativos que temos algum saldo
15 for ativo in lista_ativos:
16     if float(ativo["free"]) > 0:
17         print(ativo)
18
19 # criar uma ordem dentro da binance
20 from binance.enums import *
21 order = client.create_order(
22     symbol='BNBBRL',
23     side=SIDE_BUY,
24     type=ORDER_TYPE_MARKET,
25     quantity=0.01,
26 )
27 print(order)
    
```

```

Run: main
{'asset': 'BTC', 'free': '0.00000225', 'locked': '0.00000000'}
{'asset': 'BNB', 'free': '0.02694976', 'locked': '0.00000000'}
{'asset': 'XEM', 'free': '0.25200000', 'locked': '0.00000000'}
{'asset': 'BRL', 'free': '62.77078694', 'locked': '0.00000000'}
{'asset': 'XYM', 'free': '8739.25200000', 'locked': '0.00000000'}
{'symbol': 'BNBBRL', 'orderId': 58241274, 'orderListId': -1, 'clientOrderId': 'Erjtn44tpBRo0YivzCjKIi', 'transactTime': 1637612512168, 'p
    
```

Process finished with exit code 0

Ao enviar uma ordem a mercado, você deve informar apenas o símbolo e a quantidade desejada através do método **create_order**. O retorno dessa função é um **dicionário** contendo detalhes da transação, como o **ID** da ordem, o status da execução (que deve aparecer como **FILLED**) e as taxas de comissão pagas. Após a execução, você pode verificar a alteração imediata nos seus saldos de ativos através de uma nova consulta à conta. Para acompanhar o histórico de atividades, a **API** oferece as funções **get_all_orders** e **get_my_trades**, que retornam **listas** de dicionários com os registros de ordens enviadas e negociações finalizadas.

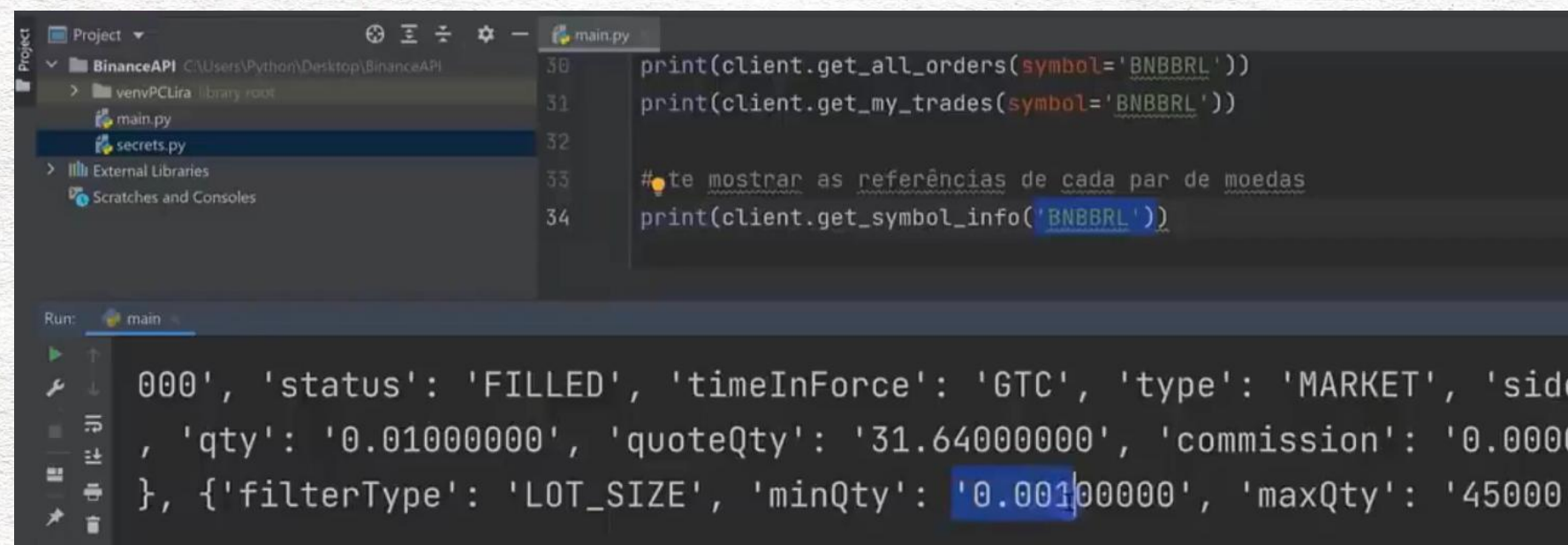
```

25 # quantity=0.01,
26 # )
27 # print(order)
28
29 # visualizar as ordens executadas
30 print(client.get_all_orders(symbol='BNBBRL'))
31 print(client.get_my_trades(symbol='BNBBRL'))
    
```

```

Run: main
permissions
{'asset': 'BTC', 'free': '0.00000225', 'locked': '0.00000000'}
{'asset': 'BNB', 'free': '0.02693478', 'locked': '0.00000000'}
{'asset': 'XEM', 'free': '0.25200000', 'locked': '0.00000000'}
{'asset': 'BRL', 'free': '62.69078694', 'locked': '0.00000000'}
{'asset': 'XYM', 'free': '8739.25200000', 'locked': '0.00000000'}
[{'symbol': 'BNBBRL', 'orderId': 58233083, 'orderListId': -1, 'clientOrderId': 'V
[{'symbol': 'BNBBRL', 'id': 5614723, 'orderId': 58233083, 'orderListId': -1, 'pr
    
```

Além disso, é crucial respeitar os limites operacionais da corretora, como o lote mínimo de negociação e o valor financeiro mínimo, que pode ser de dez reais para alguns pares. Essas restrições e precisões decimais podem ser consultadas programaticamente através da função `get_symbol_info`.



```

30 print(client.get_all_orders(symbol='BNBBRL'))
31 print(client.get_my_trades(symbol='BNBBRL'))
32
33 #te mostrar as referências de cada par de moedas
34 print(client.get_symbol_info('BNBBRL'))

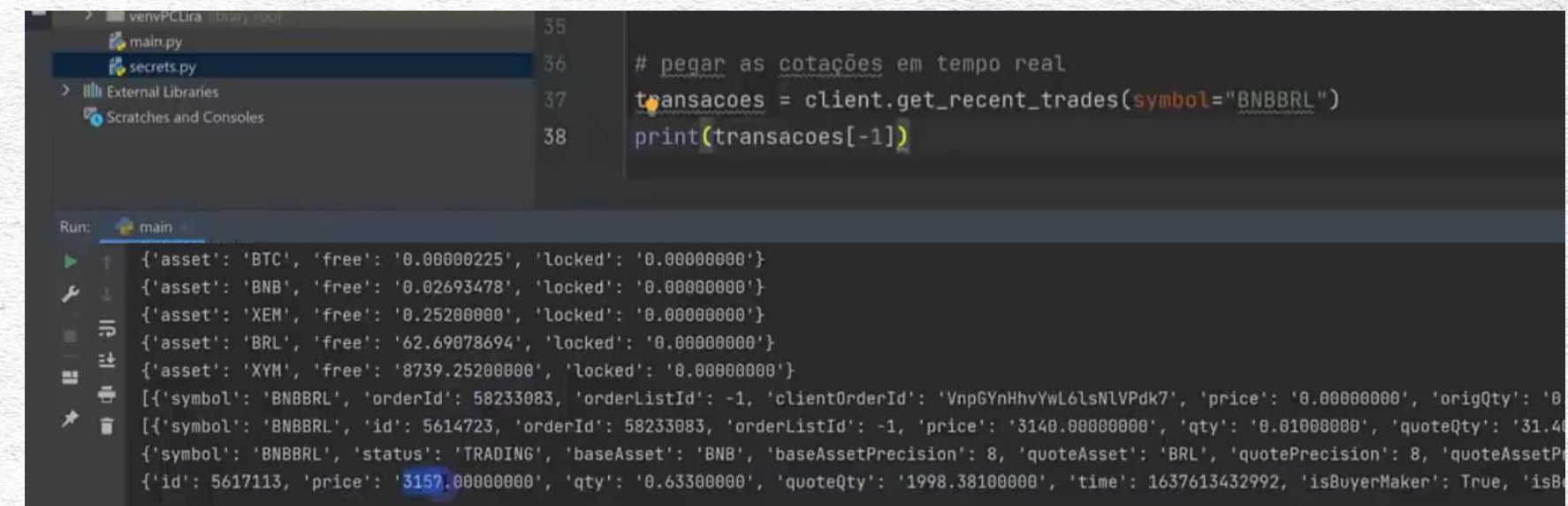
```

```

000', 'status': 'FILLED', 'timeInForce': 'GTC', 'type': 'MARKET', 'side
, 'qty': '0.01000000', 'quoteQty': '31.64000000', 'commission': '0.0000
}, {'filterType': 'LOT_SIZE', 'minQty': '0.00100000', 'maxQty': '45000

```

Para monitorar o mercado e tomar decisões de investimento, você pode capturar as cotações das criptomoedas em tempo real utilizando a função `get_recent_trades`, passando o símbolo do par desejado como parâmetro. Essa função retorna uma **lista** com as últimas transações ocorridas, fornecendo dados como o preço individual do ativo e a quantidade negociada naquela transação específica. Ao acessar o último item dessa **lista** por meio do **índice -1**, você obtém o preço de negociação mais recente do par no mercado.



```

35
36 # pegar as cotações em tempo real
37 transacoes = client.get_recent_trades(symbol="BNBBRL")
38 print(transacoes[-1])

```

```

{'asset': 'BTC', 'free': '0.00000225', 'locked': '0.00000000'}
{'asset': 'BNB', 'free': '0.02693478', 'locked': '0.00000000'}
{'asset': 'XEM', 'free': '0.25200000', 'locked': '0.00000000'}
{'asset': 'BRL', 'free': '62.69078694', 'locked': '0.00000000'}
{'asset': 'XYM', 'free': '8739.25200000', 'locked': '0.00000000'}
[{'symbol': 'BNBBRL', 'orderId': 58233083, 'orderListId': -1, 'clientOrderId': 'VnpGYnHhvYwL6lsNLVPdk7', 'price': '0.00000000', 'origQty': '0
[{'symbol': 'BNBBRL', 'id': 5614723, 'orderId': 58233083, 'orderListId': -1, 'price': '3140.00000000', 'qty': '0.01000000', 'quoteQty': '31.4
{'symbol': 'BNBBRL', 'status': 'TRADING', 'baseAsset': 'BNB', 'baseAssetPrecision': 8, 'quoteAsset': 'BRL', 'quotePrecision': 8, 'quoteAssetP
{'id': 5617113, 'price': '3157.00000000', 'qty': '0.63300000', 'quoteQty': '1998.38100000', 'time': 1637613432992, 'isBuyerMaker': True, 'isB

```

Essas informações de preço são entregues em um **dicionário** dentro do **Python**, permitindo que você trate esses dados para criar automações, como disparar compras quando o preço atingir um determinado valor ou registrar o histórico de oscilações em um banco de dados ou planilha. Caso a velocidade de execução seja uma prioridade para o seu código, você pode adicionar o parâmetro **limit=1** na requisição para que o sistema da **Binance** retorne apenas a última transação, tornando o processo mais leve e rápido.

As requisições feitas ao sistema da corretora possuem limites de frequência impostos pela própria instituição, portanto, é recomendável estruturar o seu código para fazer chamadas eficientes e apenas com os dados necessários. Com o domínio dessas funções fundamentais, você estabelece a base para integrar o Python à API da Binance e automatizar operações.

O **Firebase** é uma tecnologia do **Google** que oferece diversas funcionalidades, sendo o seu banco de dados uma excelente opção para armazenar informações online de maneira eficiente. Diferente dos bancos de dados tradicionais que utilizam a estrutura relacional do **SQL** baseada em tabelas e linhas, o **Firebase** utiliza um modelo não relacional organizado em formato de árvore. Isso significa que os dados são estruturados em níveis hierárquicos, semelhantes a pastas e arquivos em um computador.

Para utilizar essa ferramenta, você deve criar um projeto no console do **Firebase** e ativar o **Realtime Database**. Durante a configuração inicial, recomenda-se o uso do **modo de teste** para as regras de segurança, permitindo que você realize operações de leitura e escrita sem restrições complexas de autenticação nesta fase de aprendizado. No banco de dados, você pode criar categorias principais, como vendas ou produtos, e inserir subníveis com identificadores exclusivos para cada registro.

×

Create a project

Let's start with a name for your project^②

Project name

meuprojetofirebase

✎

meuprojetofirebase-f776a

☒

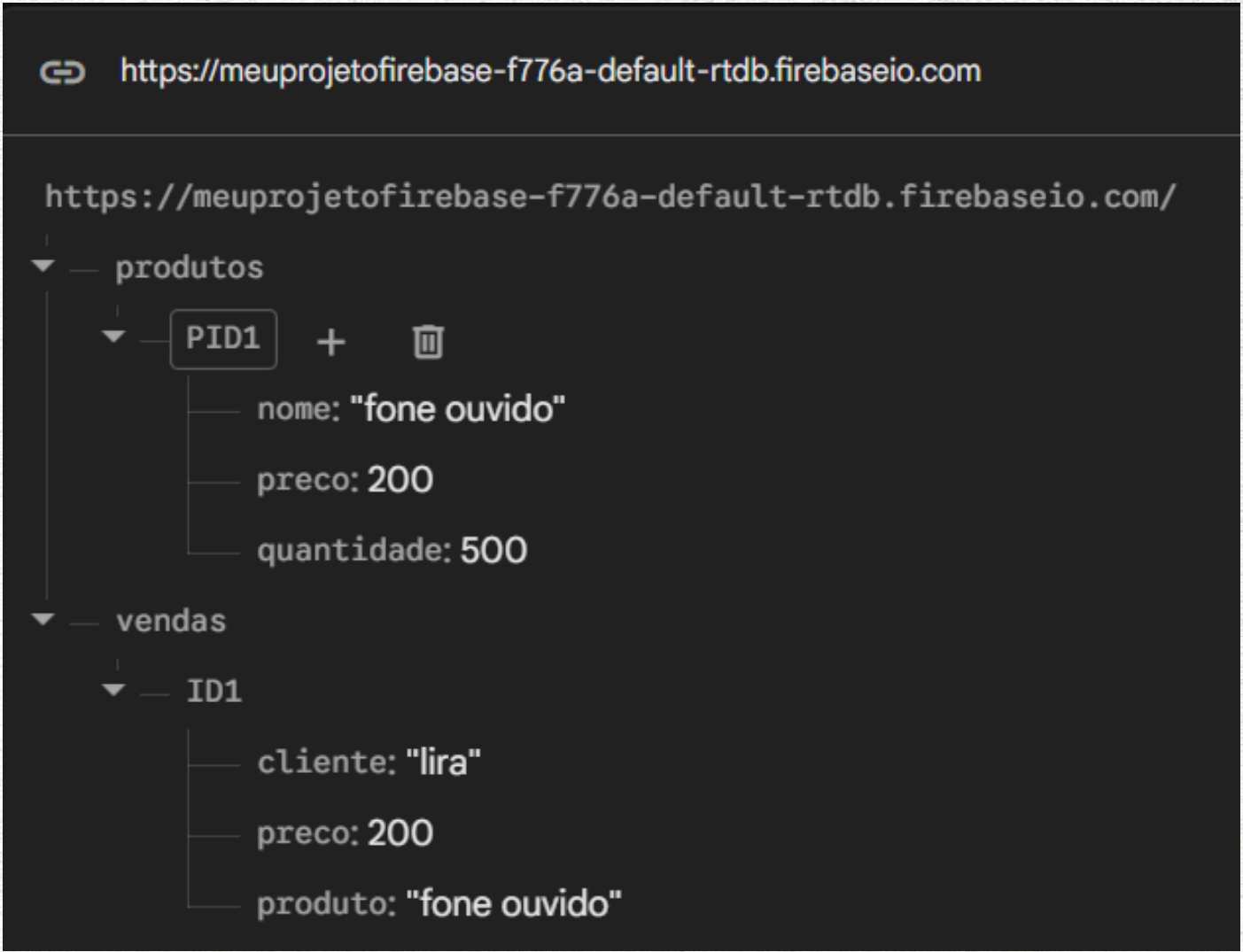
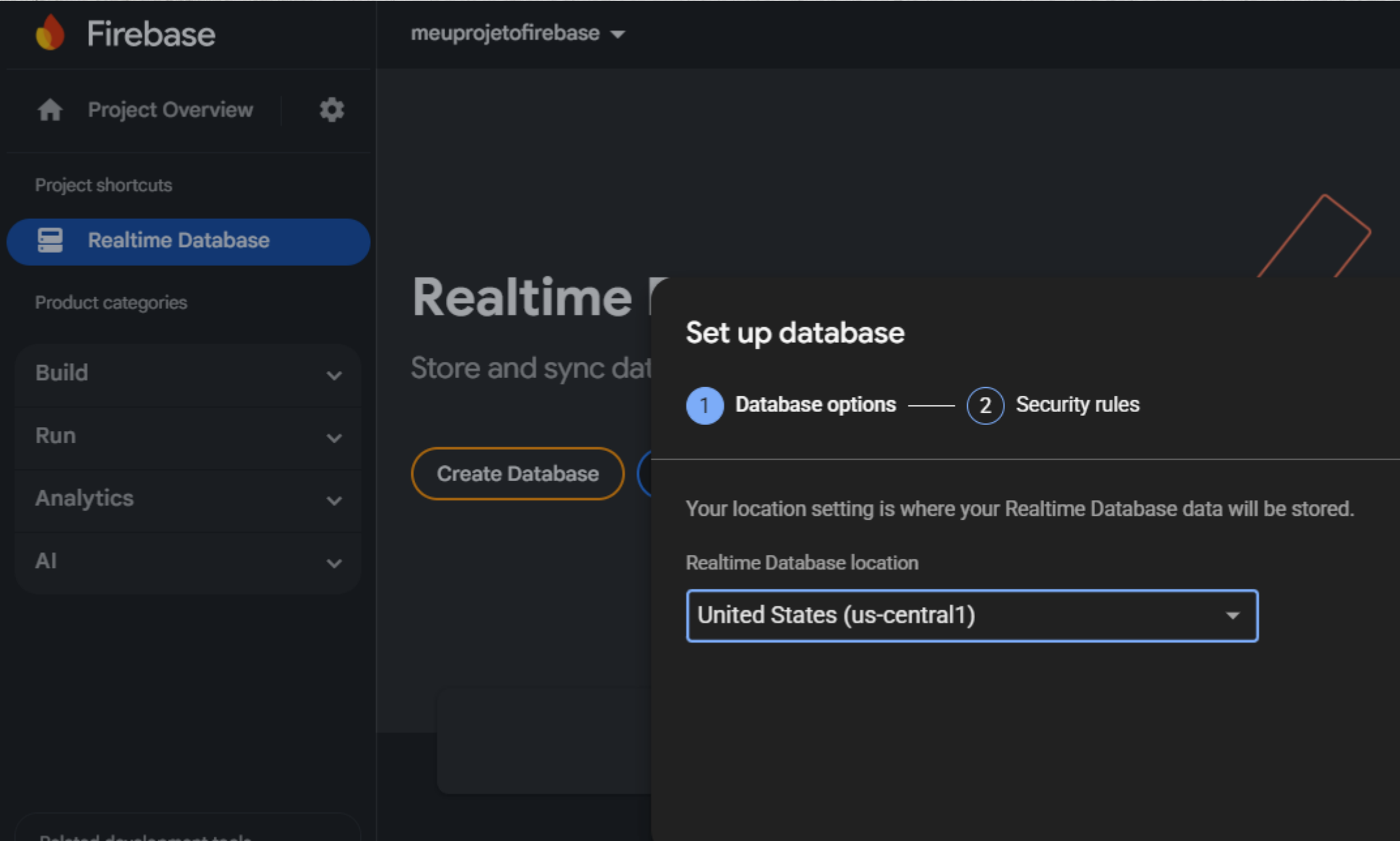
I accept the [Firebase terms](#)

☒

Join the [Google Developer Program](#) to enrich your developer journey with access to AI assistance, learning resources, profile badges, and more!

Already have a Google Cloud project?
[Add Firebase to Google Cloud project](#)

Continue



A integração com o **Python** é feita através de uma **REST API**, que permite a comunicação com o banco de dados por meio de requisições padrão de internet. A biblioteca principal para este processo é a **requests**, que deve ser instalada com o comando **pip install requests**. Toda a comunicação com o **Firebase** é baseada em um link principal fornecido pelo console, e a regra fundamental da documentação é que todas as requisições devem obrigatoriamente terminar com a extensão **.json** na **URL**.

(c) Microsoft Corporation. All rights reserved.

```
C:\Users\Manfredi\Desktop\Codigos>pip install requests
```

```
Collecting requests
```

```
Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
```

```
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\manfredi\appdata\local\progr
```

```
Requirement already satisfied: idna<4,>=2.5 in c:\users\manfredi\appdata\local\programs\python\p
```

```
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\manfredi\appdata\local\programs\py
```

```
Requirement already satisfied: certifi>=2017.4.17 in c:\users\manfredi\appdata\local\programs\py
```

```
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
```

```
Installing collected packages: requests
```

```
Successfully installed requests-2.32.5
```

Para manipular os dados, você utilizará quatro tipos principais de requisição que compõem o ciclo de vida da informação no banco: o **post** para criar novos dados, o **get** para ler informações, o **patch** para atualizar campos específicos e o **delete** para remover registros. Para criar uma nova informação, como uma venda, você utiliza o método **requests.post** enviando um dicionário contendo os dados.

É essencial utilizar a função **json.dumps** da biblioteca **json** para converter o dicionário do **Python** para o formato **JSON** antes de enviá-lo no parâmetro **data** da requisição.

```
firebase.py > ...
1  # Criar o projeto no Firebase
2
3  # Link documento REST API Firebase
4  # https://firebase.google.com/docs/reference/rest/database
5
6  # Criar o Database (atenção às regras de segurança)
7  # Estrutura de árvore
8
9  # Interação com o Database (REST API)
10 import requests
11 import json
12
13 link = "https://meuprojetofirebase-f776a-default-rtdb.firebaseio.com"
14
15 # Criar uma venda (POST)
16 dados = {'cliente': 'alon', 'preco': 150, 'produto': 'teclado'}
17 requisicao = requests.post(f'{link}/vendas/.json', data=json.dumps(dados))
18 print(requisicao)
19 print([requisicao.text])
20
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

Microsoft Windows [Version 10.0.26200.7623]

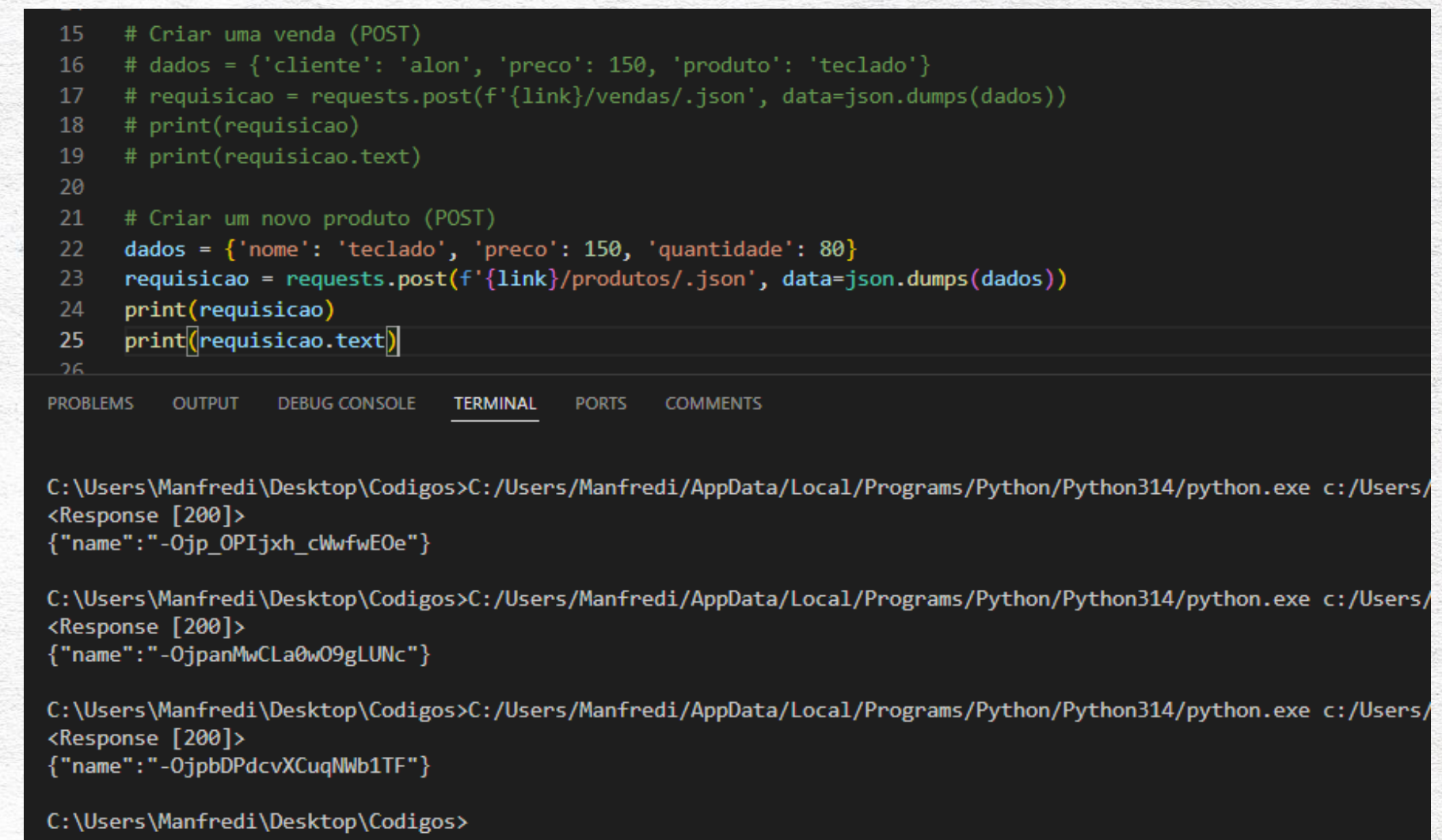
(c) Microsoft Corporation. All rights reserved.

```
C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/AppData/Local/Programs/Python/Python314/python.exe c:/Us
```

```
<Response [200]>
```

```
{"name": "-0jp_OPIjxh_cWfwEOe"}
```

```
C:\Users\Manfredi\Desktop\Codigos>
```



O método **post** cria automaticamente um identificador único para cada novo registro inserido no **Firebase**, garantindo que não existam conflitos de dados mesmo que as informações internas sejam idênticas. Caso você precise editar uma informação já existente, como alterar o nome de um cliente em uma venda específica, deve utilizar o método **patch**. Diferente de uma criação, o **patch** requer que você informe o caminho exato do registro na **URL**, incluindo o seu **ID**, e ele alterará apenas os campos enviados no dicionário, preservando os demais dados do registro.

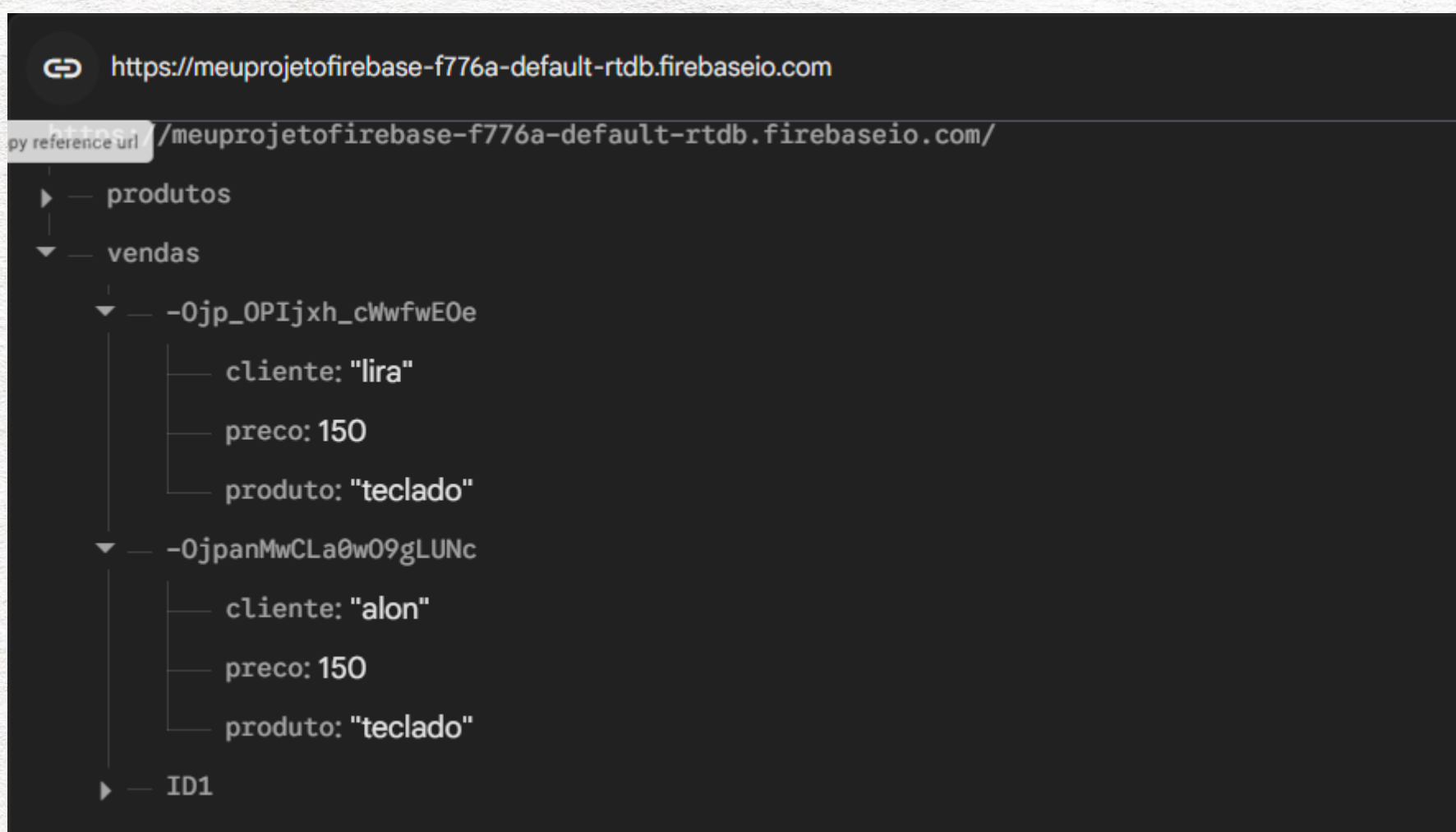
```
27 # Editar a venda (PATCH)
28 dados = {'cliente': 'lira'}
29 requisicao = requests.patch(f'{link}/vendas/-Ojp_OPIjxh_cWfwEOe/.json', data=json.dumps(dados))
30 print(requisicao)
31 print(requisicao.text)
32
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/AppData/Local/Programs/Python/Python314/python.exe c:/Users
<Response [200]>
{"cliente":"lira"}

C:\Users\Manfredi\Desktop\Codigos>



Para realizar a leitura das informações, utiliza-se o método **get**, que não exige o envio de dados no corpo da requisição, apenas o link do recurso desejado terminado em **.json**. O retorno de um **get** pode ser capturado através do método **.json()** da resposta, que transforma o texto recebido em um **dicionário** nativo do **Python**, facilitando a manipulação dos dados no código. Você pode filtrar o nível de detalhamento da busca alterando o link: buscar pela raiz traz o banco completo, enquanto buscar por um caminho específico, como **/vendas**, retorna apenas os registros daquela categoria.

Como o banco de dados é uma estrutura de árvore, ao buscar uma categoria inteira, você recebe um **dicionário** onde as **chaves** são os **IDs** dos registros e os valores são as informações internas. Isso permite que você realize buscas manuais percorrendo o dicionário com um loop e aplicando condições **if** para encontrar, por exemplo, o **ID** de uma venda associada a um determinado cliente. Essa técnica de filtragem manual é útil para localizar identificadores dinâmicos que o sistema gerou automaticamente.

```

32
33 # Pegar uma venda específico ou todas as vendas (GET)
34 requisicao = requests.get(f'{link}/vendas/.json')
35 print(requisicao)
36 dic_requisicao = requisicao.json()
37 id_alon = None
38 for id_venda in dic_requisicao:
39     cliente = dic_requisicao[id_venda]['cliente']
40     if cliente == "alon":
41         print(id_venda)
42         id_alon = id_venda
43

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS COMMENTS

Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/AppData/Local/Programs/Python/Python314/python.exe c:/Us
<Response [200]>
-OjpanMwCLa0w09gLUNc

C:\Users\Manfredi\Desktop\Codigos>

Para remover uma informação, você utiliza o comando **requests.delete**, passando na **URL** o caminho completo até o item ou categoria que deseja excluir. Se a operação for bem-sucedida, o código de status retornado será **200** e o dado desaparecerá instantaneamente do console do **Firebase**. Dominar essas operações básicas de criação, leitura, atualização e exclusão permite que você construa sistemas completos e inteligentes onde a lógica do **Python** gerencia e organiza os dados em tempo real na nuvem.

```

firebase.py > ...
32
33 # Pegar uma venda específico ou todas as vendas (GET)
34 requisicao = requests.get(f'{link}/vendas/.json')
35 print(requisicao)
36 dic_requisicao = requisicao.json()
37 id_alon = None
38 for id_venda in dic_requisicao:
39     cliente = dic_requisicao[id_venda]['cliente']
40     if cliente == "alon":
41         print(id_venda)
42         id_alon = id_venda
43
44 # Deletar uma venda (DELETE)
45 requisicao = requests.delete(f'{link}/vendas/{id_alon}/.json')
46 print(requisicao)
47 print(requisicao.text)

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS COMMENTS

Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/AppData/Local/Programs/Python/Python314/python.exe c:/Us
<Response [200]>
-OjpanMwCLa0w09gLUNc

C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/AppData/Local/Programs/Python/Python314/python.exe c:/Us
<Response [200]>
-OjpanMwCLa0w09gLUNc
<Response [200]>
null

C:\Users\Manfredi\Desktop\Codigos>

<https://meuprojetofirebase-f776a-default-rtdb.firebaseio.com>

<https://meuprojetofirebase-f776a-default-rtdb.firebaseio.com/>

▼ — produtos

▼ — -OjpbDPdcvXCuqNMb1TF

— nome: "teclado"

— preco: 150

— quantidade: 80

▼ — PID1

— nome: "fone ouvido"

— preco: 200

— quantidade: 500

▼ — vendas

▼ — -Ojp_OPIjxh_cWfwEOe

— cliente: "lira"

— preco: 150

— produto: "teclado"

▼ — ID1

— cliente: "lira"

— preco: 200

— produto: "fone ouvido"

Com o conhecimento das requisições fundamentais, você pode expandir a estrutura do seu banco de dados para comportar múltiplas categorias, como vendedores, lojas e estoques. A lógica central de um aplicativo robusto reside na interação entre essas diferentes partes: ao registrar uma venda via **post**, o seu código **Python** pode simultaneamente realizar um **patch** na categoria de estoques para subtrair a quantidade de produtos vendidos. Todas essas ações são orquestradas por meio de requisições sequenciais enviadas aos **links** correspondentes em seu banco de dados.

Embora o modo de teste facilite o desenvolvimento inicial, para aplicações reais é indispensável configurar as regras de segurança no console do **Firebase**. Nessas regras, você define quem possui autorização para ler ou editar cada parte da árvore de dados, protegendo as informações contra acessos não autorizados. O uso de autenticação de usuários é um recurso avançado que também opera via requisições, onde o sistema valida um **token** de acesso para garantir a identidade do usuário antes de permitir qualquer operação no banco de dados.

A **RapidAPI** funciona como um hub centralizado que reúne milhares de **APIs** de diversos temas, como finanças, previsão do tempo, esportes e geolocalização, facilitando a integração de serviços externos em seus projetos. Uma **API** é um serviço online que permite ao seu código consultar sistemas externos para obter ou enviar informações, e a vantagem de utilizar um hub é ter um local único para buscar e gerenciar esses diferentes serviços. Cada **API** listada na plataforma possui indicadores de performance, como popularidade, latência e o status de funcionamento (health check).

Ao selecionar uma **API**, como a de previsão do tempo, você tem acesso a uma documentação que inclui exemplos práticos de código em várias linguagens, incluindo **Python** com a biblioteca **requests**. Para integrar o serviço, a **RapidAPI** gera automaticamente os cabeçalhos de segurança, conhecidos como **headers**, que contêm a sua **chave** pessoal (API Key) e o **host** específico daquele serviço. Essas credenciais são fundamentais para que o hub identifique a sua conta e valide as permissões de acesso ao realizar a requisição para a **URL** do **end point**.

The screenshot displays the RapidAPI page for the 'Forecast' API. At the top, the API is identified as 'Forecast' by 'wetter.com', with a 'FREEMIUM' status and a 'Verified' badge. Performance metrics are shown: Popularity (9.8 / 10), Latency (116ms), Service Level (100%), and Health (N/A). Navigation links for 'Endpoints', 'About', 'Tutorials', 'Discussions', and 'Pricing' are present. The main description states: 'Get 16 days weather forecast data - worldwide - geographical coordinates...'. Below this, a paragraph explains the API's history and purpose. A section titled 'V1 (Current)' contains a search bar for endpoints. On the left, a sidebar lists 'API Information' with 'GET Index' and 'GET GetStatus' endpoints, and a 'RapidApi' section. The main content area shows the 'GET Index' endpoint with a description: 'See the current API version. Here is the link to the documentation'. Below this, a 'Personal Account' section for 'Python Impressionador' is visible, showing a 'RapidAPI App' with a 'default-application_8511565' key and a 'REQUIRED' status. On the right, a 'Code Snippets' section shows a Python example using the 'requests' library to call the API, with a 'Copy Code' button.

```
main.py > ...
1 import requests
2
3 url = "https://forecast9.p.rapidapi.com/"
4
5 headers = {
6     "X-RapidAPI-Key": "4ad83ddea8msh757f582db25d70bp1605bf
7     "X-RapidAPI-Host": "forecast9.p.rapidapi.com"
8 }
9
10 response = requests.get(url, headers=headers)
11
12 print(response.json())
```

Muitas **APIs** operam no modelo **Freemium**, o que significa que oferecem um plano gratuito com um limite determinado de requisições diárias e planos pagos para volumes maiores ou recursos adicionais. Antes de realizar testes no seu código, é necessário realizar a inscrição ou **subscribe** no plano desejado dentro da plataforma. Você deve observar os diferentes **end points** disponíveis em cada serviço, pois cada um possui uma **URL** específica para finalidades distintas, como obter um resumo por localização ou verificar o histórico de dados.

Verified ✓

Pricing

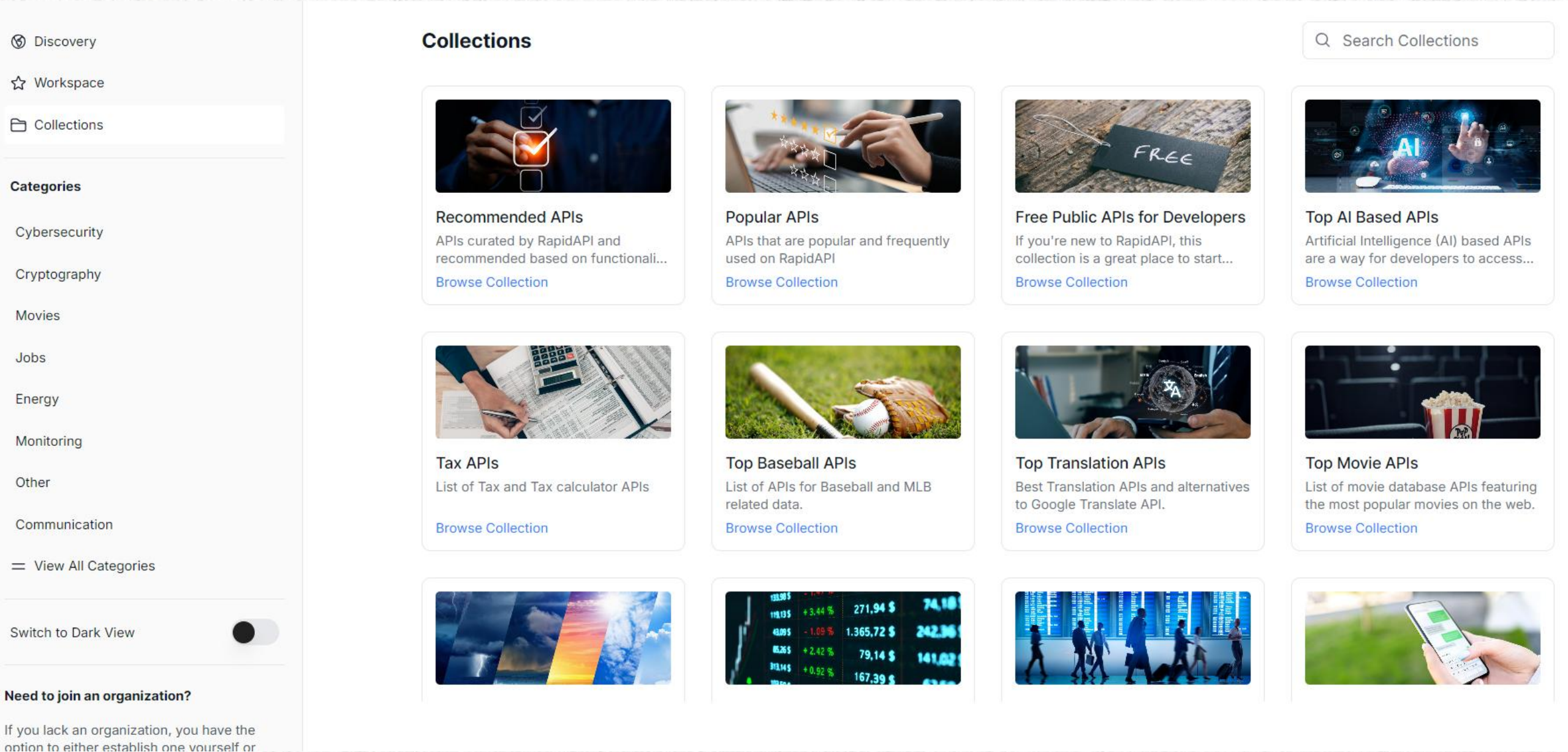
Choose the Right Plan For You

partners directly with API providers to give you no-fuss, transparent pricing. Find a plan that best matches the scale you need for your application.

 Python Impressionador ▾

Basic	Pro	Recommended Ultra	Mega
\$0.00 / mo	\$50.00 / mo	\$200.00 / mo	\$500.00 / mo
Subscribe	Subscribe	Subscribe	Subscribe
10 / day Hard Limit	1.000 / day + US\$ 0,01 each other	10.000 / day + US\$ 0,01 each other	50.000 / day + US\$ 0,001 each other
×	×	✓ Hourly Forecast (14 days)	✓ Hourly Forecast (14 days)

A plataforma permite filtrar buscas por categorias ou coleções de **APIs** públicas e gratuitas para desenvolvedores. Em alguns casos de serviços totalmente gratuitos, você pode testar os **end points** diretamente pelo site antes mesmo de implementar a solução no **Python**. Independentemente do modelo de cobrança, a lógica de implementação permanece padronizada: você copia a estrutura de requisição fornecida pelo hub, ajusta os parâmetros necessários em seu código e utiliza a resposta do sistema para resolver desafios específicos em suas aplicações.





PYTHON

Módulo 34

INTEGRAÇÃO PYTHON PARA FINANÇAS



Atenção!! Este módulo **não** é sobre finanças ou mercado financeiro.

Nosso objetivo aqui é entender como interagir via Python com ferramentas, bases comumente utilizadas no mercado financeiro.

Essencialmente, o que veremos aqui são métodos ou bibliotecas já conhecidas aplicadas a essa realidade.

Dito isso, vamos começar 😊 !

Como sempre vamos iniciar pela importação de bibliotecas que nos permitirão analisar dados de ativos financeiros.

As importações estão destacadas ao lado.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pandas_datareader.data as web
```

Além do nosso bom e velho Pandas, vamos importar também pandas_datareader que nos permite ler dados da web ([doc](#))

Apelido normalmente utilizado

Para essa integração, usaremos o [Yahoo Finance](#).

Por isso, vai ser necessário conhecer a nomenclatura utilizada por esse site.

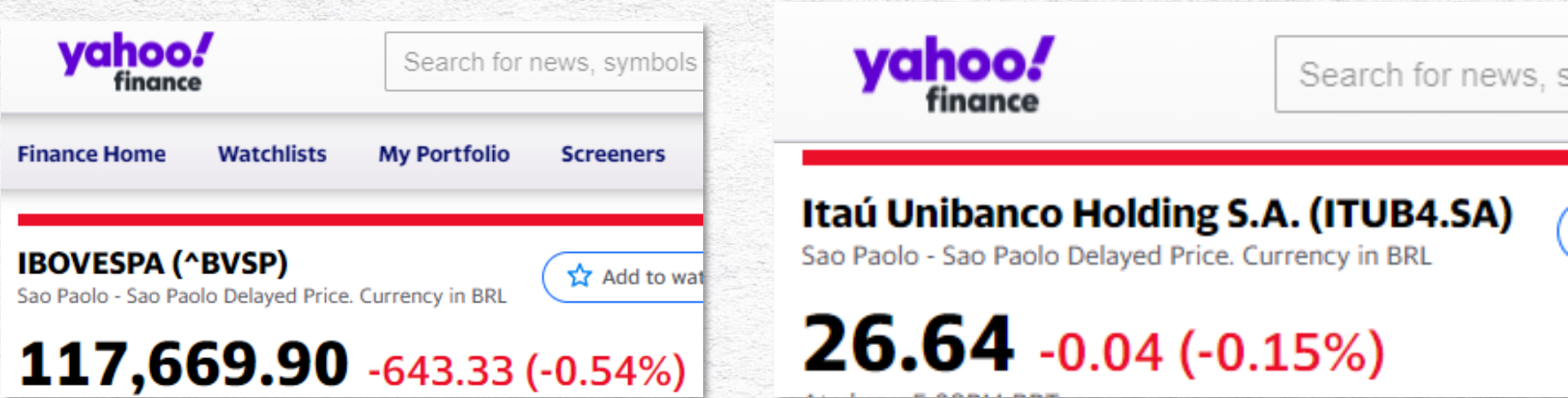
Por exemplo, se queremos saber as cotações da IBOVESPA no Yahoo Finance iremos usar o código ^BVSP.

Outro exemplo, são em casos de ações brasileiras como a ITUB4(nome original usado na Bovespa). No Yahoo Finance, utilizaremos ITUB4.SA.

Sabendo dessas informações, podemos usar nosso primeiro método do pandas_datareader:

.DataReader()

Esse método nos permite buscar os dados de cotação da web do período solicitado conforme apresentado ao lado.



Código do ativo

data_source: indica que será puxada do yahoo

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pandas_datareader.data as web

cotacao_ibov = web.DataReader('^BVSP', data_source='yahoo', start='2020-1-1', end='2020-11-10')
display(cotacao_ibov)
```

start: início do período desejado;
end: fim do período desejado

	High	Low	Open	Close	Volume	Adj Close
Date						
2020-01-02	118573.0	115649.0	115652.0	118573.0	5162700.0	118573.0
2020-01-03	118792.0	117341.0	118564.0	117707.0	6834500.0	117707.0
2020-01-06	117707.0	116269.0	117707.0	116878.0	6570000.0	116878.0

Dados Coletados do Yahoo Finance

Feito essa importação dos dados, podemos trata-los de acordo com nosso interesse. Para isso, usaremos as outras bibliotecas que foram importadas.

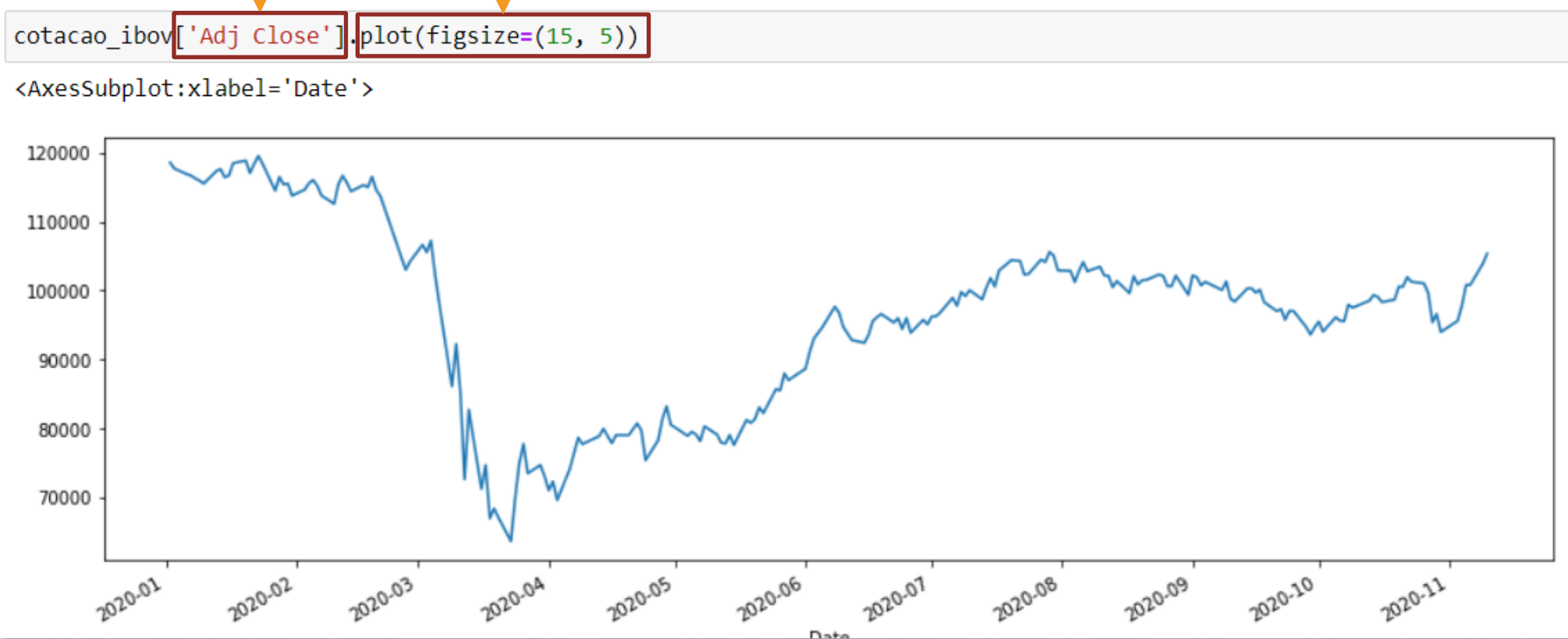
No exemplo ao lado fizemos um gráfico usando o **matplotlib** a partir dos dados da coluna **Adj Close**.

Coluna a ser considerada para o gráfico

Figsize: permite ajustar o tamanho do gráfico criado

Date	High	Low	Open	Close	Volume	Adj Close
2020-01-02	118573.0	115649.0	115652.0	118573.0	5162700.0	118573.0
2020-01-03	118792.0	117341.0	118564.0	117707.0	6834500.0	117707.0
2020-01-06	117707.0	116269.0	117707.0	116878.0	6570000.0	116878.0
2020-01-07	117076.0	115965.0	116872.0	116662.0	4854100.0	116662.0
2020-01-08	117335.0	115693.0	116667.0	116247.0	5910500.0	116247.0
...	...	...	...	...	...	...
2020-11-04	98296.0	95987.0	95992.0	97811.0	10704600.0	97811.0
2020-11-05	100922.0	97872.0	97873.0	100774.0	10455300.0	100774.0
2020-11-06	100928.0	99837.0	100751.0	100799.0	8382800.0	100799.0
2020-11-09	105147.0	100954.0	100954.0	103913.0	17410500.0	103913.0
2020-11-10	105758.0	103453.0	103516.0	105351.0	16665200.0	105351.0

214 rows × 6 columns



Módulo 34 – Integração Python para Finanças – Retorno IBOV e Média Móvel (1/2)

Tendo as informações extraídas no Yahoo Finance, podemos agora realizar diversas análises.

Vamos começar por uma muito comum e bem simples. O cálculo do retorno da Bovespa para o período definido anteriormente.

Como queremos o retorno do período, vamos calcular o ganho percentual entre a cotação do último dia do período e o primeiro dia do período. O cálculo do retorno será dado por:

$$\text{Retorno} = \frac{\text{Última cotação do período}}{\text{Primeira cotação do período}} - 1$$

Para acessarmos a **última cotação** usaremos a posição **[-1]** da coluna **['Adj Close']**, já para a primeira cotação do período, usaremos a posição **[0]**.

Date	High	Low	Open	Close	Volume	Adj Close
2020-01-02	118573.0	115649.0	115652.0	118573.0	5162700.0	118573.0
2020-01-03	118792.0	117341.0	118564.0	117707.0	6834500.0	117707.0
2020-01-06	117707.0	116269.0	117707.0	116878.0	6570000.0	116878.0
2020-11-06	100928.0	99837.0	100751.0	100799.0	8382800.0	100799.0
2020-11-09	105147.0	100954.0	100954.0	103913.0	17410500.0	103913.0
2020-11-10	105758.0	103453.0	103516.0	105351.0	16665200.0	105351.0

Posição [0] - Primeira posição

Posição [-1] - Última posição

```

retorno_ibov = cotacao_ibov['Adj Close'][-1] / cotacao_ibov['Adj Close'][0] - 1
print('Retorno de {:.2%}'.format(retorno_ibov))

Retorno de -11.15%

```

Módulo 34 – Integração Python para Finanças – Retorno IBOV e Média Móvel (2/2)

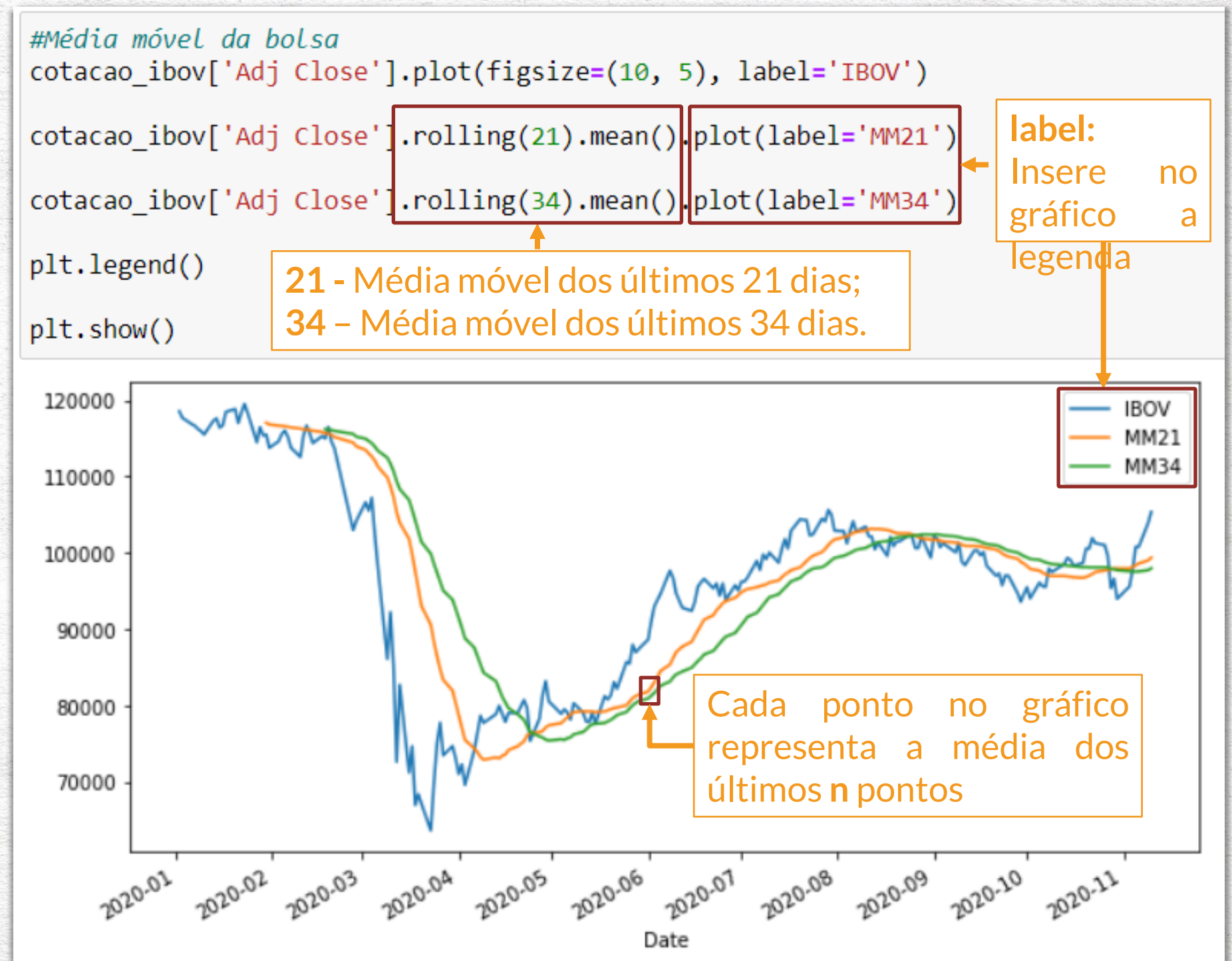
Calculado o retorno, vamos para outra análise muito comum pelos investidores de bolsa de valores. A Média Móvel.

Para isso, usaremos o Pandas. Ao invés de definirmos uma fórmula, usaremos dois métodos já existentes dentro dessa biblioteca:

rolling().mean()

Ela nos permitirá definir as médias apenas informando o número de pontos que desejamos. Nesse caso, como **cada ponto representa um dia**, ao definirmos 21 no método rolling, estamos definindo 21 dias.

Veja o exemplo ao lado, além do Pandas também utilizamos o Matplotlib para plotarmos no gráfico as médias móveis definidas por nós.



Para esse exemplo vamos usar uma carteira de investimentos “Carteira do Lira” (obviamente não é a real 😊)

Nosso objetivo é criar um dataframe de todos os ativos da nossa carteira com as cotações do período indicado.

Usaremos um pouco do conhecimento adquirido anteriormente nesse módulo, mas vamos precisar adicionar alguns outros elementos.

Para que o entendimento, fique mais fácil vamos dividir a explicação em X partes:

Passo 1 : Importar as bibliotecas e os dados da carteira contidos no arquivo “Carteira.xlsx”;

Passo 2 : Criar um dataframe vazio que receberá os ativos e as cotações “puxadas” do Yahoo Finance;

Passo 3 : Definir linha de código de preenchimento de apenas 1 dos ativos no Dataframe;

Passo 4: Usando o FOR, criar um código que preencha todos os ativos no dataframe criado.

	BOVA11	SMAL11	MGLU3	BBDC4	ITUB4	ENEV3	MOVI3	BPAC11	GNDI3	NTCO3	BCRI11	VILG11	KNRI11	
Date														
2020-01-02	114.239998	139.500000	12.010077	37.109173	37.184467	43.860001	19.449736	74.693604	71.413658	38.330002	126.190002	149.000000	199.600006	14
2020-01-03	113.800003	140.899994	11.902953	37.126244	36.793362	44.270000	19.966278	74.910362	72.749046	41.000000	127.699997	147.000000	199.600006	14
2020-01-06	112.589996	139.100006	11.912691	36.463108	36.245815	45.400002	20.040001	74.801979	69.749390	40.130001	126.699997	148.559998	197.979996	14
2020-01-07	112.239998	139.399994	11.878607	35.829655	35.404934	45.060001	20.000000	75.100418	67.367607	40.779999	126.870003	143.830002	198.750000	14
2020-01-08	111.949997	138.199997	12.243802	35.275387	34.828045	44.849998	20.100000	74.434845	67.766235	40.990002	124.570000	139.470001	187.500000	14
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2020-11-04	94.419998	112.459999	26.190001	20.680000	25.000000	57.599998	18.540001	75.139999	71.790001	47.669998	107.500000	126.970001	158.380005	12
2020-11-05	96.699997	115.580002	27.450001	21.000000	25.590000	58.709999	19.620001	78.459999	72.720001	50.279999	107.500000	128.380005	158.179993	12
2020-11-06	97.190002	117.510002	27.000000	20.900000	25.670000	59.700001	20.290001	79.320000	72.820000	50.419998	107.489998	129.009995	158.449997	12
2020-11-09	99.769997	120.110001	26.450001	22.940001	27.629999	57.580002	19.290001	78.739998	72.099998	49.150002	107.500000	128.210007	158.149994	12
2020-11-10	101.449997	120.510002	25.219999	24.420000	28.879999	57.169998	19.620001	76.290001	69.360001	48.450001	107.500000	128.520004	159.500000	12

210 rows x 14 columns

Módulo 34 – Integração Python para Finanças – Pegando Cotações Carteira com Python (2/5)

Passo 1: Importar as bibliotecas e os dados da carteira contidos no arquivo “Carteira.xlsx”;

Assim como nos exemplos anteriores deste módulo, usaremos o pandas.reader para a busca de informações no site do Yahoo Finance e o pandas para o tratamento dos dados dentro de um dataframe.

Além disso, usaremos posteriormente o numpy e matplotlib para elaboração de gráficos.

Como já vimos no módulo sobre PANDAS, para importarmos as informações de um Excel, usaremos o método .read_excel().

O exemplo ao lado, apresenta o código em Python para a conclusão do PASSO 1.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pandas_datareader.data as web
```

```
carteira = pd.read_excel('Carteira.xlsx')
display(carteira)
```

Informações importadas do arquivo excel e armazenados no dataframe 'carteira'.

	Ativos	Tipo	Qtde
0	BOVA11	ETF	100
1	SMAL11	ETF	100
2	MGLU3	Ação	1000
3	BBDC4	Ação	100
4	ITUB4	Ação	100
5	ENEV3	Ação	300
6	MOVI3	Ação	100
7	BPAC11	Ação	100
8	GNDI3	Ação	100
9	NTCO3	Ação	100
10	BCRI11	FII	100
11	VILG11	FII	100
12	KNRI11	FII	100
13	XPLG11	FII	100

Passo 2 : Criar um dataframe vazio que receberá os ativos e as cotações “puxadas” do Yahoo Finance;

Esta é a linha menos intuitiva de todas, então vamos avaliá-la por partes.

Se usarmos essencialmente o mesmo código utilizado anteriormente, conseguimos buscar as cotações, mas além disso, também obtemos uma série de informações que não nos interessa por enquanto (high, low, open, close, volume).

De todas as informações apresentadas ao lado, apenas a última coluna nos interessa e além disso, pensando que vamos ter vários ativos no nosso dataframe, será importante renomeá-lo de acordo com o nome do ativo. Nesse caso **‘BOVA11.SA’**.

Vamos entender como fazer essa etapa na próxima página 😊.

```
cotacoes_carteira = pd.DataFrame()
cotacoes_carteira = web.DataReader('BOVA11.SA', data_source='yahoo', start='2020-01-01', end='2020-11-10')
display(cotacoes_carteira)
```

	High	Low	Open	Close	Volume	Adj Close
Date						
2020-01-02	114.239998	112.129997	112.449997	114.239998	5684380.0	114.239998
2020-01-03	114.500000	112.800003	112.930000	113.800003	6602450.0	113.800003
2020-01-06	113.449997	112.019997	113.000000	112.589996	6771940.0	112.589996
2020-01-07	112.900002	111.589996	112.900002	112.239998	6096900.0	112.239998
2020-01-08	113.099998	111.400002	112.650002	111.949997	6472610.0	111.949997
...	...	...	...	...	...	...
2020-11-04	94.629997	92.849998	93.510002	94.160004	12844442.0	94.160004
2020-11-05	97.169998	95.379997	95.769997	96.949997	12692233.0	96.949997
2020-11-06	97.339996	96.089996	96.089996	97.190002	8765724.0	97.190002
2020-11-09	101.150002	99.269997	100.389999	99.500000	19613783.0	99.500000
2020-11-10	101.820000	99.730003	99.930000	101.000000	13893583.0	101.000000

214 rows × 6 columns

Passo 3 : Definir linha de código de preenchimento de apenas 1 dos ativos no Dataframe;

Para facilitar a visualização desta etapa, criamos uma variável **'extracao'** que armazenará o dataframe apresentado anteriormente.

Como vimos, queremos que no nosso dataframe **cotacoes_carteira** exista uma coluna com o nome do ativo **['BOVA11.SA']** mas que apenas a coluna **['Adj Close']** seja utilizada.

O exemplo ao lado, apresenta como realizar esta etapa.



ATENÇÃO!

Por fins didáticos, dividimos essa linha em 2 etapas, mas seria possível realizar essa etapa diretamente sem a necessidade da criação da variável **extracao**.

```
cotacoes_carteira = pd.DataFrame()

extracao = web.DataReader('BOVA11.SA', data_source='yahoo', start='2020-01-01', end='2020-11-10')
cotacoes_carteira['BOVA11.SA'] = extracao['Adj Close']
display(cotacoes_carteira)
```

BOVA11.SA	
Date	
2020-01-02	114.239998
2020-01-03	113.800003
2020-01-06	112.589996
2020-01-07	112.239998
2020-01-08	111.949997
...	...
2020-11-04	94.160004
2020-11-05	96.949997
2020-11-06	97.190002
2020-11-09	99.500000
2020-11-10	101.000000

214 rows × 1 columns

Retira apenas a coluna **['Adj Close']** do dataframe **extracao** criado.

Cria a coluna **[BOVA11.SA]** que receberá os valores das cotações.

Passo 4: Usando o FOR, criar um código que preencha todos os ativos no dataframe criado.

Como temos vários ativos, podemos utilizar o FOR para percorrer todos esses ativos adicionando seus dados ao nosso dataframe `cotações_carteira`.

```
cotacoes_carteira = pd.DataFrame()
for ativo in carteira['Ativos']:
    extracao = web.DataReader('{}SA'.format(ativo), data_source='yahoo', start='2020-01-01', end='2020-11-10')
    cotacoes_carteira[ativo] = extracao['Adj Close']
display(cotacoes_carteira)
```

For permite percorrer toda lista de ativos existentes no dataframe `carteira`

Ao invés de utilizar o nome do ativo, é utilizada a **variável ativo** que possui o nome correspondente àquela iteração.

	BOVA11	SMAL11	MGLU3	BBDC4	ITUB4	ENEV3	MOVI3	BPAC11	GNDI3	NTCO3	BCRI11	
Date												
2020-01-02	114.239998	139.500000	12.038459	33.042130	36.671249	10.9650	19.307058	74.173401	71.413658	38.330002	126.190002	149.
2020-01-03	113.800003	140.899994	11.931082	33.058857	36.285545	11.0675	19.819813	74.388657	72.749046	41.000000	127.699997	147.
2020-01-06	112.589996	139.100006	11.940844	32.468361	35.745567	11.3500	19.892994	74.281029	69.749390	40.130001	126.699997	148.
2020-01-07	112.239998	139.399994	11.906678	31.904306	34.916294	11.2650	19.853287	74.577377	67.367607	40.779999	126.870003	143.
2020-01-08	111.949997	138.199997	12.272737	31.410761	34.347374	11.2125	19.952553	73.916443	67.766235	40.990002	124.570000	139.

Ativo existente na carteira e todas as cotações do período extraídas do Yahoo Finance.

Assim como vimos no módulo do Pandas, é sempre importante **analisarmos a qualidade dos dados** antes de iniciar a **análise dos dados**.

Um método já conhecido por nós que nos permite ter uma visão geral do dados é o método `.info()` conforme apresentado ao lado.

Podemos perceber que aparentemente todos os dados foram trazidos corretamente, com exceção do ativo XPLG11. Nesse caso temos apenas 176 dados não nulos enquanto os outros ativos, 210.

Possivelmente algum problema ocorreu na própria base do Yahoo Finance. Nesse caso, não temos muito o que fazer além de tratarmos esses dados faltantes usando alguma premissa.

```
cotacoes_carteira.info()

<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 210 entries, 2020-01-02 to 2020-11-10
Data columns (total 14 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   BOVA11   210 non-null    float64
 1   SMAL11   210 non-null    float64
 2   MGLU3    210 non-null    float64
 3   BBDC4    210 non-null    float64
 4   ITUB4    210 non-null    float64
 5   ENEV3    210 non-null    float64
 6   MOVI3    210 non-null    float64
 7   BPAC11   210 non-null    float64
 8   GNDI3    210 non-null    float64
 9   NTCO3    210 non-null    float64
10  BCRI11   210 non-null    float64
11  VILG11   210 non-null    float64
12  KNRI11   210 non-null    float64
13  XPLG11   176 non-null    float64
dtypes: float64(14)
memory usage: 24.6 KB
```

Valores faltantes no ativo XPLG11. Era esperado 210 assim como os outros ativos.

Para o ajuste destes dados faltantes, poderíamos usar algumas premissas. Segue abaixo a listagem de algumas mais comuns :

- Excluir dados faltantes;
- Utilizar a média de todos ou uma dos valores;
- Replicar o dado anterior existente mais próximo.

Não existe uma abordagem 100% correta, qualquer uma que venha a ser adotada será uma premissa utilizada e qual utilizar irá variar de projeto para projeto.

No nosso caso, usaremos o valor existente imediatamente anterior ao dado faltante. Ou seja, O valor da cotação do dia anterior ao do dado faltante.

Para isso, usaremos o método `.ffill()` que nos permite preencher os espaços vazios com os dados existentes na linha anterior.

```
cotacoes_carteira = cotacoes_carteira.ffill()
cotacoes_carteira.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 210 entries, 2020-01-02 to 2020-11-10
Data columns (total 14 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   BOVA11   210 non-null    float64
 1   SMAL11   210 non-null    float64
 2   MGLU3    210 non-null    float64
 3   BBDC4    210 non-null    float64
 4   ITUB4    210 non-null    float64
 5   ENEV3    210 non-null    float64
 6   MOV13    210 non-null    float64
 7   BPAC11   210 non-null    float64
 8   GNDI3    210 non-null    float64
 9   NTCO3    210 non-null    float64
10  BCRI11   210 non-null    float64
11  VILG11   210 non-null    float64
12  KNRI11   210 non-null    float64
13  XPLG11   210 non-null    float64
dtypes: float64(14)
memory usage: 24.6 KB
```

Apesar de não serem dados reais, a premissa nos permite igualar a quantidade de dados existentes.

Para conseguirmos analisar nossos ativos em um mesmo gráfico é interessante normalizarmos os valores.

O que isso significa? Vamos considerar o primeiro valor conhecido da nossa base de ativos como a linha de base. Todas as variações (positivas ou negativas) serão referencias a essa linha de base. Isso nos permitirá avaliar a evolução de todos os ativos em um mesmo gráfico independentemente do seus valores serem, em valores absolutos, muito distintos.

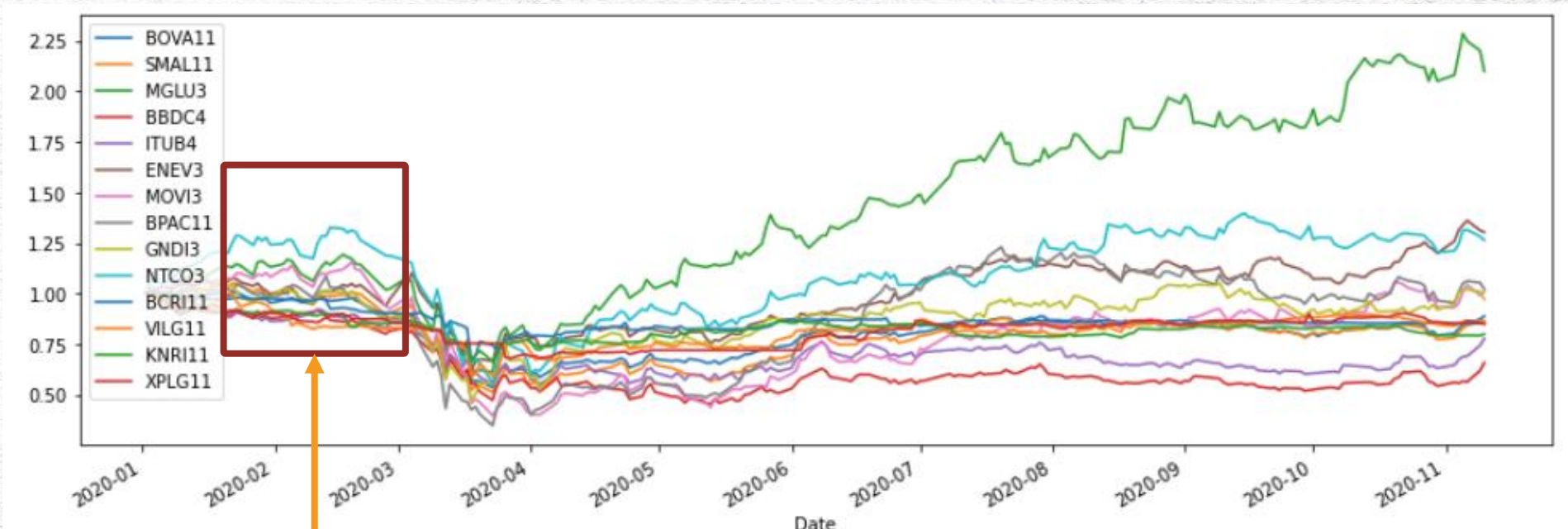
Como normalizar? Considerando nosso caso, vamos criar um novo dataframe que será:

$$\frac{\text{Valor da cotação do dia}}{\text{Valor da cotação do 1º dia da base de dados}}$$

Chamaremos esse novo dataframe de **carteira_norm** conforme apresentado ao lado.

```
carteira_norm = cotacoes_carteira / cotacoes_carteira.iloc[0]  
carteira_norm.plot(figsize=(15, 5))  
plt.legend(loc='upper left')
```

Cotação do 1º dia. O `iloc[0]` significa que estamos pegando a primeira linha do nosso dataframe



Os dados normalizados nos permitem ver a evolução em % dos ativos. Perceba que todos valores se iniciam com valor no eixo $y = 1$.

Sem a normalização a diferença entre valores absoluto dificultaria muito a análise conjunta em um único gráfico

A primeira parte é a mesma que já fizemos anteriormente. Puxar do Yahoo Finance usando o `pandas.reader`(definido como `web` nesse exemplo) para realizar a importação dos dados.

Nosso objetivo é comparar a rentabilidade entre a Bovespa e nossa carteira. Já aprendemos como calcular essa rentabilidade anteriormente, mas consideramos que todos os ativos possuíam a mesma quantidade, o que dificilmente é realidade em um caso real.

Portanto, vamos precisar incorporar a quantidade dos ativos a nossa análise.

Para isso, vamos criar um novo dataframe chamado **valor_investido** conforme apresentado ao lado, onde a quantidade do ativo apresentada no dataframe **carteira** é multiplicada pelo valor da cotação do ativo no dataframe **cotacoes_carteira**.

Vamos entender um pouco melhor este código na próxima página

```
cotacao_ibov = web.DataReader('^BVSP', data_source='yahoo', start='2020-01-01', end='2020-11-10')
display(cotacao_ibov)
```

	High	Low	Open	Close	Volume	Adj Close
Date						
2020-01-02	118573.0	115649.0	115652.0	118573.0	5162700.0	118573.0
2020-01-03	118792.0	117341.0	118564.0	117707.0	6834500.0	117707.0
2020-01-06	117707.0	116269.0	117707.0	116878.0	6570000.0	116878.0
2020-01-07	117076.0	115965.0	116872.0	116662.0	4854100.0	116662.0
2020-01-08	117335.0	115693.0	116667.0	116247.0	5910500.0	116247.0
...	...	...	...	...	...	...
2020-11-04	98296.0	95987.0	95992.0	97811.0	10704600.0	97811.0
2020-11-05	100922.0	97872.0	97873.0	100774.0	10455300.0	100774.0

Vamos utilizar a mesma estrutura de for utilizada anteriormente para “preencher” nosso novo dataframe **valor_investido**. A parte que é menos intuitiva nesse caso é como buscar as informações do nosso dataframe **carteira**. Como podemos ver no extrato apresentado abaixo, a orientação do nosso dataframe é diferente. Os ativos não são colunas e sim parte dos dados que são apresentados em linhas.

Assim, usaremos o **.loc** para acessar os dados. A informação desejada, SEMPRE, estará na coluna **Qtde**, e o que nos permite chegar no valor certo da coluna e o **nome do ativo**. Assim, usaremos a estrutura abaixo para acessar os dados.

```
valor_investido = pd.DataFrame()

for ativo in carteira['Ativos']:
    valor_investido[ativo] = cotacoes_carteira[ativo] * carteira.loc[carteira['Ativos']==ativo, 'Qtde'].values[0]
display(valor_investido)
```

carteira: nome do dataframe;
carteira['Ativos']==ativo: o valor da coluna 'Ativos' do df carteira que for IGUAL ao valor de ativo nesta iteração do for;
'Qtde': Coluna que desejamos
.values[0] = indica que queremos o valor do df conforme as condições dos itens anteriores.

	Ativos	Tipo	Qtde
0	BOVA11	ETF	100
1	SMAL11	ETF	100
2	MGLU3	Ação	1000
3	BBDC4	Ação	100

	BOVA11	SMAL11	MGLU3	BBDC4	ITUB4	ENEV3	MOVI3	BPAC11	GNDI3	NTCO3	BCRI11
Date											
2020-01-02	11423.999786	13950.000000	12010.077477	3710.917282	3718.446732	13158.000183	1944.973564	7469.360352	7141.365814	3833.000183	12619.000244

Valores de cotação multiplicados pela quantidade do ativo

Agora que temos o valor investido por carteira, vamos descobrir o valor total da carteira.

Para isso, vamos criar uma coluna de **TOTAL** no nosso dataframe **valor_investido**.

Vamos calculá-la utilizando o método `.sum()`. No entanto, esse método por padrão soma colunas e não linhas... Vamos precisar fornecer um argumento para especial para nosso método que nos permita somar as linhas que representarão o valor total da carteira/dia.

Esse argumento será **axis=1**.

Criação de uma coluna ['Total'] no dataframe **valor_investido**

```
valor_investido['Total'] = valor_investido.sum(axis=1)
```

axis=1 permite somar linhas ao invés de colunas (axis=0) que é a configuração padrão do método `sum()`.

Criada nossa coluna Total, podemos replicar os conhecimentos das páginas anteriores realizando as etapas abaixo:

- 1) Normalizar valores para plotagem dos gráficos;
- 2) Plotar gráfico acrescentando as **labels** necessárias para identificação;
- 3) Utilizando a fórmula abaixo, calcular o retorno:

$$\text{Retorno} = \frac{\text{Última cotação do período}}{\text{Primeira cotação do período}} - 1$$

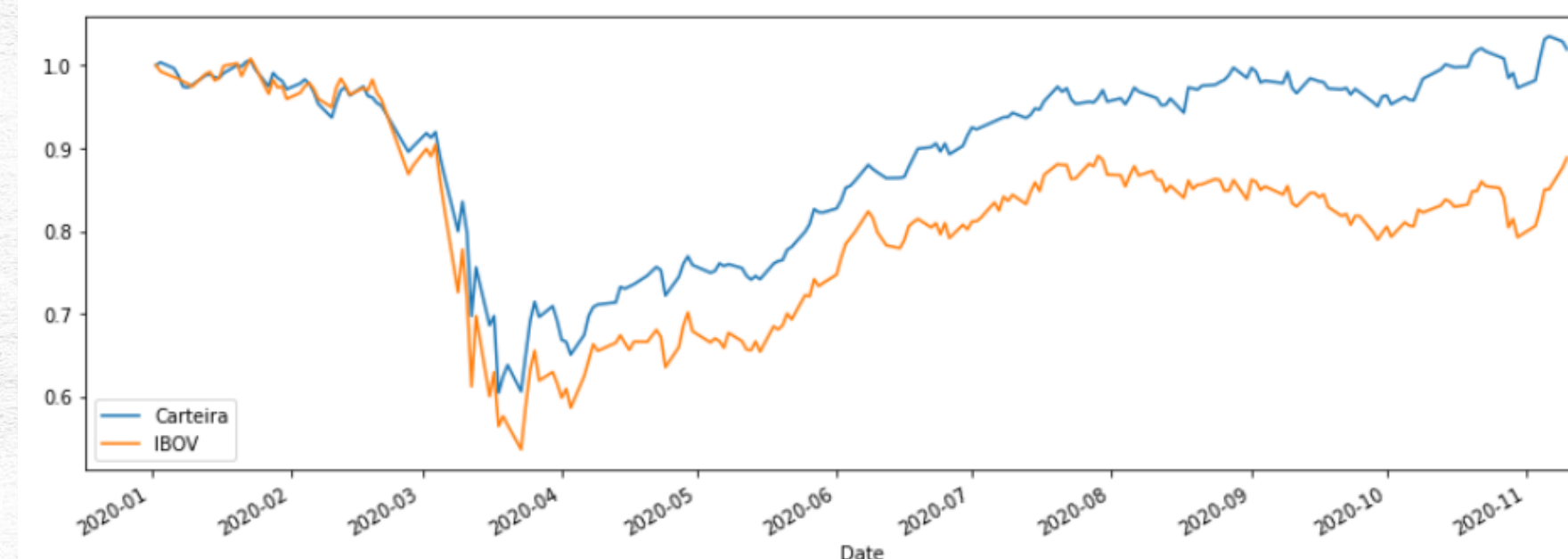
Após a realização das etapas acima teremos o gráfico apresentado ao lado assim como os retornos indicados na parte inferior.

Podemos perceber que os gráficos parecem relacionados e vamos aproveitar isso para aprender mais um método do PANDAS, o `.corr()`.

```
valor_investido['Total'] = valor_investido.sum(axis=1)

valor_investido_norm = valor_investido / valor_investido.iloc[0]
cotacao_ibov_norm = cotacao_ibov / cotacao_ibov.iloc[0]

valor_investido_norm['Total'].plot(figsize=(15, 5), label='Carteira')
cotacao_ibov_norm['Adj Close'].plot(label='IBOV')
plt.legend()
plt.show()
```



```
retorno_carteira = valor_investido['Total'][-1] / valor_investido['Total'][0] - 1
retorno_ibov = cotacao_ibov['Adj Close'][-1] / cotacao_ibov['Adj Close'][0] - 1
print('Retorno da Carteira: {:.2%}'.format(retorno_carteira))
print('Retorno IBOV: {:.2%}'.format(retorno_ibov))
```

Retorno da Carteira: 1.93%
Retorno IBOV: -11.15%

Retorno da carteira e Bovespa para o período estudado.

Os valores de correlação, variam sempre entre -1 e 1.

Valores próximos de 1 indicam que existe uma relação forte e diretamente proporcional, ou seja, se um índice sobe, o outro também sobe.

Valores próximos de -1 indicam que existe uma relação forte mas inversamente proporcional, ou seja, se um índice sobe, o outro desce.

Na imagem ao lado apresentamos a correlação alcançada no nosso exemplo utilizando o método `.corr()`.

Como já era esperado pela nossa análise gráfica, podemos ver que esta correlação é bastante forte e de proporcionalidade positiva.

Compara a coluna 'Adj Close' com a variação do valor de fechamento da carteira calculado na coluna 'Total'

```
correlacao = valor_investido['Total'].corr(cotacao_ibov['Adj Close'])  
print(correlacao)  
0.8879301020686045
```



PYTHON

Módulo 35

AMBIENTES VIRTUAIS



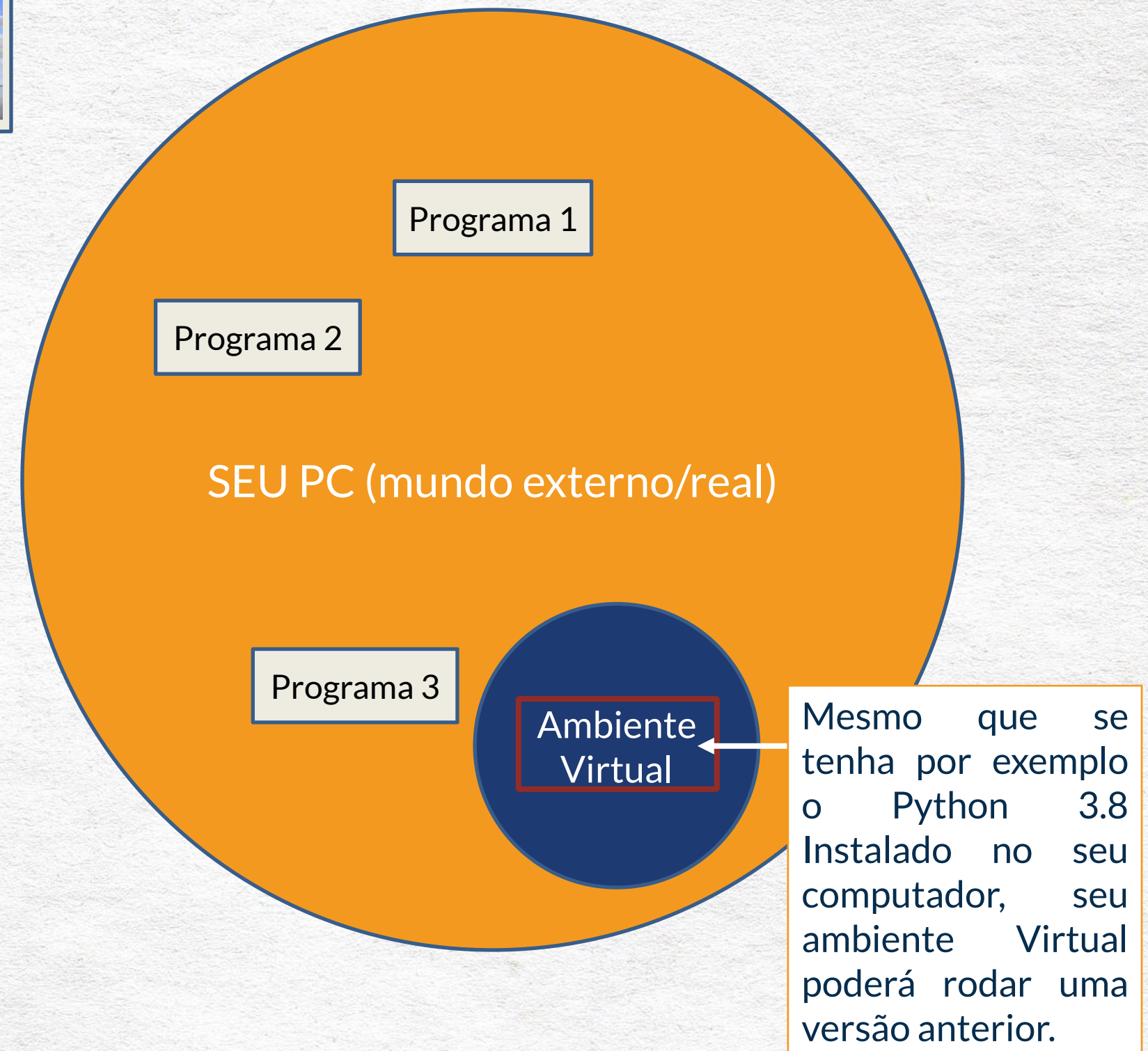
Caso você goste de filmes/séries(**SPOILER ALERT**), imagine um ambiente virtual como o mundo do [Show de Truman](#) ou a cidade criada pela Wanda de [WandaVision](#).



O que acontece nesses casos é que são criadas bolhas dentro do mundo real. Lá, as regras são outras, as condutas são outras sem a interferência do que está fora da bolha (mundo real).

O que isso significa em termos de programação e Python?

O ambiente virtual quando criado, não recebe por definição os programas instalados do seu computador(mundo externo), dentro desse ambiente, você poderá instalar outros programas, bibliotecas, até mesmo versões mais antigas do Python que só existirão dentro daquele ambiente.

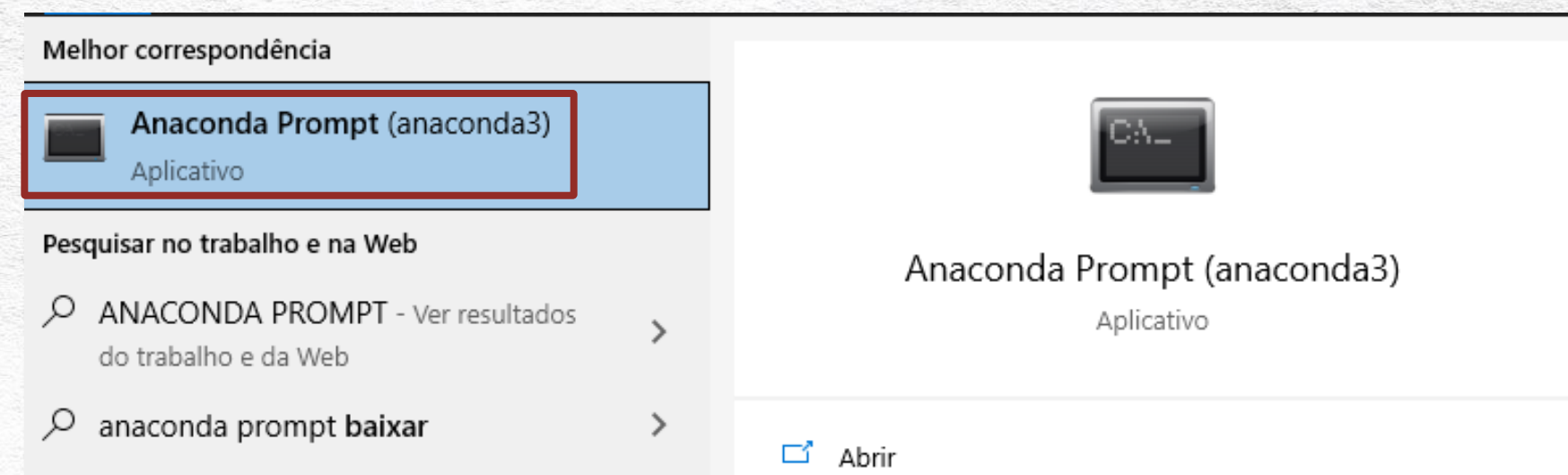


Imagino que esteja pensando... OK... E daí? Para que serve, de que se alimenta e no fim das contas como que eu consigo usar esse tal de Ambiente Virtual?

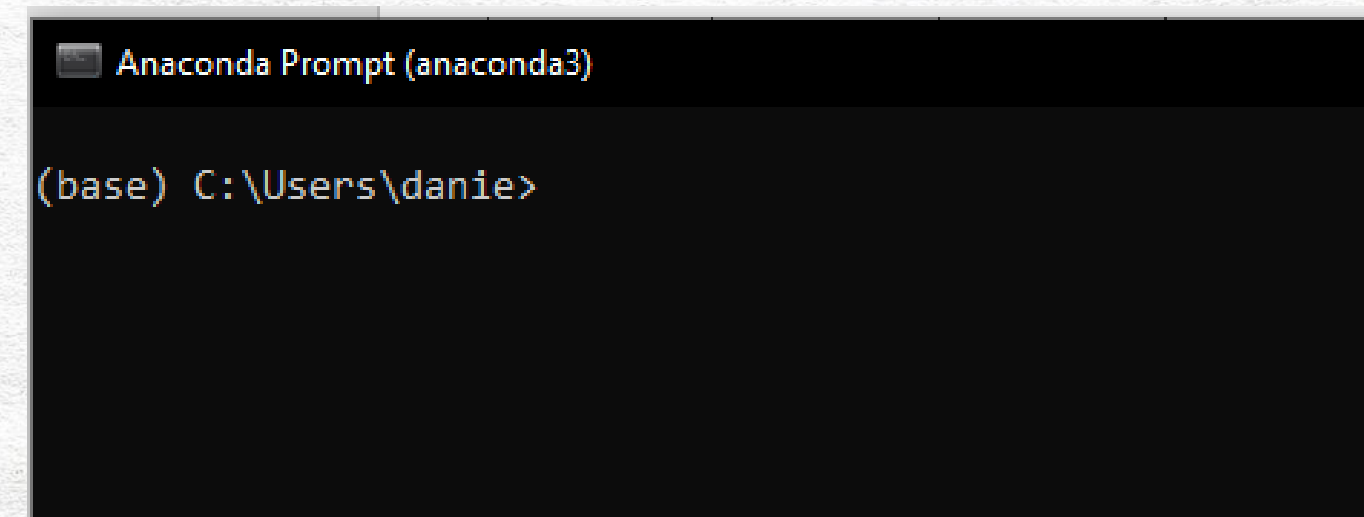
Tudo se inicia no **PROMPT DE COMANDO** talvez você não se lembre, mas já acessamos o prompt outras vezes durante o curso... É aquela tela preta cheia de texto que as vezes usamos para instalar uma biblioteca através do **pip** lembra?

Se não lembra, não tem problema, vamos entender primeiro como abrir o prompt e depois como navegar até conseguirmos criar nosso ambiente virtual.

Para acessar o prompt basta escrever no seu menu iniciar **ANACONDA PROMPT** e clicar no ícone indicado ao lado.



ANACONDA PROMPT



Vamos entender o que estamos vendo no Prompt.

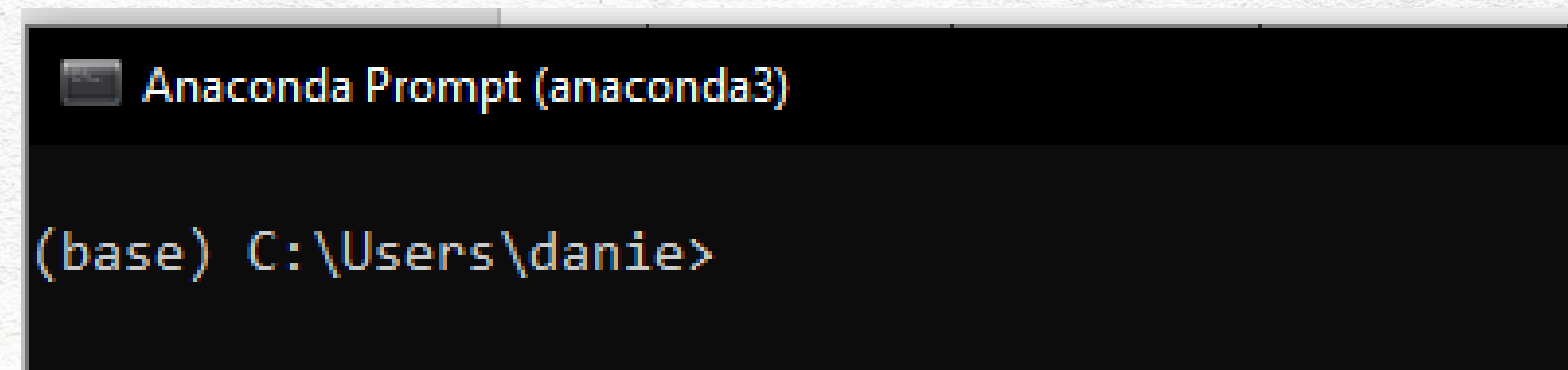
(base) -> Indica que estamos trabalhando no ambiente base do Anaconda que está sendo aplicado a todo o Anaconda;

C:\Users\danie> - > Indica em que pasta do nosso PC estamos.

Agora vamos definir onde gostaríamos de criar nosso ambiente virtual (nossa “bolha”). No nosso caso, vamos criar dentro de uma pasta do nosso curso.

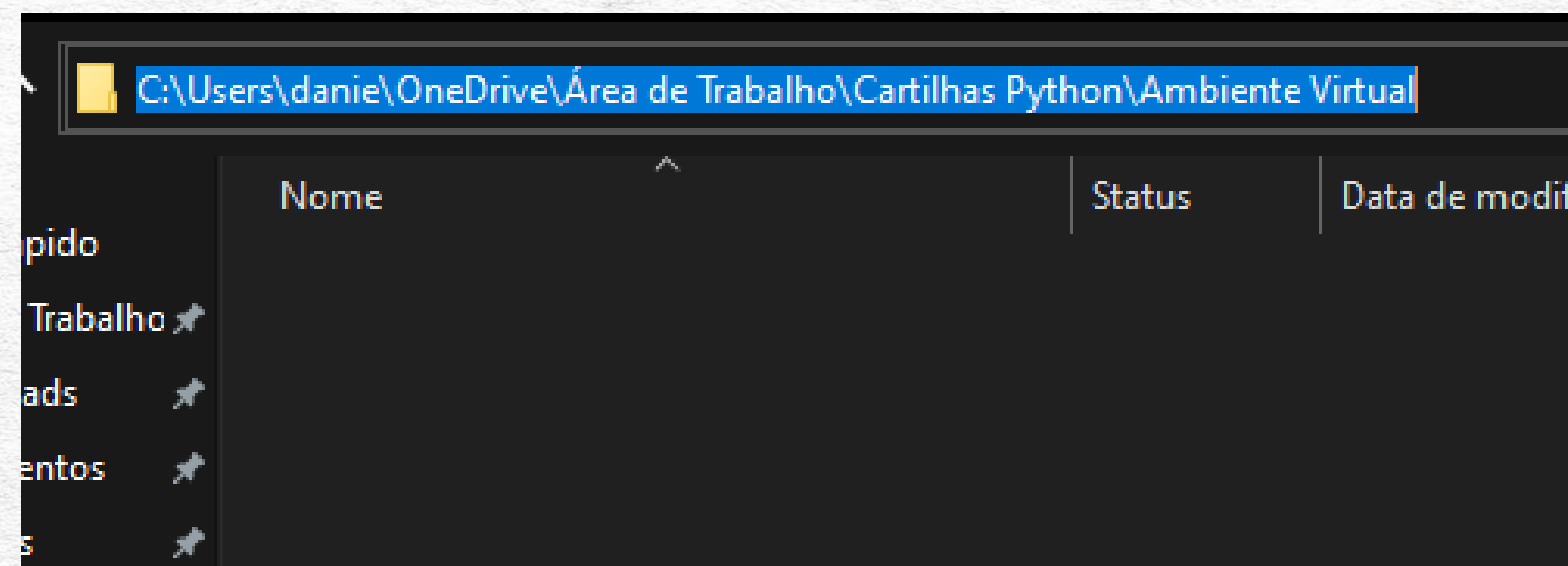
No caso do rapaz que escreve essa apostila, esse ambiente está dentro do caminho assinalado na imagem ao lado. NO ENTANTO, VOCÊ PODERÁ ESCOLHER O LOCAL!

Escolhido o lugar, vamos precisar navegar através do nosso prompt até a pasta escolhida. Para isso, vamos precisar aprender como navegar no prompt. Essa navegação é feita exclusivamente pelo TECLADO através de comandos. É isso que vamos ver na próxima página.



```
Anaconda Prompt (anaconda3)

(base) C:\Users\danie>
```



Módulo 35 – Ambientes Virtuais – Navegando no Prompt de Comando (3/3)

DIR:

- Esse comando nos permite acessar todos os arquivos existentes na pasta onde ele é acionado. Portanto, ao usar **dir** estamos listando tudo que existe na pasta **Cartilhas Python**.

CD (LS para MAC e LINUX):

- Esse comando nos permite abrir pastas do nosso computador. As pastas são identificadas pelo **<DIR>**. Como é de nosso interesse entrar na pasta **Ambiente Virtual**, basta usarmos o comando **cd Ambiente Virtual**.
- O comando **cd..** Permitirá voltar para a pasta mãe da pasta em que se está. Ou seja, se estivermos dentro de **Ambiente Virtual** e usarmos **cd ..**, iremos **VOLTAR** para a pasta **Cartilhas Python**

```
(base) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python>dir
O volume na unidade C é Windows
O Número de Série do Volume é 7234-2414

Pasta de C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python

11/04/2021  16:01    <DIR>          .
11/04/2021  16:01    <DIR>          ..
24/03/2021  13:34    <DIR>          .ipynb_checkpoints
11/04/2021  16:01    <DIR>          Ambiente Virtual
04/04/2021  23:17    <DIR>          Arquivos apostila
23/03/2021  22:48                3.945 Gabarito - 07.07.02 Exercícios Listas.ipynb
23/03/2021  22:41                2.539 Gabarito - Dicionario 06.ipynb
23/03/2021  22:09                2.939 Gabarito - While 01.ipynb
24/03/2021  13:48    <DIR>          Suporte
25/03/2021  01:56                795 Untitled.ipynb
                        4 arquivo(s)          10.218 bytes
                        6 pasta(s) 845.870.264.320 bytes disponíveis
```

```
rea de Trabalho\Cartilhas Python>cd "Ambiente Virtual"
rea de Trabalho\Cartilhas Python\Ambiente Virtual>cd..
rea de Trabalho\Cartilhas Python>cd "Ambiente Virtual"
rea de Trabalho\Cartilhas Python\Ambiente Virtual>
```

Abrindo a pasta Ambiente Virtual

Volta para a pasta acima de Ambiente Virtual (Cartilhas Python)

Abrindo novamente a pasta Ambiente Virtual

Indica que a pasta 'Ambiente Virtual' está ativa e aguardando comando



Vamos agora para a criação do nosso ambiente virtual.

Aqui, vamos focar nos comandos mais usados e logo, mais importantes, mas caso queira se aprofundar sobre o tema, é só clicar aqui na [documentação](#) 😊

Para a criação do nosso ambiente virtual, vamos voltar para o nosso prompt de comando dentro da pasta **Ambiente Virtual** e usar o comando:

conda create -n

No nosso caso, vamos instalar o Python 3.6 não é a versão mais atualizada do Python. Por que faríamos isso? Alguns programas (Ex: ArcGis) por uma questão de atualização, muitas vezes não conseguem ser executados para as versões mais recentes do Python.

Instalando versões mais antigas conseguimos realizar integrações para esses casos.

```
has Python\Ambiente Virtual>conda create -n novoambiente python=3.6
```

conda create: comando de criação de um ambiente virtual;
-n: indica que daremos um nome personalizado a nosso ambiente virtual;
novoambiente: nome do nosso ambiente virtual;
python=3.6: definição de qual versão do Python usaremos dentro desse ambiente virtual.

```
vc          pkgs/  
vs2015_runtime pkgs/  
wheel       pkgs/  
wincertstore pkgs/  
  
Proceed ([y]/n)? y
```

Após o prompt definir o que será instalado, será perguntado se você deseja seguir ou não. Escreva **Y** no seu teclado e aperte **ENTER**

Agora temos nosso ambiente virtual instalado, ,mas como saber se funcionou e já estamos dentro dele?

Se olharmos no nosso prompt ainda estaremos com **(base)** isso significa que ainda não estamos dentro do nosso ambiente.

Para “entrarmos” no ambiente virtual precisamos antes ativá-lo. Para isso, usaremos o comando:

conda activate novoambiente

(base) indica que o ambiente virtual por mais que já tenha sido criado, não foi ativado.

Ativando nosso ambiente virtual

```
(base) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Ambiente Virtual>conda activate novoambiente
```

```
(novoambiente) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Ambiente Virtual>python -V  
Python 3.6.13 :: Anaconda, Inc.
```

(novoambiente) indica que estamos agora **dentro** do ambiente virtual criado.

Ao usarmos o comando **python - V** temos como retorno a versão do python instalada nesse ambiente. Podemos ver, que conforme solicitado durante a criação do ambiente a versão do Python é a 3.6.

Vamos aproveitar para conhecermos outros 2 comandos que são opostos aos que acabamos de aprender:

- **conda deactivate** : desativa o ambiente virtual;
- **conda remove -- name XXX -- all** : deleta o ambiente virtual criado

```
(novoambiente) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Ambiente Virtual>conda deactivate  
(base) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Ambiente Virtual>conda remove --name foo --all
```

Desativando o ambiente virtual

Após a desativação, podemos perceber que voltamos para o ambiente **(base)**

Remoção de um ambiente virtual chamado **foo**

Módulo 35 – Ambientes Virtuais – Observações Importantes de Ambientes Virtuais

Lembram que falamos antes que o ambiente virtual funciona como uma espécie de “Bolha” onde as regras do “mundo externo” não influenciam o ambiente virtual?

Vamos agora entender o que isso significava.

Primeiro vamos abrir o Jupyter Notebook usando o ambiente **(base)**(o “mundo externo”) e depois tentando fazer a mesma ação dentro do ambiente virtual **(novoambiente)**

Por que isso acontece? Ao criarmos o ambiente, apenas instalamos o Python, não instalamos outros programas como o Jupyter... Mesmo que ele exista no mundo externo, ele não acessa esse nosso ambiente virtual. Portanto, precisamos instalar esses programas antes de abri-los 😊

```
(base) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Ambiente Virtual>jupyter notebook
```

A partir da (base) conseguimos acessar o Jupyter Notebook



```
(novoambiente) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Ambiente Virtual>jupyter notebook
'jupyter' não é reconhecido como um comando interno
ou externo, um programa operável ou um arquivo em lotes.
```

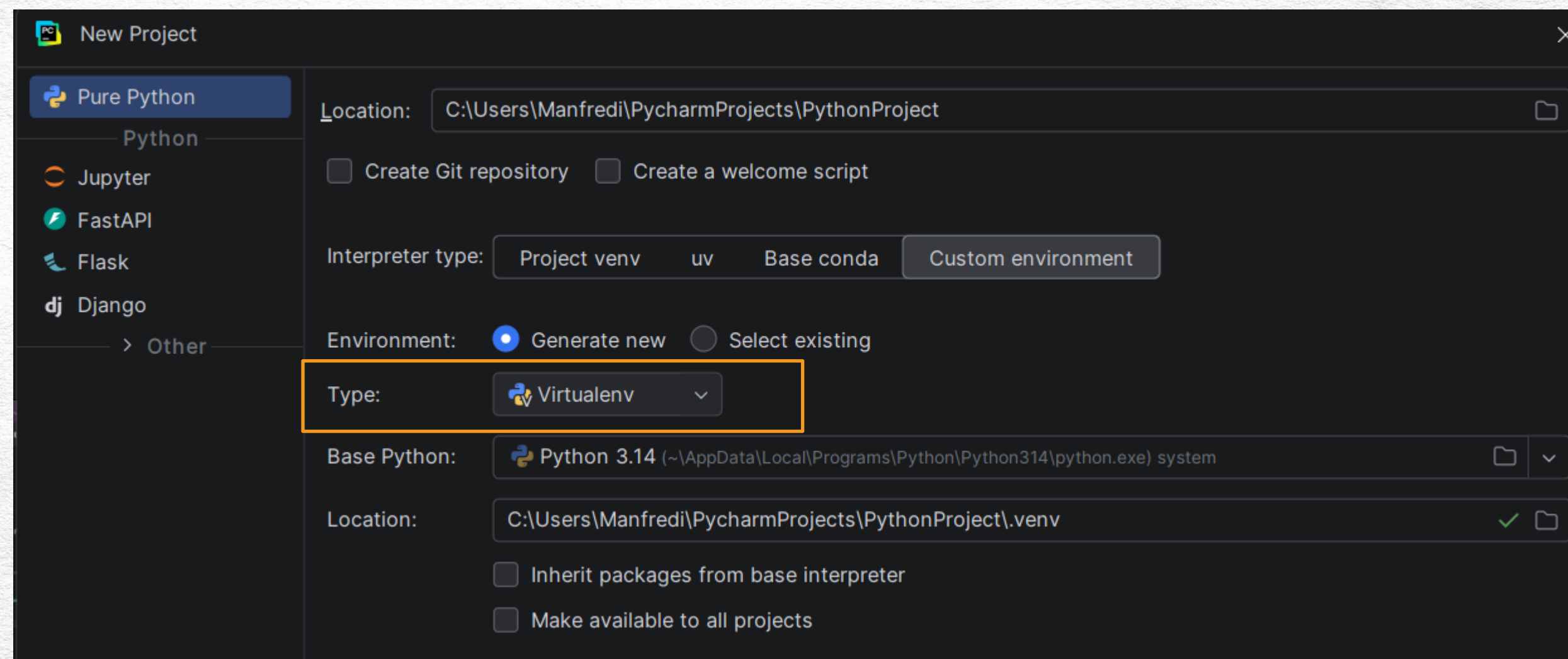
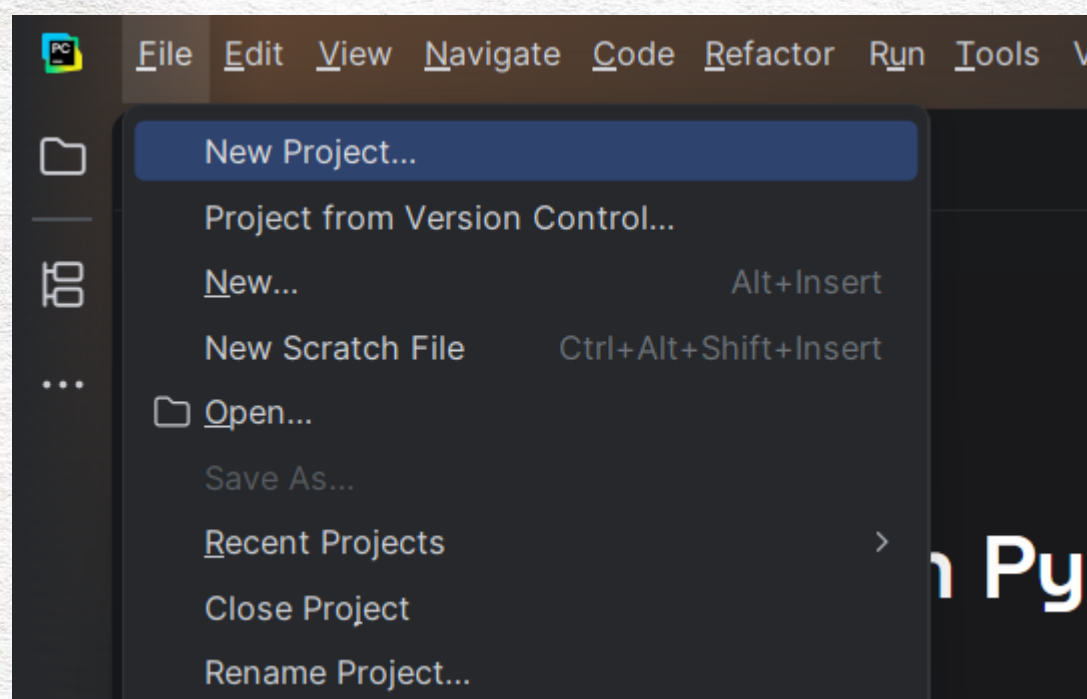
No entanto, não é possível dentro do **(novoambiente)** pois o Jupyter ainda não está instalado.

```
(novoambiente) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Ambiente Virtual>pip install jupyter
```

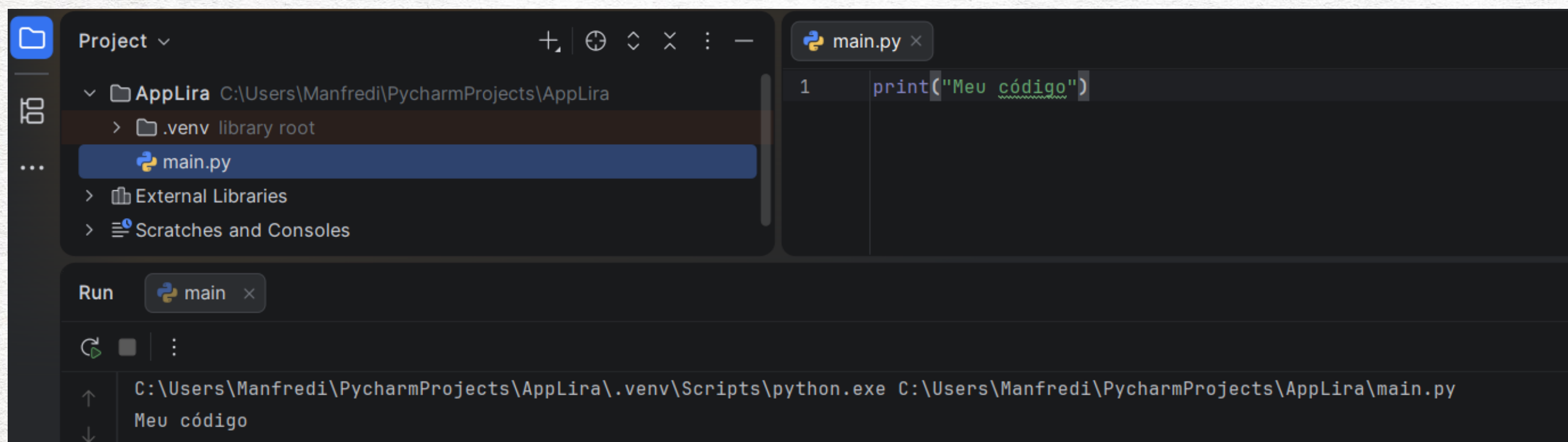
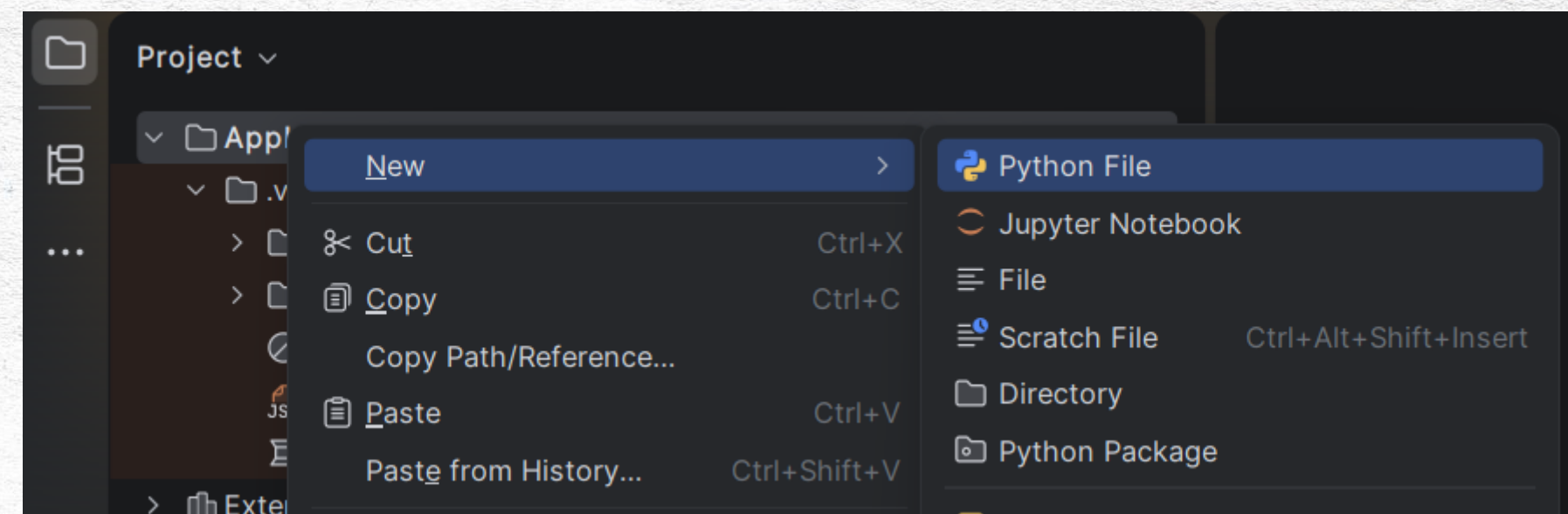
```
(novoambiente) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Ambiente Virtual>jupyter notebook
```

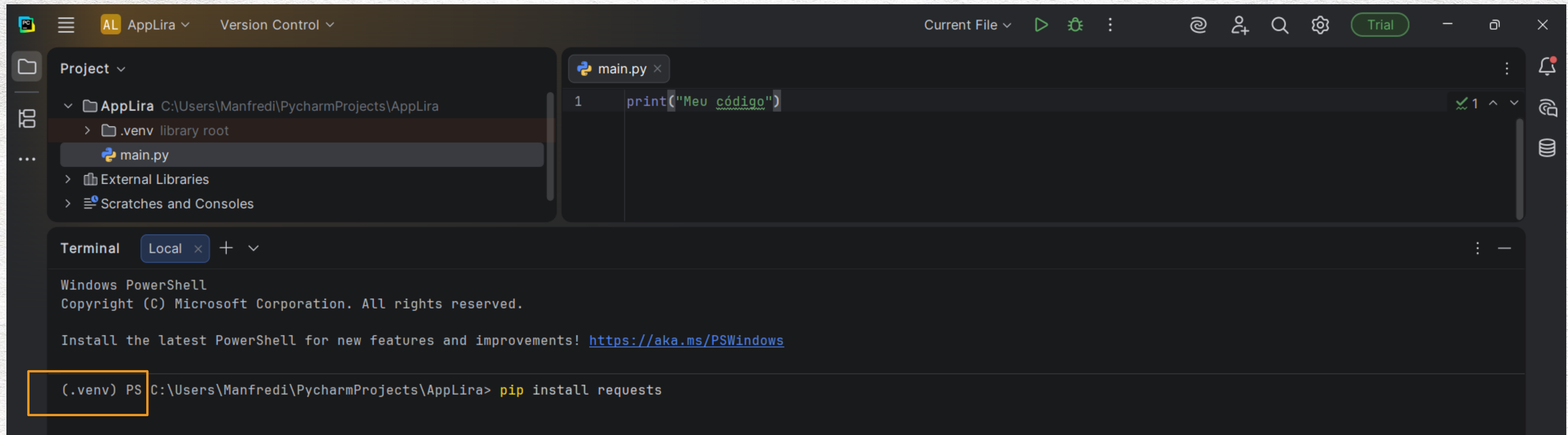


Para criar um ambiente virtual no **PyCharm**, você deve acessar o menu superior em **File** e selecionar a opção **New Project**. Ao definir o local do seu arquivo, a ferramenta apresentará opções de configuração do interpretador, onde você deve optar por **New environment** para que o **Python** base seja utilizado na criação de uma estrutura isolada. Caso existam múltiplas versões do **Python** instaladas em seu computador, é possível selecionar qual delas servirá como base para o novo projeto. O **PyCharm** criará automaticamente uma pasta denominada **venv** dentro do diretório do seu projeto, utilizando a ferramenta **Virtualenv** para a instalação padrão ou o **Conda**, caso você utilize o **Anaconda**. Após definir o nome do projeto e clicar em **Create**, o processo de criação pode levar alguns instantes, pois o sistema está configurando uma instalação independente do **Python** naquela pasta específica.



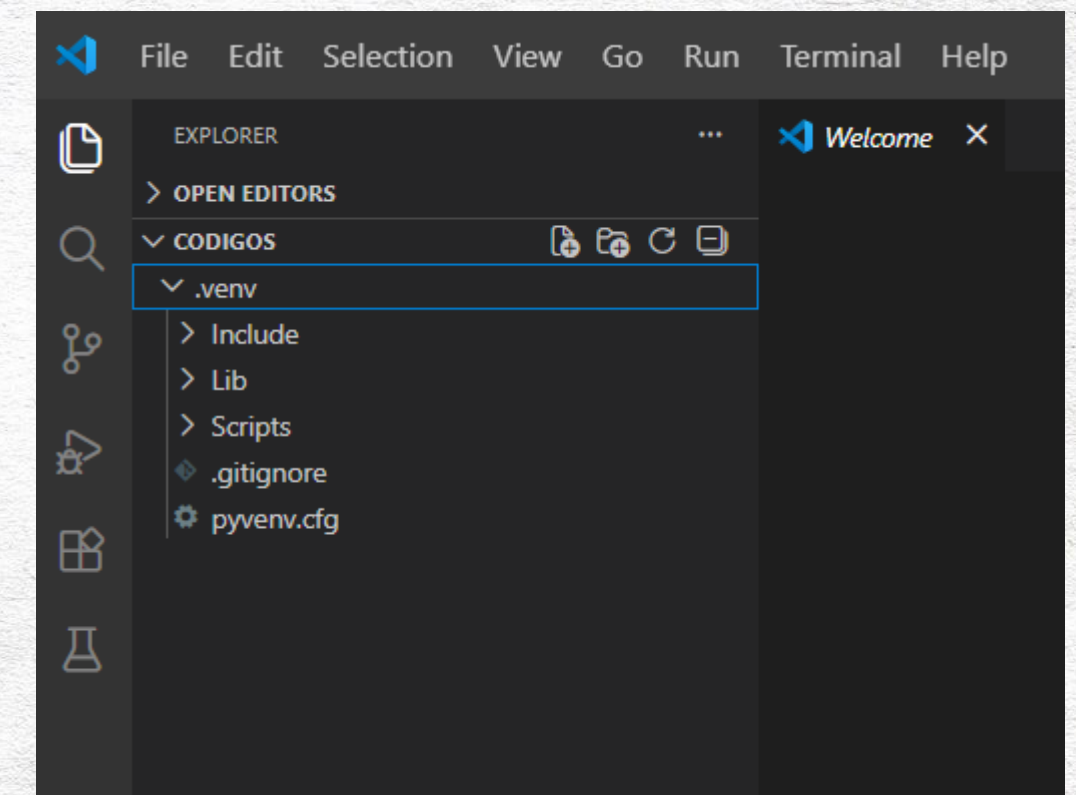
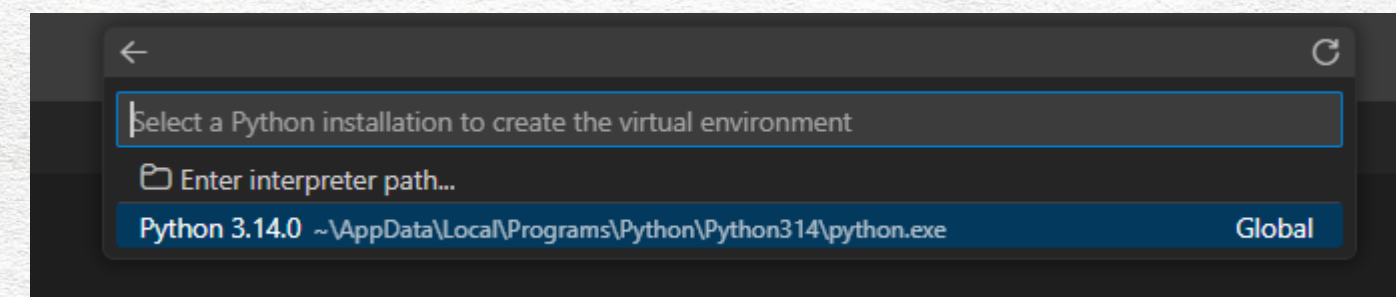
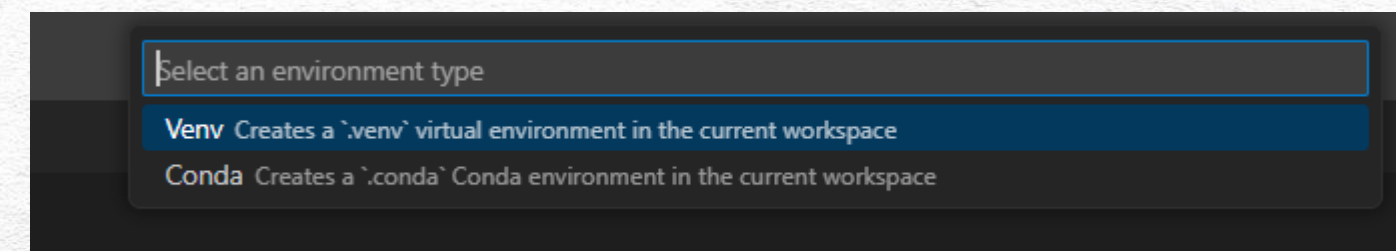
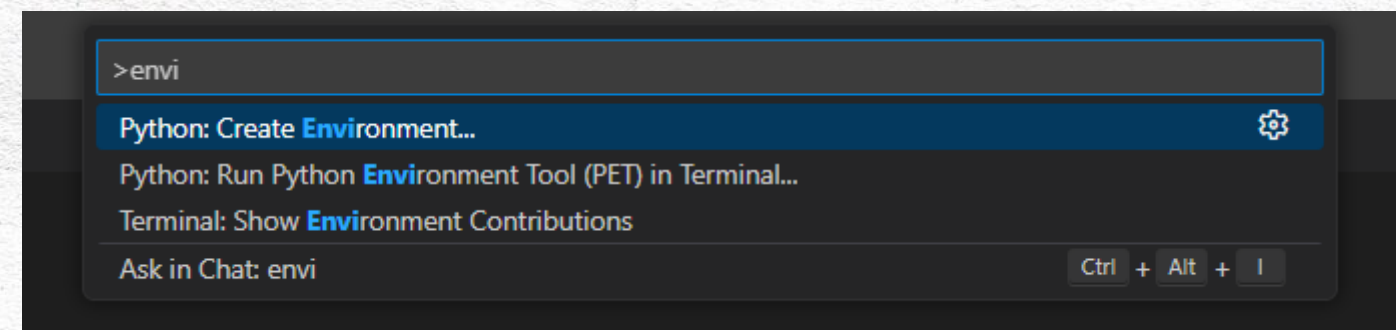
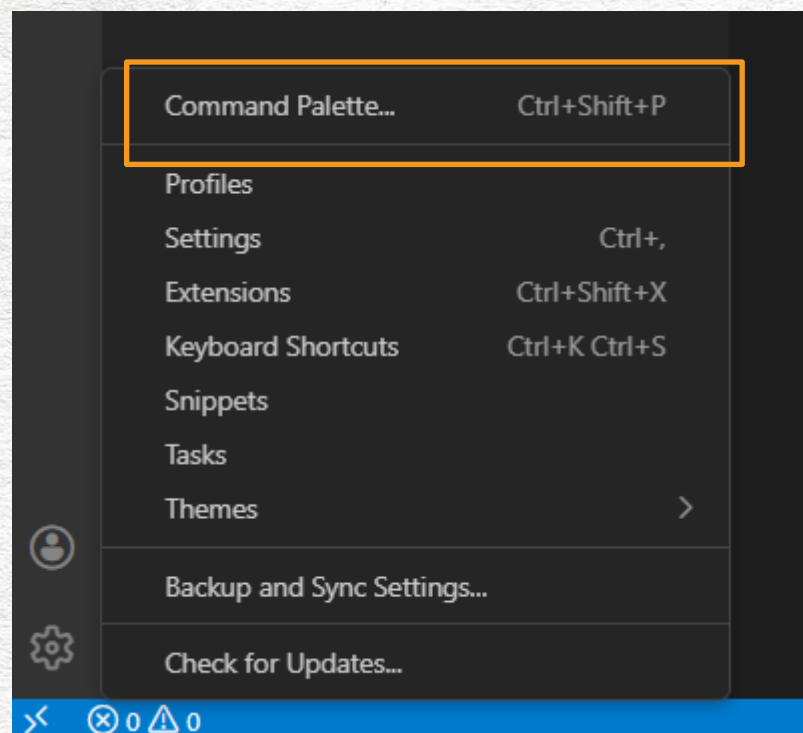
Ao finalizar, você notará a presença da pasta **venv** no navegador de arquivos do projeto. Para testar o ambiente, você pode criar um novo arquivo com a extensão **.py**, como o **main.py**, e executar um comando simples de **print**. Um dos benefícios da integração do **PyCharm** é que, ao clicar no botão de execução, o editor utiliza automaticamente o executável **python.exe** localizado dentro da pasta de scripts do seu ambiente virtual. No entanto, ao realizar a instalação de novas bibliotecas, é fundamental acessar o **Terminal** embutido na parte inferior da interface. Você deve sempre garantir que o prefixo (**venv**) esteja visível na linha de comando antes de prosseguir. Se o prefixo não estiver presente, as bibliotecas instaladas via **pip install** serão direcionadas ao **Python** global do sistema e não ao projeto atual.



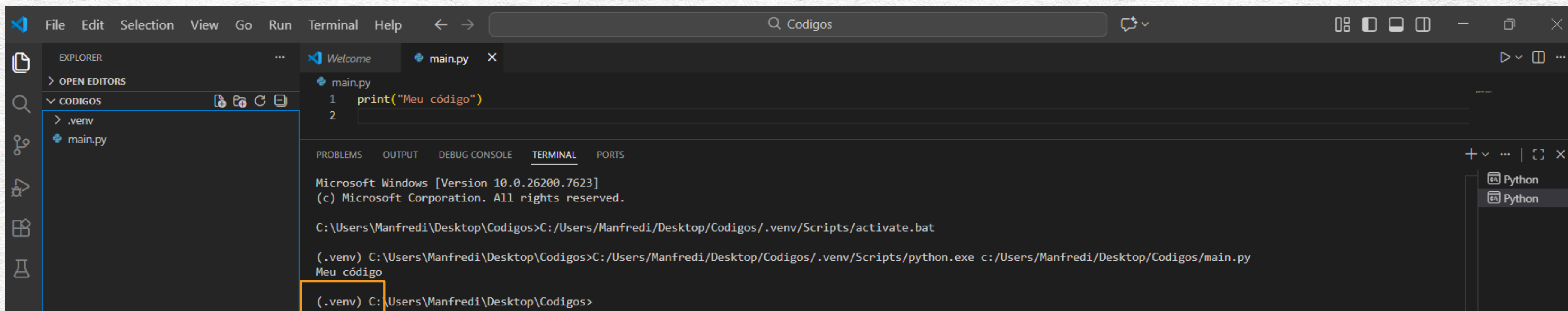


Caso encontre problemas de reconhecimento do comando **pip** ou se a biblioteca não for identificada após a instalação no **PowerShell**, a recomendação técnica é abrir uma nova instância do terminal selecionando o **Command Prompt** (ou **cmd**), que costuma carregar o ambiente virtual de forma mais consistente. Partir de uma configuração limpa criando um novo projeto é sempre preferível a tentar corrigir ambientes virtuais em projetos já existentes que apresentem conflitos. Esse isolamento é essencial para garantir que o projeto contenha apenas as dependências necessárias, tornando-o mais leve e rápido para futuras exportações ou transformações em executáveis.

A criação de ambientes virtuais no **VS Code** é facilitada pelas funcionalidades embutidas na própria **IDE**, dispensando o uso manual excessivo do prompt de comando. Para iniciar o processo, você deve abrir a paleta de comandos utilizando o atalho **Ctrl + Shift + P** ou acessando o ícone de engrenagem em **Manage** e selecionando **Command Palette**. Na barra de busca, ao digitar **environment**, selecione a opção **Python: Create Environment**. O editor permitirá que você escolha entre o uso do **Venv**, que utiliza a instalação padrão do **Python**, ou o **Conda**, caso possua o **Anaconda** configurado. É necessário selecionar a versão base do **Python** que será replicada dentro do ambiente isolado, permitindo a segmentação precisa das bibliotecas por projeto. Durante a criação, uma pasta oculta chamada **.venv** será gerada no diretório raiz do projeto.



Após a conclusão do processo, o **VS Code** sinalizará que o ambiente foi selecionado e você poderá verificar a indicação do interpretador no canto inferior da tela, exibindo o caminho relacionado à pasta **.venv**. Ao executar seu código através do botão de reprodução da interface, o editor invocará automaticamente o interpretador correto. Entretanto, um cuidado especial deve ser tomado ao utilizar o terminal para gerenciar dependências. Muitas vezes, o terminal padrão aberto pode não ativar automaticamente o ambiente virtual, resultando em uma instalação global ao utilizar comandos como **pip install requests**.



The screenshot shows the Visual Studio Code interface with a dark theme. The Explorer sidebar on the left shows a project folder with a subfolder **.venv** and a file **main.py**. The main editor area displays **main.py** with the following code:

```
1 print("Meu código")
2
```

The bottom panel shows the **TERMINAL** tab. The terminal output is as follows:

```
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

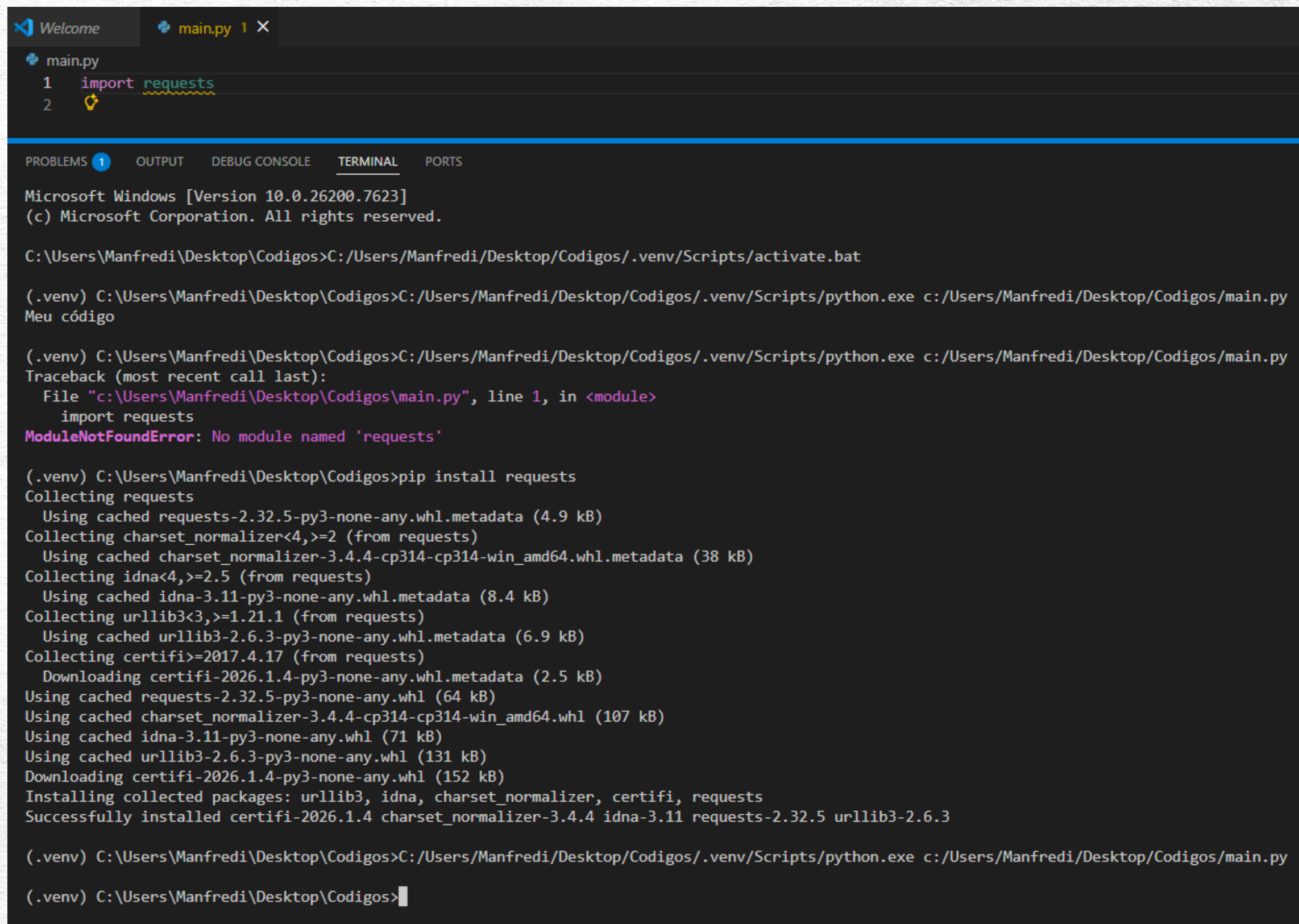
C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/Desktop/Codigos/.venv/Scripts/activate.bat

(.venv) C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/Desktop/Codigos/.venv/Scripts/python.exe c:/Users/Manfredi/Desktop/Codigos/main.py
Meu código

(.venv) C:\Users\Manfredi\Desktop\Codigos>
```

The prompt **(.venv) C:\Users\Manfredi\Desktop\Codigos>** is highlighted with an orange box, indicating that the virtual environment is active. On the right side of the bottom panel, the Python interpreter selection menu shows **Python** selected for the current file.

Se você instalar uma biblioteca e o código ainda assim apresentar um erro de **ModuleNotFoundError**, significa que o pacote não foi inserido no ambiente isolado. A solução definitiva para este comportamento é clicar na seta ao lado do botão de novo terminal e escolher o **Command Prompt**. Ao abrir este terminal específico, o ambiente virtual será carregado automaticamente, sendo identificado pelo prefixo **(.venv)** antes do caminho do diretório. Certificando-se de que o prefixo está presente no **cmd**, as instalações via **pip** funcionarão corretamente e estarão disponíveis para uso imediato no desenvolvimento do seu projeto.



```
main.py
1 import requests
2

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/Desktop/Codigos/.venv/Scripts/activate.bat

(.venv) C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/Desktop/Codigos/.venv/Scripts/python.exe c:/Users/Manfredi/Desktop/Codigos/main.py
Meu código

(.venv) C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/Desktop/Codigos/.venv/Scripts/python.exe c:/Users/Manfredi/Desktop/Codigos/main.py
Traceback (most recent call last):
  File "c:\Users\Manfredi\Desktop\Codigos\main.py", line 1, in <module>
    import requests
ModuleNotFoundError: No module named 'requests'

(.venv) C:\Users\Manfredi\Desktop\Codigos>pip install requests
Collecting requests
  Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Using cached charset_normalizer-3.4.4-cp314-cp314-win_amd64.whl.metadata (38 kB)
Collecting idna<4,>=2.5 (from requests)
  Using cached idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,>=1.21.1 (from requests)
  Using cached urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2017.4.17 (from requests)
  Downloading certifi-2026.1.4-py3-none-any.whl.metadata (2.5 kB)
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
Using cached charset_normalizer-3.4.4-cp314-cp314-win_amd64.whl (107 kB)
Using cached idna-3.11-py3-none-any.whl (71 kB)
Using cached urllib3-2.6.3-py3-none-any.whl (131 kB)
Downloading certifi-2026.1.4-py3-none-any.whl (152 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2026.1.4 charset_normalizer-3.4.4 idna-3.11 requests-2.32.5 urllib3-2.6.3

(.venv) C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/Desktop/Codigos/.venv/Scripts/python.exe c:/Users/Manfredi/Desktop/Codigos/main.py

(.venv) C:\Users\Manfredi\Desktop\Codigos>
```



PYTHON

Módulo 36

INTEGRAÇÃO PYTHON COM POWER BI



Existem 2 formas de integrar o Python com Power BI.

Uma delas é usar todo nosso conhecimento adquirido em tratamento de dados para criar uma automação que nos permita criar base de dados que servirão como base para o Power BI. O que não seria diretamente uma integração e sim uma opção por usar o Power BI para gráficos etc ao invés de usar outras bibliotecas.

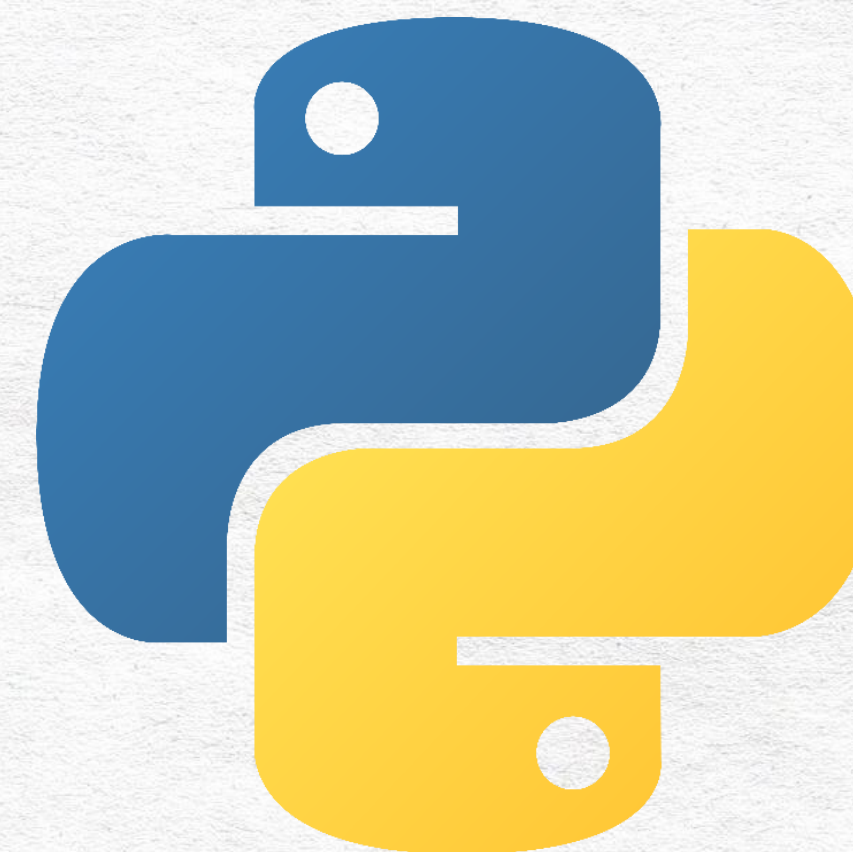
A outra opção, é usarmos o Python **via Power BI**. Para isso, você precisará ter o Power BI instalado no seu computador. Aqui embaixo vamos deixar o link de instalação e um vídeo que explica como instalar 😊.

Link para download:

<https://powerbi.microsoft.com/pt-br/downloads/>

Link de instalação com o MESTRE ALON:

<https://www.youtube.com/watch?v=YO1Tn1z2grE>



Assim como fizemos com o ArcGIS, iremos precisar de um ambiente virtual dedicado a essa integração pois existem(pelo menos no momento da elaboração dessa apostila) alguns *bugs* entre a versão mais recente do Python e o Power BI.

Então, iremos criar uma pasta chamada Python e Power BI onde criaremos um ambiente virtual específico com a versão 3.6 do Python.

Além disso, vamos instalar as bibliotecas que usaremos nessa integração:

- Jupyter Notebook;
- Pandas;
- Matplotlib.

Feito essas importações vamos indicar para o Power BI que queremos usar esse ambiente virtual como ambiente de integração com o Power BI. Em geral, seus ambientes virtuais estarão no caminho abaixo:

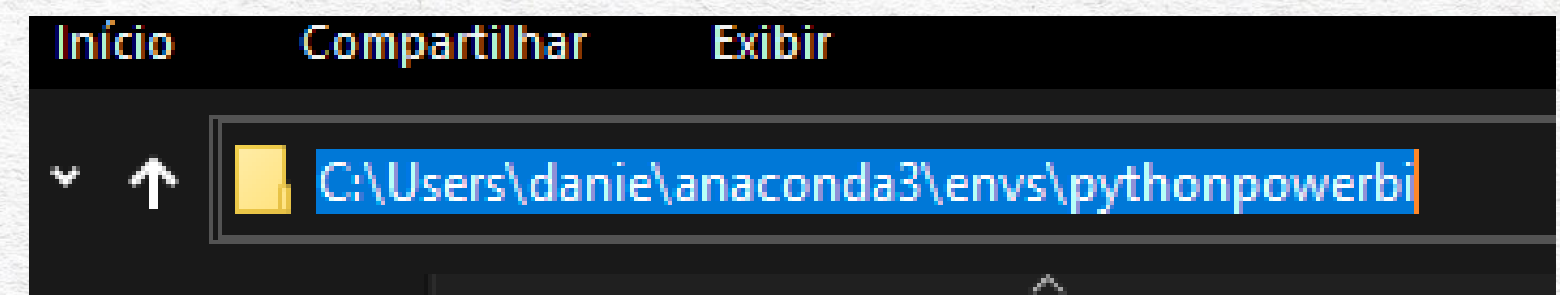
C:\Users\SEU USUÁRIO\anaconda3\envs\

```
Python\Python e Power BI>conda create -n pythonpowerbi python=3.6
```

```
(pythonpowerbi)\Python e Power BI>pip install jupyter notebook
```

```
(pythonpowerbi)\Python e Power BI>pip install pandas
```

```
(pythonpowerbi)\Python e Power BI>pip install matplotlib
```



Iremos indicar o ambiente virtual que desejamos dentro na parte **Opções e configurações** dentro da aba **arquivo** do Power BI.

The image shows the 'Opções e configurações' (Options and configurations) dialog box in Power BI. The left sidebar contains a list of options: 'Opções' (highlighted with a red box), 'Configurações da fonte de dados', 'Novo', 'Abrir relatório', 'Salvar', 'Salvar como', 'Obter dados', 'Importar', 'Exportar', 'Publicar', 'Opções e configurações' (highlighted with a red box), and 'Introdução'. The main area is titled 'Opções' and has a sidebar with 'GLOBAL' and 'Scripts do Python' (highlighted with a red box). The 'Scripts do Python' section is expanded, showing 'Opções de scripts do Python'. It includes a dropdown menu for 'Diretórios base do Python detectados:' with 'Outro' selected (highlighted with a red box and an orange arrow pointing to it with the text 'Selecione a opção Outro'). Below this is a text field for 'Defina um diretório raiz do Python:' containing the path 'C:\Users\danie\anaconda3\envs\pythonpowerbi' (highlighted with a red box and an orange arrow pointing to it with the text 'Insira o caminho da pasta do seu ambiente virtual criado'). A 'Procurar' button is next to the text field. At the bottom, there is a dropdown for 'IDEs do Python Detectados:' with 'Programa do SO padrão para arquivos .PY' selected.

Opções e configurações

Opções

Configurações da fonte de dados

Opções

Scripts do Python

Opções de scripts do Python

Para escolher um diretório raiz para o Python, selecione uma instalação do Python detectada na lista suspensa ou selecione Outro e navegue até o local desejado.

Diretórios base do Python detectados:

Outro

Defina um diretório raiz do Python:

C:\Users\danie\anaconda3\envs\pythonpowerbi

Procurar

Como instalar o Python

Para escolher qual IDE (ambiente de desenvolvimento integrado) do Python você deseja que o Power BI Desktop inicie, selecione um IDE detectado na lista suspensa ou selecione Outro para procurar outro IDE no computador.

IDEs do Python Detectados:

Programa do SO padrão para arquivos .PY

Selecione a opção Outro

Insira o caminho da pasta do seu ambiente virtual criado.

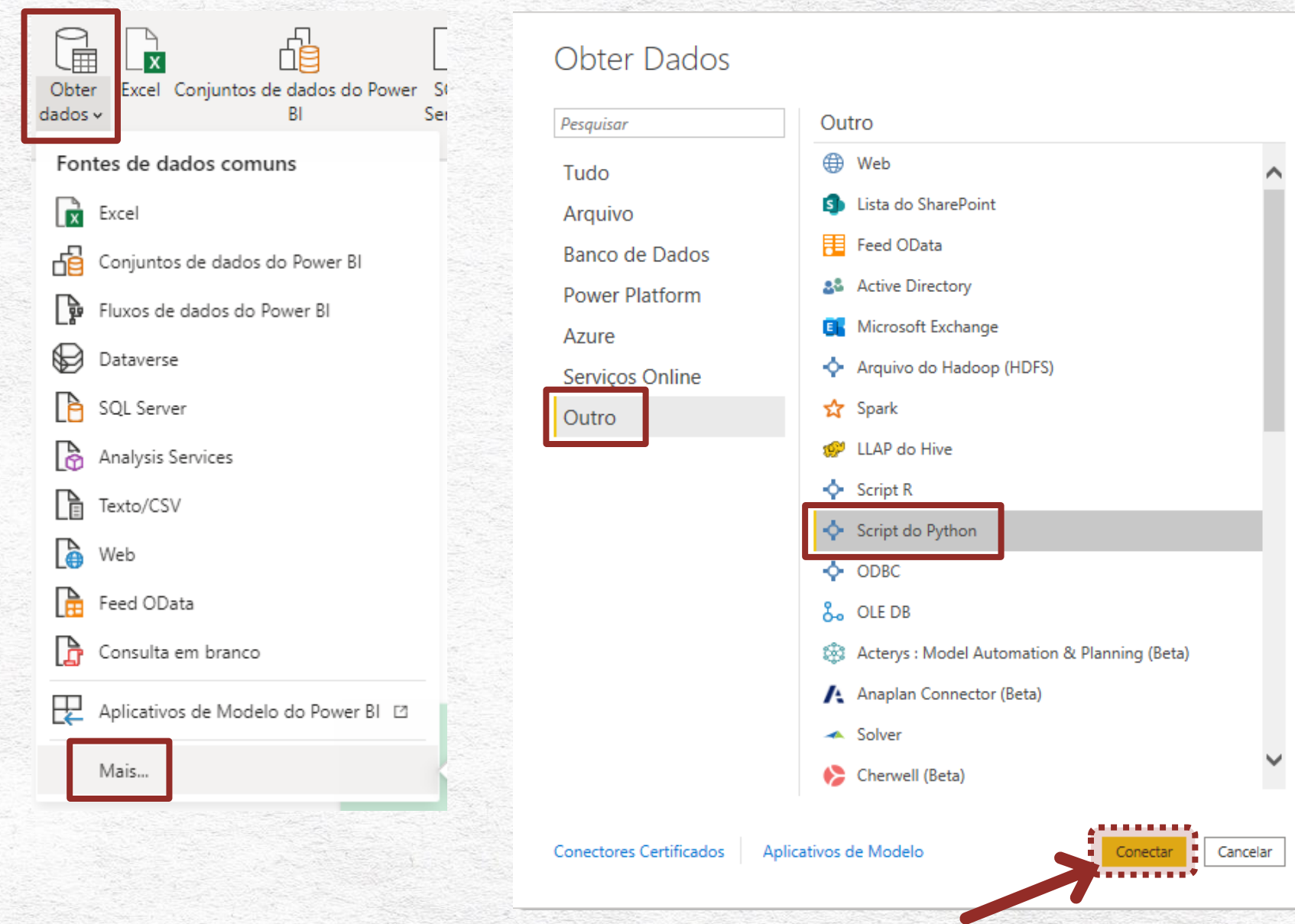
Vamos agora fazer alguns testes para saber se todas as configurações estão corretas.

Dentro do Power BI:

- 1) Clique **Obter Dados**;
- 2) Clique na opção **Mais...**;
- 3) Após a abertura da Janela procure a opção **Outro**;
- 4) Depois selecione a opção **Script do Python**;
- 5) Clique em **Conectar**;
- 6) Dentro do campo **Script** vamos escrever nosso código Python de importação do Pandas e do arquivo CSV. Dica: sempre colocar o caminho completo do arquivo.
- 7) Clique em OK para rodar

Se rodar, sabemos que a integração está funcionando.

Caso não rode, precisamos entender o que está acontecendo, lendo o erro indicado. Se tiver dúvida, lembre-se que temos o suporte na plataforma para te ajudar 😊.



Script do Python

Script

```
import pandas as pd  
  
vendas_df = pd.read_csv(r'C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Python e Power BI\Dados\ vendas.csv')
```

OK

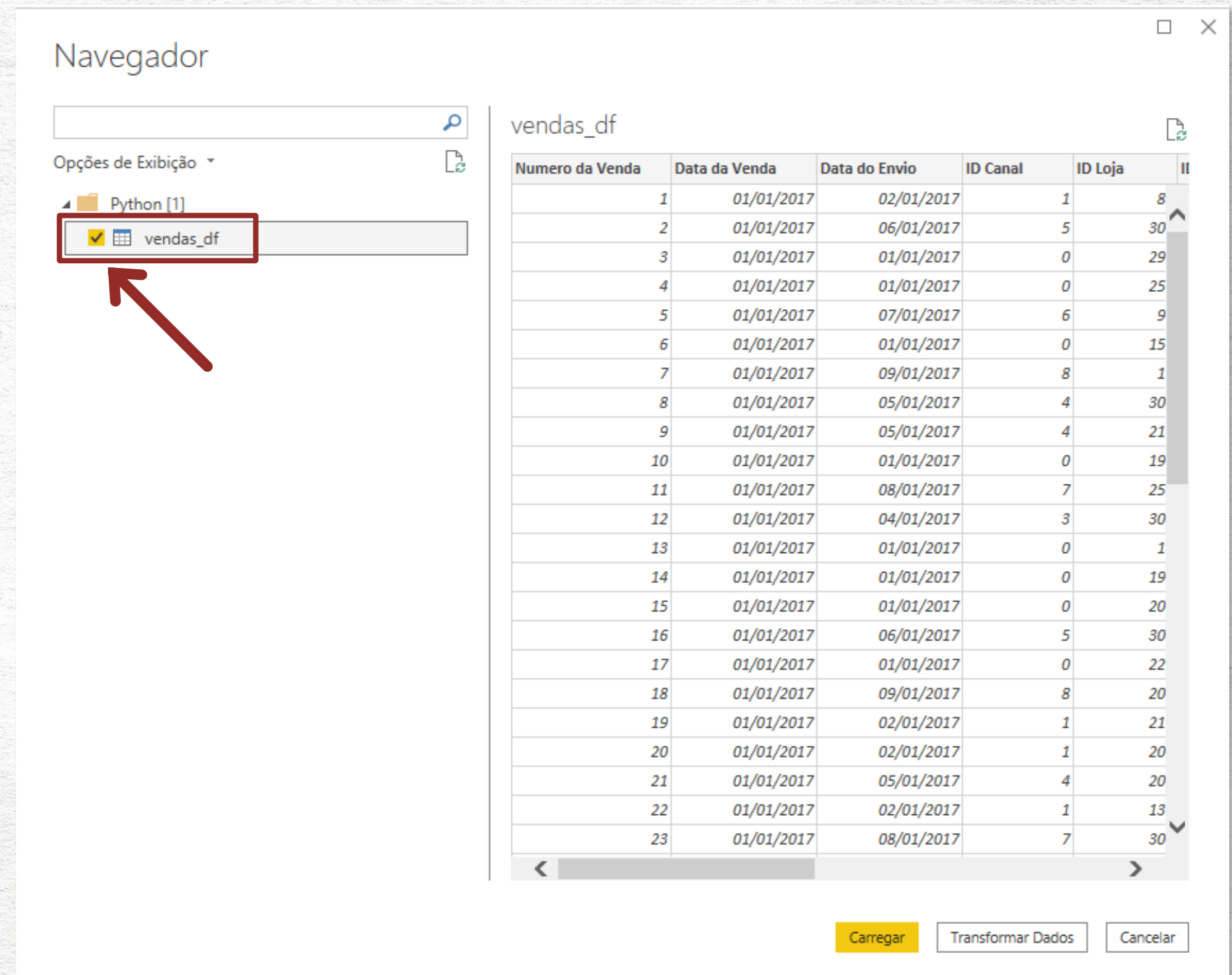
Cancelar



Se ao rodar o script Python dentro do Power BI surgiu uma janela como a apresentada ao lado, sabemos que tanto nossa integração quanto nosso código estão funcionando.

Por default, vendas_df não estará marcada (). ☒ vendas_df checkbox, nosso vendas_df aparecerá no display do lado direito da janela.

Feita a configuração, agora é só executar nossos códigos. E é isso que vamos ver até o fim deste módulo.



Navegador

Opções de Exibição ▾

Python [1]

☒ vendas_df

vendas_df

Numero da Venda	Data da Venda	Data do Envio	ID Canal	ID Loja	II
1	01/01/2017	02/01/2017	1	8	
2	01/01/2017	06/01/2017	5	30	
3	01/01/2017	01/01/2017	0	29	
4	01/01/2017	01/01/2017	0	25	
5	01/01/2017	07/01/2017	6	9	
6	01/01/2017	01/01/2017	0	15	
7	01/01/2017	09/01/2017	8	1	
8	01/01/2017	05/01/2017	4	30	
9	01/01/2017	05/01/2017	4	21	
10	01/01/2017	01/01/2017	0	19	
11	01/01/2017	08/01/2017	7	25	
12	01/01/2017	04/01/2017	3	30	
13	01/01/2017	01/01/2017	0	1	
14	01/01/2017	01/01/2017	0	19	
15	01/01/2017	01/01/2017	0	20	
16	01/01/2017	06/01/2017	5	30	
17	01/01/2017	01/01/2017	0	22	
18	01/01/2017	09/01/2017	8	20	
19	01/01/2017	02/01/2017	1	21	
20	01/01/2017	02/01/2017	1	20	
21	01/01/2017	05/01/2017	4	20	
22	01/01/2017	02/01/2017	1	13	
23	01/01/2017	08/01/2017	7	30	

Carregar Transformar Dados Cancelar

Feita a configuração inicial, vamos criar um código de importação de dados.



DICA

Dica 1: Teste seus códigos no Jupyter e quando tudo estiver funcionando, cole no campo de Script Python do Power BI.

Dica 2: Geralmente damos print ou display no Jupyter. Retire essas linhas de código quando colar no Power BI.

```
import pandas as pd
import os
#importando os arquivos
caminho_padrao = r'C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Python e Power BI'
vendas_df = pd.read_csv(os.path.join(caminho_padrao, r'Contoso - Vendas - 2017.csv'), sep=';')
produtos_df = pd.read_csv(os.path.join(caminho_padrao, r'Contoso - Cadastro Produtos.csv'), sep=';')
lojas_df = pd.read_csv(os.path.join(caminho_padrao, r'Contoso - Lojas.csv'), sep=';')
clientes_df = pd.read_csv(os.path.join(caminho_padrao, r'Contoso - Clientes.csv'), sep=';')

#limpando apenas as colunas que queremos
clientes_df = clientes_df[['ID Cliente', 'E-mail']]
produtos_df = produtos_df[['ID Produto', 'Nome do Produto']]
lojas_df = lojas_df[['ID Loja', 'Nome da Loja']]

#mesclando e renomeando os dataframes
vendas_df = vendas_df.merge(produtos_df, on='ID Produto')
vendas_df = vendas_df.merge(lojas_df, on='ID Loja')
vendas_df = vendas_df.merge(clientes_df, on='ID Cliente').rename(columns={'E-mail': 'E-mail do Cliente'})
display(vendas_df)
```

	Numero da Venda	Data da Venda	Data do Envio	ID Canal	ID Loja	ID Produto	ID Promocao	ID Cliente	Quantidade Vendida	Quantidade Devolvida	Nome do Produto	Nome da Loja	E-mail do Cliente
0	1	01/01/2017	02/01/2017	1	86	981	2	6825	9	1	A. Datum Advanced Digital Camera M300 Pink	Loja Contoso Austin	rbrumfieldmy@ameblo.jp
1	880458	23/11/2017	23/11/2017	0	306	235	10	6825	8	0	Litware Home Theater System 7.1	Loja Contoso Europe	rbrumfieldmy@ameblo.jp



Script do Python

Script

```
clientes_df = pd.read_csv(os.path.join(caminho_padrao, r'Contoso - Clientes.csv'), sep=';')

#limpando apenas as colunas que queremos
clientes_df = clientes_df[['ID Cliente', 'E-mail']]
produtos_df = produtos_df[['ID Produto', 'Nome do Produto']]
lojas_df = lojas_df[['ID Loja', 'Nome da Loja']]

#mesclando e renomeando os dataframes
vendas_df = vendas_df.merge(produtos_df, on='ID Produto')
vendas_df = vendas_df.merge(lojas_df, on='ID Loja')
vendas_df = vendas_df.merge(clientes_df, on='ID Cliente').rename(columns={'E-mail': 'E-mail do Cliente'})
```

O script será executado com a seguinte instalação do Python C:\Users\danie\anaconda3\envs\pythonpowerbi.
Para definir suas configurações e alterar qual instalação do Python você deseja executar, acesse Opções e configurações.

OK Cancelar



Perceba que automaticamente foram importadas as 4 bases em CSV. Se clicarmos em Transformar Dados, vamos poder ir para o Power Query, que é uma ferramenta que existe no Power BI.

Navegador

Opções de Exibição

Python [4]

☒ clientes_df

☒ lojas_df

☒ produtos_df

☒ vendas_df

vendas_df

Numero da Venda	Data da Venda	Data do Envio	ID Canal	ID Loja	ID
1	01/01/2017	02/01/2017	1	8	
880458	23/11/2017	23/11/2017	0	30	
191019	20/03/2017	21/03/2017	1	17	
18610	08/01/2017	10/01/2017	2	20	
287704	23/04/2017	26/04/2017	3	7	
373600	22/05/2017	29/05/2017	7	3	
873068	21/11/2017	22/11/2017	1	27	
26350	12/01/2017	18/01/2017	6	2	
180875	16/03/2017	16/03/2017	0	26	
413808	04/06/2017	06/06/2017	2	18	
860046	16/11/2017	16/11/2017	0	19	
415344	04/06/2017	05/06/2017	1	23	
811927	29/10/2017	31/10/2017	2	10	
364459	19/05/2017	20/05/2017	1	2	
836335	07/11/2017	08/11/2017	1	23	
638297	27/08/2017	27/08/2017	0	26	
372597	21/05/2017	22/05/2017	1	8	
614980	18/08/2017	18/08/2017	0	30	
786402	21/10/2017	25/10/2017	4	30	
383897	25/05/2017	28/05/2017	3	30	
645873	31/08/2017	31/08/2017	0	30	
169169	11/03/2017	14/03/2017	3	2	
333103	09/05/2017	09/05/2017	0	19	

Carregar

Transformar Dados

Cancelar



POWER QUERY:

Consultas [4]

clientes_df

lojas_df

produtos_df

vendas_df

= Fonte([Name="vendas_df"])[Value]

	Numero da Venda	Data da Venda	Data do Envio	ID Canal	ID Loja	ID Produto
1	1	01/01/2017	02/01/2017	1	86	981
2	880458	23/11/2017	23/11/2017	0	306	235
3	191019	20/03/2017	21/03/2017	1	172	376
4	18610	08/01/2017	10/01/2017	2	200	448
5	287704	23/04/2017	26/04/2017	3	76	280
6	373600	22/05/2017	29/05/2017	7	33	1284
7	873068	21/11/2017	22/11/2017	1	275	246
8	26350	12/01/2017	18/01/2017	6	21	568
9	180875	16/03/2017	16/03/2017	0	268	975
10	413808	04/06/2017	06/06/2017	2	185	1197
11	860046	16/11/2017	16/11/2017	0	191	578
12	415344	04/06/2017	05/06/2017	1	231	997
13	811927	29/10/2017	31/10/2017	2	100	431
14	364459	19/05/2017	20/05/2017	1	26	845
15	836335	07/11/2017	08/11/2017	1	237	322
16	638297	27/08/2017	27/08/2017	0	262	356
17	372597	21/05/2017	22/05/2017	1	86	981
18	614980	18/08/2017	18/08/2017	0	306	1621
19	786402	21/10/2017	25/10/2017	4	306	226
20	383897	25/05/2017	28/05/2017	3	306	347
21	645873	31/08/2017	31/08/2017	0	309	968
22	169169	11/03/2017	14/03/2017	3	22	437
23	333103	09/05/2017	09/05/2017	0	199	591

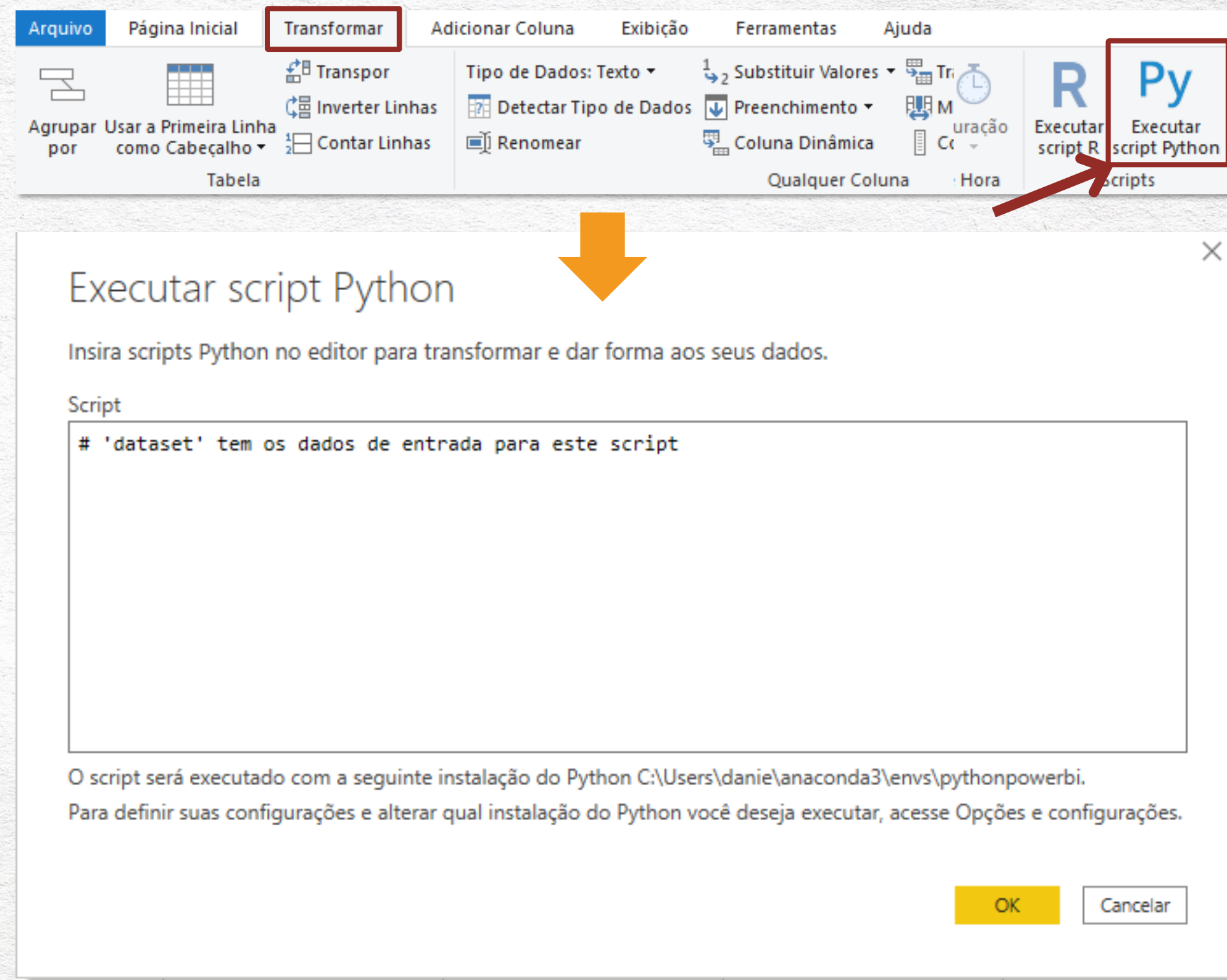


É muito comum no Power BI, antes de carregarmos os dados para elaboração de relacionamentos, gráficos, etc, fazer a edição desses dados para garantirmos que todos os dados estão corretos e formatados da forma que nos interessa.

Podemos fazer essa edição via Python! Na guia **Transformar** do Power Query clique na opção **Executar script Python**.

Nesse campo, é possível criar seus códigos que rodarão dentro do Power Query para edição dos dados.

Dica: Crie seus códigos no Jupyter e só depois coloque-o no script do Python.



Agora já sabemos onde colocar nosso código, vamos voltar para o Jupyter para escrever nosso código e quando tudo estiver certinho, a gente volta e coloca no campo **Script Python**.

Dentro do PowerBI os dataframes são chamados de **dataset** conforme apresentado no print ao lado.

Sabendo dessa informação, vamos usar esse nome para realizar nosso código.

Nosso objetivo é filtrar apenas as lojas com ID IGUAL a 86, 306 ou 172. Ao executarmos o código e termos sucesso, podemos copiar a linha de código e colá-la dentro do campo **Script Python** do Power BI.

Executar script Python

Insira scripts Python no editor para transformar e dar forma aos seus dados.

Script

'dataset' tem os dados de entrada para este script

```
dataset = vendas_df
tres_lojas_df = dataset[dataset['ID Loja'].isin([86, 306, 172])]
display(tres_lojas_df)
```

	Numero da Venda	Data da Venda	Data do Envio	ID Canal	ID Loja	ID Produto	ID Promocao	ID Cliente	Quantidade Vendida	Quantidade Devolvida	Nome do Produto	Nome da Loja	E-mail do Cliente
0	1	01/01/2017	02/01/2017	1	86	981	2	6825	9	1	A. Datum Advanced Digital Camera M300 Pink	Loja Contoso Austin	rbrumfieldmy@ameblo.jp
1	880458	23/11/2017	23/11/2017	0	306	235	10	6825	8	0	Litware Home Theater System 7.1 Channel M710 B...	Loja Contoso Europe Online	rbrumfieldmy@ameblo.jp
2	191019	20/03/2017	21/03/2017	1	172	376	2	6825	9	0	Adventure Works Laptop12 M1201 Silver	Loja Contoso Hartford	rbrumfieldmy@ameblo.jp

Executar script Python

Insira scripts Python no editor para transformar e dar forma aos seus dados.

Script

'dataset' tem os dados de entrada para este script

tres_lojas_df = dataset[dataset['ID Loja'].isin([86, 306, 172])]

Código a ser levado para o Power Query/Power BI

Agora é só clicar em **Fechar e Aplicar** no Power Query e nosso dataframe já estará disponível dentro do Power BI.



Numero da Venda	Data da Venda	Data do Envio	ID Canal	ID Loja	ID Produto	ID Promocao	ID Cliente	Quantidade Vendida	Quantidade Devolvida	Nome do Produto	Nome da Loja	E-mail do Cliente
880458	quinta-feira, 23 de novembro de 2017	quinta-feira, 23 de novembro de 2017	0	306	235	10	6825	8	0	Litware Home Theater System 7.1 Channel M710 Brown	Loja Contoso Europe Online	rbrumfi@contoso.com
818047	terça-feira, 31 de outubro de 2017	terça-feira, 31 de outubro de 2017	0	306	1008	10	26471	8	0	A. Datum Consumer Digital Camera M300 Orange	Loja Contoso Europe Online	jessica44@contoso.com
36490	segunda-feira, 16 de janeiro de 2017	segunda-feira, 16 de janeiro de 2017	0	306	196	10	28318	8	0	Litware Home Theater System 2.1 Channel E211 Black	Loja Contoso Europe Online	arianna4@contoso.com
911713	terça-feira, 5 de dezembro de 2017	terça-feira, 5 de dezembro de 2017	0	306	1380	10	31431	8	0	Contoso Bedroom Phone with AM/FM Stereo and Call Waiting	Loja Contoso Europe Online	cody19@contoso.com
951107	quarta-feira, 20 de dezembro de 2017	quarta-feira, 20 de dezembro de 2017	0	306	566	10	31431	8	0	Proseware Projector 720p DLP56 Silver	Loja Contoso Europe Online	cody19@contoso.com
802072	quinta-feira, 26 de outubro de 2017	quinta-feira, 26 de outubro de 2017	0	306	855	10	20064	8	0	Contoso Multimedia Speakers M25 Blue	Loja Contoso Europe Online	jimlacke@contoso.com
58933	quarta-feira, 25 de janeiro de 2017	quarta-feira, 25 de janeiro de 2017	0	306	279	10	25296	8	0	Contoso Home Theater System 2.1 Channel E1200 Brown	Loja Contoso Europe Online	sophia17@contoso.com
732741	quarta-feira, 4 de outubro de 2017	quarta-feira, 4 de outubro de 2017	0	306	710	10	10683	8	0	Proseware Scan Jet Digital Flat Bed Scanner M300 White	Loja Contoso Europe Online	ncelloj0@contoso.com
767650	domingo, 15 de outubro de 2017	domingo, 15 de outubro de 2017	0	306	510	10	24700	8	0	Adventure Works LCD19W M100 White	Loja Contoso Europe Online	alfredo1@contoso.com
955271	sexta-feira, 22 de dezembro de 2017	sexta-feira, 22 de dezembro de 2017	0	306	405	10	24612	8	0	Proseware Laptop15 M510 Black	Loja Contoso Europe Online	alan29@contoso.com
944485	segunda-feira, 18 de dezembro de 2017	segunda-feira, 18 de dezembro de 2017	0	306	1192	10	30129	8	0	Fabrikam Home and Vacation Moviemaker 1" 25mm M400	Loja Contoso Europe Online	brandy1@contoso.com
864903	sábado, 18 de novembro de 2017	sábado, 18 de novembro de 2017	0	306	388	10	23763	8	0	Adventure Works Laptop12 M1201 Blue	Loja Contoso Europe Online	kathleen@contoso.com
874760	terça-feira, 21 de novembro de 2017	terça-feira, 21 de novembro de 2017	0	306	626	10	13494	8	0	WWI Projector 480p LCD12 White	Loja Contoso Europe Online	hgaddnr@contoso.com
934010	quinta-feira, 14 de dezembro de 2017	quinta-feira, 14 de dezembro de 2017	0	306	939	10	11959	8	0	SV Wireless LAN PCI Network Card Adapter E901 Black	Loja Contoso Europe Online	cdackey@contoso.com
961828	domingo, 24 de dezembro de 2017	domingo, 24 de dezembro de 2017	0	306	1007	10	10009	8	0	A. Datum Consumer Digital Camera E100 Orange	Loja Contoso Europe Online	wprettej@contoso.com
783258	sexta-feira, 20 de outubro de 2017	sexta-feira, 20 de outubro de 2017	0	306	880	10	24994	8	0	Contoso Optical Wheel OEM PS/2 Mouse E60 Black	Loja Contoso Europe Online	jenny17@contoso.com
957969	sábado, 23 de dezembro de 2017	sábado, 23 de dezembro de 2017	0	306	203	10	13636	8	0	Litware Home Theater System 7.1 Channel M710 Black	Loja Contoso Europe Online	cdohmer@contoso.com
61331	quinta-feira, 26 de janeiro de 2017	quinta-feira, 26 de janeiro de 2017	0	306	1201	10	13816	8	0	Fabrikam Trendsetter 1/2" 3mm X300 Grey	Loja Contoso Europe Online	gtownen@contoso.com
750080	segunda-feira, 9 de outubro de 2017	segunda-feira, 9 de outubro de 2017	0	306	1592	10	29975	8	0	SV DVD 48 DVD Storage Binder M50 Red	Loja Contoso Europe Online	casey0@contoso.com
824394	quinta-feira, 2 de novembro de 2017	quinta-feira, 2 de novembro de 2017	0	306	675	10	29975	8	0	Proseware Laser Jet All in one X300 Grey	Loja Contoso Europe Online	casey0@contoso.com
1821	domingo, 1 de janeiro de 2017	domingo, 1 de janeiro de 2017	0	306	667	10	26250	8	0	Proseware Office Jet Wireless All-in-One Inkjet Printer M61	Loja Contoso Europe Online	xavier33@contoso.com

Módulo 36 – Integração Python com Power BI – Usando Python para criar Gráficos no Power BI (1/6)

183

Nessa parte do módulo, iremos fazer uma integração do Python com a parte gráfica.

Usaremos a guia **Relatório** do Power BI (indicada abaixo) e o ícone **Visual Python** para criar um gráfico.

The screenshot shows the Microsoft Power BI Desktop application. The ribbon at the top includes 'Arquivo', 'Página Inicial', 'Inserir', 'Modelagem', 'Exibição', 'Ajuda', 'Formato', and 'Dados / Analisar'. The 'Página Inicial' ribbon is active, showing options like 'Colar', 'Recortar', 'Copiar', 'Pincel de formatação', 'Obter dados', 'Excel', 'Conjuntos de dados do Power BI', 'SQL Server', 'Inserir', 'Datastage', 'Fontes recentes', 'Transformar dados', 'Atualizar dados', 'Novo visual', 'Caixa de texto', 'Mais visuais', 'Nova medida rápida', 'Medida rápida', 'Confidencialidade (versão prévia)', and 'Publicar'.

Annotations with orange boxes and arrows point to specific elements:

- Guia Relatório do Power BI onde são criados os gráficos:** Points to the 'Relatório' icon in the left-hand navigation pane.
- Gráfico criado pelo Visual Python. Ainda sem conteúdo:** Points to a blank chart area in the main workspace that features a Python logo.
- Ícone de criação de Gráfico via Python (Visual Python):** Points to the 'Visual Python' icon in the 'Visualizações' pane on the right.
- Editor de código Python:** Points to the 'Editor de scripts Python' window at the bottom of the screen, which contains the instruction: 'Arraste os campos para a área Valores no painel Visualização para iniciar a criação de scripts.'



Módulo 36 – Integração Python com Power BI – Usando Python para criar Gráficos no Power BI (2/6)

184

Usando a interface do próprio Power BI, vamos **arrastar** a coluna **Data da Venda** para o campo **Valores** de código do Python.

Automaticamente o campo de edição de código do Python criará algumas linhas de código.

A recuperação automática contém alguns arquivos recuperados que não foram abertos.

Exibir arquivos recuperados

Pesquis...

Filtros neste visual

Data da Venda - Ano é (Tudo)

Data da Venda - Dia é (Tudo)

Data da Venda - Mês é (Tudo)

Data da Venda - Trime... é (Tudo)

Adicionar os campos de da...

Filtros nesta página

Adicionar os campos de da...

Filtros em todas as páginas

Visualizações

Pesquis...

Valores

Data da Venda

Ano

Trimestre

Mês

Dia

Drill-through

Relatório cruzado

Desativado

Manter todos os filtros

Ativado

Adicionar os campos de dr...

Campos

Pesquisar

vendas_df

Data da Ve...

Data do En...

E-mail do C...

ID Canal

ID Cliente

ID Loja

ID Produto

ID Promocao

Nome da L...

Nome do P...

Numero da...

Quantidade...

Quantidade...

Editor de scripts Python

As linhas duplicadas serão removidas dos dados.

```
1 # O código a seguir para criar um dataframe e remover as linhas duplicadas sempre é executado e age como um preâmbulo para o script:
2
3 # dataset = pandas.DataFrame(Ano, Trimestre, Mês, Dia)
4 # dataset = dataset.drop_duplicates()
5
6 # Cole ou digite aqui seu código de script:
```

Valores criados a partir da escolha da coluna "Data da Venda" como Valor

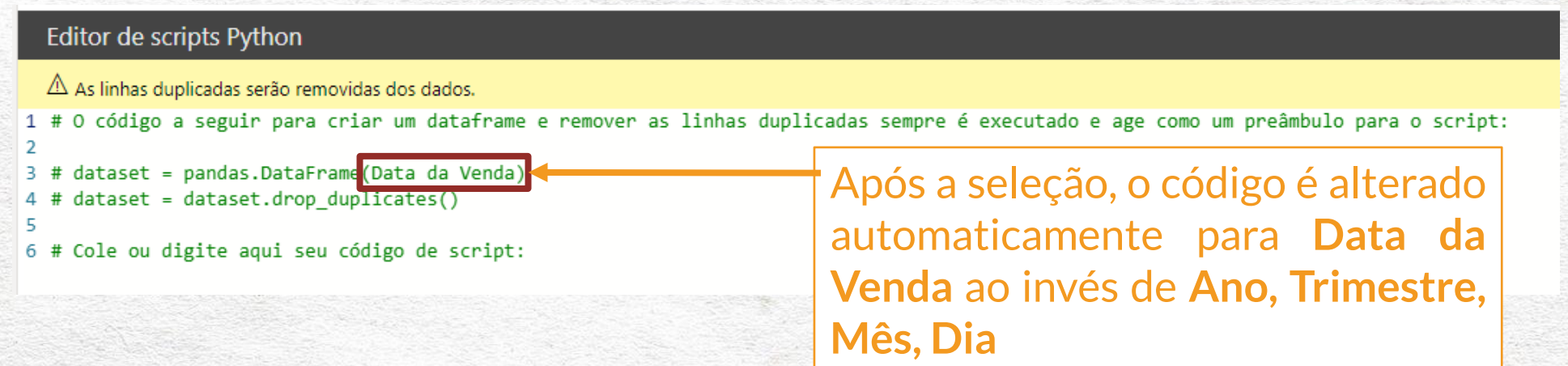
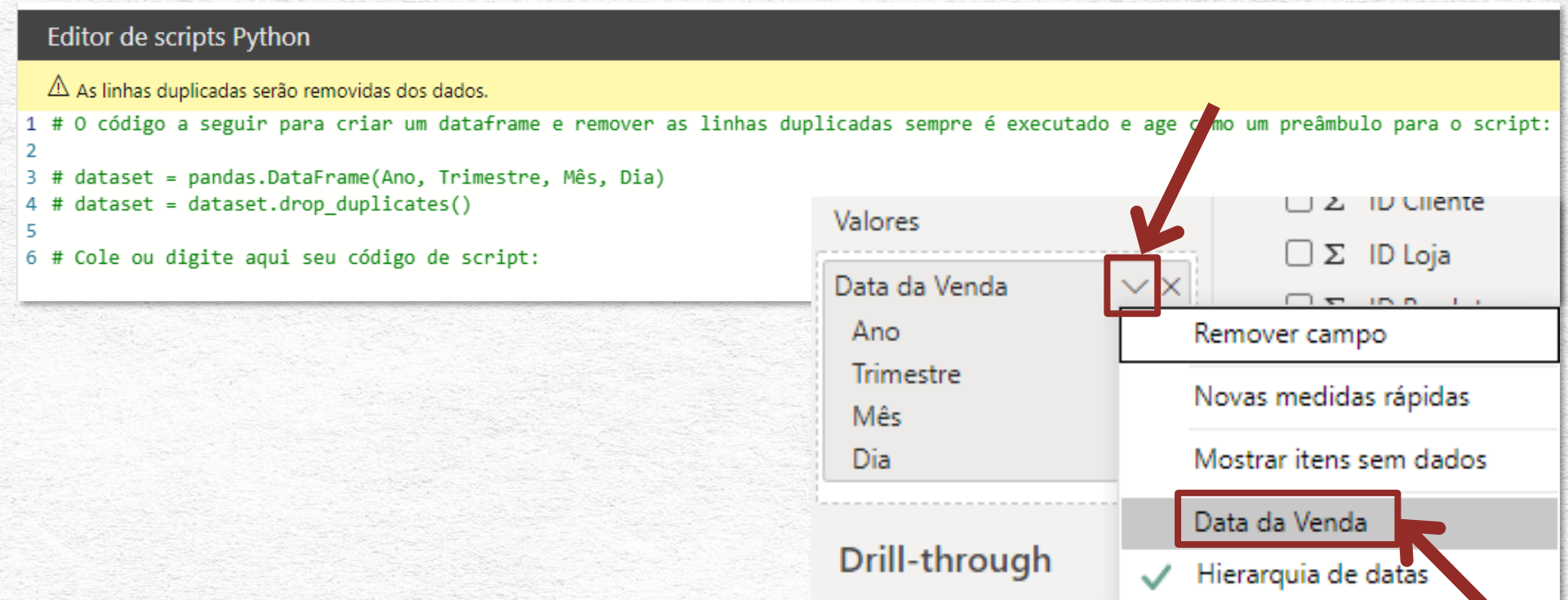


Analizando o código criado, percebemos que nosso DataFrame está considerando ao invés da Data exata da venda as informações de Ano, Trimestre, Mês e Dia.

Isso acontece pois o Power BI tende a trabalhar os dados de forma que para ELE faça mais sentido.

Para retornarmos essa informação para a Data da Venda original, basta clicarmos na seta para baixo da coluna **Data da Venda** e selecionar a opção **Data da Venda**.

Como podemos ver no exemplo ao lado, o Power BI estava considerando a opção **Hierarquia de datas** o que não nos atende nesse exemplo.

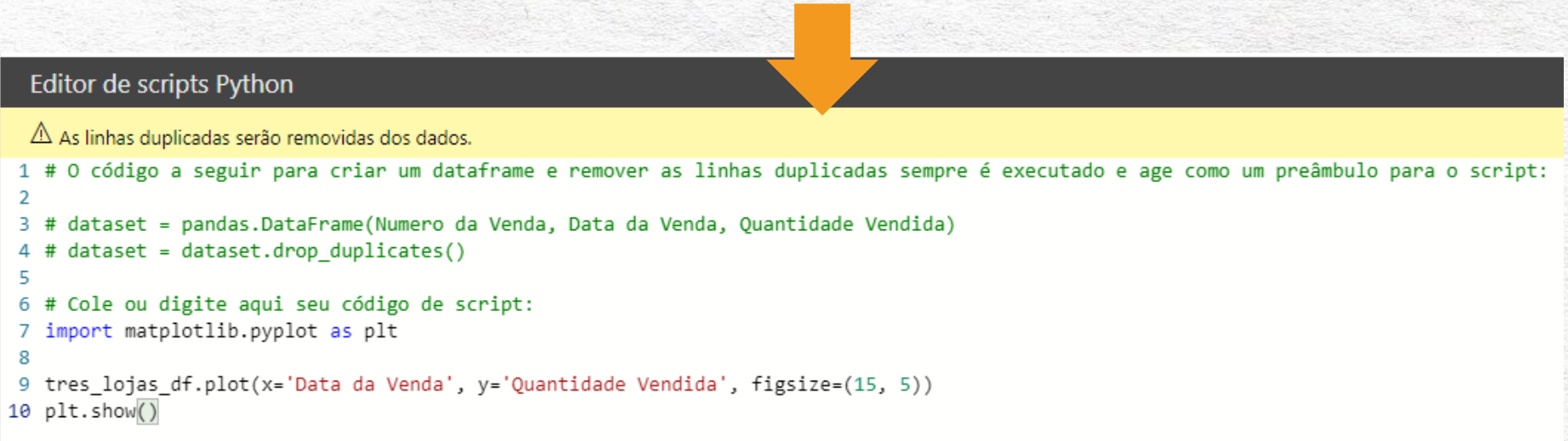
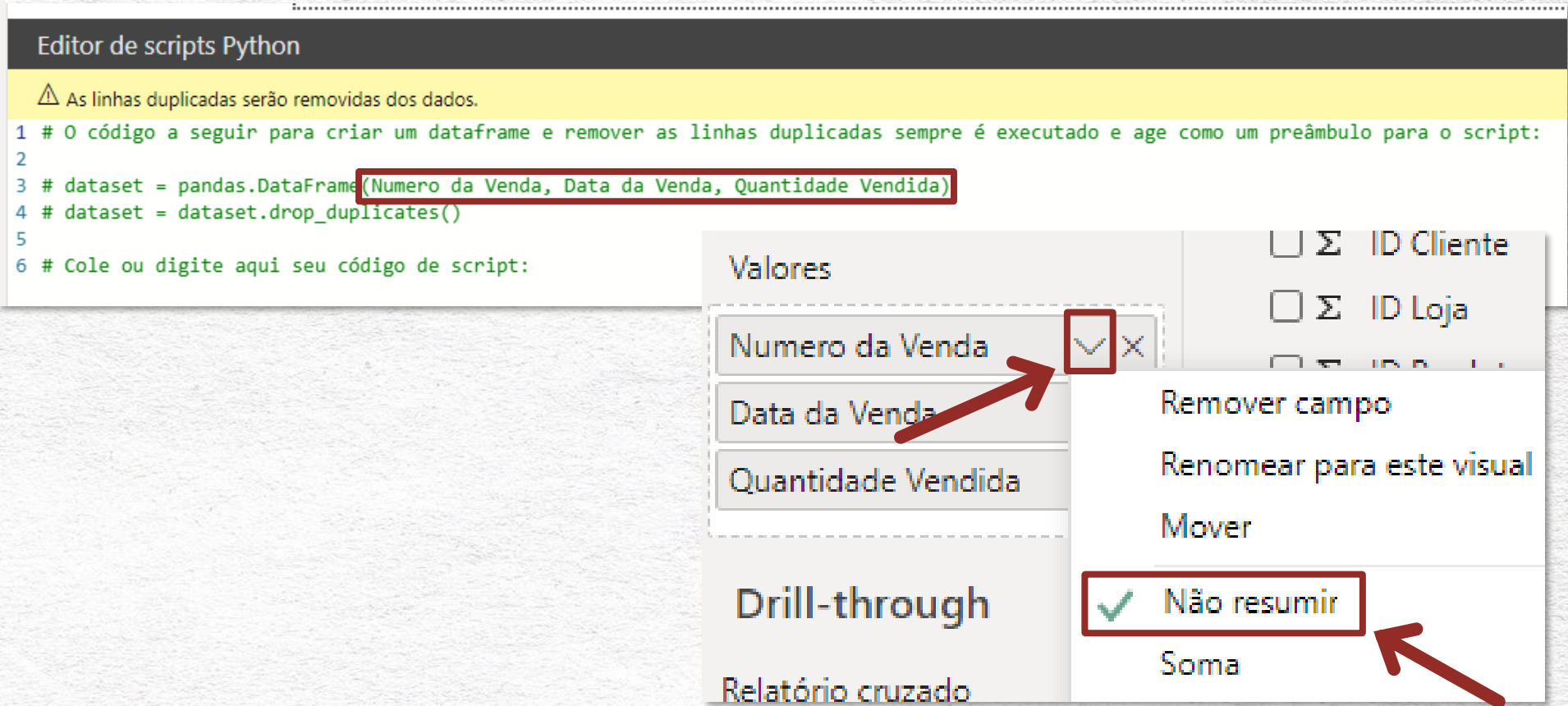


Vamos agora adicionar outras colunas ao nosso gráfico: **Numero da Venda** e **Quantidade Vendida**.

Assim como fizemos em **Data da Venda**, vamos alterar a configuração sugerida pelo Power BI para a coluna **Numero da Venda**.

Feito isso, vamos voltar para o nosso Jupyter e validar o código que usaremos dentro do Power BI na criação do Gráfico (Lembrando que não é uma obrigação. O código poderá ser feito diretamente no Power BI)

Código rodando, vamos agora levar esse código para o nosso Power BI.



Antes de rodarmos o código, precisamos tomar alguns cuidados:

- 1) Nome do Dataframe: No Jupyter estávamos usando `tres_lojas_df`, no entanto, por padrão, o PowerBI está chamando essa variável de `dataset`;
- 2) Os valores de `x= 'Data da Venda'` e `y='Quantidade Vendida'`, só funcionam pois essas duas colunas foram incluídas no gráfico (passos anteriores);
- 3) O uso do `plt.show()` é obrigatório para a execução do código.

Editor de scripts Python

⚠ As linhas duplicadas serão removidas dos dados.

```
1 # O código a seguir para criar um dataframe e remover as linhas duplicadas sempre  
2  
3 # dataset = pandas.DataFrame(Numero da Venda, Data da Venda, Quantidade Vendida)  
4 # dataset = dataset.drop_duplicates()  
5  
6 # Cole ou digite aqui seu código de script:  
7 import matplotlib.pyplot as plt  
8  
9 tres_lojas_df.plot(x='Data da Venda', y='Quantidade Vendida', figsize=(15, 5))  
10 plt.show()
```

Colunas foram incluídas no gráfico, por isso podemos usá-las no código

Tres_lojas_df precisa ser substituída por dataset.plot

```
6 # Cole ou digite aqui seu código de script:  
7 import matplotlib.pyplot as plt  
8  
9 dataset.plot(x='Data da Venda', y='Quantidade Vendida', figsize=(15, 5))  
10 plt.show()
```

Ao rodar o código, temos o nosso gráfico plotando dentro do Power BI.

Sabemos que não é dos mais bonitos, mas vamos entender como melhorar a cara dele mais para frente nesse módulo 😊.



Nessa parte do módulo, nosso objetivo é mostrar como usar gráficos mais interessantes que o usado anteriormente e aproveitar para apresentar uma nova biblioteca de gráficos chamada **Seaborn** ([doc](#)).

Vamos lembrar que estamos trabalhando em um ambiente virtual específico para a integração com o Power BI. Portanto, ainda não temos a biblioteca do Seaborn instalada.

Usando o pip, faça a instalação dessa biblioteca.

ATENÇÃO: Essa instalação precisa ser feita dentro do ambiente virtual. Caso esteja com dúvidas, retorne ao módulo 30 😊 para consulta.

Como sempre, vamos usar o Jupyter para validar nosso código e só depois vamos para o Power BI.

O código que usaremos é o código apresentado ao lado.

```
(pythonpowerbi) | \Python e Power BI> pip install seaborn
```

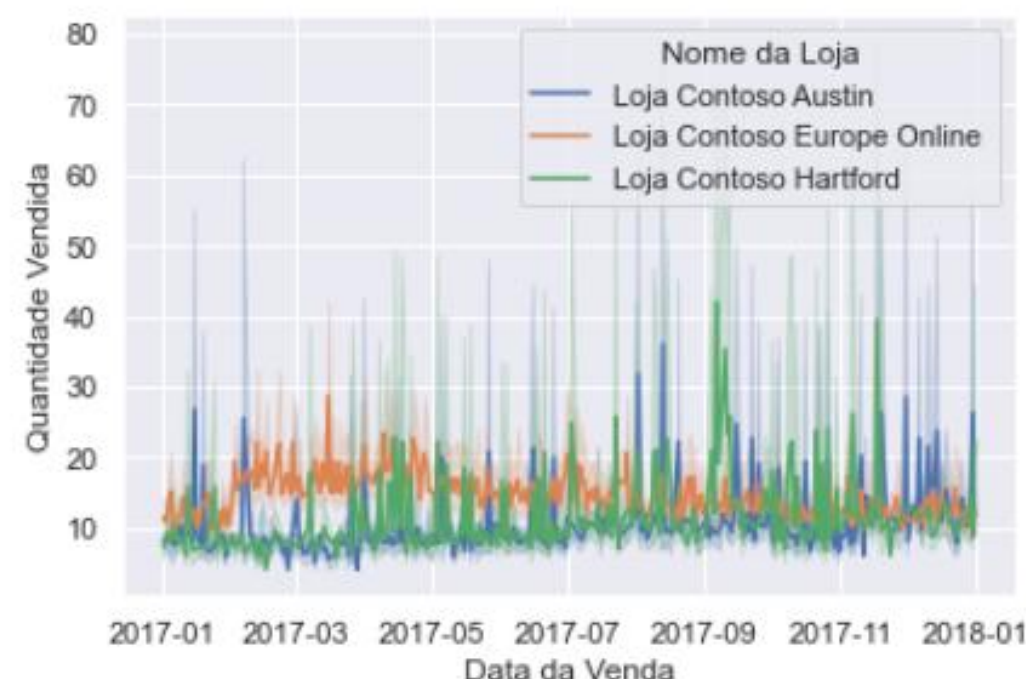
```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sns.set_theme(style="darkgrid")
```

Define a cor de fundo do gráfico

```
sns.lineplot(x="Data da Venda", y="Quantidade Vendida", hue="Nome da Loja", data=tres_lojas_df)
```

```
plt.show()
```



Código rodando, vamos para o Power BI!

Vamos abrir uma nova aba na guia relatórios e criar um gráfico usando o **Visual Python**.

The screenshot shows the Power BI Desktop interface. The ribbon at the top includes 'Arquivo', 'Página Inicial', 'Inserir', 'Modelagem', 'Exibição', 'Ajuda', 'Formato', and 'Dados / Analisar'. The 'Dados / Analisar' ribbon is active, showing options like 'Obter dados', 'Excel', 'Conjuntos de dados do Power BI', 'SQL Server', 'Inserir', 'Datastage', 'Fontes recentes', 'Transformar', 'Atualizar', 'Novo visual', 'Caixa de texto', 'Mais visuais', 'Nova medida rápida', 'Medida rápida', 'Confidencialidade (versão prévia)', and 'Publicar'. On the left sidebar, the 'Relatórios' view is selected, indicated by a red box and an arrow pointing to a callout box. In the center workspace, a new visual is being created, shown as a grey box with a Python logo, highlighted by a red box and an arrow pointing to a callout box. On the right sidebar, the 'Visualizações' pane is open, showing various chart types. The 'Visual Python' icon, which is a blue square with a white 'Py', is highlighted by a red box and an arrow pointing to a callout box. Below the main workspace, the 'Editor de scripts Python' is open, showing a message: 'Arraste os campos para a área Valores no painel Visualização para iniciar a criação de scripts.'

Guia **Relatório** do Power BI onde são criados os gráficos

Gráfico criado pelo **Visual Python**. Ainda sem conteúdo.

Ícone de criação de Gráfico via Python (**Visual Python**)

Editor de código Python

Módulo 36 – Integração Python com Power BI – Gráficos Seaborn no Power BI (3/4)

Selecione as colunas de interesse (Número da Venda, Data da Venda, Nome da Loja e Quantidade Vendida)

ATENÇÃO: Certifique-se que as colunas estão com a exibição correta:

- Número da Venda->Não resumir;
- Data da Venda ->Data da Venda

Feito isso, cole o código em Python que elaboramos no Jupyter e execute o Python Script.

The screenshot displays the Power BI interface. On the left, a line chart titled 'Numero da Venda, Data da Venda, Nome da Loja e Quantidade Vendida' shows 'Quantidade Vendida' on the y-axis (0 to 80) and 'Data da Venda' on the x-axis. The chart is faceted by 'Nome da Loja' with three series: 'Loja Contoso Austin' (blue), 'Loja Contoso Europe Online' (orange), and 'Loja Contoso Hartford' (green). On the right, the 'Visualizações' pane shows the 'Campos' section for the 'vendas_df' table. The fields 'Data da Ve...', 'Nome da L...', and 'Numero da...' are selected. The 'Valores' section shows 'Numero da Venda' set to 'Não resumir', 'Data da Venda' set to 'Data da Venda', 'Nome da Loja' set to 'Nome da Loja', and 'Quantidade Vendida' set to 'Quantidade Vendida'. Below the chart, the 'Editor de scripts Python' pane shows the following code:

```

1 # O código a seguir para criar um dataframe e remover as linhas duplicadas sempre é executado e age c
2
3 # dataset = pandas.DataFrame(Numero da Venda, Data da Venda, Nome da Loja, Quantidade Vendida)
4 # dataset = dataset.drop_duplicates()
5
6 # Cole ou digite aqui seu código de script:
7 import matplotlib.pyplot as plt
8 import seaborn as sns
9
10 sns.set_theme(style="darkgrid")
11
12 sns.lineplot(x="Data da Venda", y="Quantidade Vendida", hue="Nome da Loja", data=dataset)
13 plt.show()

```

An orange callout box points to the 'data=dataset' parameter in the line plot code, stating: 'Ao colar o código feito no Python para o Power BI, é necessário alterar o nome do dataframe para dataset'.

Gráfico criado, mas o tamanho ainda não é o ideal.

Ao invés de voltarmos para o Jupyter, vamos fazer esse tratamento diretamente no campo de Script Python do Power BI.

Altera o tamanho do gráfico.

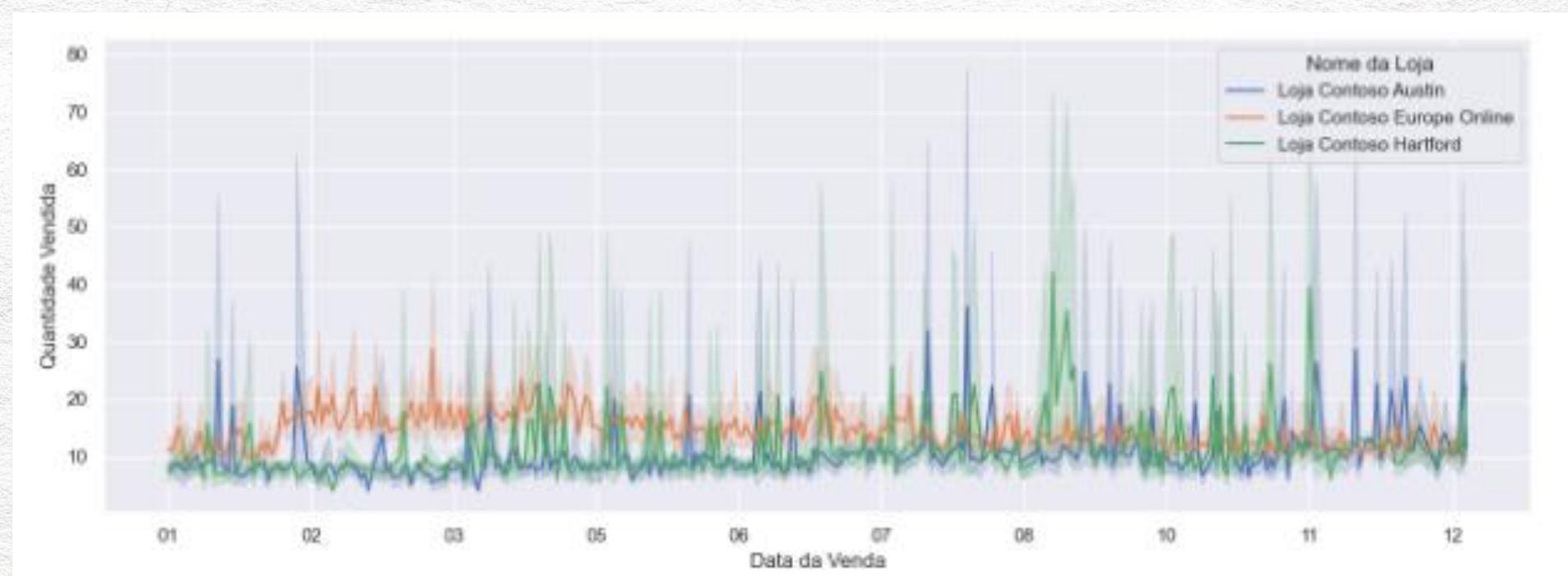
Formata os 12 dados para 12 meses

Editor de scripts Python

```
5
6 # Cole ou digite aqui seu código de script:
7 import matplotlib.pyplot as plt
8 from matplotlib.dates import DateFormatter
9 import seaborn as sns
10
11 sns.set_theme(style="darkgrid")
12
13 fig, ax = plt.subplots(figsize=(15,5))
14 sns.lineplot(x="Data da Venda", y="Quantidade Vendida", hue="Nome da Loja", data=dataset, ax=ax)
15 ax.xaxis.set_major_locator(plt.MaxNLocator(12))
16 ax.xaxis.set_major_formatter(DateFormatter('%m'))
17
18 plt.show()
```

Vamos importar uma nova biblioteca para editar o gráfico.

Altera o eixo X para apenas 12 dados





PYTHON

Módulo 37

TRANSFORMANDO PYTHON EM EXE



Nesse módulo, vamos aprender como transformar nossos códigos em programas **EXECUTÁVEIS** (que funcionam quando damos um duplo clique).

Até agora, sempre usamos o Jupyter para rodar nossos códigos, mas queremos fazer um arquivo que rode em qualquer computador mesmo que o Python não esteja instalado.

Para transformarmos nosso código em um arquivo executável, vamos seguir os passos abaixo:

- 1) **Passo 1:** Garantir que seu código **PRECISA FUNCIONAR SEMPRE**;
- 2) **Passo 2:** Transformar seu arquivo .ipynb em um arquivo .py;
- 3) **Passo 3:** Usar uma biblioteca de conversão para transformar o arquivo .py em .exe;
- 4) **Passo 4:** Testar e adaptar o que for necessário **A CADA ETAPA** de conversão.



ATENÇÃO!

Se seu PC é Linux ou MAC, esse módulo é em parte “redundante” pois por padrão esses sistemas operacionais já possuem dentro deles o Python instalado. **No entanto**, não deixe de aprender 😊

Módulo 37 – Transformando Python em exe – Python para exe com Arquivos Simples (1/4)

Antes de irmos para arquivos mais complexos, vamos pegar um código bem básico e aplicar os 4 passos citados anteriormente.

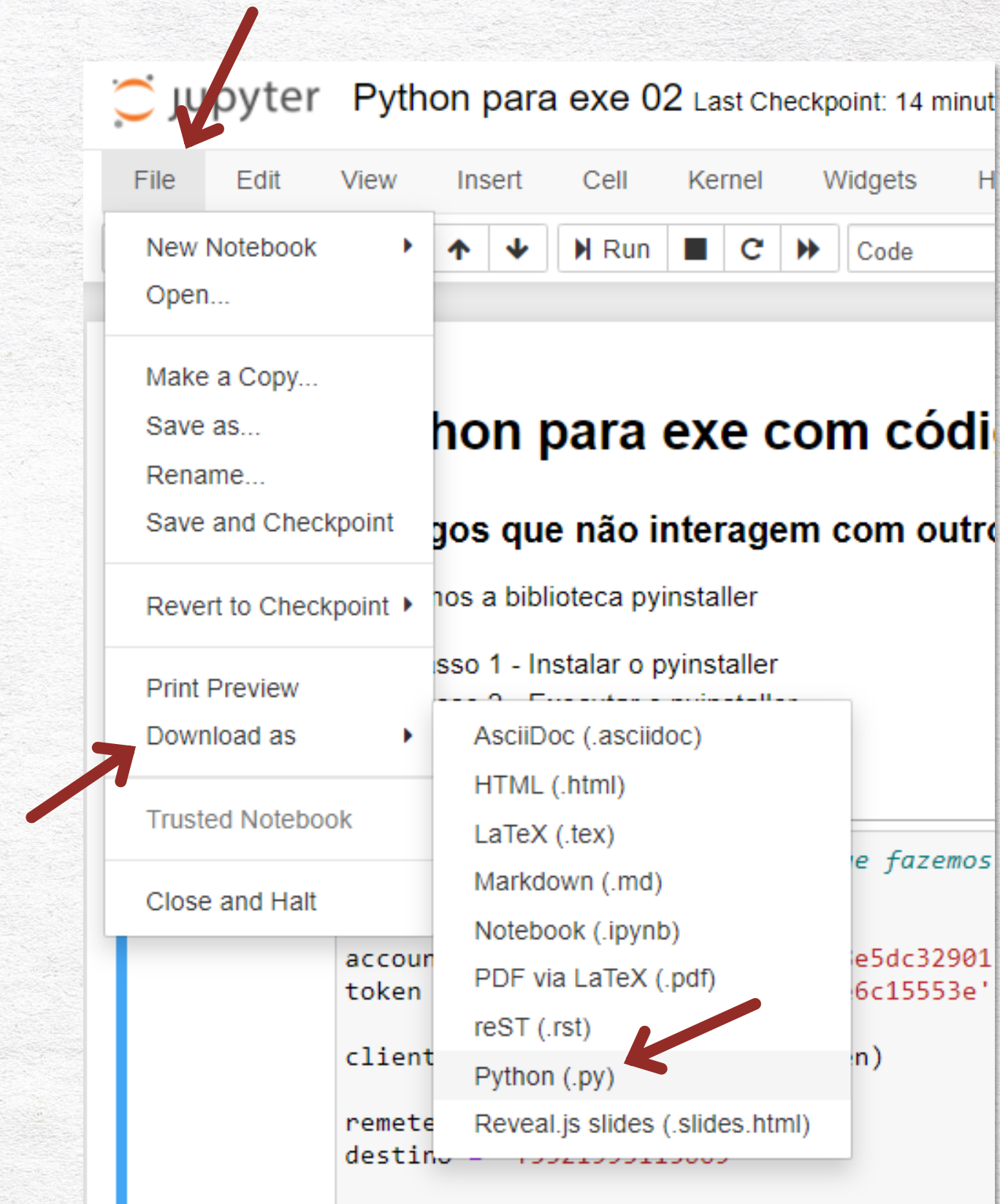
Vamos utilizar um código já conhecido por nós no módulo de APIs. Por que esse é considerando um código básico.

Essencialmente, como não estamos acessando informações dentro do nosso PC, a chance de erro é um pouco menor no caso de um executável.

No **Passo 1**: Garantir que seu código **PRECISA FUNCIONAR SEMPRE**; já está atendido, pois usamos este mesmo código no gabarito de módulos anteriores.

Para realizarmos o **Passo 2**: Transformar seu arquivo .ipynb em um arquivo .py; basta seguir os passos indicados ao lado no próprio Jupyter.

Feito isso, temos agora arquivo com extensão .py
Teste!



Baixado o arquivo **.py**, vamos executá-lo. Ele ainda não possui a extensão **.exe**, portanto, ainda precisa de um ambiente para sua execução.

Esse ambiente será o Python! E como acessar o Python fora do Jupyter?

- 1) Abra seu Anaconda Prompt;
- 2) Encontre o arquivo na pasta onde o mesmo foi baixado;
- 3) Execute-o usando o Python:

python "Python para exe 02.py"

- 4) Verifique se o código executou corretamente.

```
Pasta de C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Arquivos apostila\Módulo 29
12/04/2021 17:18 <DIR> .
12/04/2021 17:18 <DIR> ..
12/04/2021 16:58 <DIR> .ipynb_checkpoints
12/04/2021 17:13 3.072 Python para exe 02.ipynb
12/04/2021 17:14 1.676 Python para exe 02.py
2 arquivo(s) 4.748 bytes
3 pasta(s) 851.605.393.408 bytes disponíveis
```

Arquivo com extensão **.py** criado

Informando ao Anaconda Prompt que o comando será executado via Python

```
a\Módulo 29>python "Python para exe 02.py"
```

```
SMec4e91ebc0c14223ad320dec2bf6992c
```

Confirmação da execução

Agora vamos para o **Passo 3: Usar uma biblioteca de conversão para transformar o arquivo .py em .exe;**

Para isso vamos antes de tudo instalar uma biblioteca de conversão :

pyinstaller

Como já fizemos diversas vezes, vamos instalá-la através do nosso **Anaconda Prompt**.

pip install pyinstaller

Feito isso, vamos instalar nosso arquivo .py através dessa biblioteca.

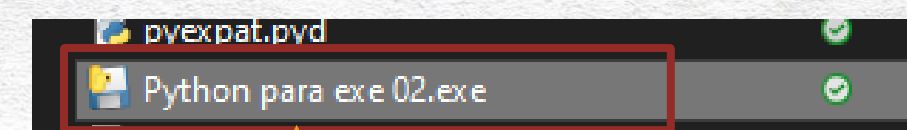
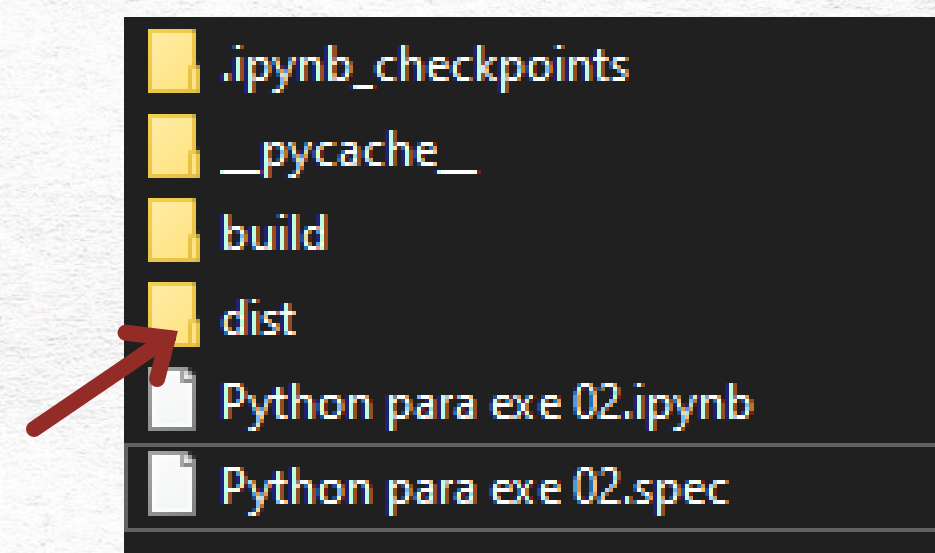
pyinstaller -w "Python para exe 02.py"

Após o processamento, o pyinstaller criou uma série de arquivos e entre eles o nosso arquivo executável 😊

Basta dar um duplo clique e agora temos um arquivo executável.

```
>pip install pyinstaller
```

```
>pyinstaller -w "Python para exe 02.py"
```



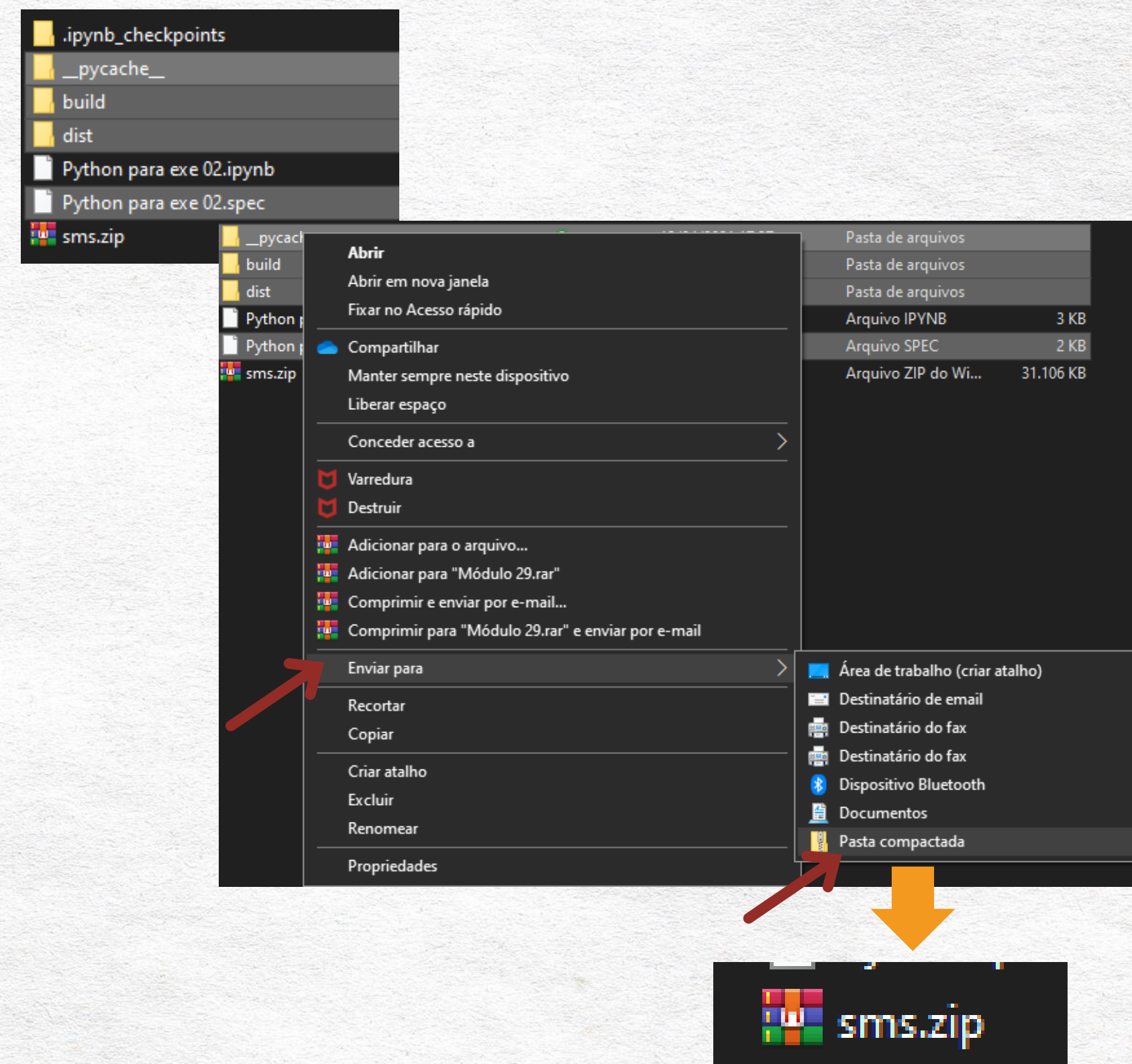
Dentro da pasta **dist** temos um arquivo executável (.exe)



Perceba que foram criadas várias pastas assim como vários arquivos. Para facilitar a vida do usuário podemos criar um atalho do arquivo para área de trabalho 😊

Assim, basta clicar para executar o programa.

Caso seja do seu interesse enviar esse programa para outras pessoas, basta juntar todos os arquivos que foram criados em uma única pasta compactada conforme apresentado ao lado.



O que permitirá reduzir o tamanho do executável será criá-lo dentro de um ambiente virtual onde apenas o que é **NECESSÁRIO** para seu funcionamento estará instalado.

Para esse exemplo, iremos criar uma pasta chamada **SMS MENOR** onde dentro dela, criaremos um ambiente virtual dedicado.

Para isso, usaremos o passo a passo aprendido no módulo 30 dessa apostila.

Como nesse caso, não é uma questão de versão do Python, vamos utilizar a versão 3.8 ao criarmos esse ambiente virtual.

```
9\SMS Menor>conda create -n programasms python=3.8
```

Criação do ambiente virtual

```
\SMS Menor>conda activate programasms
```

Ativação do ambiente virtual

```
(programasms) \SMS Menor>python SMSmenor.py
```

Testando o arquivo .py baixado do Jupyter

```
(programasms) \SMS Menor>pip install twilio
```

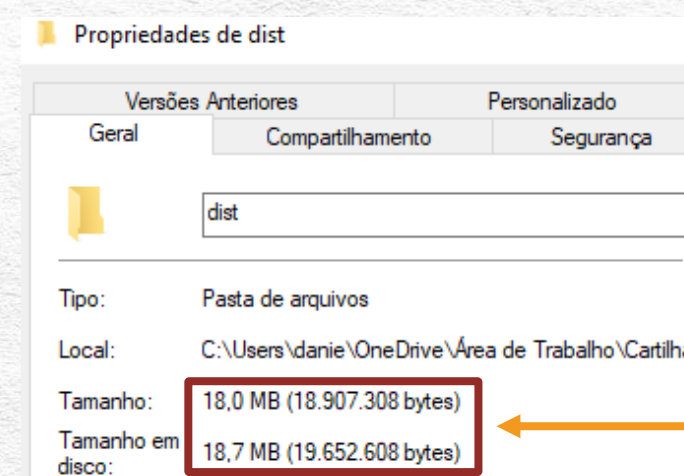
Instalando o Twilio DENTRO do ambiente virtual. Só está sendo instalado pois é necessário para execução do código

```
(programasms) \SMS Menor>pip install pyinstaller
```

Instalando pyinstaller

```
(programasms) >pyinstaller -w SMSmenor.py
```

Executando o pyinstaller



Executável com apenas 18 MB 😊

Antes de irmos para adaptação do código, vamos entender o problema e o código original.

Para a Hashtag realizar anúncios no Youtube, ela precisa identificar quais são os vídeos em que ela deseja anunciar.

Para isso, temos um código que busca os links desses vídeos de forma automática.

Se rodarmos esse código no computador da Hashtag sempre rodará pois ele está sendo rodado no computador que foi criado (nesse caso o pc do Alon, conforme exemplo ao lado).

Vamos dizer que agora queremos rodar esse código no computador do Lira, como fazer?

Vamos precisar criar um código genérico que funcione em qualquer computador.

```
#Ler csv
buscas_df = pd.read_csv(r'C:\Users\alomp\Downloads\Canais Youtube.csv', encoding = 'ISO-8859-1', sep=';')
display(buscas_df.head())
```

O fato deste código estar vinculado a uma pasta específica dentro do usuário Alon, nos impede de criar um arquivo executável que possa ser usado por diversas pessoas.

Iremos precisar criar um código genérico que se aplique a qualquer usuário

Para resolver esse problema, temos 3 soluções possíveis:

1) Indicar apenas o nome do arquivo:

Nesse caso, é necessário que o arquivo esteja OBRIGATORIAMENTE na mesma pasta do nosso programa python.

2) Usar a biblioteca 'os' ou a 'pathlib':

Essas bibliotecas, permitem buscar o caminho do computador. Sendo assim, usando esse código poderíamos o local do pc antes da execução do código em si.

3) Criar uma interface para o usuário escolher o local do arquivo:

Usando a biblioteca TKINTER, ofereceremos ao usuário indicar onde está o arquivo que será lido.

Caso 1 - Indicar apenas o nome do arquivo:

```
buscas_df = pd.read_csv(r'C:\Users\alomp\Downloads\Canais Youtube.csv', encoding = 'ISO-8859-1', sep=';')
display(buscas_df.head())
```

```
buscas_df = pd.read_csv(r'Canais Youtube.csv', encoding = 'ISO-8859-1', sep=';')
display(buscas_df.head())
```

Caso 2 - Usar a biblioteca 'os' ou a 'pathlib'

```
buscas_df = pd.read_csv(r'C:\Users\alomp\Downloads\Canais Youtube.csv', encoding = 'ISO-8859-1', sep=';')
display(buscas_df.head())
```

```
import os
```

```
caminho = os.getcwd()
```

Identifica o local do arquivo antes da execução da extração

```
buscas_df = pd.read_csv(os.join(caminho,r'Canais Youtube.csv'), encoding = 'ISO-8859-1', sep=';')
display(buscas_df.head())
```

Caso 3 -Criar uma interface para o usuário escolher o local do arquivo

```
buscas_df = pd.read_csv(r'C:\Users\alomp\Downloads\Canais Youtube.csv', encoding = 'ISO-8859-1', sep=';')
display(buscas_df.head())
```

```
from tkinter import *
import tkinter.filedialog
from tkinter import messagebox
```

```
janela = Tk()
arquivo=tkinter.filedialog.askopenfilename(title = 'Selecione o arquivo com os Canais do Youtube a serem mapeados')
janela.destroy()
```

```
buscas_df = pd.read_csv(arquivo, encoding = 'ISO-8859-1', sep=';')
display(buscas_df.head())
```



Usaremos o caso 3 daqui para frente, pois nesse caso, o risco de erro é menor visto que o usuário pode escolher o local do arquivo.

Vamos entender um pouco melhor o código apresentado anteriormente.

The diagram illustrates the execution flow of the code. It shows a sequence of steps: 'Cria Janela Tkinter' (Create Tkinter Window) and 'Destrói a janela Tkinter' (Destroy Tkinter Window). The code snippet shows the creation of a Tkinter window, opening a file dialog to select a CSV file, and then displaying the first few rows of the data. The file dialog is shown as a screenshot, with the file 'Canais Youtube.csv' selected. The annotation explains that the Tkinter window is used to open a file selection dialog for the user.

```
from tkinter import *
import tkinter.filedialog
from tkinter import messagebox

janela = Tk()
arquivo = tkinter.filedialog.askopenfilename(title = 'Selecione o arquivo com os Canais do Youtube a serem mapeados')
janela.destroy()

buscas_df = pd.read_csv(arquivo, encoding = 'ISO-8859-1', sep=';')
display(buscas_df.head())
```

Através do TkInter abre uma janela para seleção do arquivo desejado pelo usuário

Cria Janela Tkinter

Destrói a janela Tkinter

Selecione o arquivo com os Canais do Youtube a serem mapeados

Nome	Status	Data de modifica
.ipynb_checkpoints	✓	12/04/2021 18:57
Canais Youtube.csv	✓	12/04/2021 18:42
Gabarito - Python para exe 04.ipynb	✓	12/04/2021 18:56
Python para exe 04 (1).ipynb	✓	15/04/2021 11:10

No entanto, a partir do momento que o usuário interage com o nosso código, é importante que ele também seja comunicado que o mesmo foi executado com sucesso.

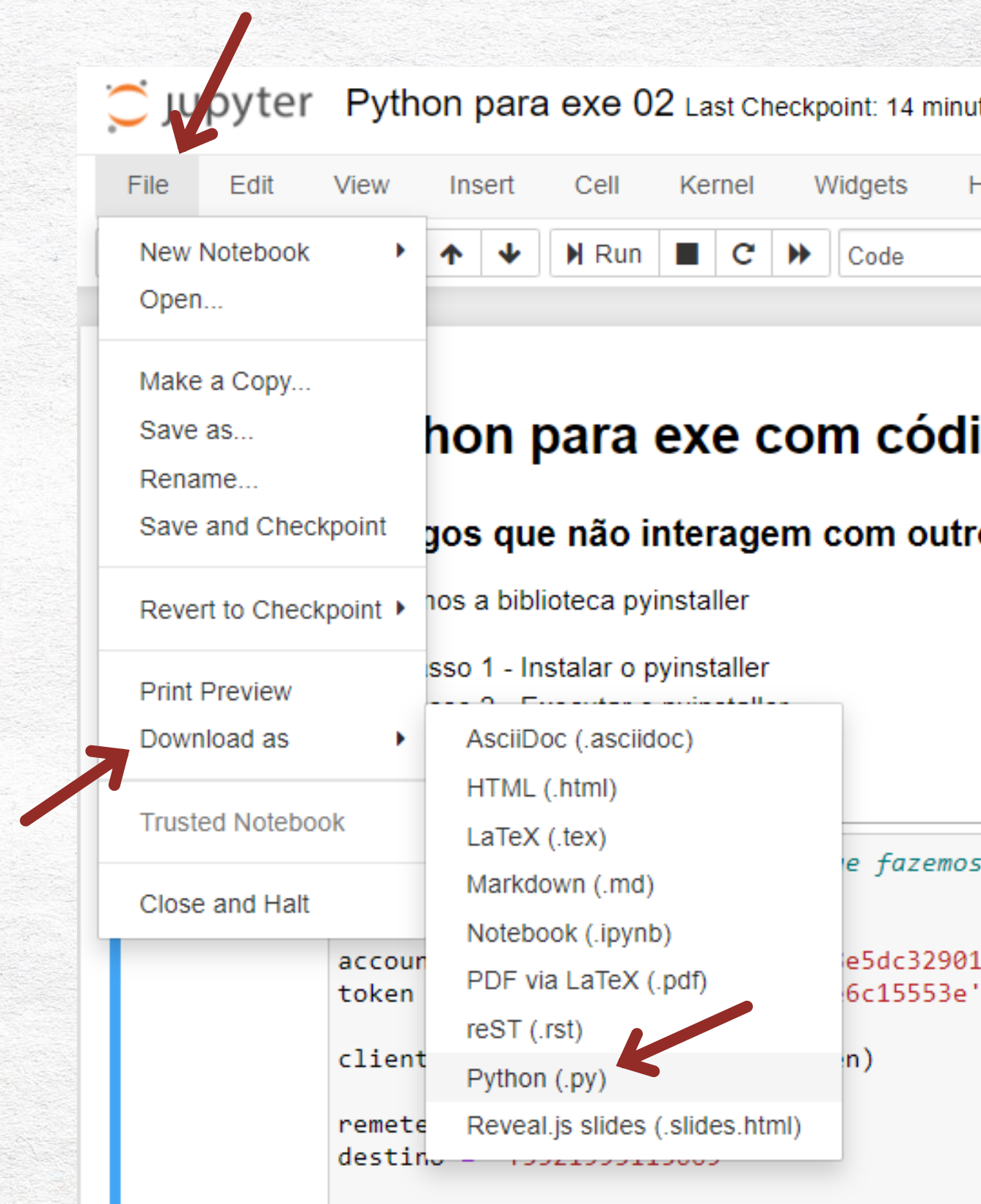
Para isso, vamos usar novamente o Tkinter, mas agora apresentando uma mensagem conforme indicado abaixo.

Com isso, podemos garantir que o programa rodará em qualquer computador ou qualquer usuário que venha a utilizar o programa.

```
#salvando o resultado em um csv
hrefs_df = pd.DataFrame(hrefs)
hrefs_df.to_csv(r'Canais Prontos.csv', sep=',', encoding='utf-8')
root = Tk()
messagebox.showinfo("Programa Finalizado com Sucesso", "Seu arquivo csv foi gerado com sucesso na pasta do Programa")
root.destroy()
```

Bem, agora que temos nosso programa rodando, vamos transformá-lo em um executável.

Assim como vimos anteriormente, o primeiro passo é baixar o nosso arquivo para a versão **.py**, conforme apresentado ao lado.

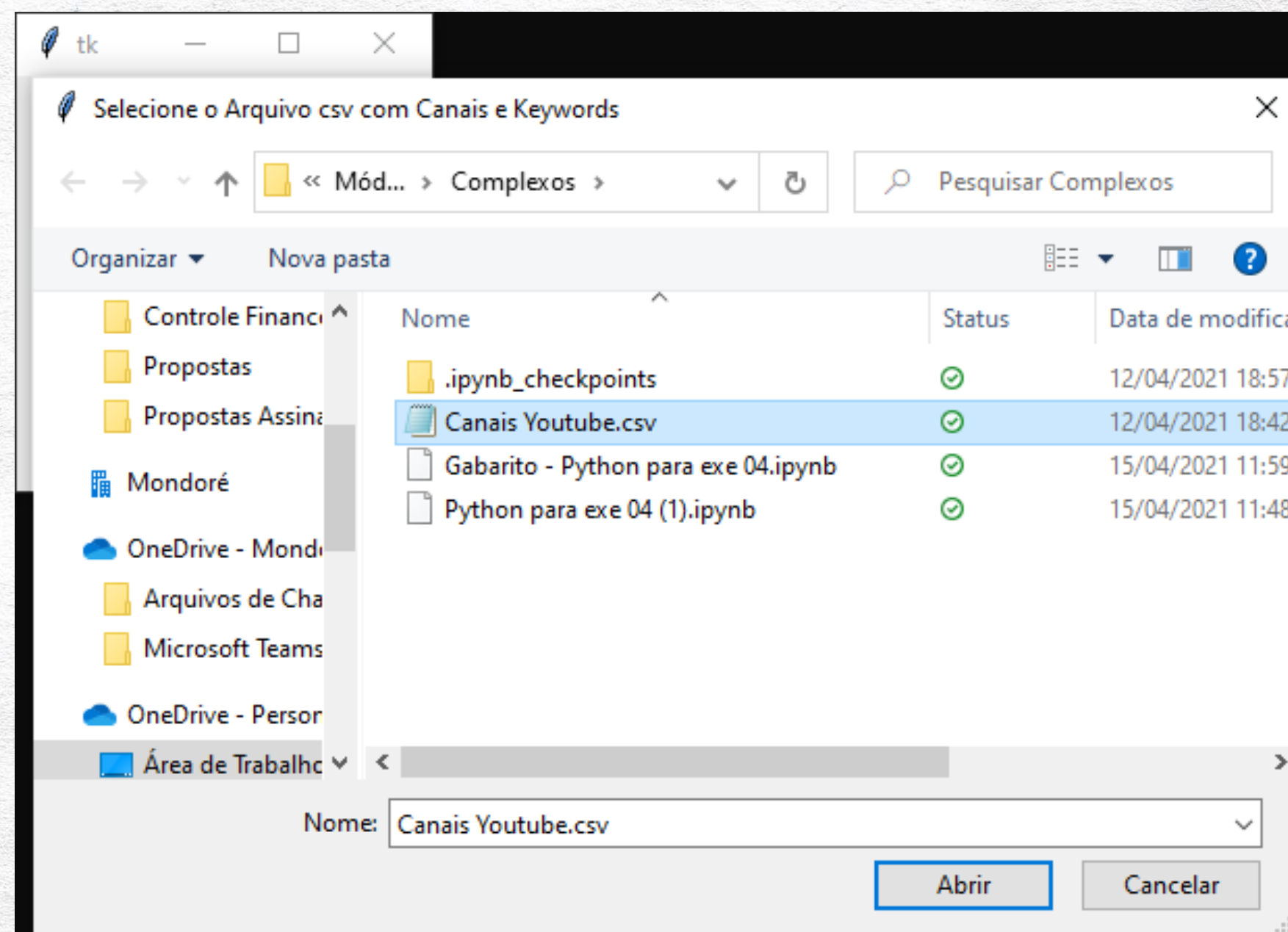


Usando o **Anaconda Prompt** execute seu arquivo.py e veja se tudo funciona bem.

Ocorrendo tudo certo, podemos ir para o próximo passo que já vimos anteriormente.

Transformar esse arquivo em um arquivo executável.

```
(base) >python "Gabarito - Python para exe 04.py"
```



Para isso usaremos uma nova biblioteca:

auto-py-to-exe

Assim como fizemos em exemplos anteriores, vamos instalar essa biblioteca utilizando o comando abaixo:

pip install auto-py-to-exe

Após a instalação, basta executarmos nossa biblioteca via prompt de comando do Anaconda usando:

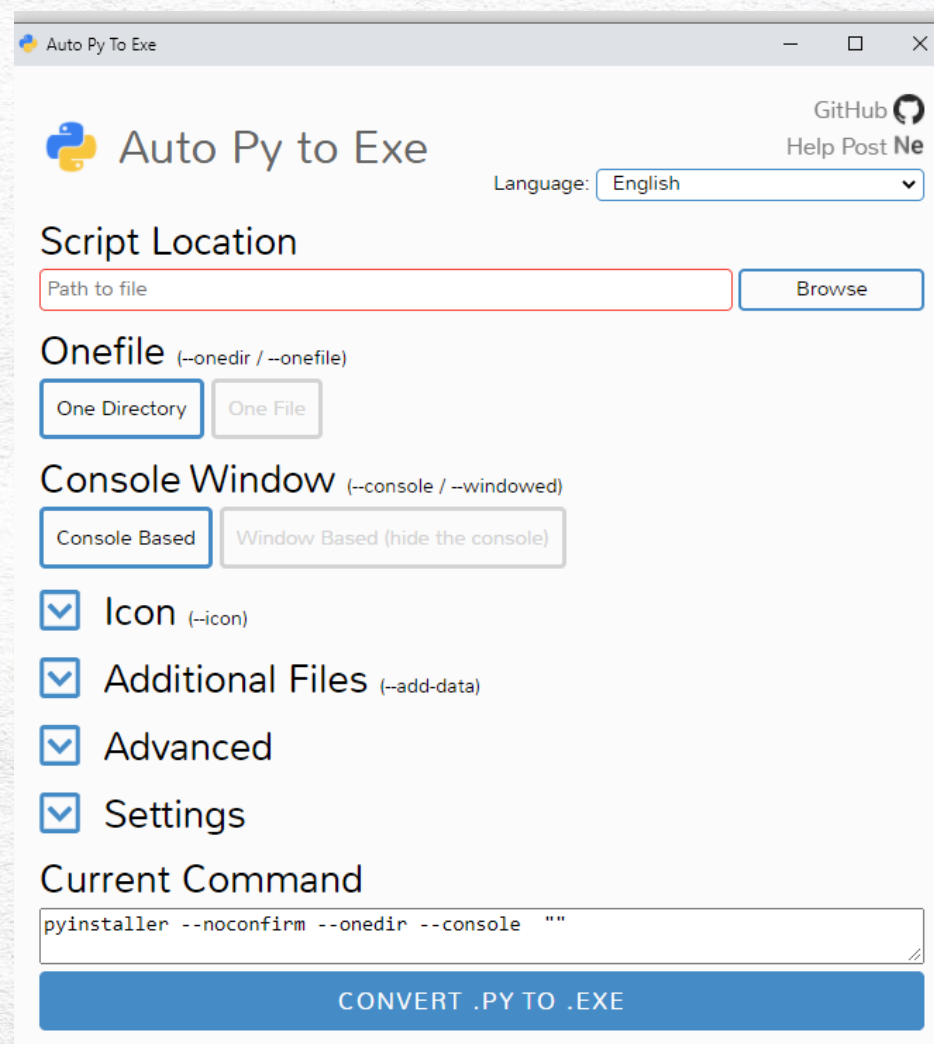
auto-py-to-exe

Essa biblioteca, nos oferece uma interface gráfica para criarmos esse novo executável, conforme apresentado ao lado.

Vamos entender no próximo slide como utilizá-lo.

```
(base) pip install auto-py-to-exe
```

```
(base) auto-py-to-exe
```



Script Location: Qual o script que você deseja transformar em executável. Para nosso caso específico usaremos "Gabarito - Python para exe 04.py"

Onefile: Decisão se tudo deverá ser comprimido em um arquivo único ou uma pasta com tudo;

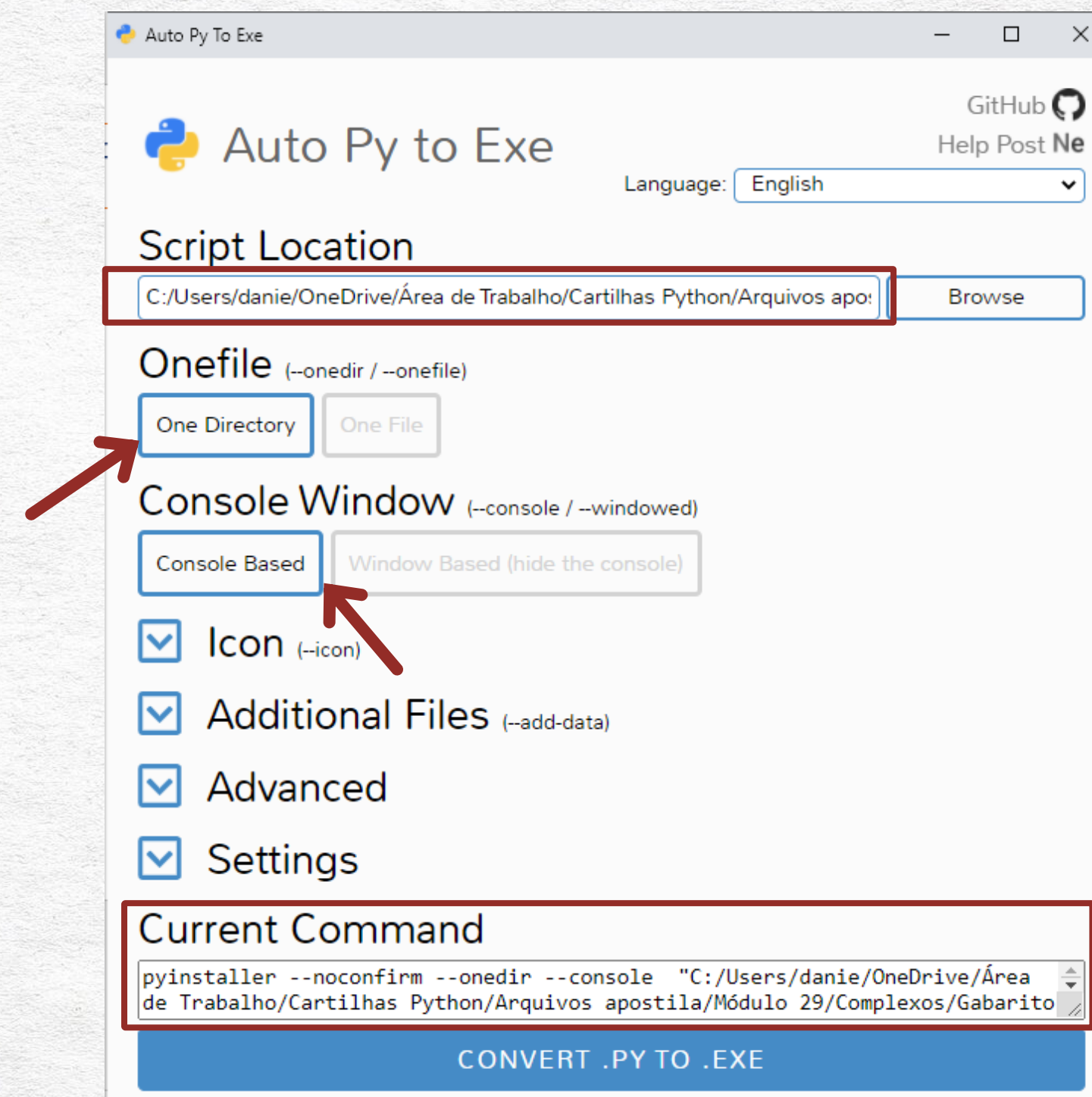
Console Window: Indicado usar Console Based;

Icon: Caso você queira usar um ícone especial para o seu programa, é só importar a imagem;

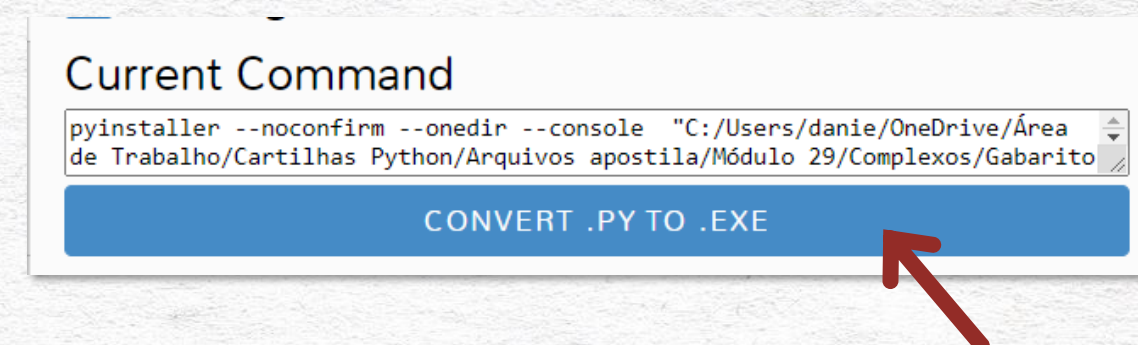
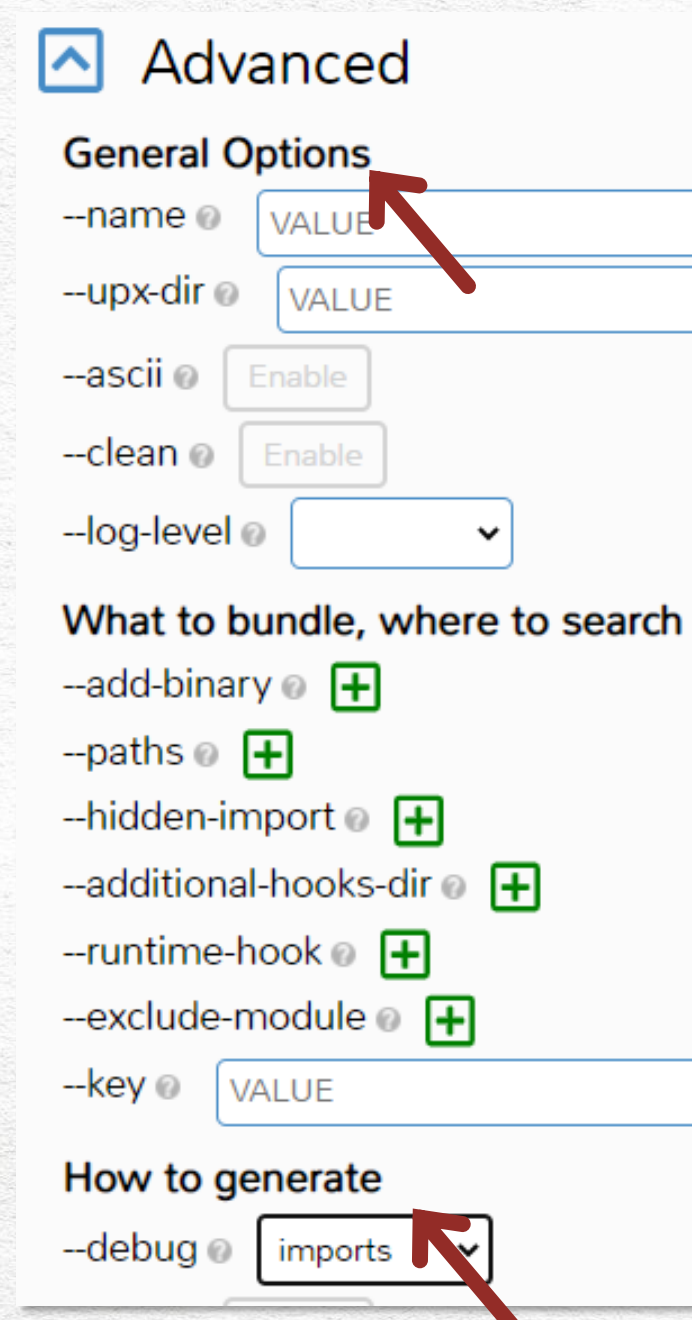
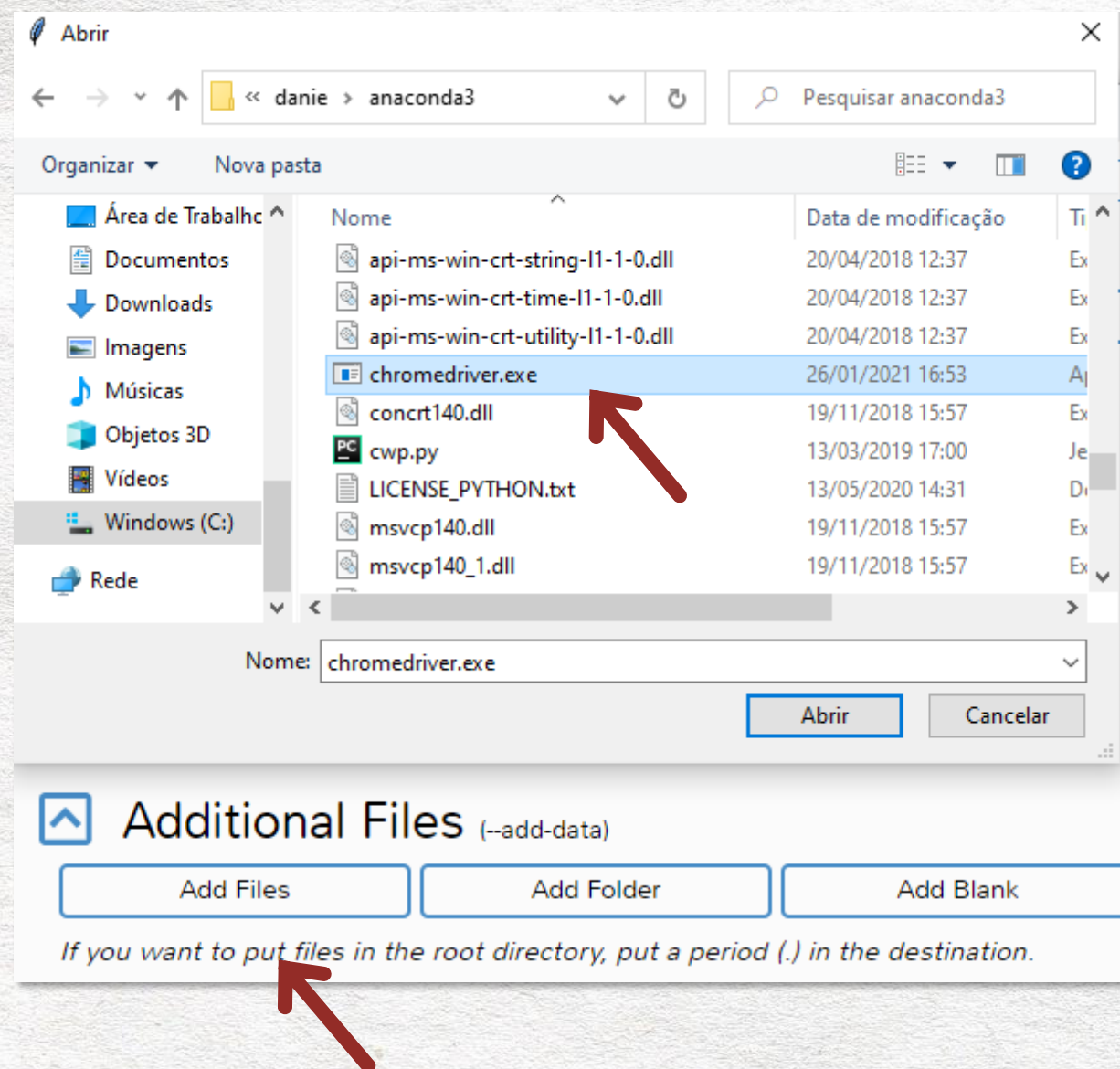
Additional Files: Onde se insere arquivos que são necessários para execução do código. Para nosso exemplo o chromedriver.exe por exemplo;

Advanced: Acionar -debug todas as opções são possíveis com exceção de **vazio**;

Current Command: Cria um código para ser executado no prompt do anaconda (perceba que se utiliza do **pyinstaller**)



Aqui colocamos todas configurações necessárias. Feitas as configurações, basta clicar em **convert.py to.exe**.



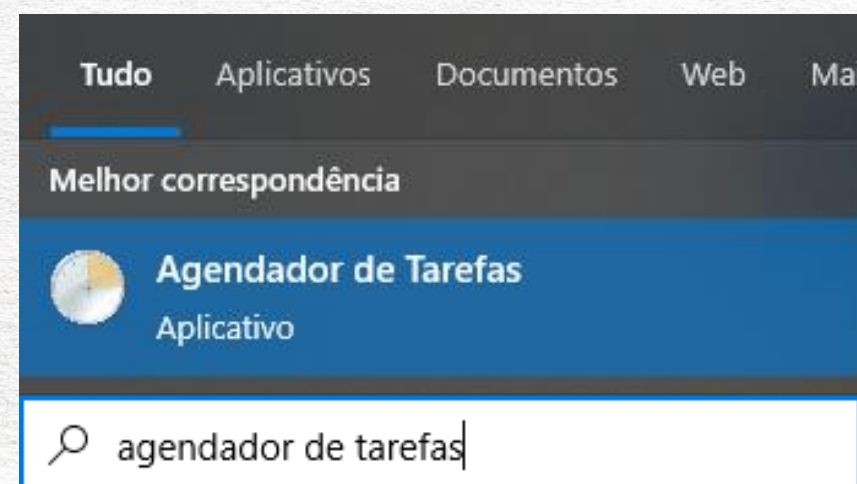
Pronto! Agora basta acessar a pasta **OUTPUT** criada pelo **Auto-py-to-exe** e executar 😊

Agora vamos ver como rodar o seu código automaticamente, sempre que você precisar. Lembrando que o seu computador precisa estar ligado para rodar. O passo a passo que será mostrado aqui, funcionará para o Windows. Para o Linux ou Mac, é recomendado que você encontre uma ferramenta parecida com a desta aula.

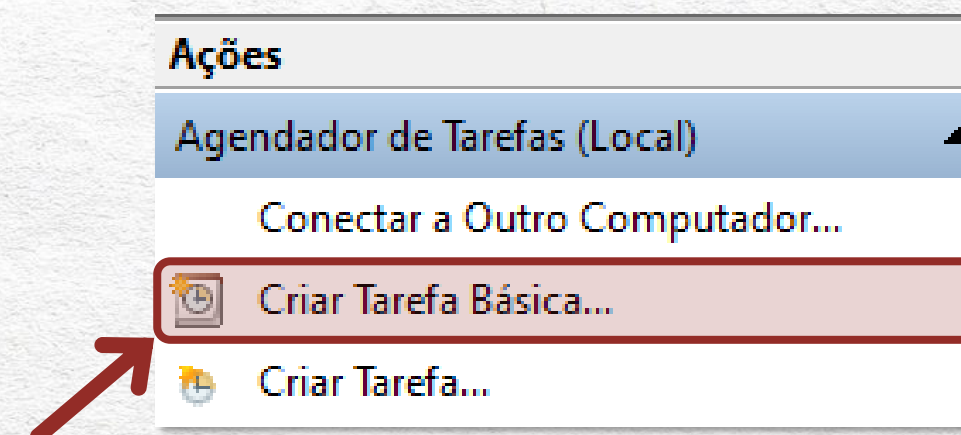
Vamos usar o Agendador de Tarefas do Windows. O agendador de tarefas permite agendar uma tarefa para que ela seja executada com uma determinada frequência (semanalmente, diariamente, sempre que ligar o computador, etc).

Para seguir esse passo a passo, você precisará ter um arquivo `.py` do seu código. Se você estiver usando o Jupyter, é só transformar o seu arquivo `.ipynb` em um arquivo `.py` clicando em **File > Download as > Python (.py)**.

1 Abra o Agendador de Tarefas



2 Clique em “Criar Tarefa Básica”



Após isso, uma tela se abrirá e você deverá escrever o nome da sua tarefa no campo **Nome** e clicar em **Avançar** para prosseguir.

Assistente para Criação de Tarefa Básica

Criar uma Tarefa Básica

Criar uma Tarefa Básica
Disparador
Ação
Concluir

Use este assistente para agendar rapidamente uma tarefa comum. Para obter mais opções ou configurações avançadas, como ações ou disparadores de várias tarefas, use o comando Criar Tarefa do painel Ações.

Nome:

Descrição:

< Voltar **Avançar >** Cancelar

Nesta tela precisamos escolher a frequência em que o código será executado. Se você precisar que seu código rode todo dia ao meio-dia, por exemplo, escolha **diariamente**, pois depois faremos a configuração do horário. Se escolher **Ao iniciar o computador**, precisará fazer uma configuração adicional que veremos adiante.

Selecione a frequência e clicar em **Avançar** para prosseguir.



ATENÇÃO!

O Agendador de tarefas não ligará o seu computador para executar o seu código! É necessário garantir que o seu computador esteja ligado e ativo no horário agendado.

Agora escolha uma data e um horário para iniciar e escolha a cada quantos dias a ação deve se repetir.

No nosso caso, essa ação se repetirá todo dia e iniciará no dia 26 de abril de 2022 às 11 horas.

O computador deve estar ligado e ativo neste dia e horário.

Clique em **Avançar**.

Assistente para Criação de Tarefa Básica

Diariamente

Criar uma Tarefa Básica

Disparador

Iniciar: 26/04/2022 11:00:00 ☐ Sincronizar fusos horários

Diariamente

Repetir a cada: 1 dias

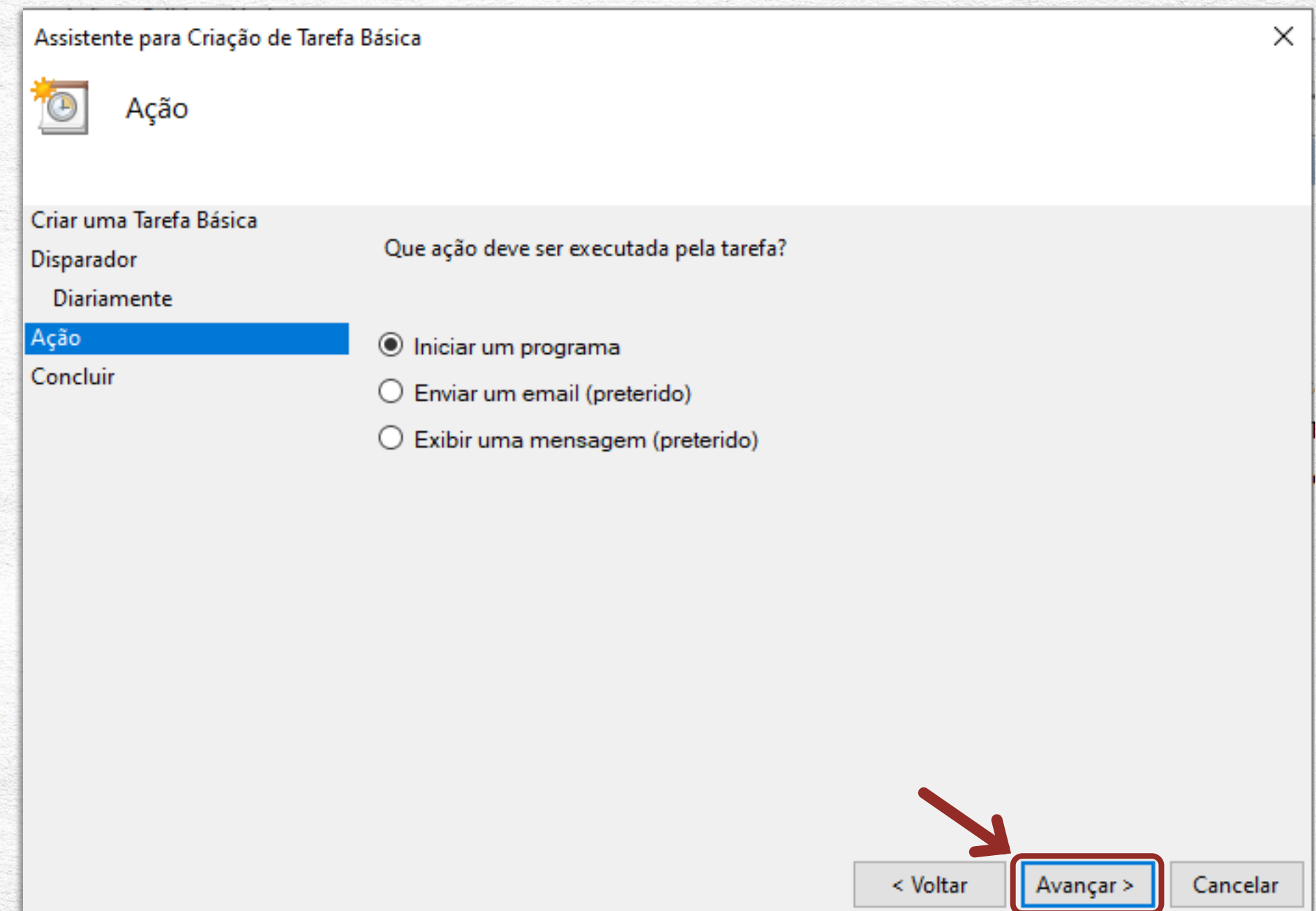
Ação

Concluir

< Voltar Avançar > Cancelar

Você deve selecionar a opção **Iniciar um programa**, porque você quer que ele rode o seu código em python.

Clique em **Avançar**.



Assistente para Criação de Tarefa Básica

Iniciar um Programa

Criar uma Tarefa Básica

Disparador

Diariamente

Ação

Iniciar um Programa

Concluir

Programa/script:

Procurar...

Adicione argumentos (opcional):

Iniciar em (opcional):

< Voltar Avançar > Cancelar

Selecionar o executável do python (**python.exe**) que o seu código usa para funcionar. Por exemplo, "C:\Users\[Nome de usuário]\anaconda3\python.exe".

Nome do arquivo que você quer executar. Por exemplo, "**Gabarito.py**".

Local onde está o arquivo que você quer executar. Por exemplo, local do "**Gabarito.py**".

Após preencher os campos, clique em **Avançar** e depois clique em **Concluir**.

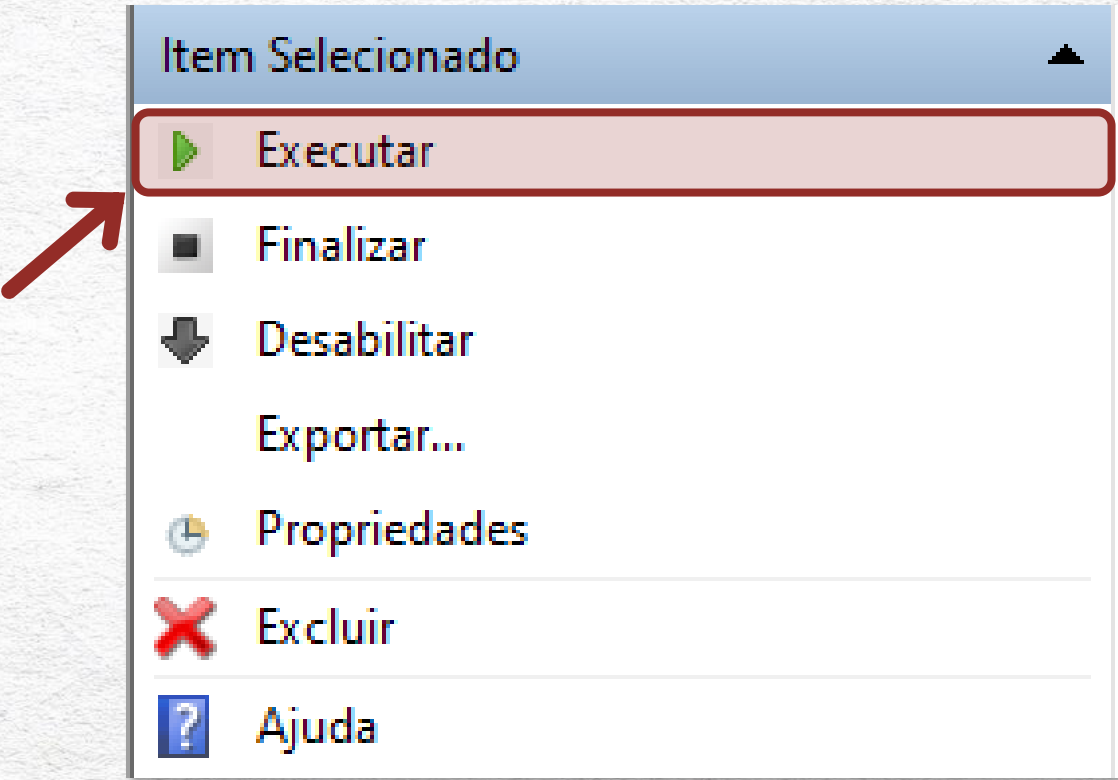
Clicando em  **Biblioteca do Agendador** as tarefas agendadas irão aparecer

Nome	Status	Disparadores	Horário da Próxima Execução
 TesteAgendador	Pronto	Todos os dias às 11:00	26/04/2022 11:00:00

Para testar se o agendador está executando seu código corretamente, clique na tarefa que você acabou de agendar e selecione **Executar**. Ele terá que rodar o seu código corretamente.

O processo é o mesmo para rodar o seu código semanalmente, mensalmente ou em qualquer outra frequência.

Pronto! Agora o seu código irá rodar automaticamente.



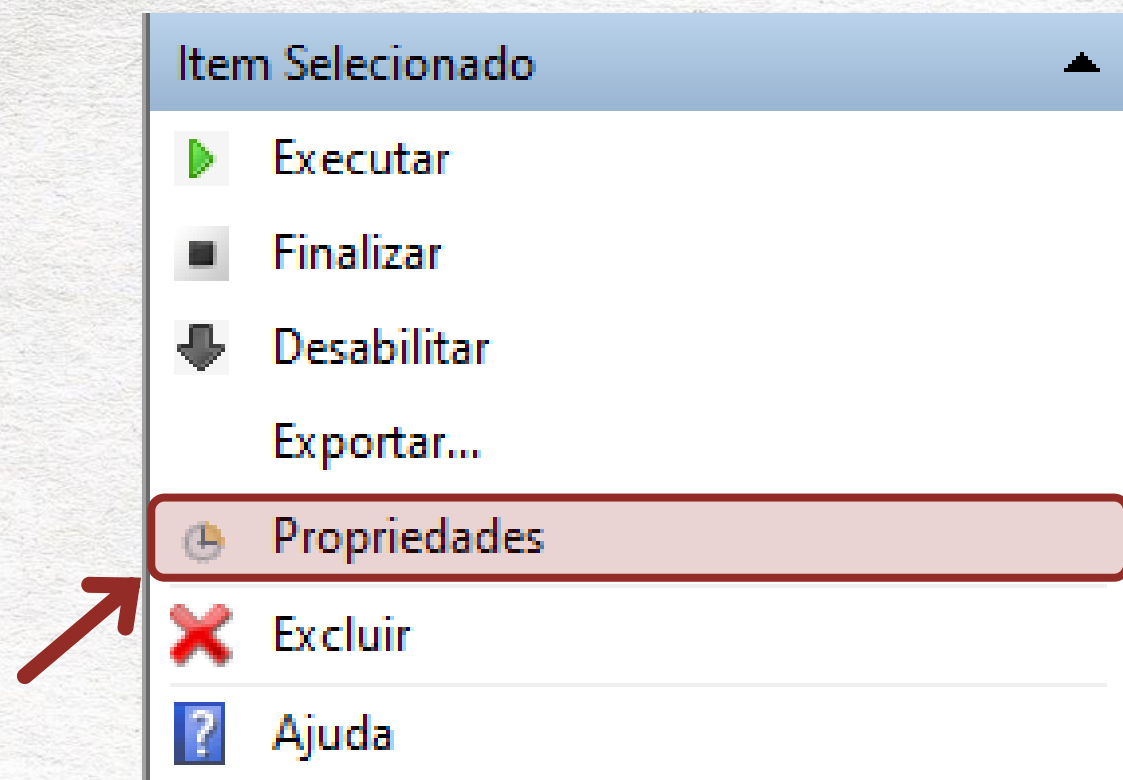
Agora vamos ver os ajustes necessários para quando quisermos que o nosso código rode sempre que iniciar o computador. Para isso, vamos criar uma outra tarefa seguindo o mesmo passo a passo anterior, com o nome da tarefa **“NovaTarefa”** e a frequência **“Ao iniciar o computador”**.

Nome	Status	Disparadores
 NovaTarefa	Pronto	Ao inicializar o sistema

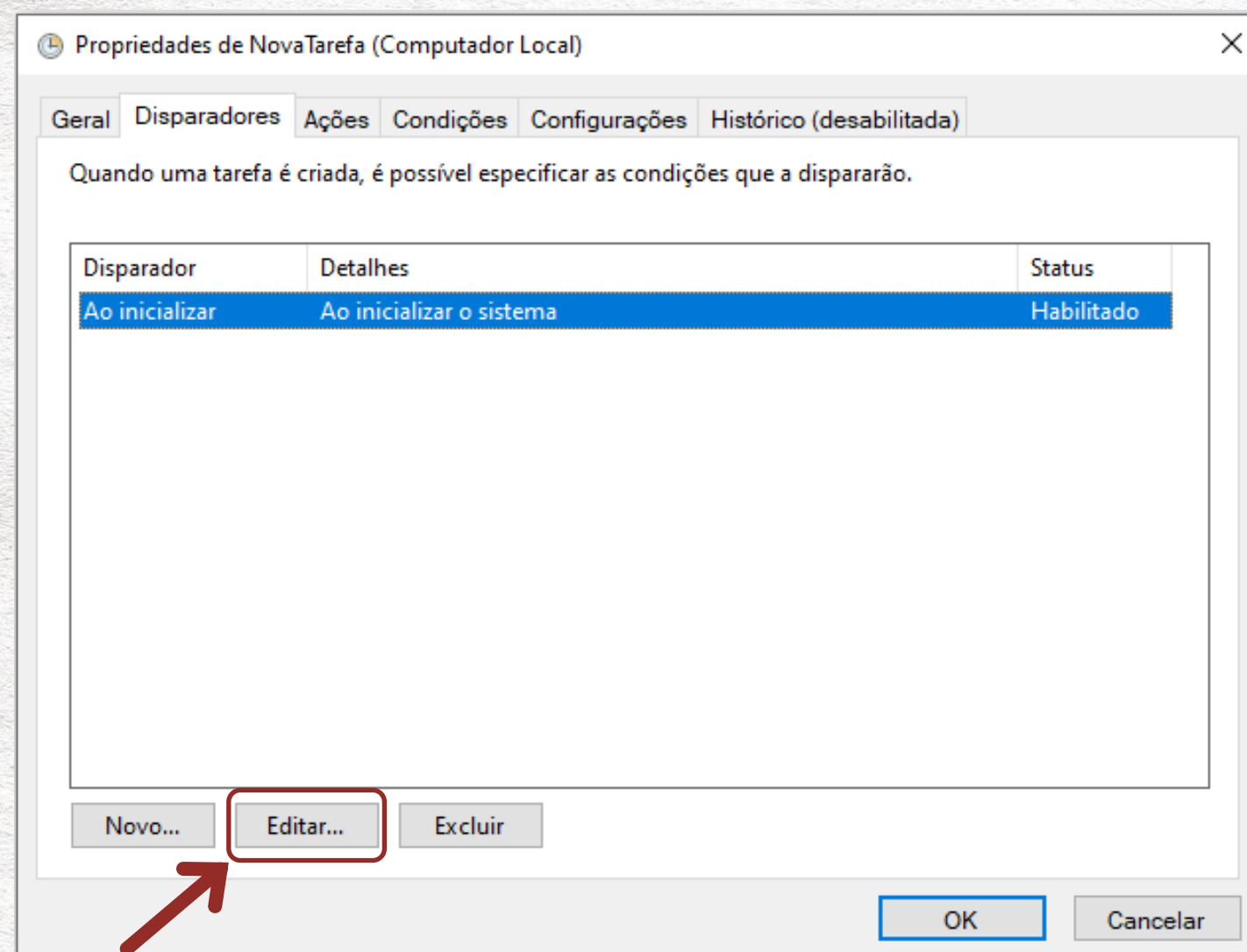
A princípio, o que esperamos é que esta tarefa funcione, mas não vai funcionar. Por padrão quando você seleciona que você quer executar uma tarefa sempre que iniciar o computador, ele executa a tarefa somente quando o usuário estiver conectado (quando você tiver feito login com o seu usuário na sua máquina). Porém quando você liga o computador, primeiro o computador é iniciado e depois você faz o login no seu usuário. A tarefa tenta executar antes de você fazer o login, o que vai gerar um erro.

Para corrigir isso, você deve selecionar a sua tarefa e clicar em propriedades

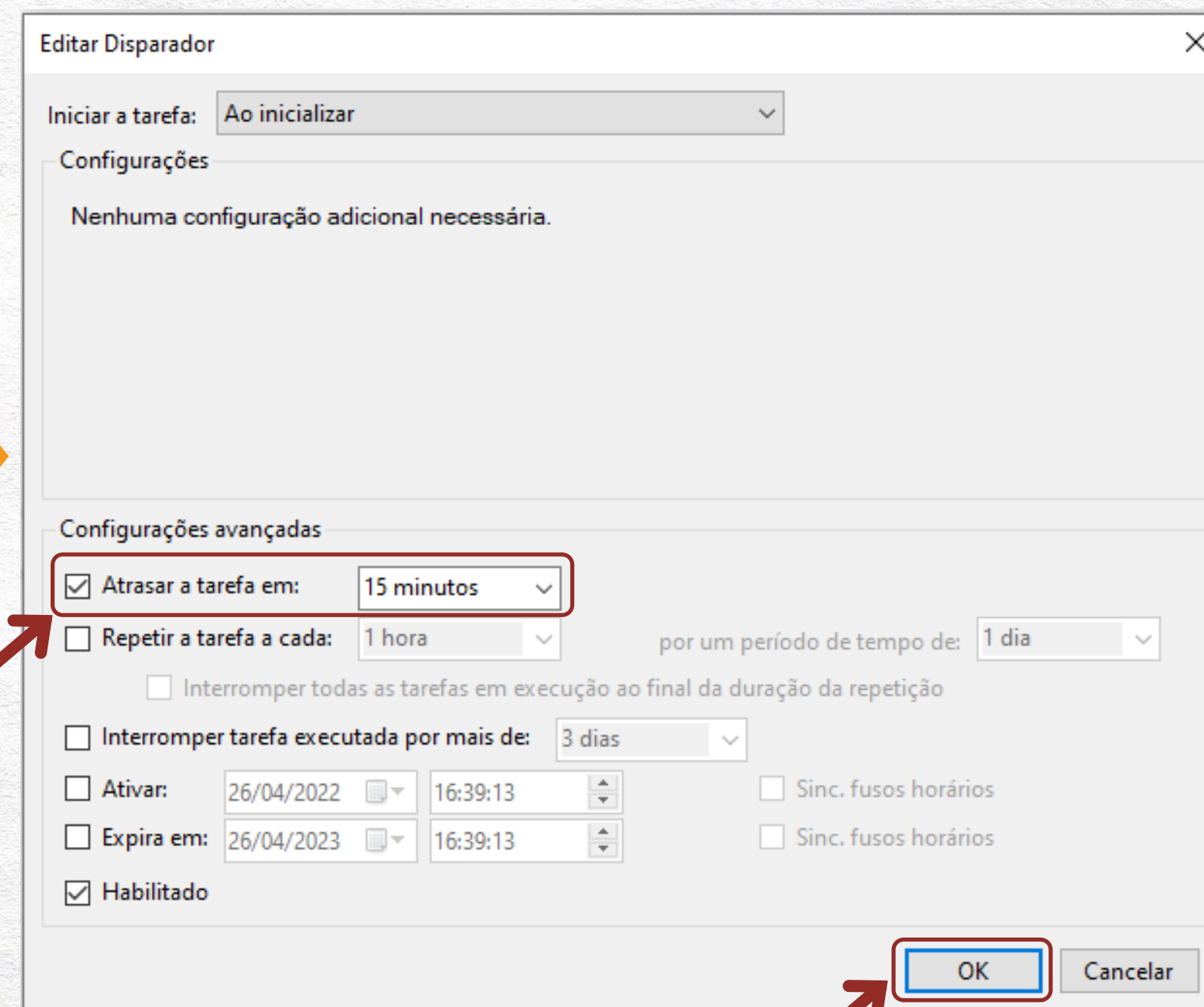
Nome	Status	Disparadores
 NovaTarefa	Pronto	Ao inicializar o sistema



Clique em **Editar**



Marque a opção **Atrasar a tarefa em** e escolha quanto tempo após a inicialização você quer que a tarefa seja executada



Agora 15 minutos após a inicialização do computador, sua tarefa será executada. Nesse tempo você pode fazer login no seu computador, começar a fazer outras coisas e 15 minutos depois da inicialização do seu computador, a sua tarefa será executada.



ATENÇÃO!

Se a sua intenção ao escolher a frequência “**Ao iniciar o computador**” for executar o seu código apenas uma vez por dia, será melhor escolher **diariamente** em um horário que você sabe que seu computador estará ligado e ativo. Porque escolhendo “**Ao iniciar o computador**”, se acontecer qualquer problema que faça seu computador desligar ou reiniciar, você terá que ligá-lo novamente. E assim, a sua tarefa será executada mais uma vez.